IoT Architectures & Protocols

By
A.Sritulasi
Assistant Professor
Dept of ECE

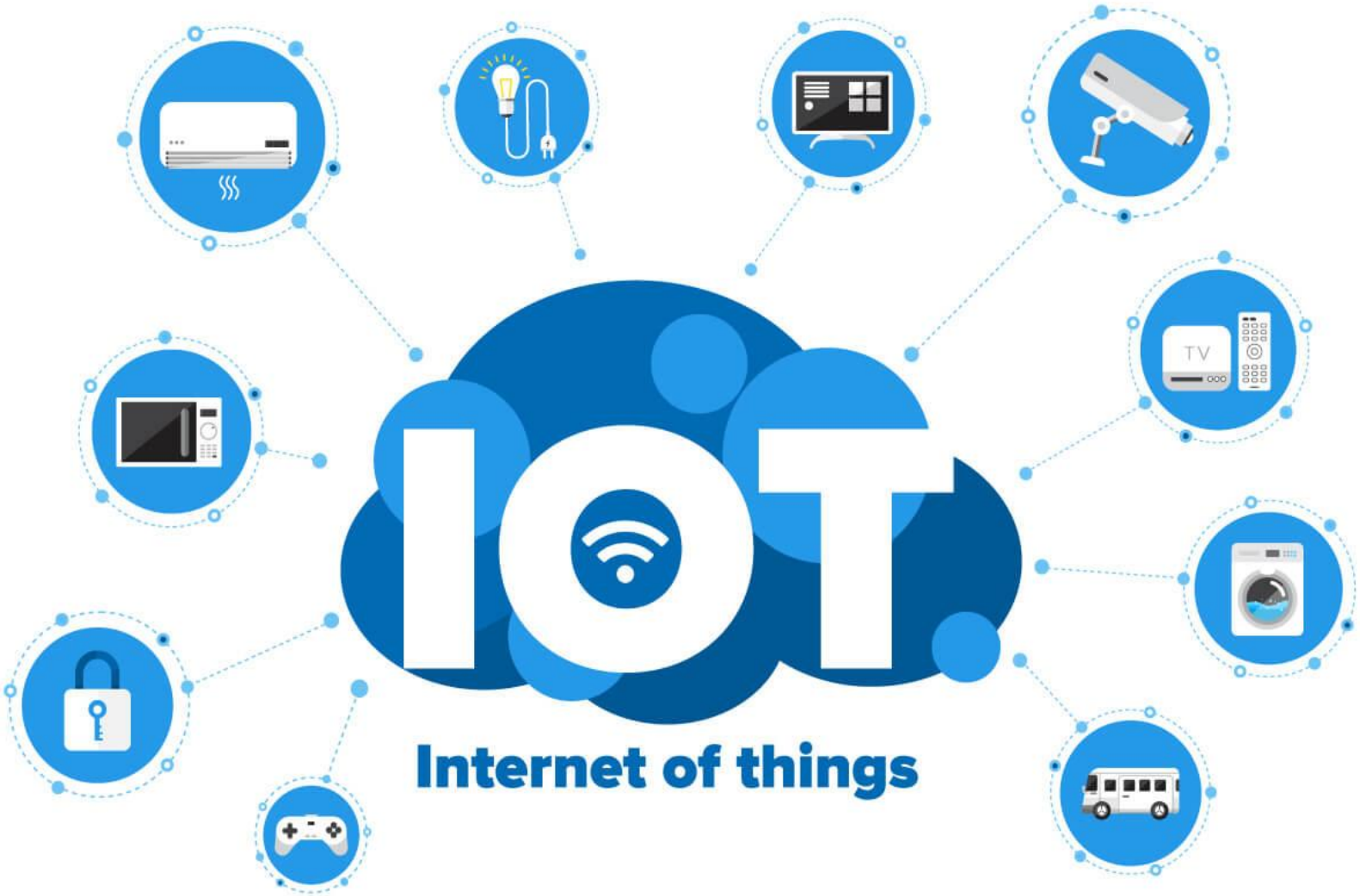
Let Us Watch



- Lloyd Wi-Fi Inverter AC advertisement-Mahesh Babu- from YouTube



What is IoT ?





Internet-Interconnected Network

- The internet, sometimes simply called the net, is a worldwide system of interconnected computer networks and electronic devices that communicate with each other using an established set of protocols.
- The internet was conceived by the Advanced Research Projects Agency (ARPA) of the U.S. government in 1969.





Why to Learn IoT ?



Everyday Things get connected

For Smarter Tomorrow

IOT in Agriculture

Embedded System

Smart Retail

Internet of Things

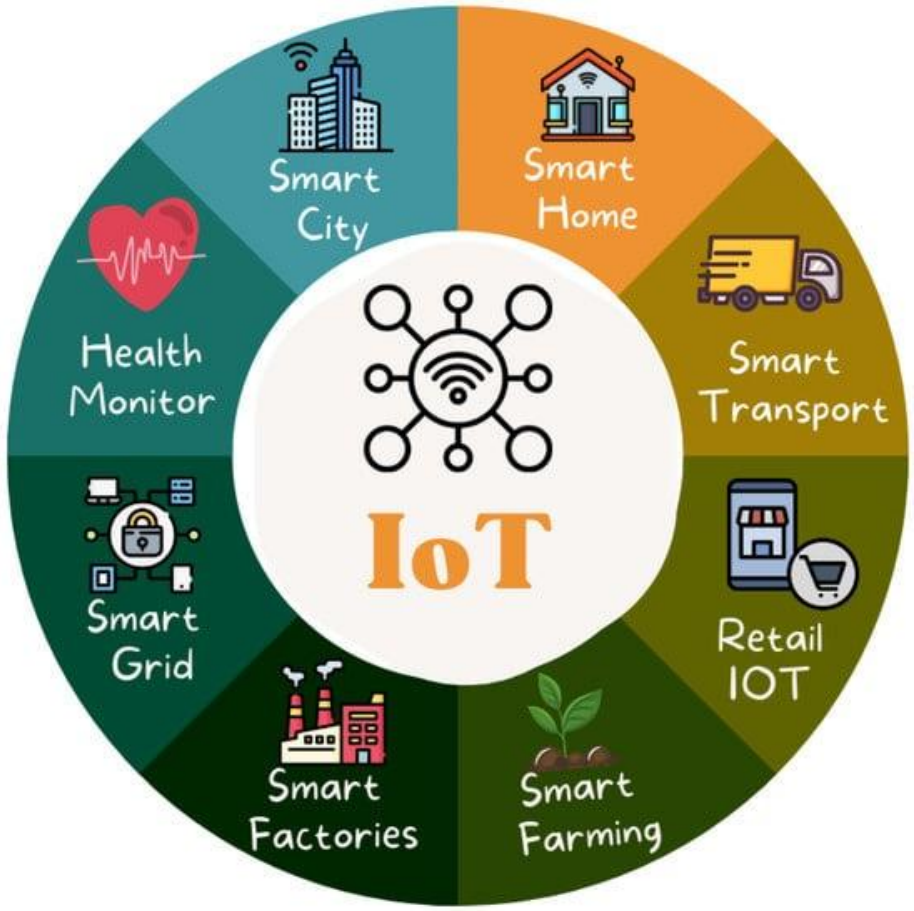
DIRECT Wi Fi

Wireless Connection

Smart Homes & Cities

Vehicle ,Asset , Pet Monitoring & Controlling

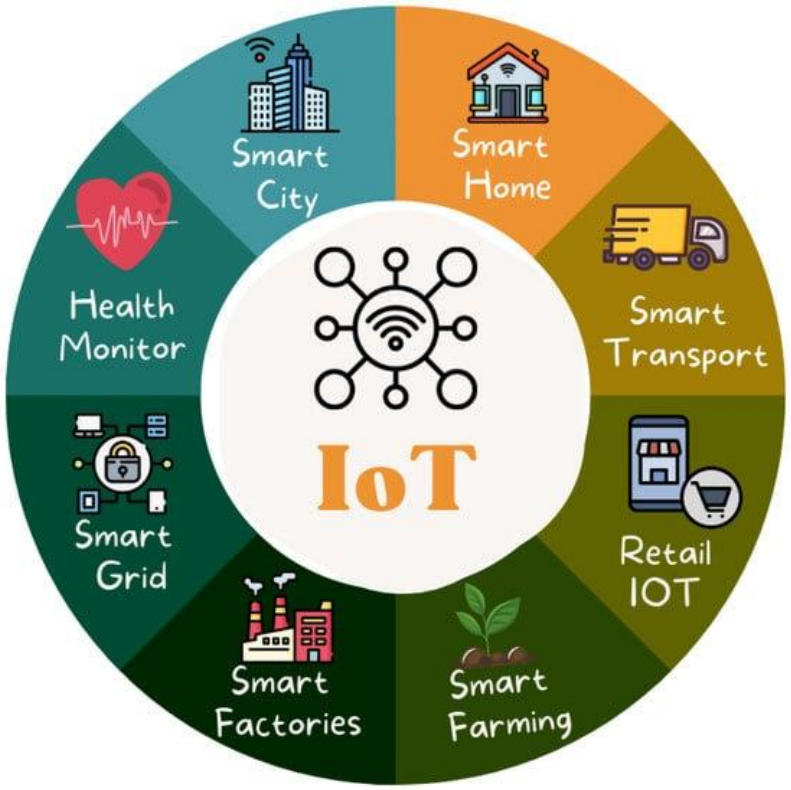
What is SMART in IoT ?





Self-
Monitoring,
Analysis, and
Reporting
Technology

What is SMART in IoT ?





Devices that have the capability to

- collect,
- process, and
- transmit data, often with the ability to
- make decisions or
- take actions based on that data, sometimes
- without direct human intervention

IOTAP-Syllabus

UNIT- I

IOT introduction:

Introduction and definition of IoT, Evolution of IoT, IoT growth, Application areas of IoT, Characteristics of IoT, IoT stack, Enabling technologies, IoT levels, IoT sensing and actuation, Sensing types, Actuator types.

UNIT - II

IOT and M2M:

M2M to IoT – A Basic Perspective- Introduction, Differences and similarities between M2M and IoT, SDN and NFV for IoT,M2M Value Chains, IoT Value Chains, An emerging industrial structure for IoT, international driven global value chain and global information monopolies.

IOT Architecture:

IoT Architecture components, Comparing IoT Architectures, A simplified IoT Architecture, core IoT functional stack, IoT data management and compute stack

UNIT- III

IOT Data link layer and Network layer protocols:

PHY/MAC Layer (3GPP MTC, IEEE 802.11, IEEE 802.15), Wireless HART, Z Wave, Bluetooth Low Energy, Zigbee Smart Energy, DASH7 - Network Layer-IPv4, IPv6, 6LoWPAN, 6TiSCH, ND, DHCP, ICMP, RPL, CORPL, CARP

IOTAP-Syllabus

UNIT- IV

Transport and Session layer protocols:

Transport Layer (TCP, MPTCP, UDP, DCCP, SCTP)-(TLS, DTLS) - Session Layer HTTP, CoAP, XMPP, AMQP, MQTT

UNIT- V

Service layer protocols and Security:

Service Layer -oneM2M, ETSI M2M, OMA, BBF - Security in IoT Protocols - MAC 802.15.4 6LoWPAN, RPL, Application Layer.

TEXT BOOKS:

1. Sudip Misra, Anandarup Mukherjee, Arijit Roy -Introduction to IOT, Cambridge University Press.
2. David Hanes, Gonzalo salgueiro, Patrick Grossetete, Rob barton, Jerome henry-IoT Fundamentals Networking Technologies, Protocols and Usecases for IoT", Cisco Press.

REFERENCE BOOKS:

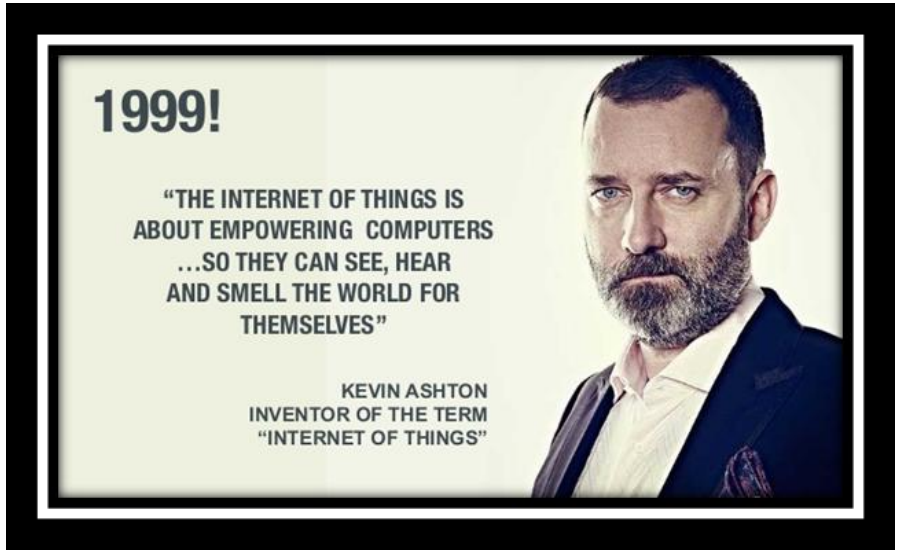
1. Cunopfister-Getting started with the internet of things, O Reilly Media, 2011
2. Francis daCosta,-Rethinking the Internet of Things: A Scalable Approach to Connecting Everything", 1 st Edition, Apress Publications.
3. Arshdeep Bahga, Vijay Madisetti -Internet of Things A Hands-on approach, Universities Press
4. Shriram K Vasudevan, RMD Sundaram, Abhishek S Nagarajan-Internet of things, John Wiley and Sons.
5. Massimo Banzi, Michael Shiloh Make: Getting Started with the Arduino, Shroff Publisher/Maker Media Publishers

UNIT- I
IOT introduction

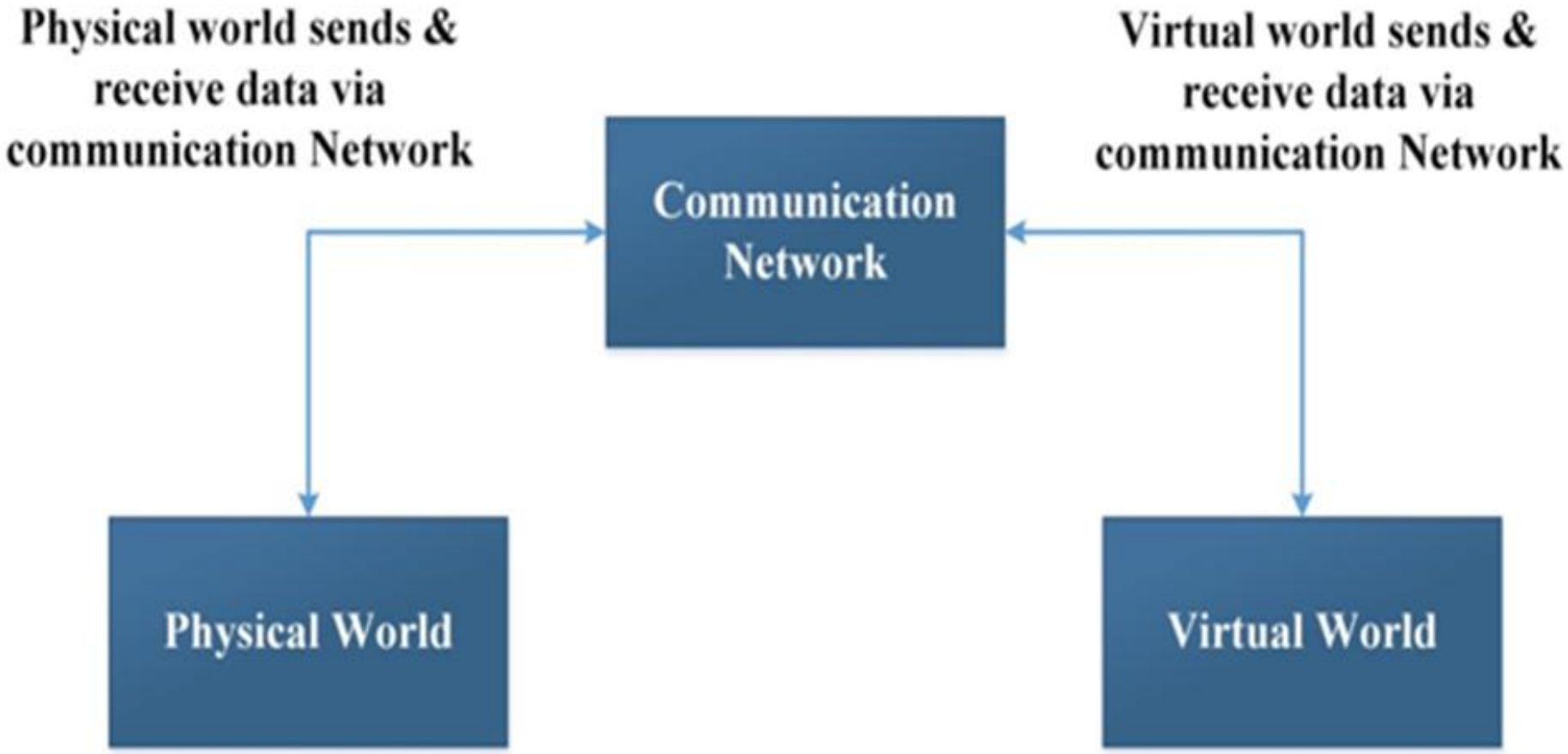
- Introduction and definition of IoT
- Evolution of IoT
- IoT growth
- Application areas of IoT
- Characteristic of IoT
- IoT stack
- Enabling technologies
- IoT levels
- IoT sensing and actuation
- Sensing types
- Actuator types

Who is the father of IoT

- **Kevin Ashton**
- Kevin Ashton is sometimes called the “Inventor of IoT”
- First used the term in 1999 to describe a system **where the Internet is connected to the physical world via ubiquitous (everywhere) sensors, including RFID**.



How IoT Works ?



Technical definition of IoT

A Dynamic global network infrastructure with self configurable capabilities based on standard / interoperable communication protocols, **where physical/virtual things have identities and use intelligent interfaces and integrated into information network**, **Often communicate data associated with users and their environments**.

Characteristics of IoT

- Dynamic & Self Adopting
- Self Configuring
- Interoperable Communication Protocols
- Unique Identity
- Integrated into Information Network

1. Dynamic and self adapting

- IoT devices and systems may have the capability to **dynamically adapt** with the **changing** **contexts and take actions**.
- Example – Surveillance system – can adapt normal (day) and night time modes .

1. Dynamic and self adapting

- Cameras should switch from lower resolution to higher resolution modes when any motion is detected and alert nearby by cameras.
- In this surveillance system is adapting itself based on the context and changing (dynamic) conditions .



1. Dynamic and self adapting

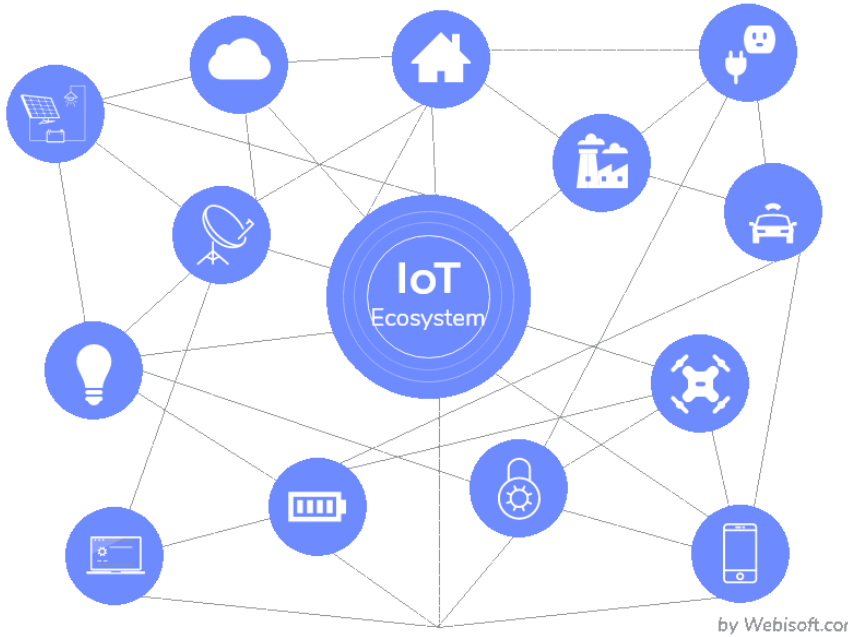


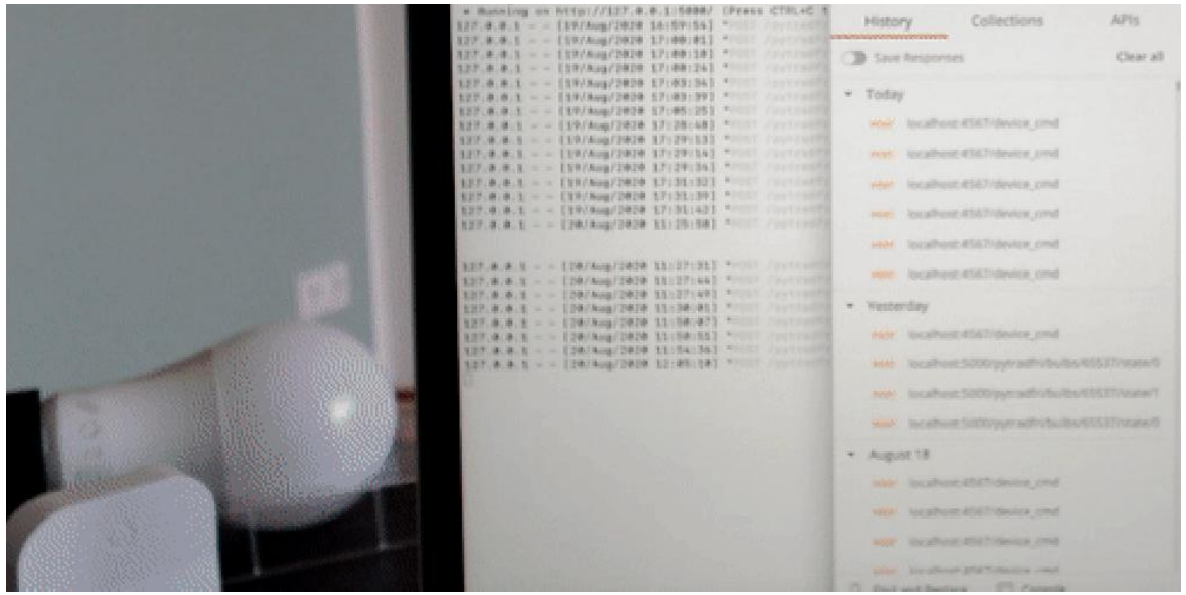
2. Self configuring

- IoT devices may have the **ability** to **configure** themselves
- Allowing a large no of devices to work together to provide certain functionality, Ex: Weather monitoring

2. Self configuring

- Setup networking
- Fetch latest software upgrades with minimal user intervention.

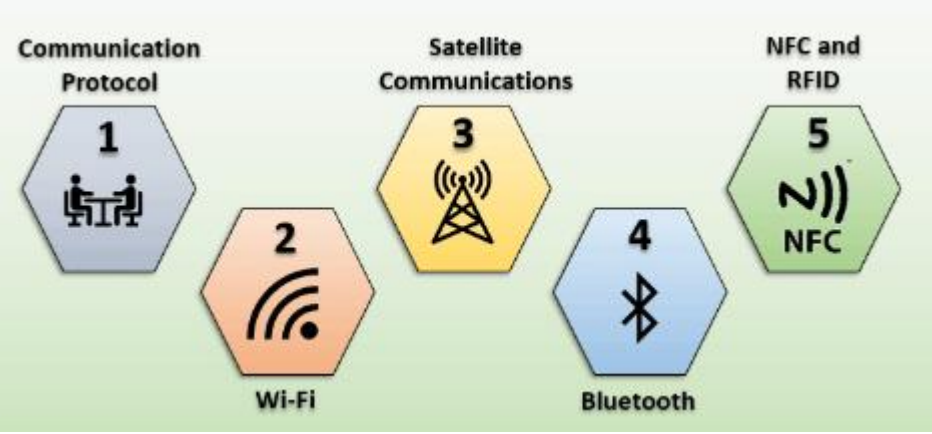




3. Interoperable Communication Protocols

-Speaking the same language

- IoT Devices may support a number of interoperable communication protocols
- Heterogeneous devices, systems, platforms or networks





4.Unique Identity

- Giving an identity to the device is an essential aspect of loT.

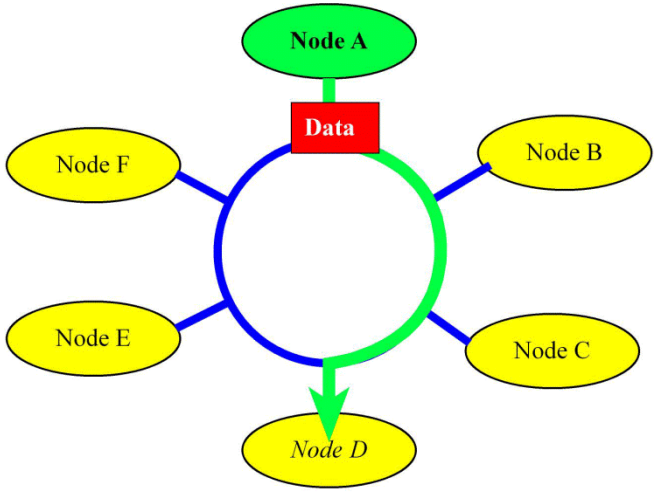
- **Identity helps** to **differentiate** between various internet devices and select the device we want to send the command.

- **It is essential** to give **a unique identity** to every device so **that we can set up passwords or other security means**.

4.Unique Identity

- Each IoT device has a unique and a unique identifier as an IP address or URL .

- For example, fingerprints, face recognition IP addresses, and Face lock systems are several means of security given to the different identified devices to protect them.



4.Unique Identity



- Each Thing in IoT has its own identity to transmit or receive data

5. Integrated into Information Network

- IoT Devices are usually integrated into the information network that allows them to communicate and exchange data with other devices and systems.
- IoT devices can be dynamically discovered in the network by other devices and /or network, and have capability to describe themselves to other devices or users.

5. Integrated into Information Network

- Example: A weather monitoring node can describe its monitoring capabilities to another connected node.
- They can communicate and exchange data
- Data from large number of connected weather monitoring IoT nodes can be aggregated and analyzed to predict the weather.

Evolution of IoT
or
A Brief History of IoT



Evolution of IoT



Evolution of IoT



- The first IoT device was a Coca-Cola vending machine installed at Carnegie Mellon University in the early 1980s.
- It was able to send information about the machine's temperature and inventory over the Internet.
- Though such functionality seemed unusual, almost unbelievable, at that time

Evolution of IoT



•1969-Invention of Internet

•1980-Coca-Cola Vending Machine at Carnegie Mellon University, Allowed users to check the status (Availability & temp) of the machine remotely

•1990- John Romkey and Simon Hackett created a toaster, could be turn ON/OFF by internet.

- 1995-Siemens introduced the first cellular module built for M2M
- 1999- The term “Internet of Things” was used by Kevin Ashton during his work at MIT’s Auto ID center, which became widely accepted

Evolution of IoT

- 2005- UN's International Telecommunications Union (ITU) published its first report on IoT
- 2008- The Internet of Things was born
- 2011-Gartner, the market research company include "The Internet of Things" technology in their research

Internet of Things—a comprehensive timeline



1989
The first IoT device is created



1993
The first online webcam is used at Cambridge



1994
Steve Mann creates WearCam



1999
Kevin Ashton coins the term Internet of Things



2008
The number of connected devices overtakes the number of people in the world and IoT is 'born'



2007
First iPhone is launched



2005
UN publishes its first report on the Internet of Things



2000
LG announces the first smart refrigerator



2009
The original Fitbit activity tracker is released



2011
IIoT comes into being



2014
Seoul becomes the world's first smart city



2015
IoT goes Mobile with smartphones



2022
World Economic Forum names IoT as one of the three most impactful technological advancements



2021
More than 10 billion active IoT devices active



2020
The number of IoT device connections increased more than 50% of the active connected devices



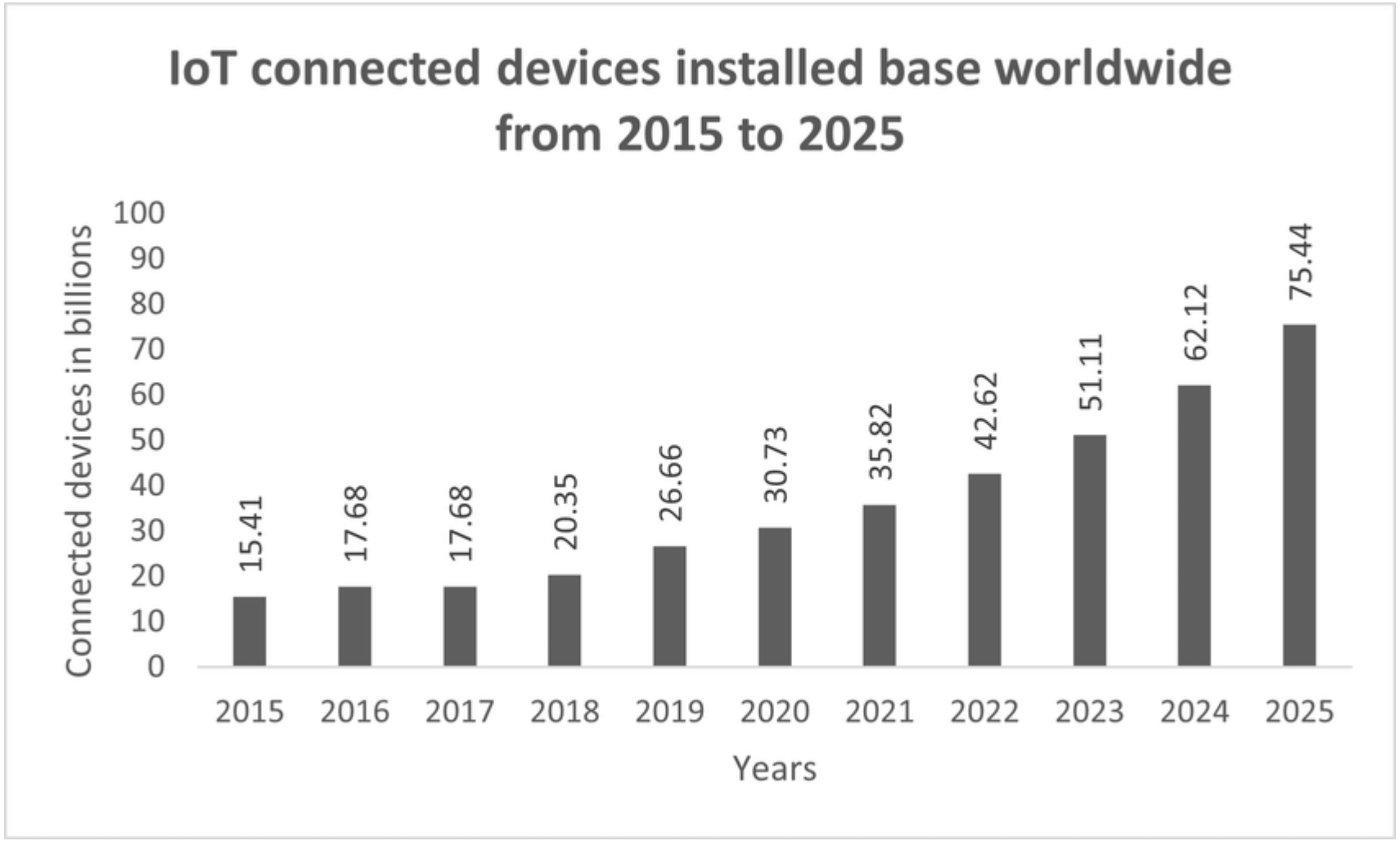
2016
AWS IoT core is launched

Factors for IoT Growth

- an expansion of networking capabilities
- the development of large-scale data analytics tools to aid in managing and interpreting data from the connected devices
- the establishment of new standards and protocols that simplify the interaction of hardware and software from multiple vendors
- the tremendous growth of the Cloud storage system
  - low-cost
  - always-on storage
  - highly available
  - easy computations from IoT devices to Cloud servers

IoT Growth

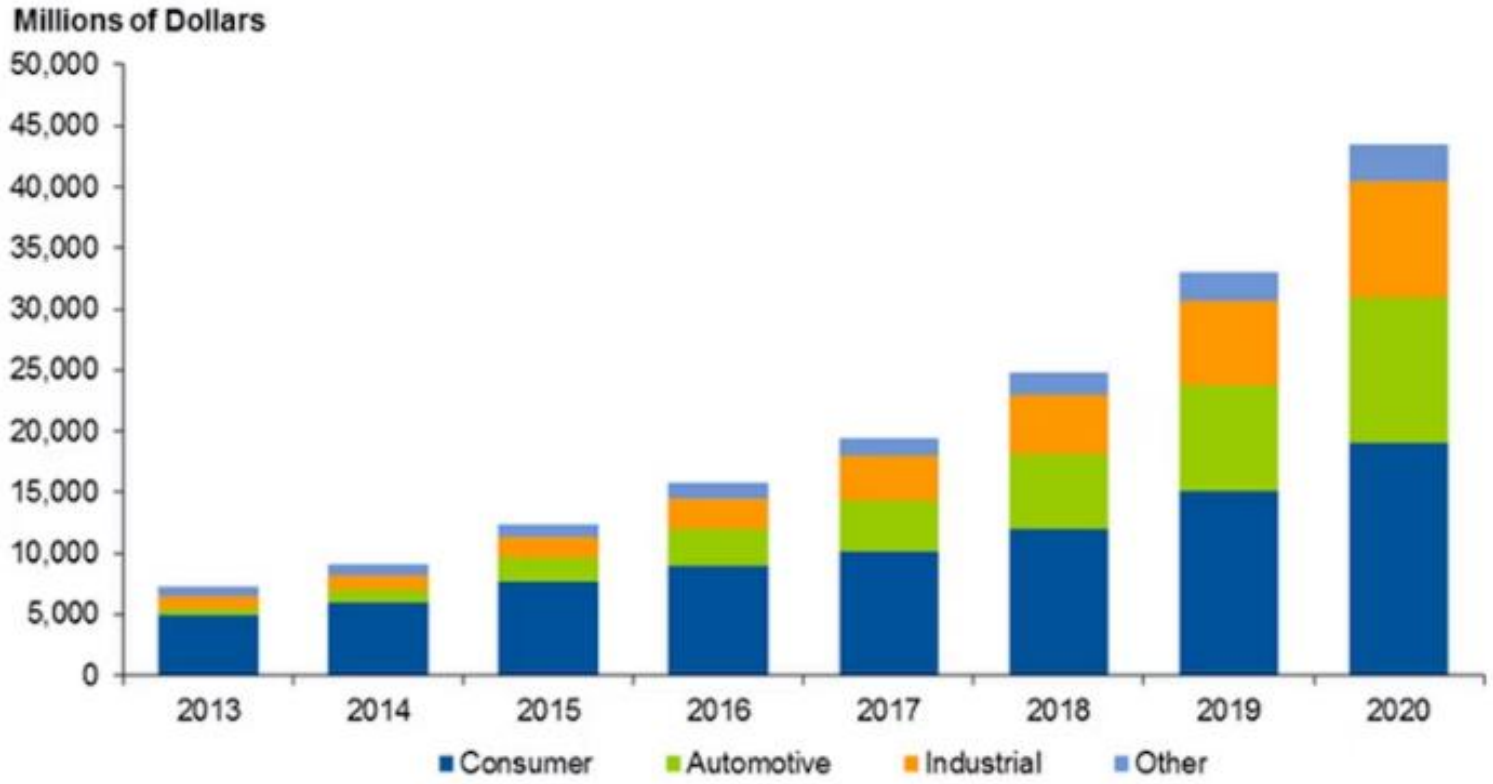
- Growth in Number of Connected Devices



Source: Researchgate.net

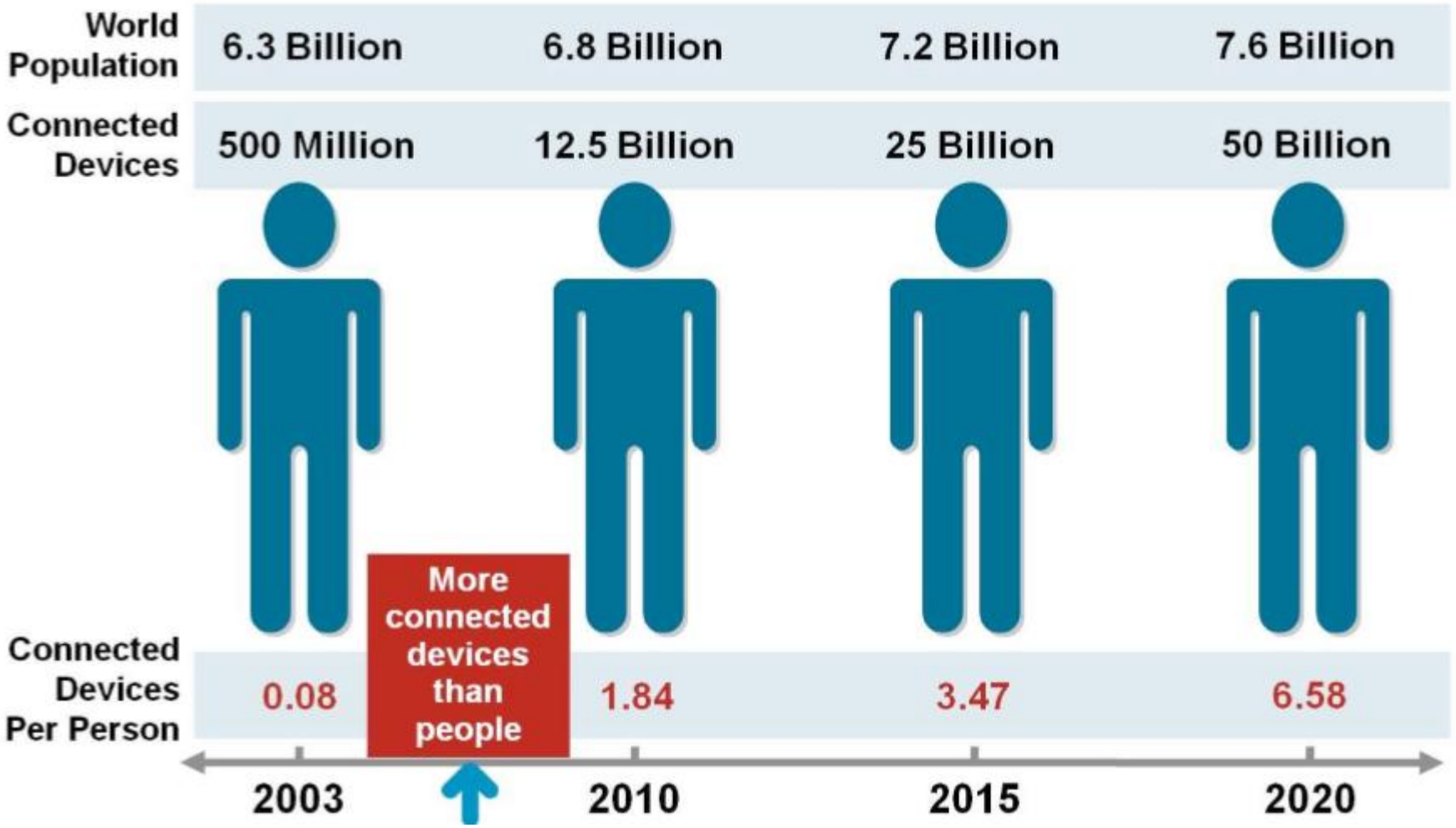
IoT Growth- By means of Revenue

Figure 1. IoT Semiconductor Revenue by Electronic Equipment (Millions of Dollars)



Source: Gartner (October 2014)

IoT Growth Vs World's Population



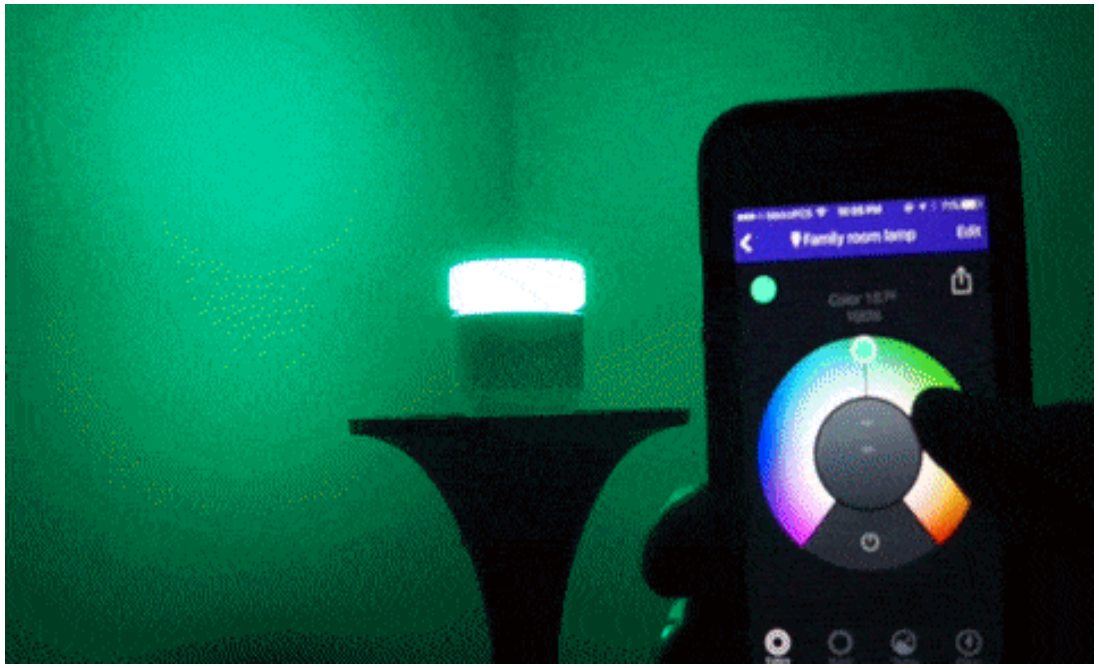
Application areas of IoT

- Home automation
- Smart Cities
- Environment
- Energy
- Retail
- Logistics
- Agriculture
- Industry
- Health & Lifestyle



Home automation

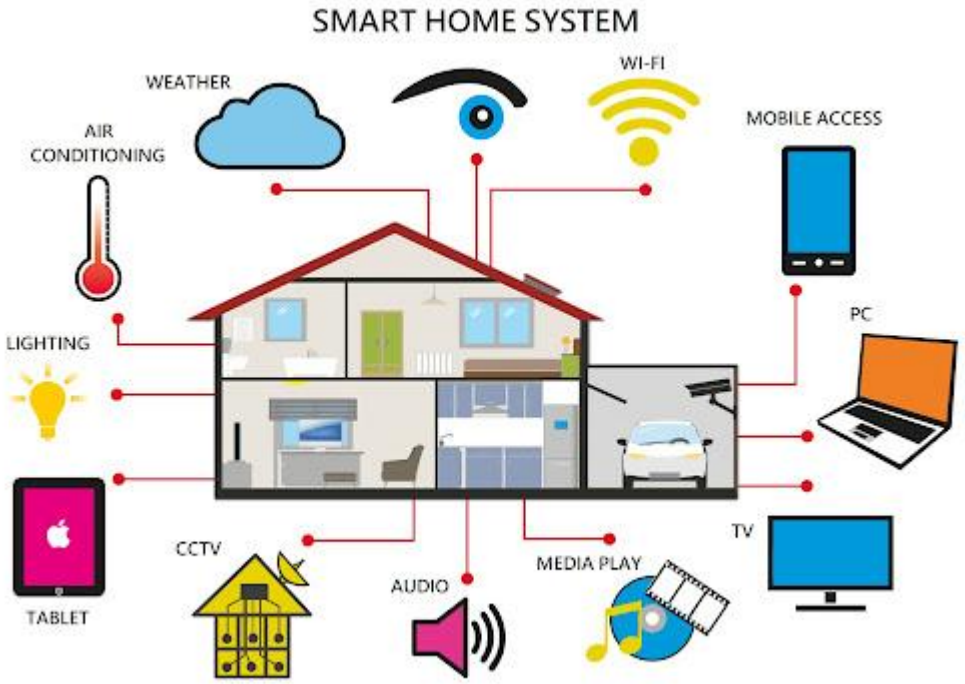
- Smart Lighting

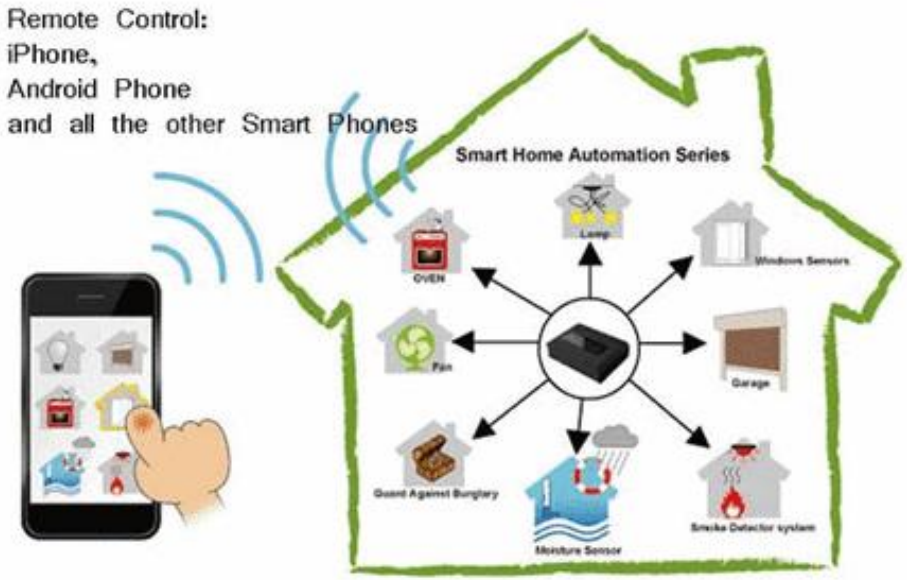




Home automation

- Smart Appliances





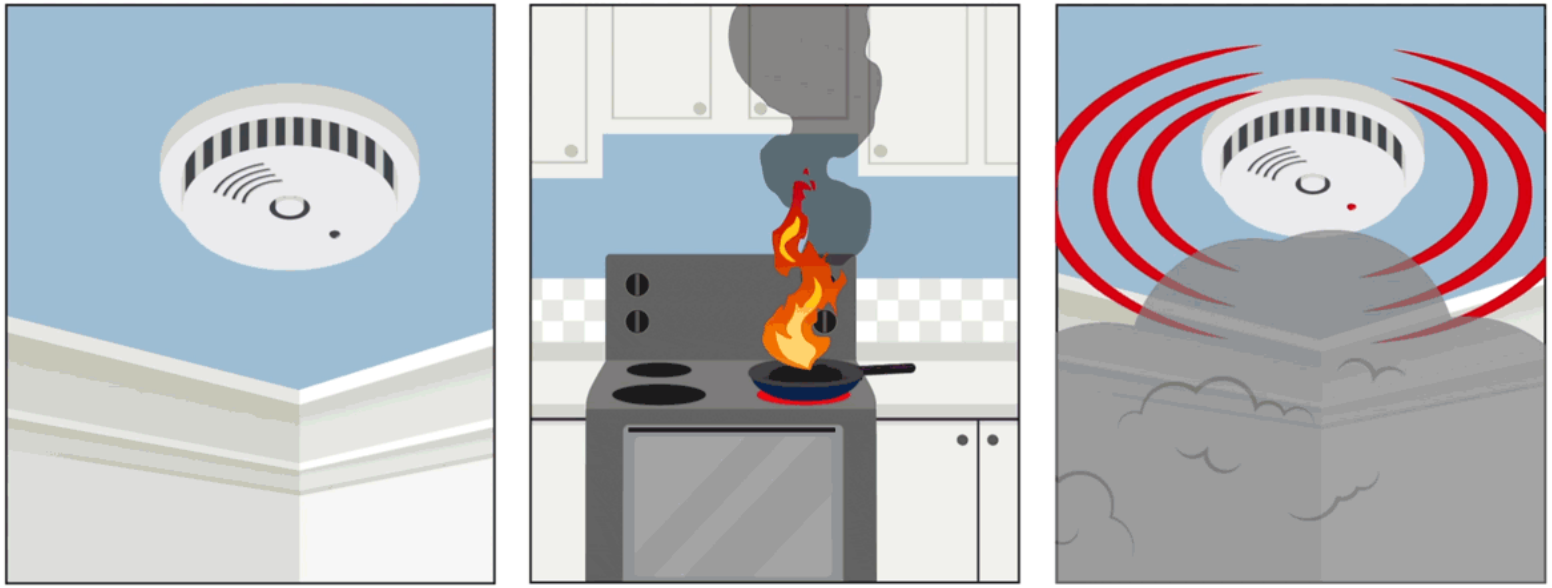
Home automation

- Intrusion Detection



Home automation

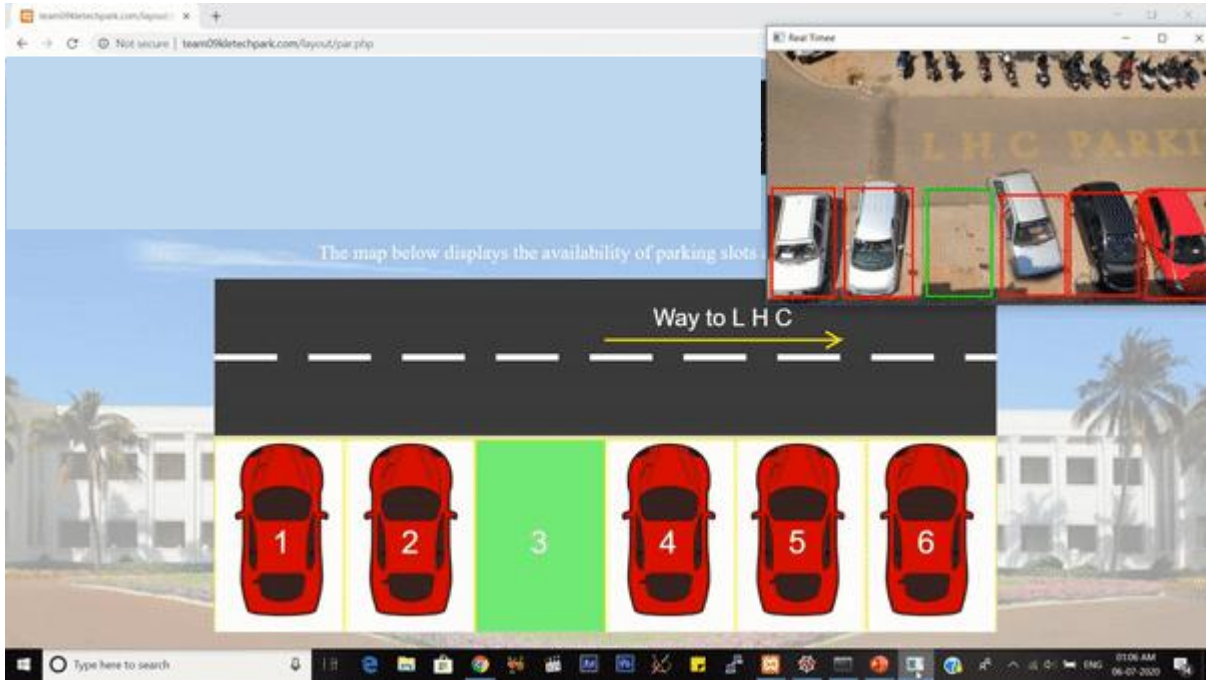
- Smoke/Gas Detectors

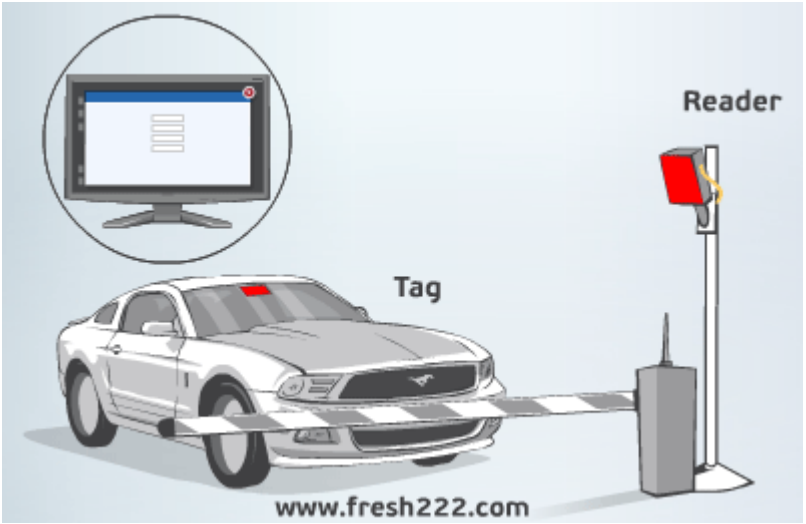


Smoke alarms beep loudly when there is a fire

Smart Cities

- Smart Parking





Smart Cities

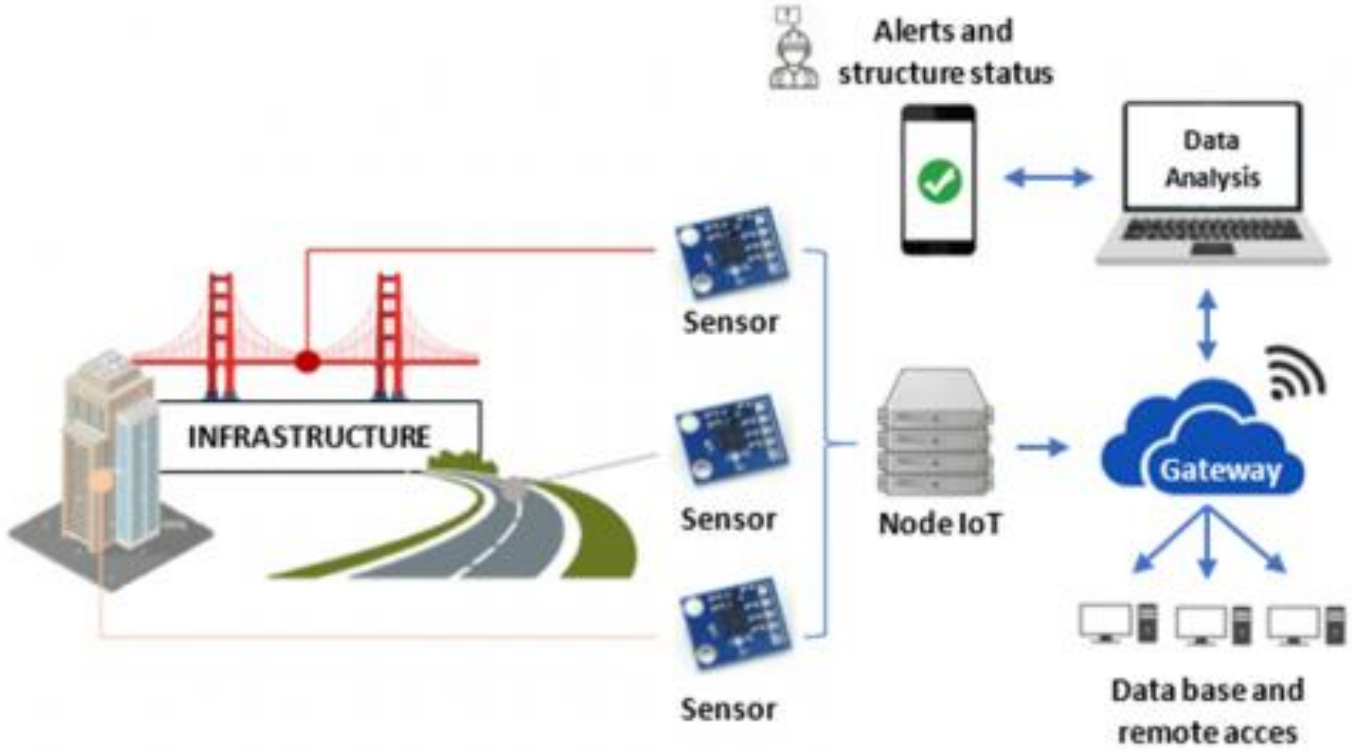
- Smart Roads





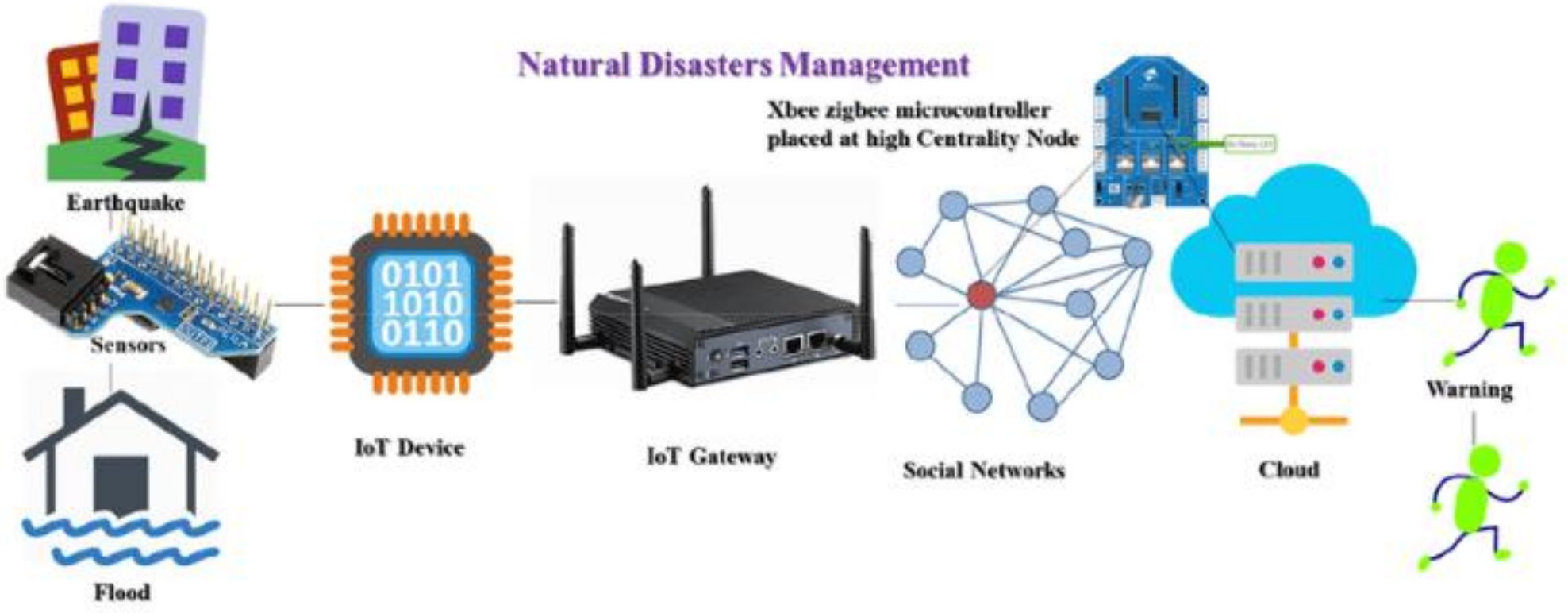
Smart Cities

- Structural Health Monitoring



Smart Cities

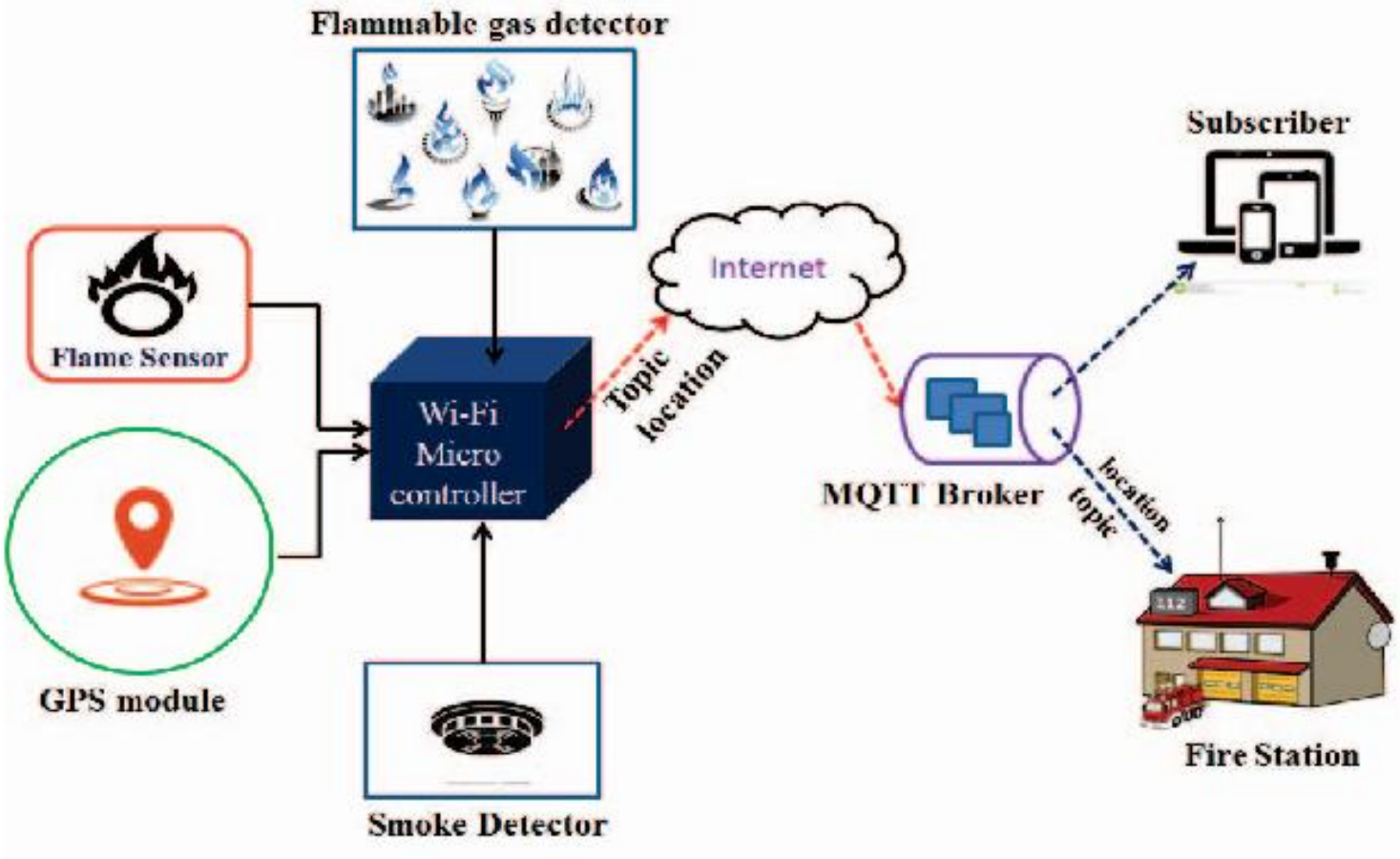
- Emergency Response



Natural disaster management by integrating internet of things with social networks

Smart Cities

- Emergency Response



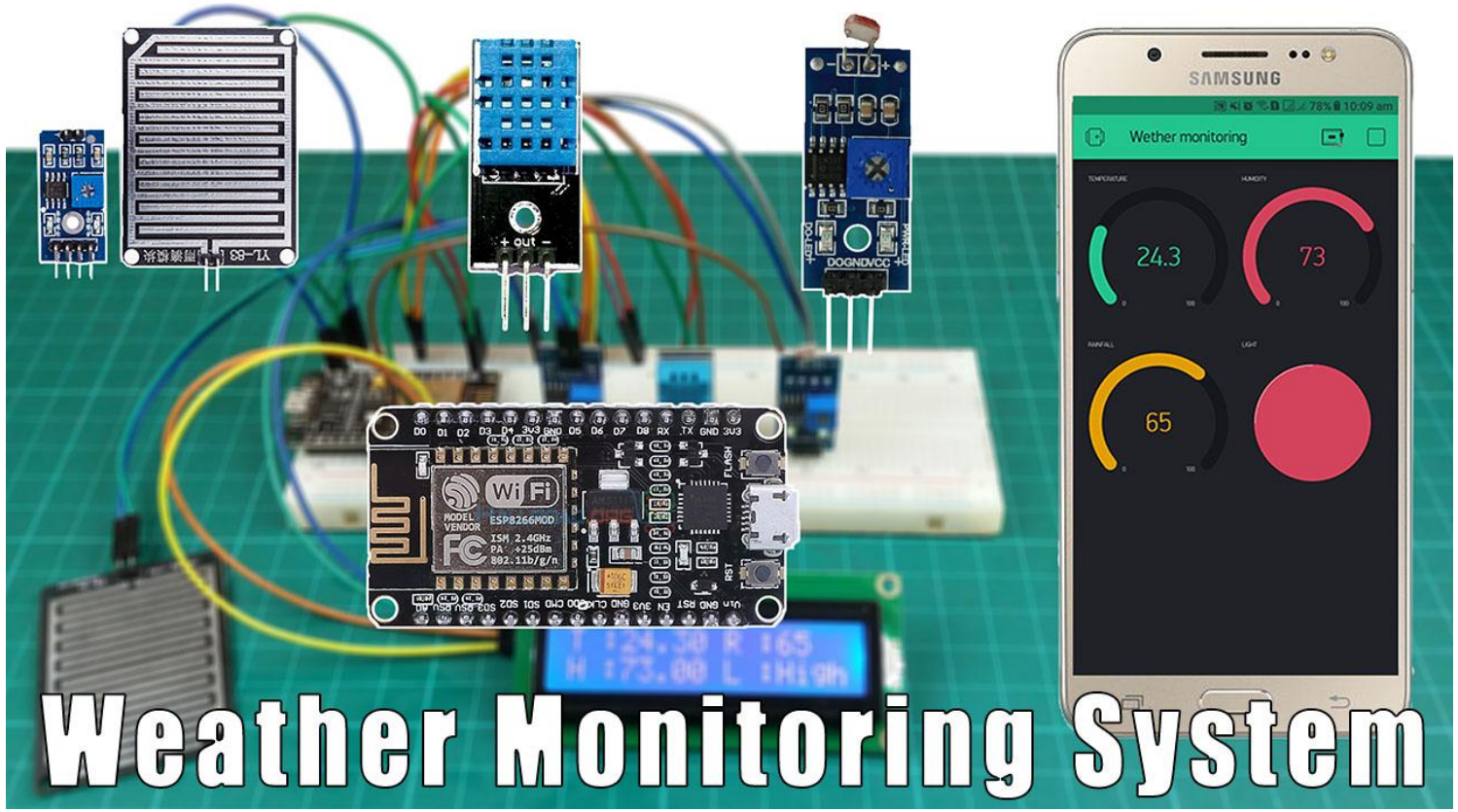
Environment

- Weather Monitoring



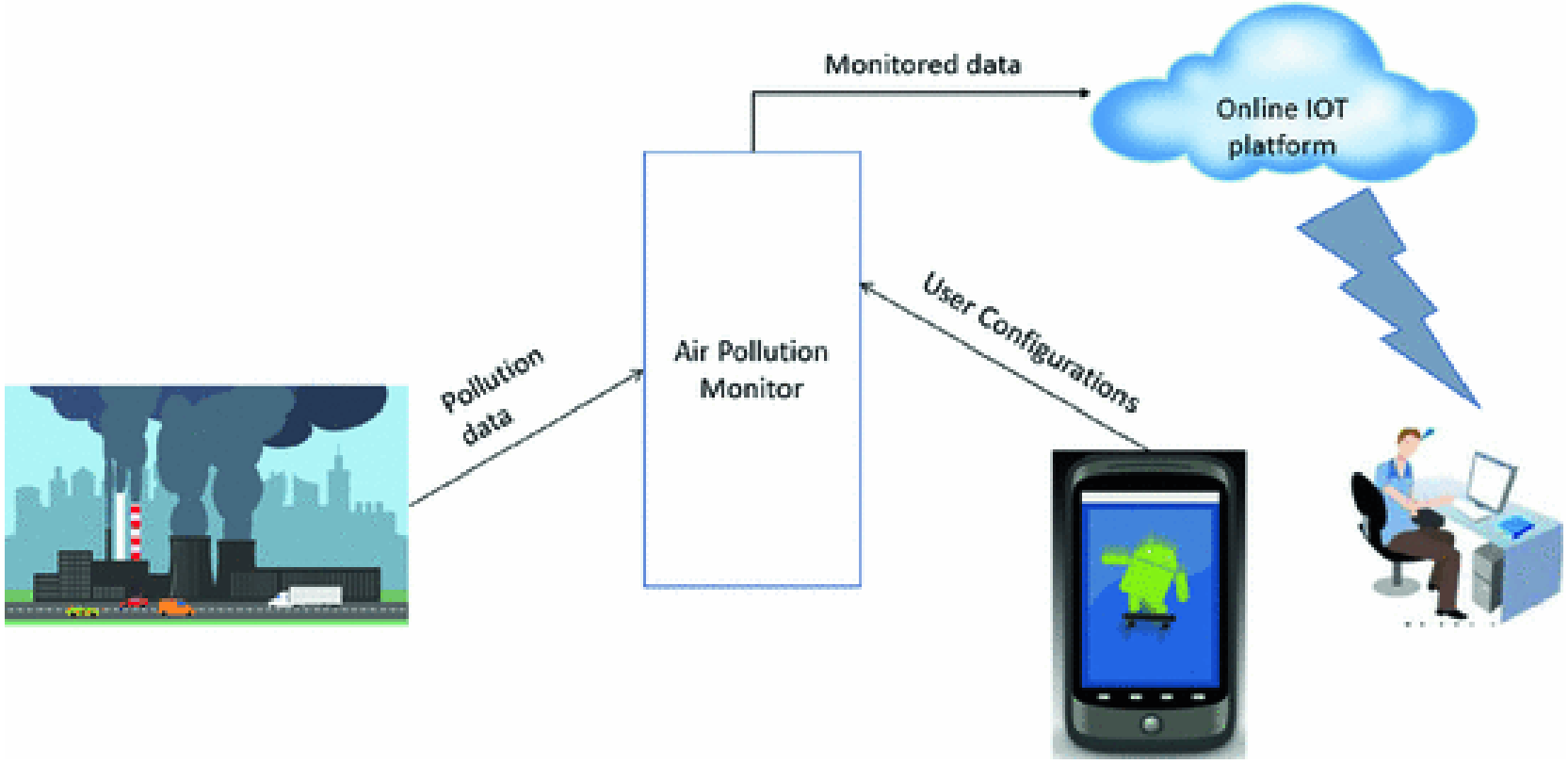
Environment

- Weather Monitoring



Environment

- Air Pollution Monitoring



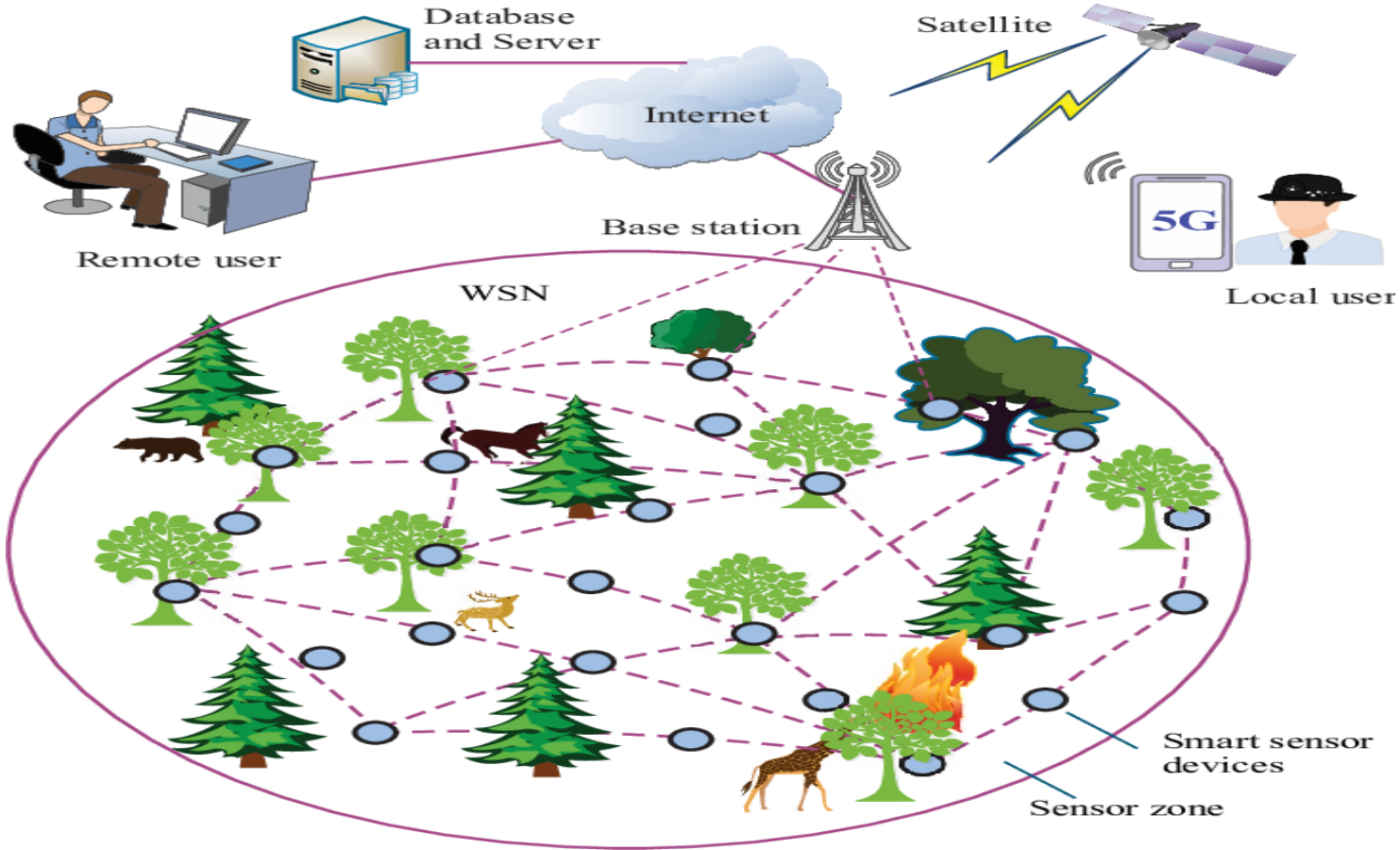
Environment

- Noise Pollution Monitoring



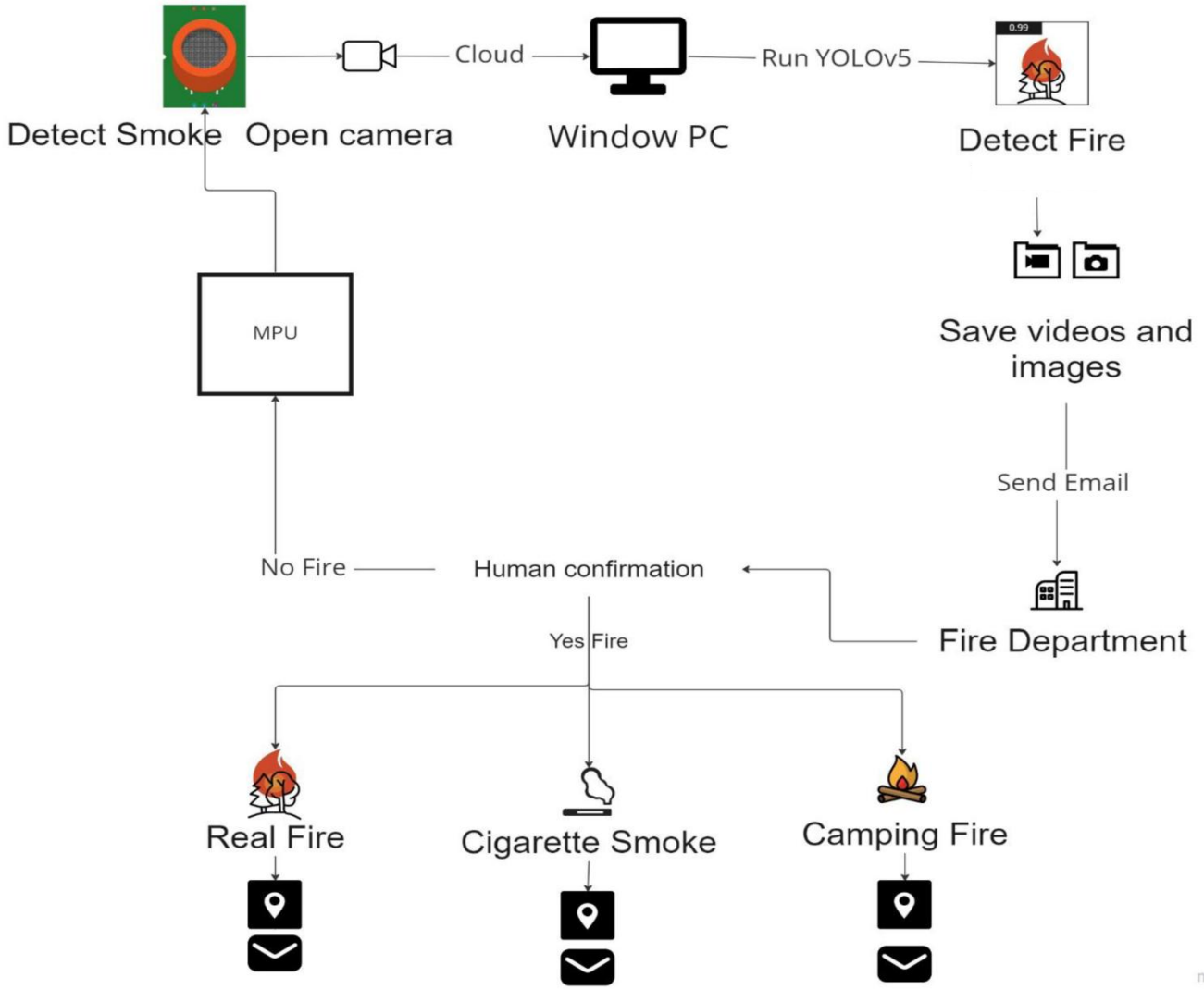
Environment

- Forest Fire Detection



Environment

- Forest Fire Detection



Energy

- Smart Grids



Energy

- Smart Grids



Energy

- Smart Grids



Energy

- Renewable Energy Systems



Energy

- Prognostics
  - An engineering discipline focused on predicting the time at which a system or a component will no longer perform its intended function.



Retail

- Inventory Management





Retail

- Smart Payments





Retail

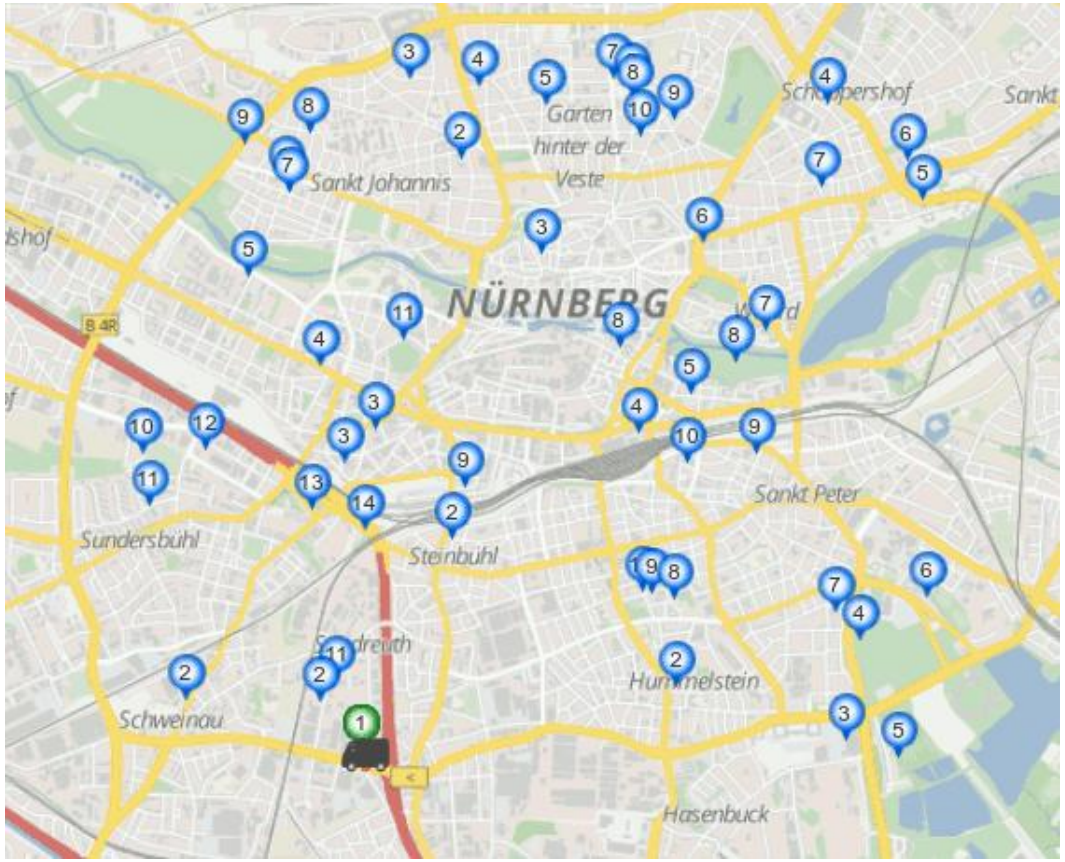
- Smart Vending Machines

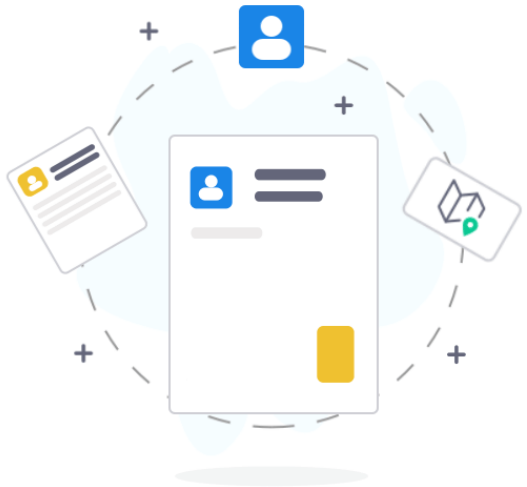




Logistics

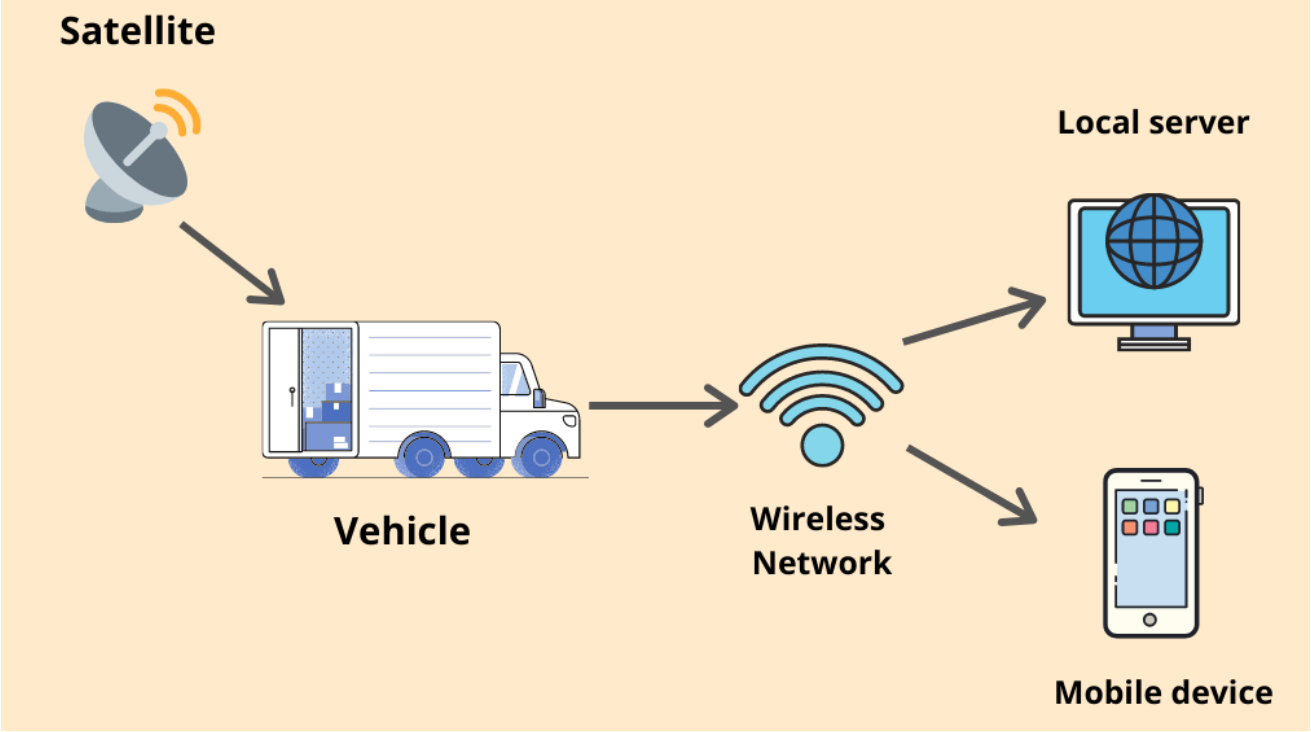
- Route Generation & Scheduling





Logistics

- Fleet Tracking

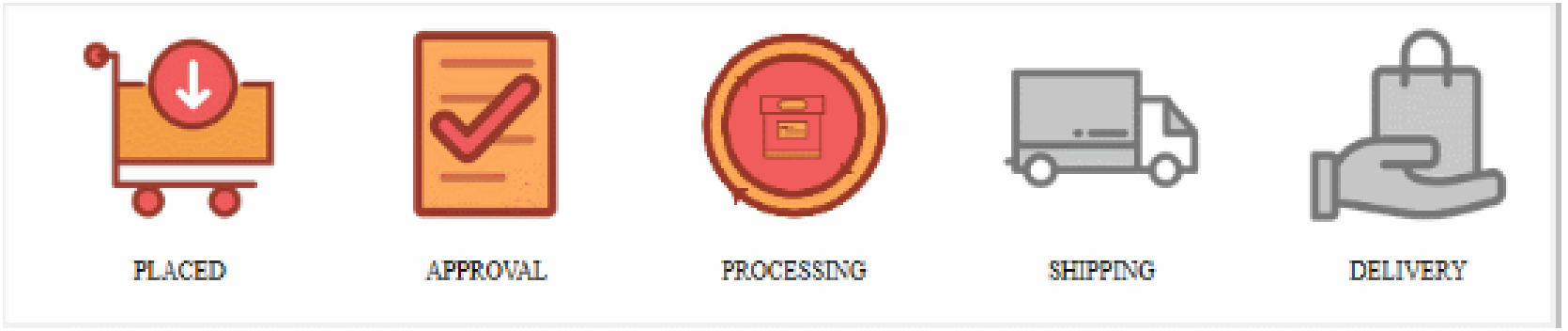




Logistics

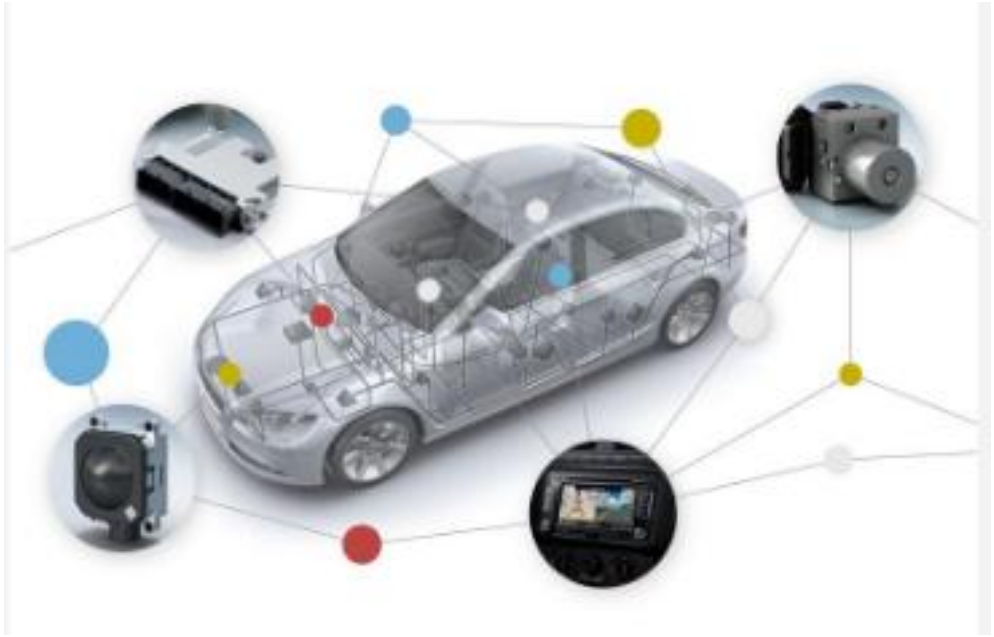
- Shipment Monitoring





Logistics

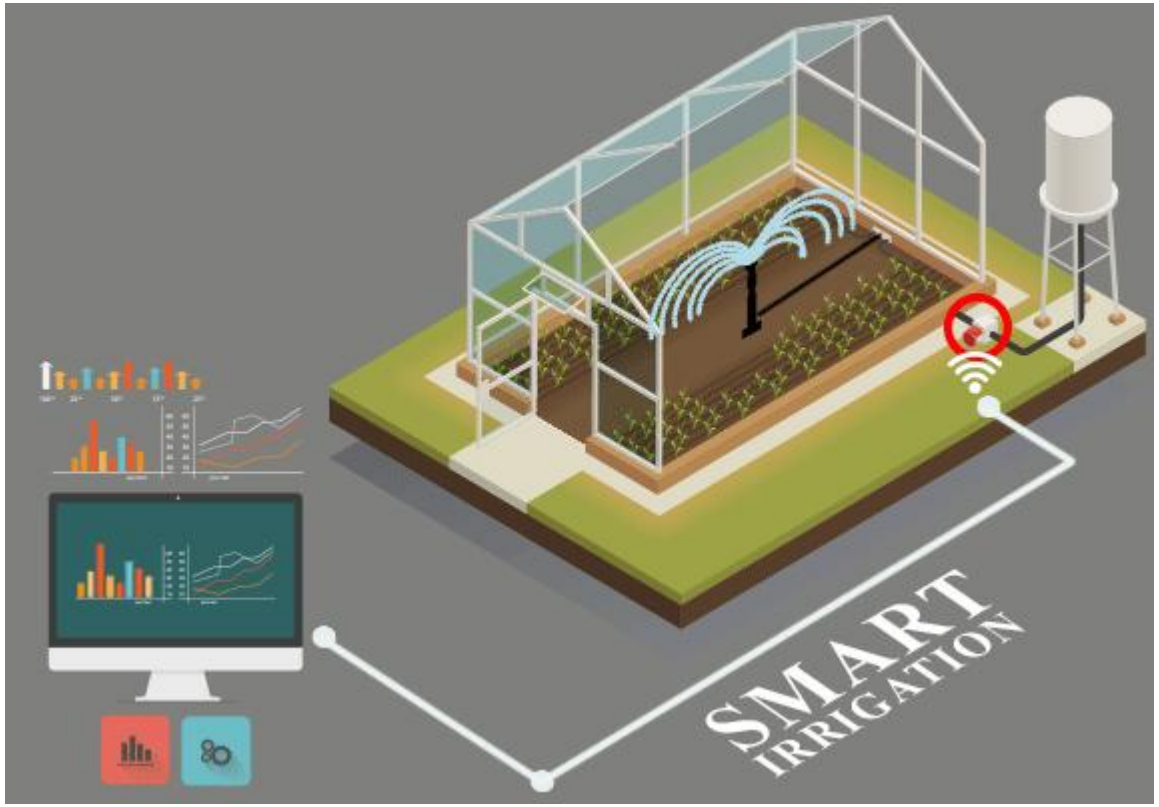
- Remote Vehicle Diagnostics

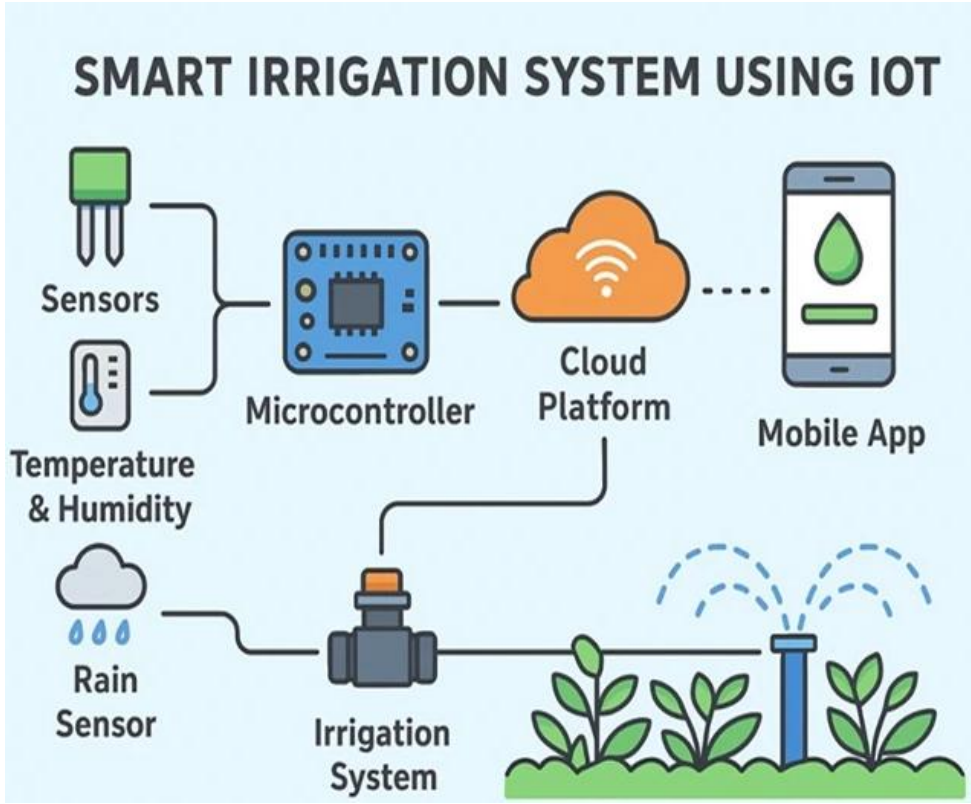




Agriculture

- Smart Irrigation





Agriculture

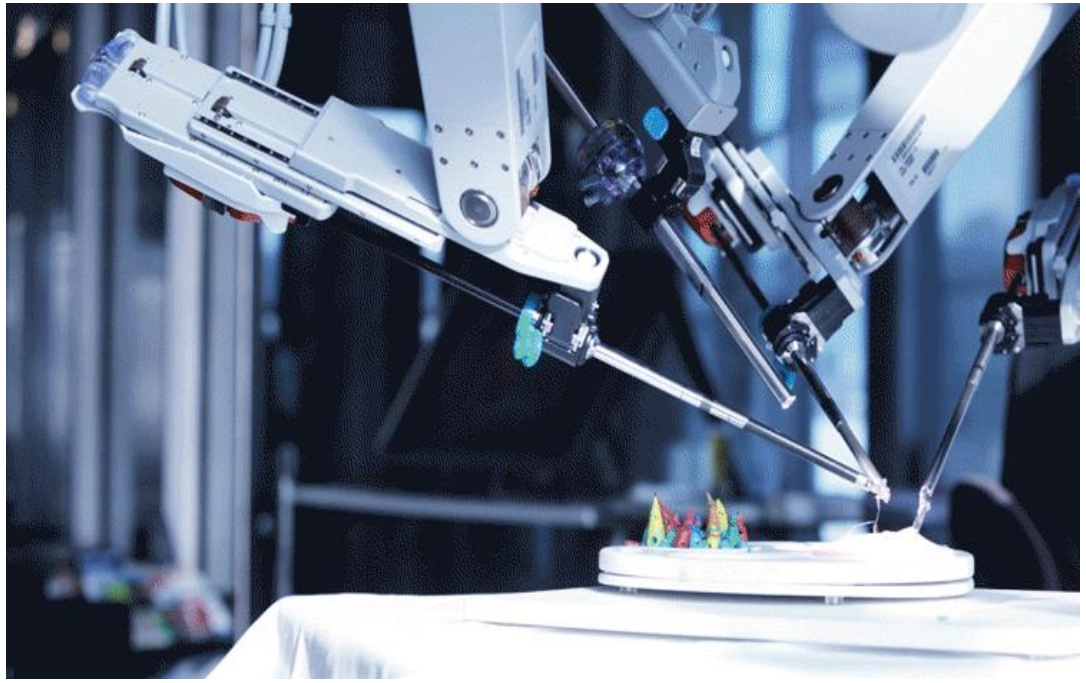
- Green House Control

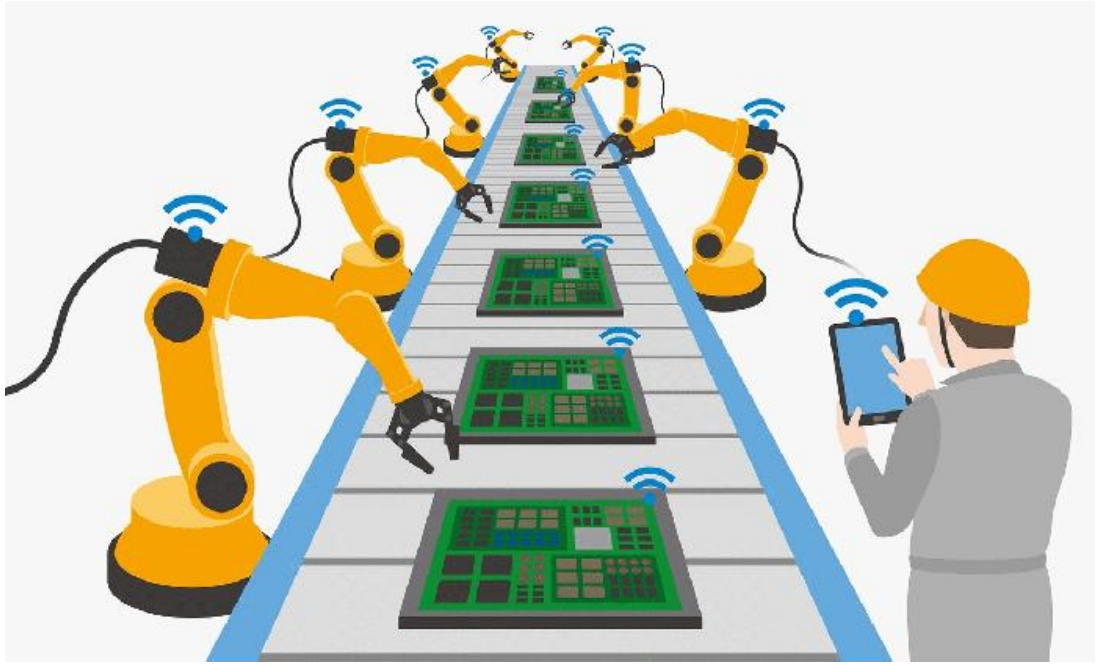




Industry

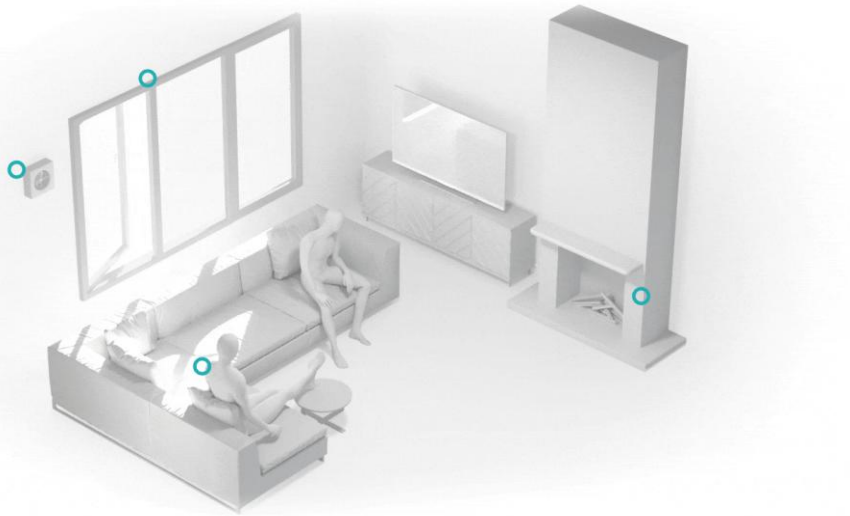
- Machine Diagnosis & Prognosis

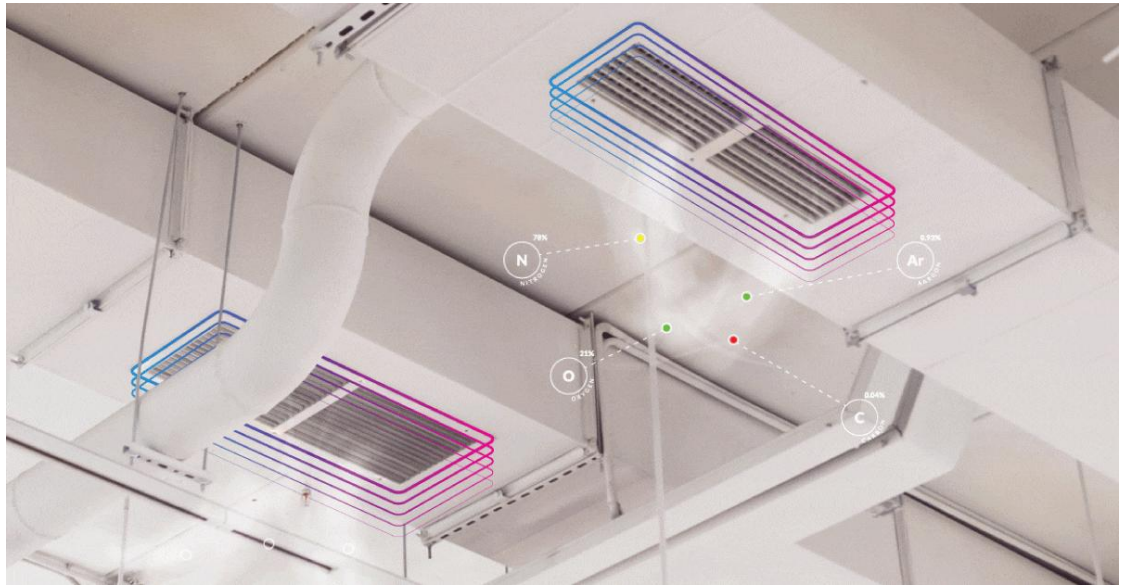




Industry

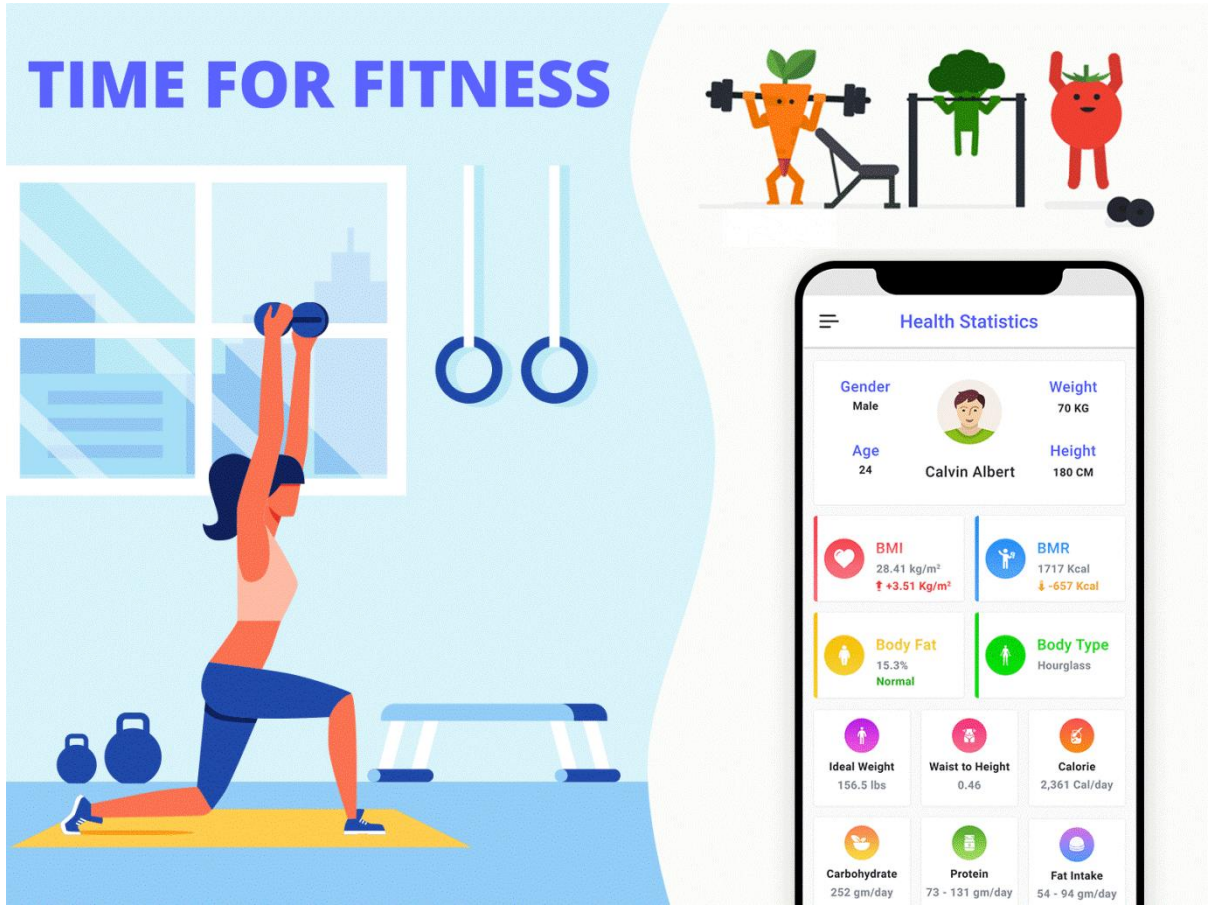
- Indoor Air Quality Monitoring





Health & Lifestyle

- Health & Fitness Monitoring





Health & Lifestyle

- Wearable Electronics





Health & Lifestyle

- Wearable Electronics



Sound Activated LED Bracelet Wristband



Portable Air Purifier Necklace

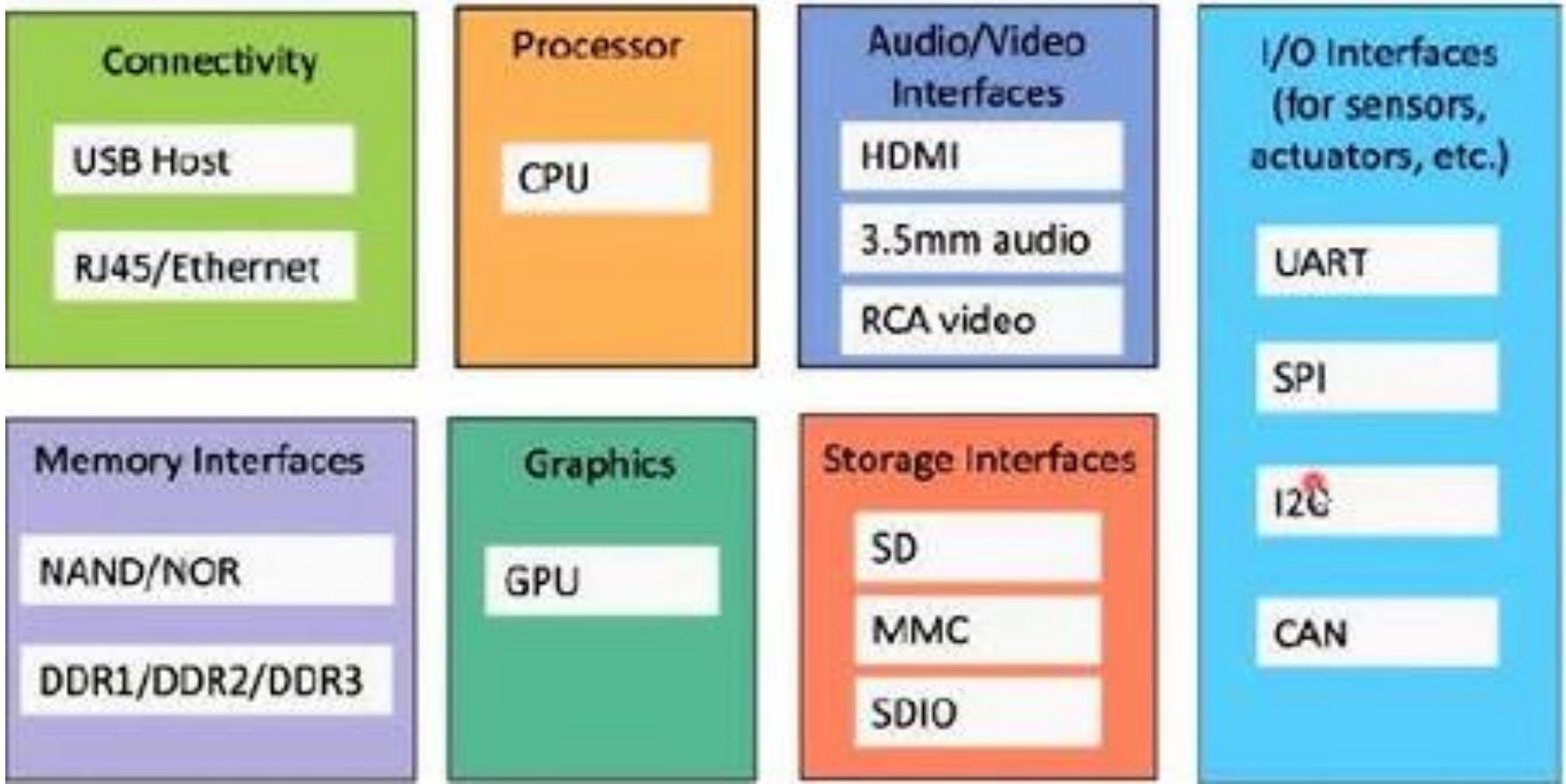
Design of IoT

- Physical Design of IoT
  - Things in IoT
    - Generic Block diagram of an IoT Device
  - IoT protocols
    - Protocol Stack

- Logical Design of IoT- Functional diagram of IoT
  - Functional blocks
  - Communication models
  - Communication APIs

Generic Block Diagram of an IoT Device

1. Connectivity interfaces
2. Memory interfaces
3. Processing interfaces
4. Graphics interfaces
5. AV interfaces
6. Storage interfaces
7. I/O interfaces



1. Connectivity interfaces

- USB Host





USB 3.0

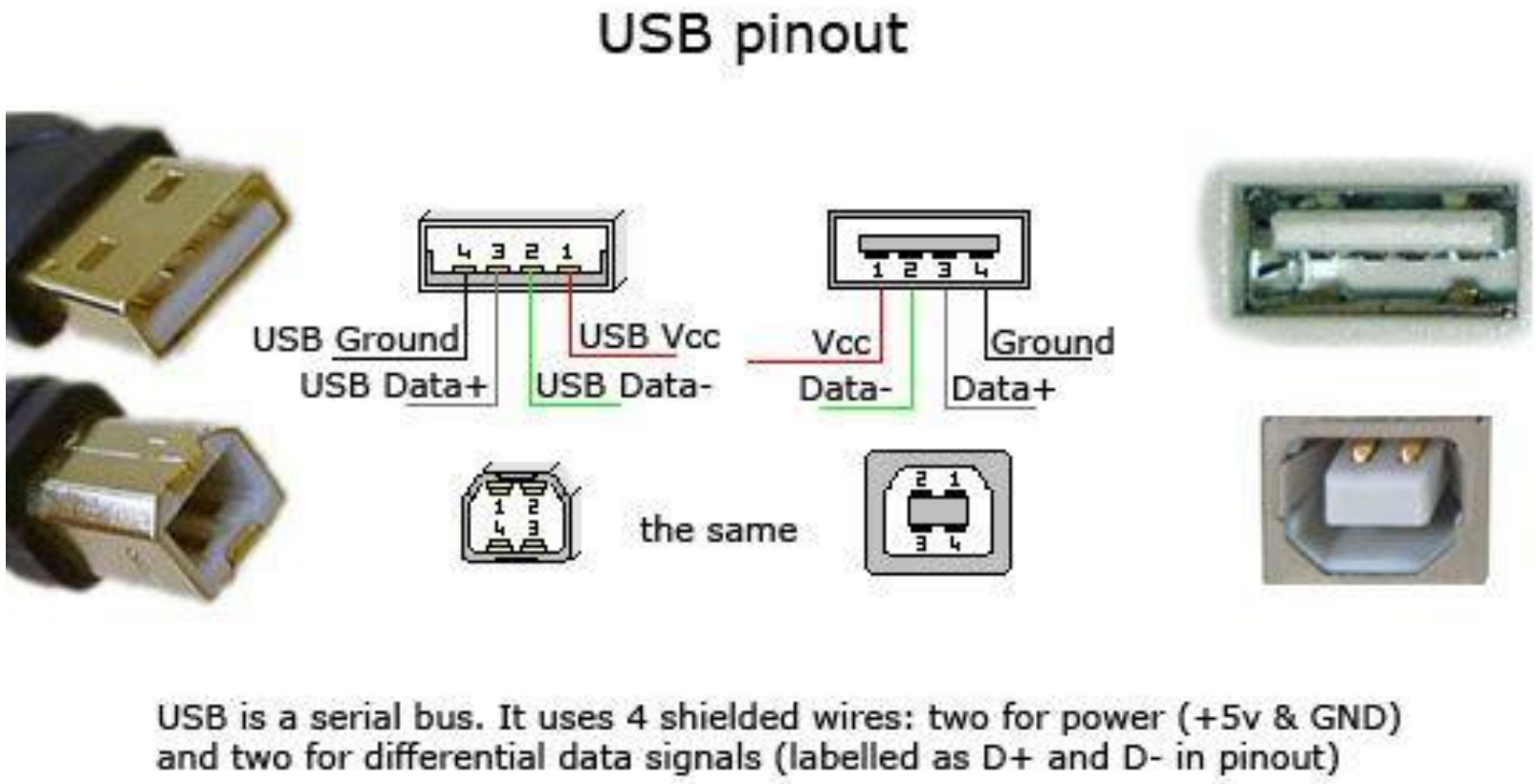


USB 2.0



1.Connectivity interfaces

- USB Host



USB is a serial bus. It uses 4 shielded wires: two for power (+5v & GND) and two for differential data signals (labelled as D+ and D- in pinout)

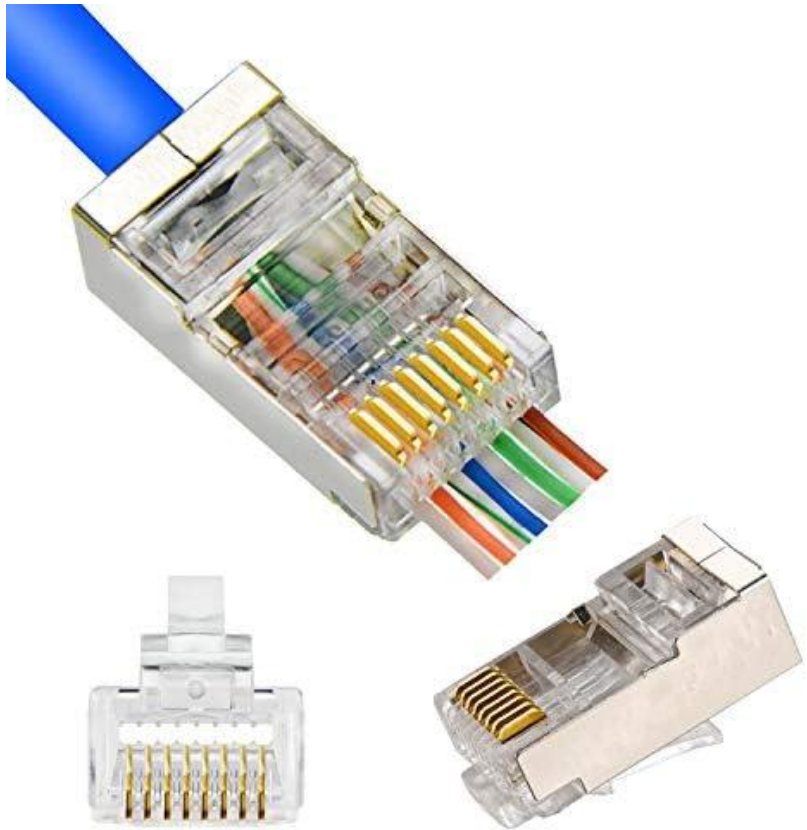
1. Connectivity interfaces

- RJ45/ Ethernet- IEEE 802.3

RJ stands for registered jack while the “45” refers to the listing number

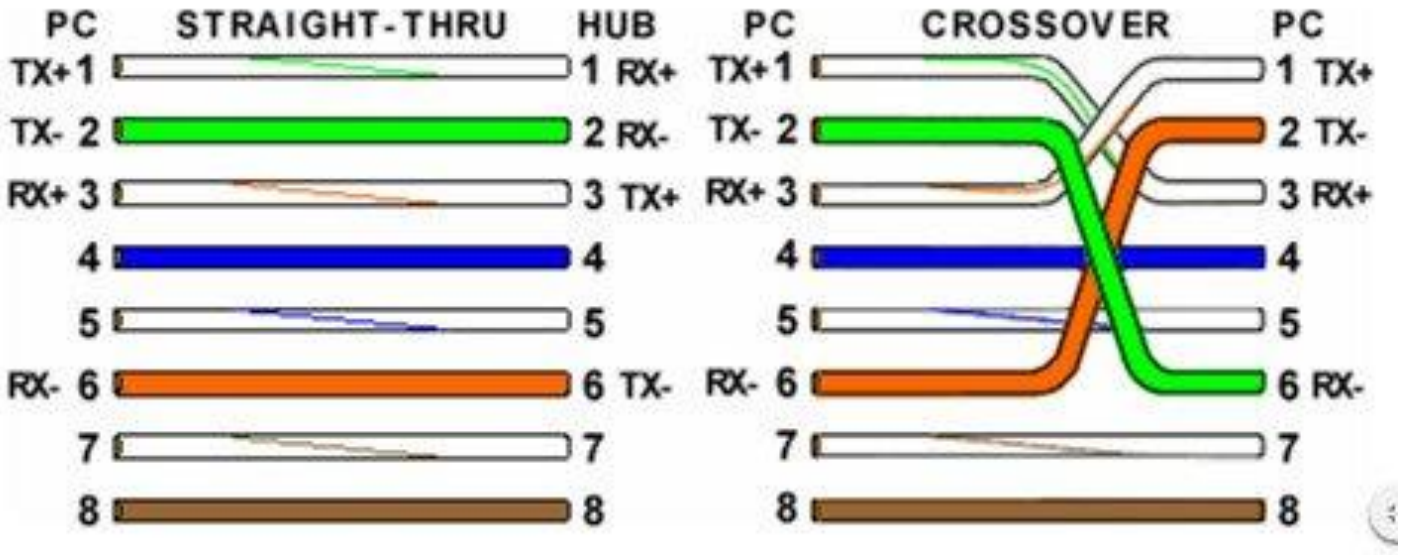




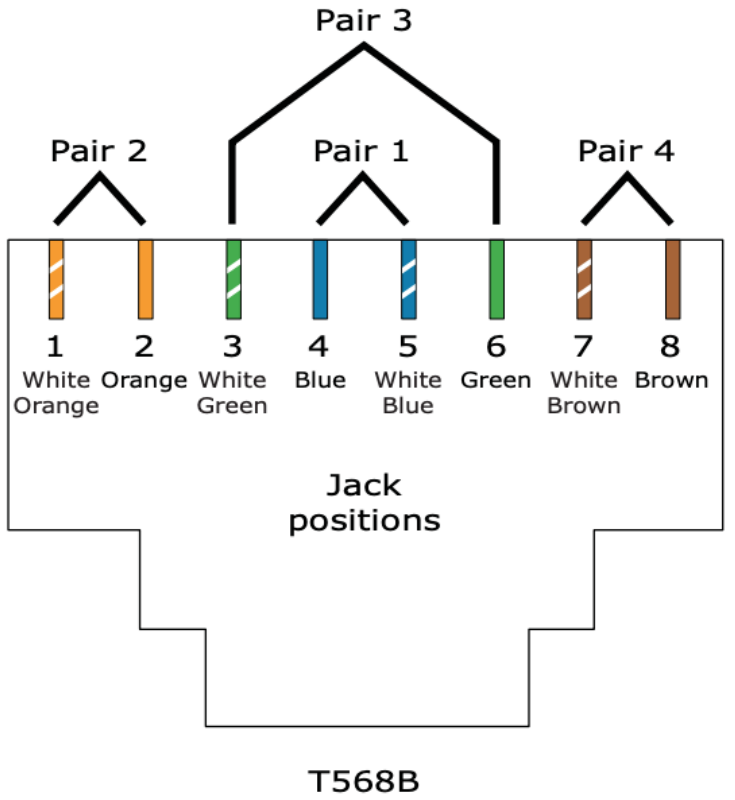


1.Connectivity interfaces

- RJ45/ Ethernet



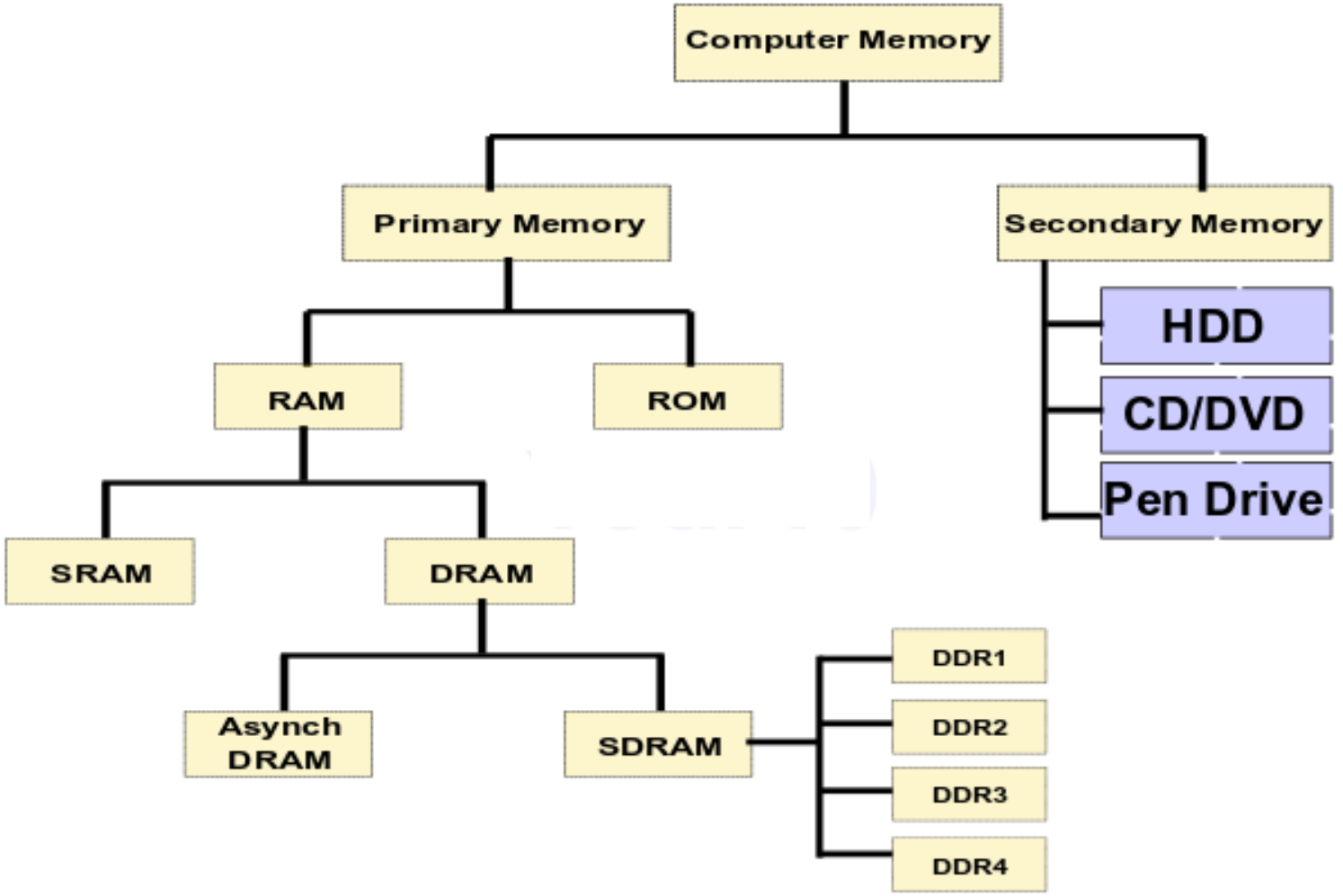
(8P8C) jack pin/pair assignments, viewed from the front



T568B

2.Memory Interfaces

- NAND/NOR
- DDR1/DDR2/DDR3



2.Memory Interfaces

- SDRAM

•Double data rate (DDR) is the advanced version of synchronous dynamic random access memory (SDRAM).

•SDRAM waits for clock signals before responding to control inputs. DDR uses both the falling and rising edges of the clock signal.



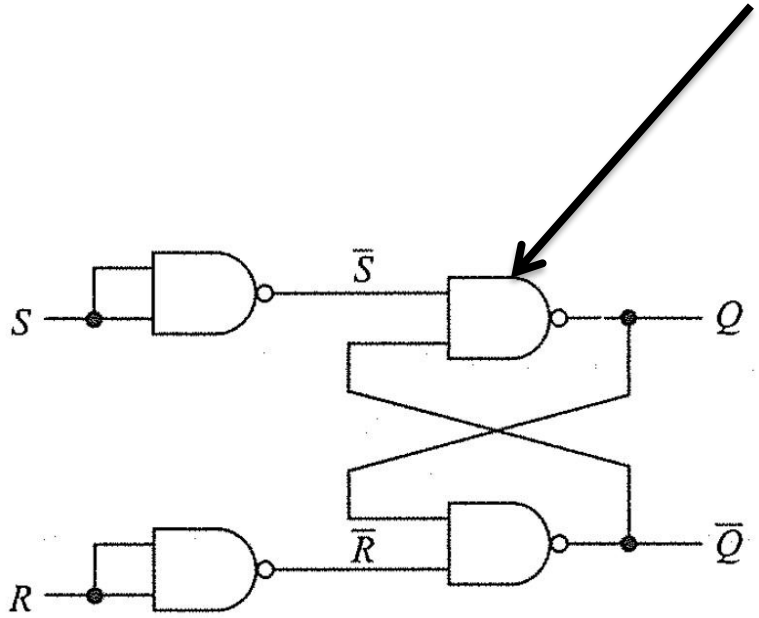
2.Memory Interfaces

- SDRAM

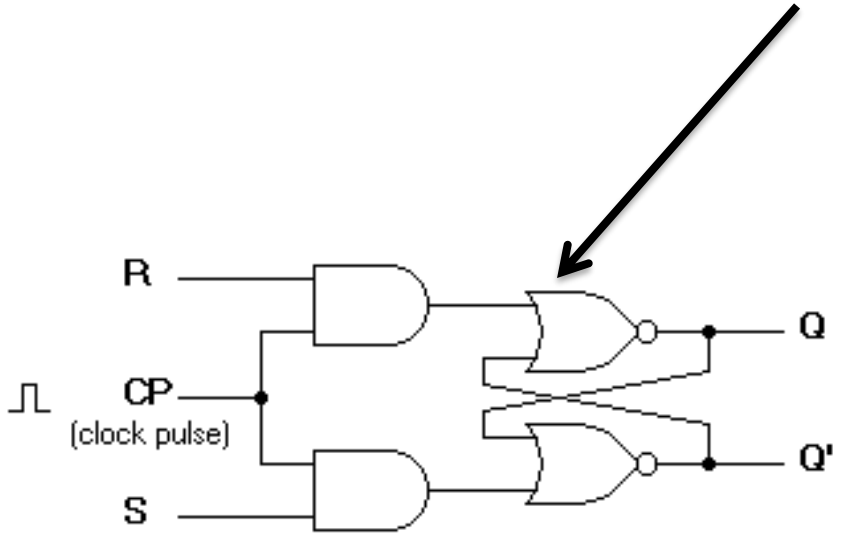
DDR SDRAM Standard	Internal rate (MHz)	Bus clock (MHz)	Prefetch	Data rate (MT/s)	Transfer rate (GB/s)	Voltage (V)
SDRAM	100-166	100-166	1n	100-166	0.8-1.3	3.3
DDR	133-200	133-200	2n	266-400	2.1-3.2	2.5/2.6
DDR2	133-200	266-400	4n	533-800	4.2-6.4	1.8
DDR3	133-200	533-800	8n	1066-1600	8.5-14.9	1.35/1.5
DDR4	133-200	1066-1600	8n	2133-3200	17-21.3	1.2

2.Memory Interfaces

- Basic Building Block of Memory is **Latch / Flip Flop**



NAND Based



NOR Based

3.Processing interfaces

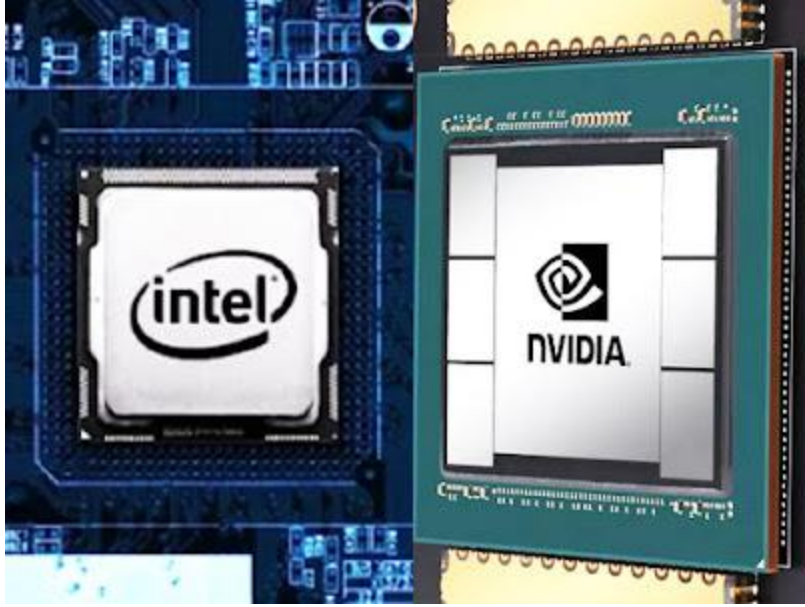
- CPU -Central Processing Unit
  - CPUs are at the heart of Internet of Things (IoT) devices
  - Can have one or more Processing Units
  - **Data Processing**
  - **Instruction Execution**
  - **Device Control**
  - **Connectivity Management**
  - **Power Management**

4. Graphics interfaces

- GPU- Graphics Processing Unit
  - GPUs are also known as video cards or graphics cards
  - In order to display pictures, videos, and 2D or 3D animations, each device uses a GPU
  - A GPU performs fast calculations of arithmetic and frees up the CPU to do different things

CPU Vs GPU

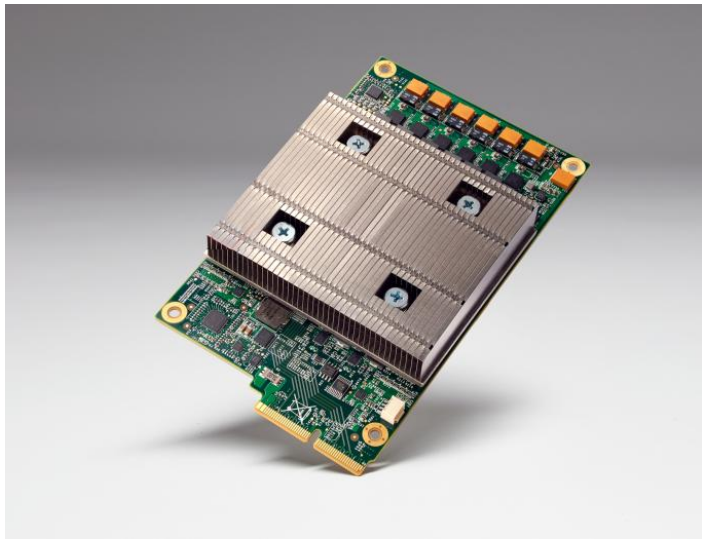
CPU	GPU
Central Processing Unit	Graphics Processing Unit
4-8 Cores	100s or 1000s of Cores
Low Latency	High Throughput
Good for Serial Processing	Good for Parallel Processing
Quickly Process Tasks That Require Interactivity	Breaks Jobs Into Separate Tasks To Process Simultaneously
Traditional Programming Are Written For CPU Sequential Execution	Requires Additional Software To Convert CPU Functions to GPU Functions for Parallel Execution



Other than CPU & GPU?

- **TPU-Tensor Processing Units**
  - It is a **processor** for Special purpose like

    ML applications of **Google**.
  - 4 trillion operations per second



Other than CPU, GPU & TPU ?



- **NPU- Neural processing unit**
  - It is a **processor** for Special purpose like ML applications of **any** other company like AMD, Intel, Apple, Samsung….
  - They are particularly adept at handling AI-related tasks, such as speech recognition, background blurring in video calls, and photo or video editing processes like object detection.
  - It's estimated that **NPUs can take as much as 10,000x less time** to accelerate machine learning tasks than a GPU

5. Audio/ Video Interfaces

- HDMI
- 3.5mm audio
- RCA video

5. Audio/ Video Interfaces

- What is HDMI?
- HDMI- High-Definition Multimedia Interface
  - a standard for transmitting high-quality audio and video signals
  - It was developed by Hitachi, Matsushita, and Sony
- What are the earlier Interfaces?
  - It was developed to replace analog audio/video cables like the DVI (Digital Visual Interface) connector or Video Graphics Array (VGA) connector

5. Audio/ Video Interfaces

- What is High Definition?

Video Resolution Chart

Resolution	Name	Aspect Ratio	Pixel Size
SD (Standard Definition)	480p	4:3	640x480
HD (High Definition)	720p	16:9	1280x720
Full HD (FHD)	1080p	16:9	1920x1080
2K (Quad HD)	2K or 1440	16:9	2560x1440
4K or Ultra HD (UHD)	4K or 2160p	1:1.9	3840x2160
8K (or Full Ultra HD	8K or 4320p	16:9	7680x4320

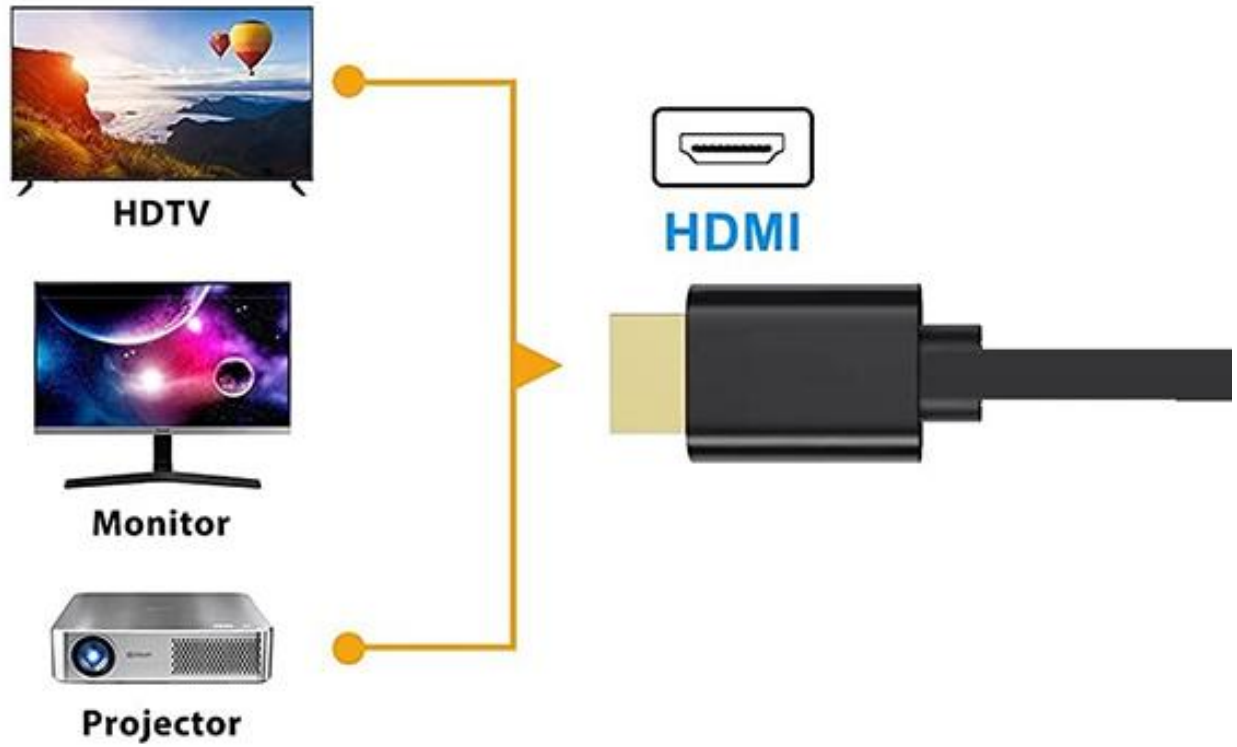
5. Audio/ Video Interfaces



- **HDMI** is a proprietary audio/video interface for transmitting uncompressed video data and compressed or uncompressed digital audio data from an HDMI-compliant source device,
- such as a display controller, to a compatible computer monitor, video projector, digital television, or digital audio device
- HDMI is a digital replacement for analog video standards.

5. Audio/ Video Interfaces

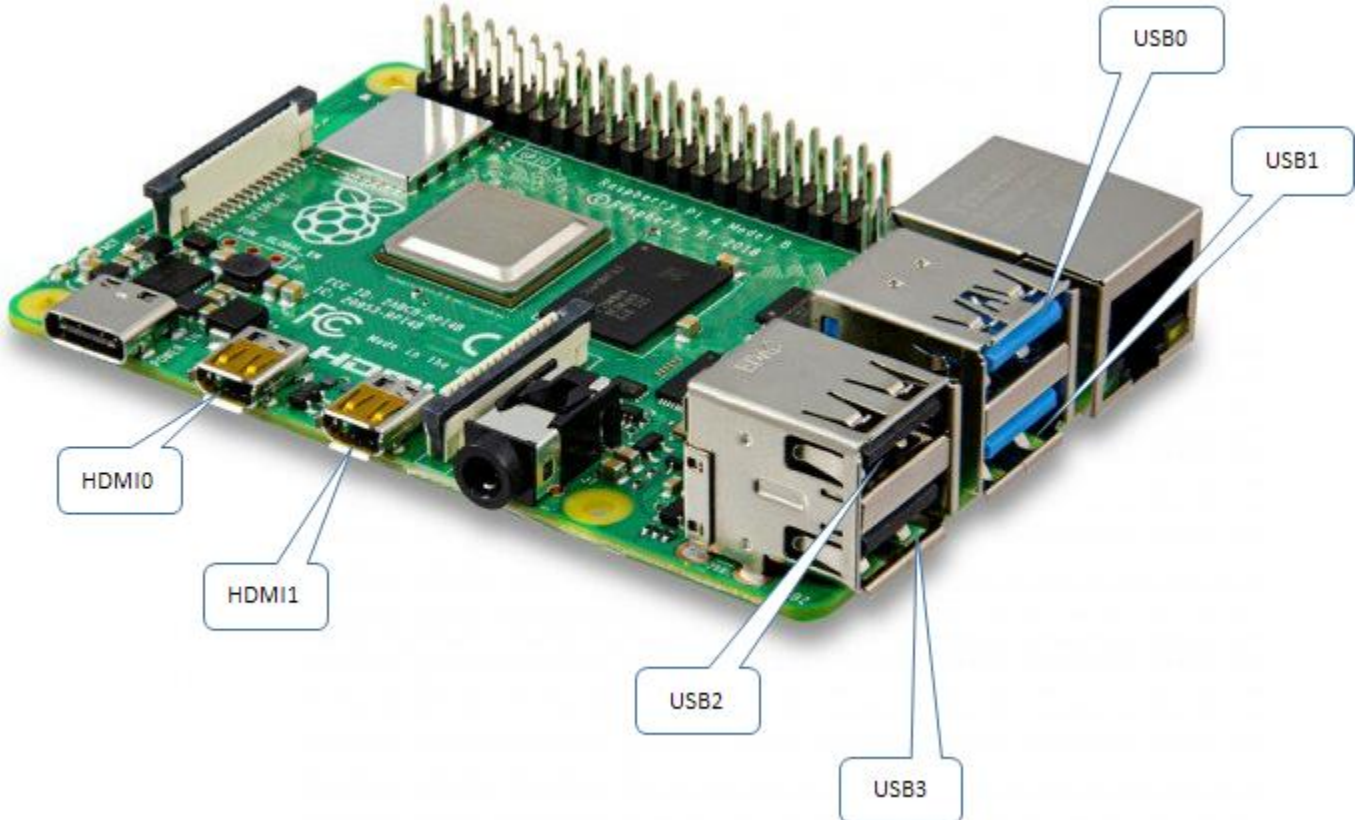
- Simultaneously transmitting digital video and audio from a source, such as a computer or TV cable box, to a computer monitor, TV or projector



5. Audio/ Video Interfaces

- Types of HDMI



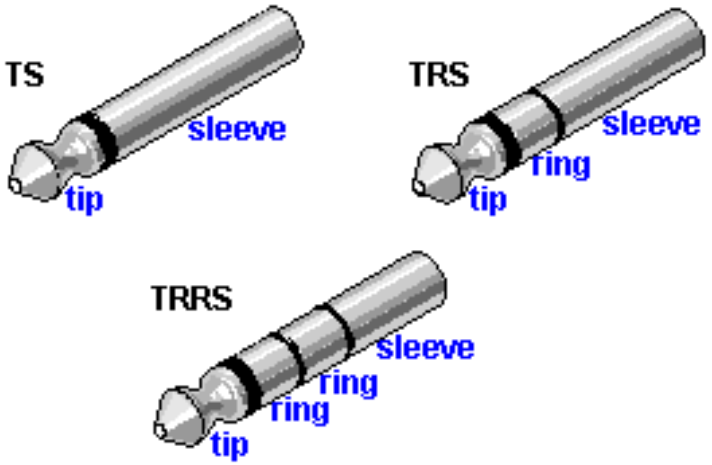


5. Audio/ Video Interfaces

- 3.5mm audio







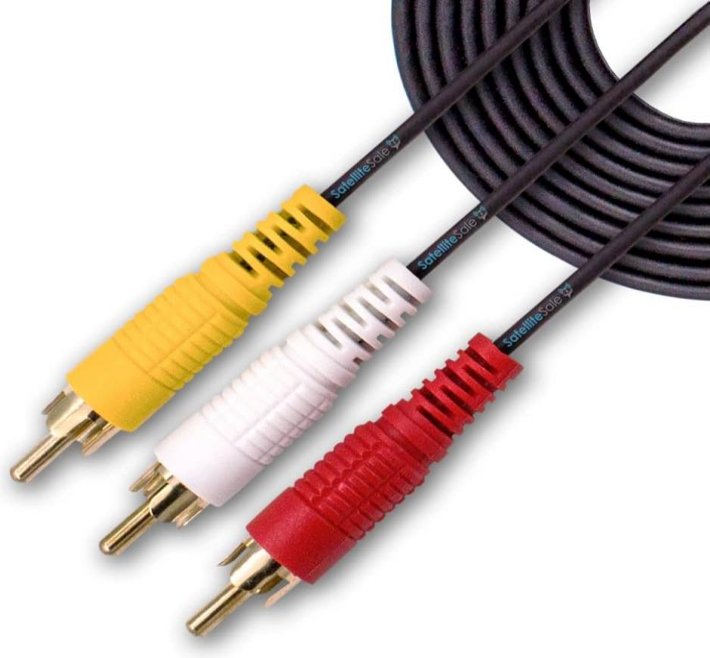
5. Audio/ Video Interfaces



- 3.5mm audio

  - To connect audio devices with 3.5mm audio output or input
  - such as smart phones, tablets, laptops, MP3 players, headphones, car stereos, speakers, etc

5. Audio/ Video Interfaces



- RCA video

  •The RCA connector is a type of electrical connector commonly used to carry audio and video signals

  •The name RCA derives from the company Radio Corporation of America, which introduced the design in the 1930s

- RCA to 3.5mm



6. Storage Interfaces

- SD
- MMC
- SDIO

6. Storage Interfaces

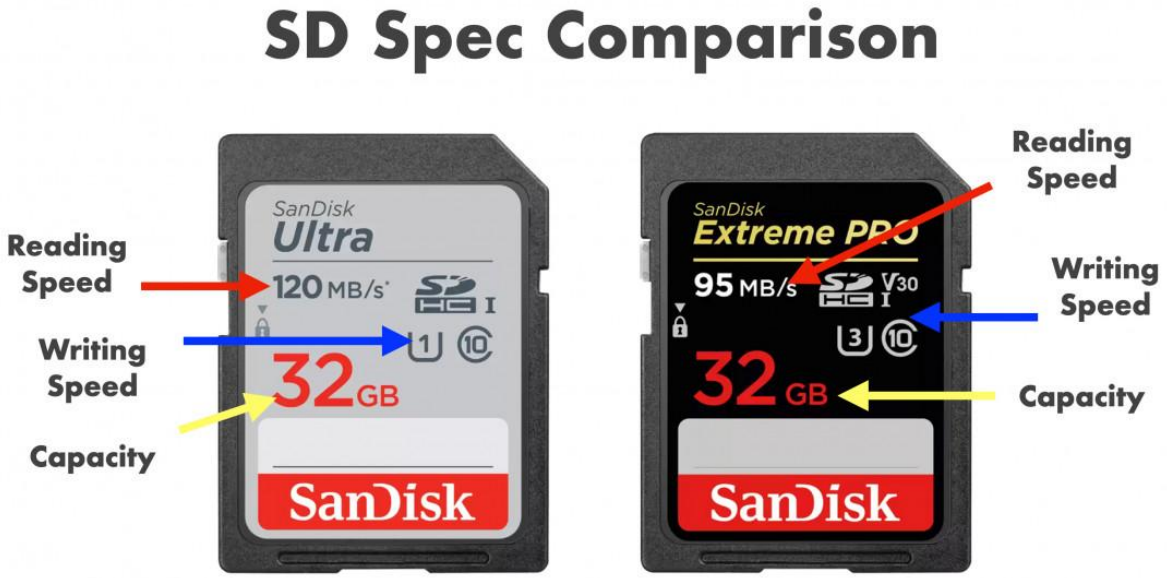
- SD-Secure Digital
- A Secure Digital (SD) card is a **tiny flash memory card** designed for **high-capacity memory** and various portable devices,

- SD is a popular type of memory card.

  - inexpensive way to store
  - Used to transfer data between computers and other digital products

6. Storage Interfaces

- Used in car navigation systems
- cellular phones
- e-books
- smart phones
- digital cameras
- music players
- digital video camcorders and
- personal computers

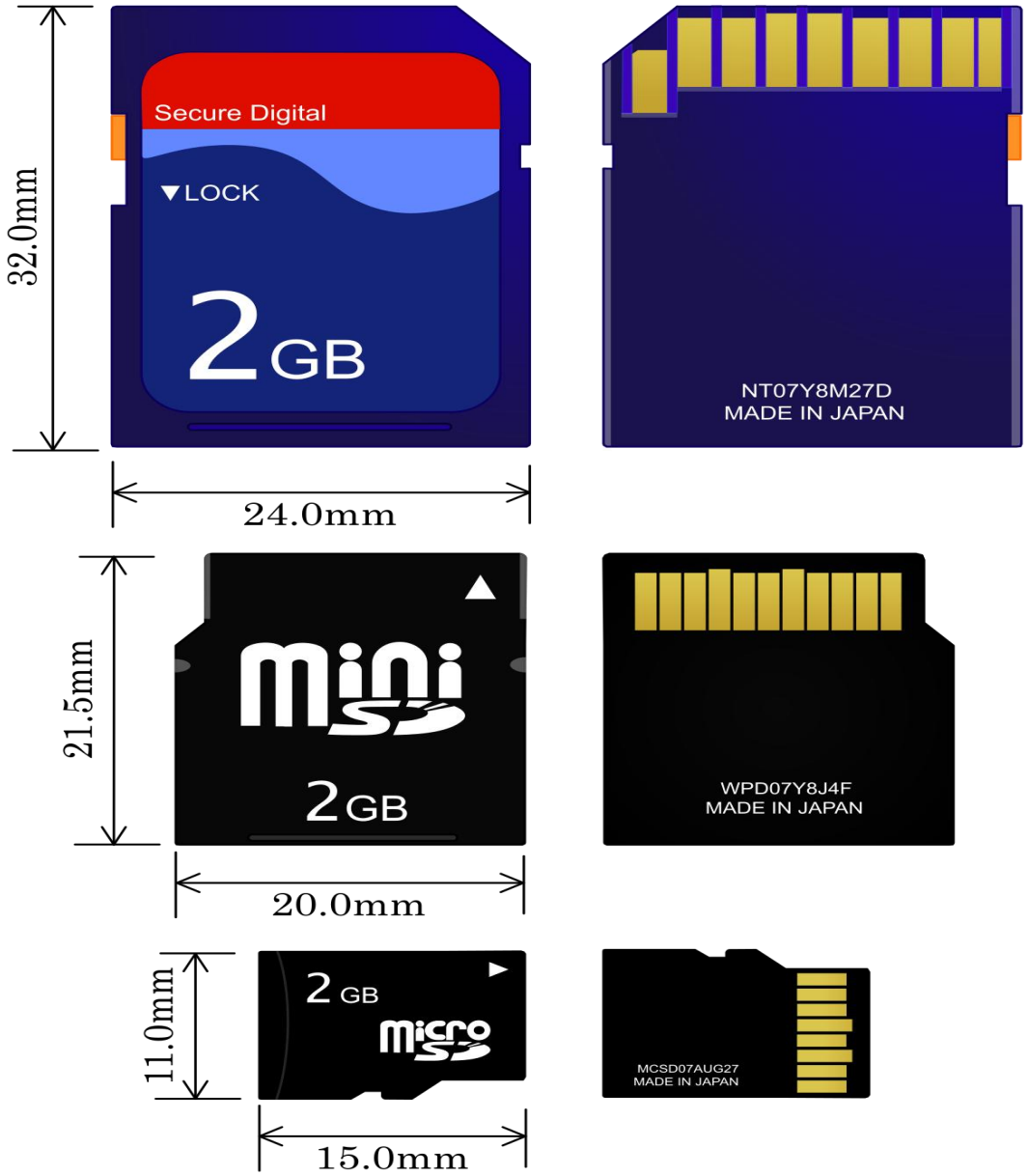


6. Storage Interfaces

- SD







6. Storage Interfaces

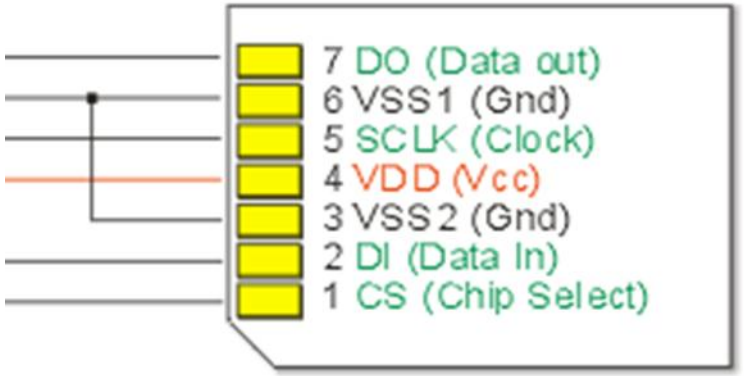
- MMC
- The **Multi Media Card**
- Unveiled in 1997

  by SanDisk and Siemens





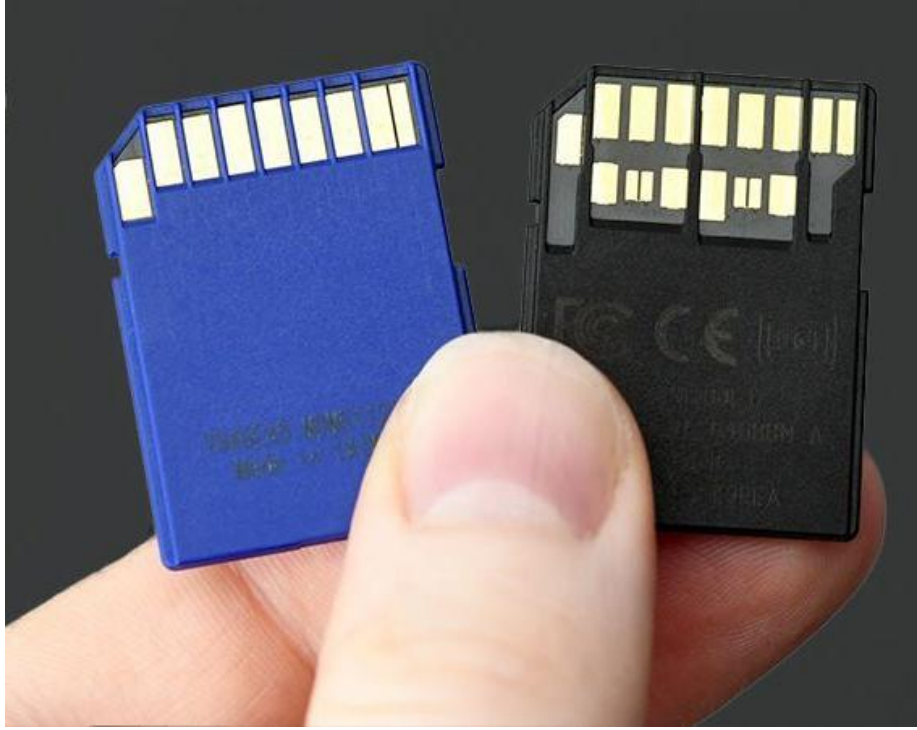
6. Storage Interfaces



- MMC
- MMC is about the size of a postage stamp: 32 mm × 24 mm × 1.4 mm.
- MMC originally used a **1-bit serial interface**, but newer versions of the specification allow transfers of **4 or 8 bits at a time**
- They are used in almost every context in which memory cards are used, like cellular phones, digital audio players, digital cameras, and PDAs
- Typically, an MMC operates as a storage medium for devices, in a form that can easily be removed for access by a PC via a connected MMC reader

6. Storage Interfaces

- SDIO- **Secure Digital Input Output**
- SDIO was introduced in 2001 as an extension of the SD standard, allowing devices to add input/output functionality through the SD card slot.



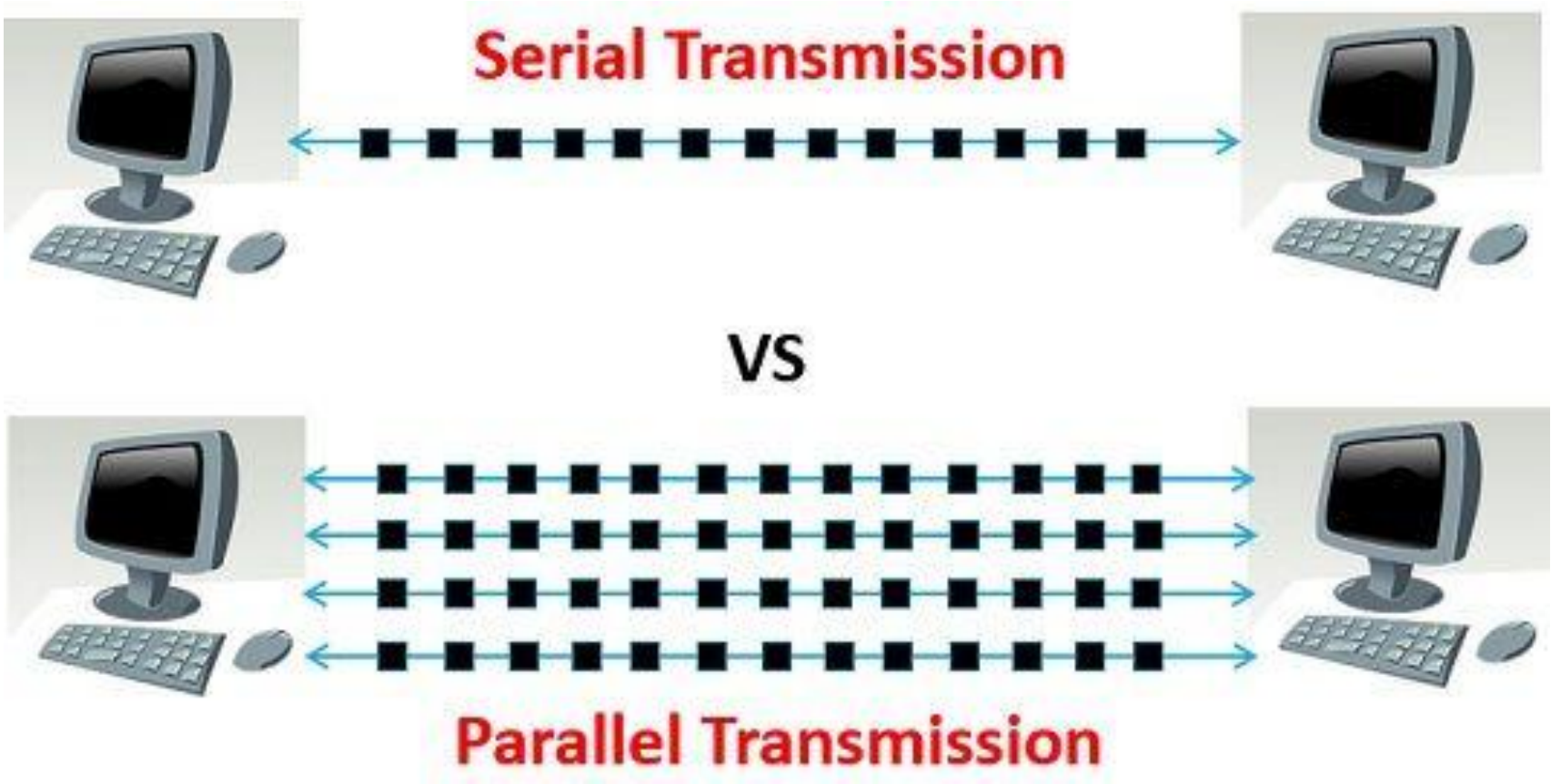
- While SDIO cards can include storage like regular SD cards, their primary function is to add I/O capabilities, so capacity varies widely depending on the application.
- SDIO cards are used to add functions like Bluetooth, Wi-Fi, GPS, and camera modules to devices with SD card slots, such as PDAs, smartphones, and laptops.

7. I/O Interfaces

- UART
- SPI
- I2C
- CAN

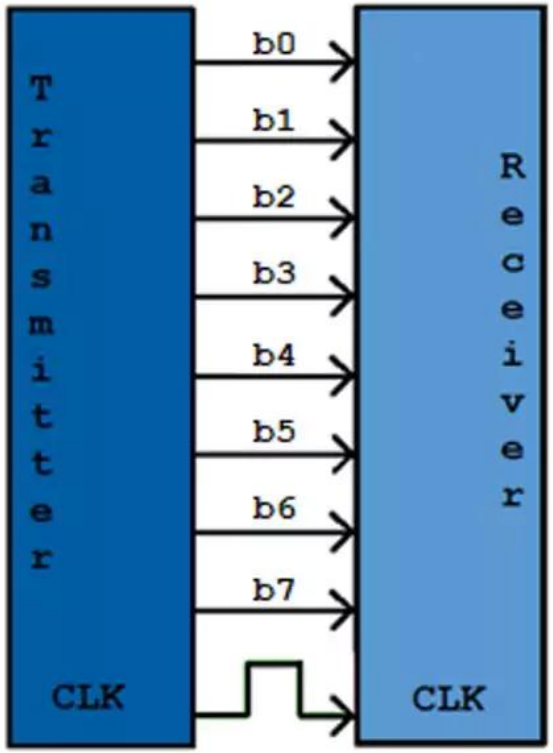
7. I/O Interfaces

- Serial Vs Parallel

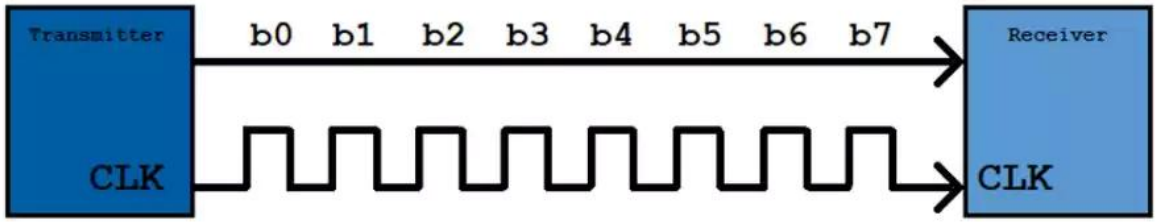


7. I/O Interfaces

- Serial Vs Parallel



8-bit Parallel Interface



8-bit Serial Interface

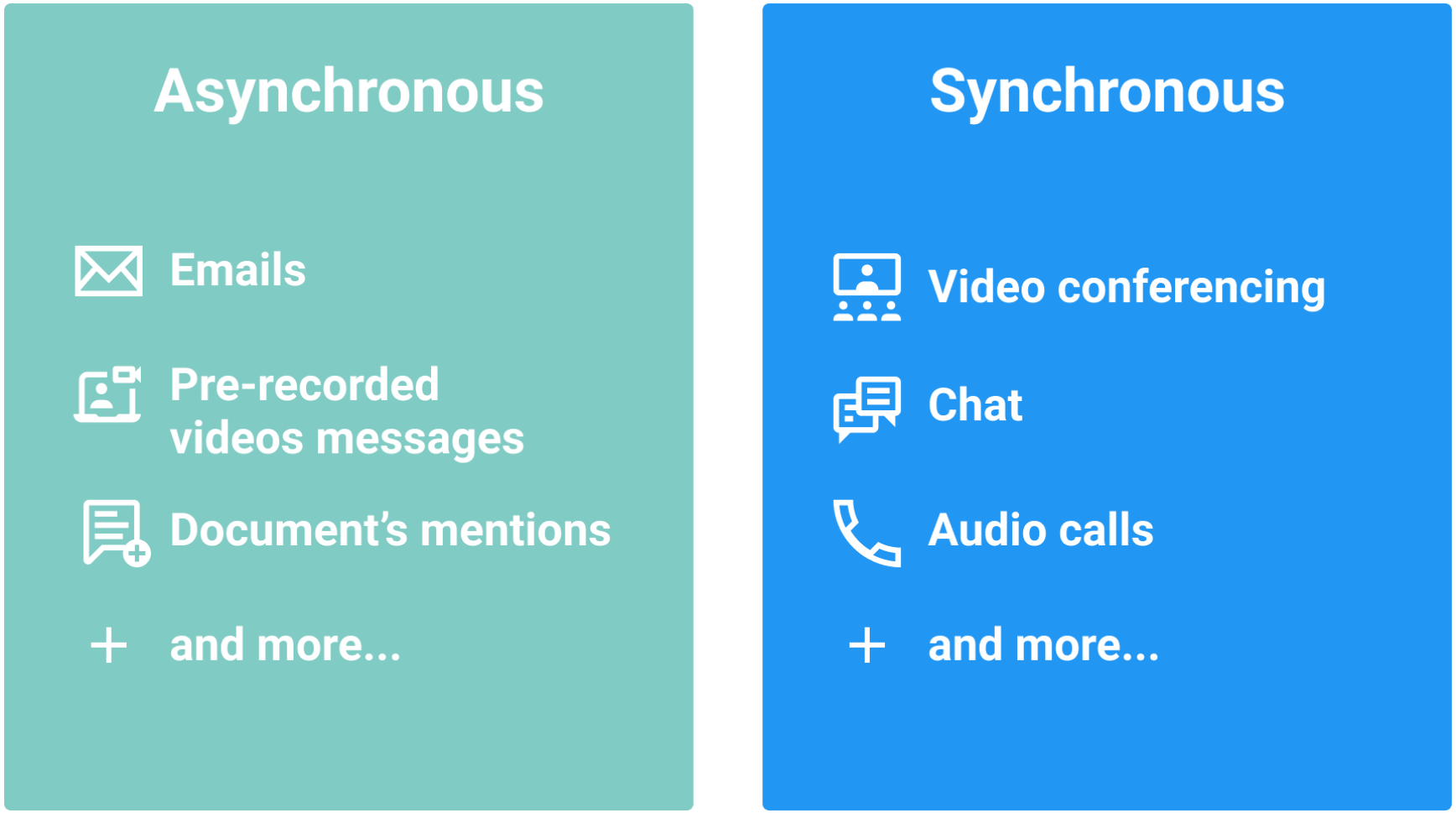
7. I/O Interfaces

- Synchronous Vs Asynchronous



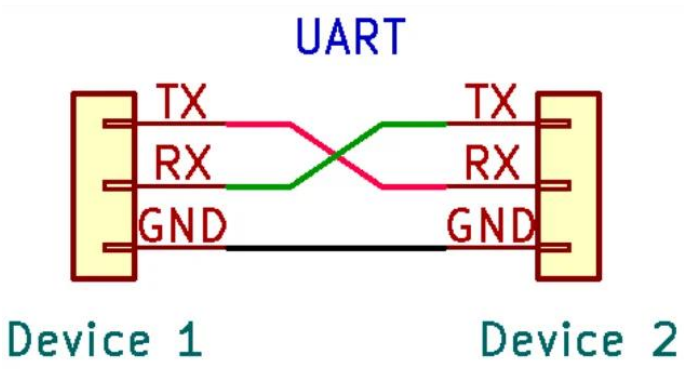
7. I/O Interfaces

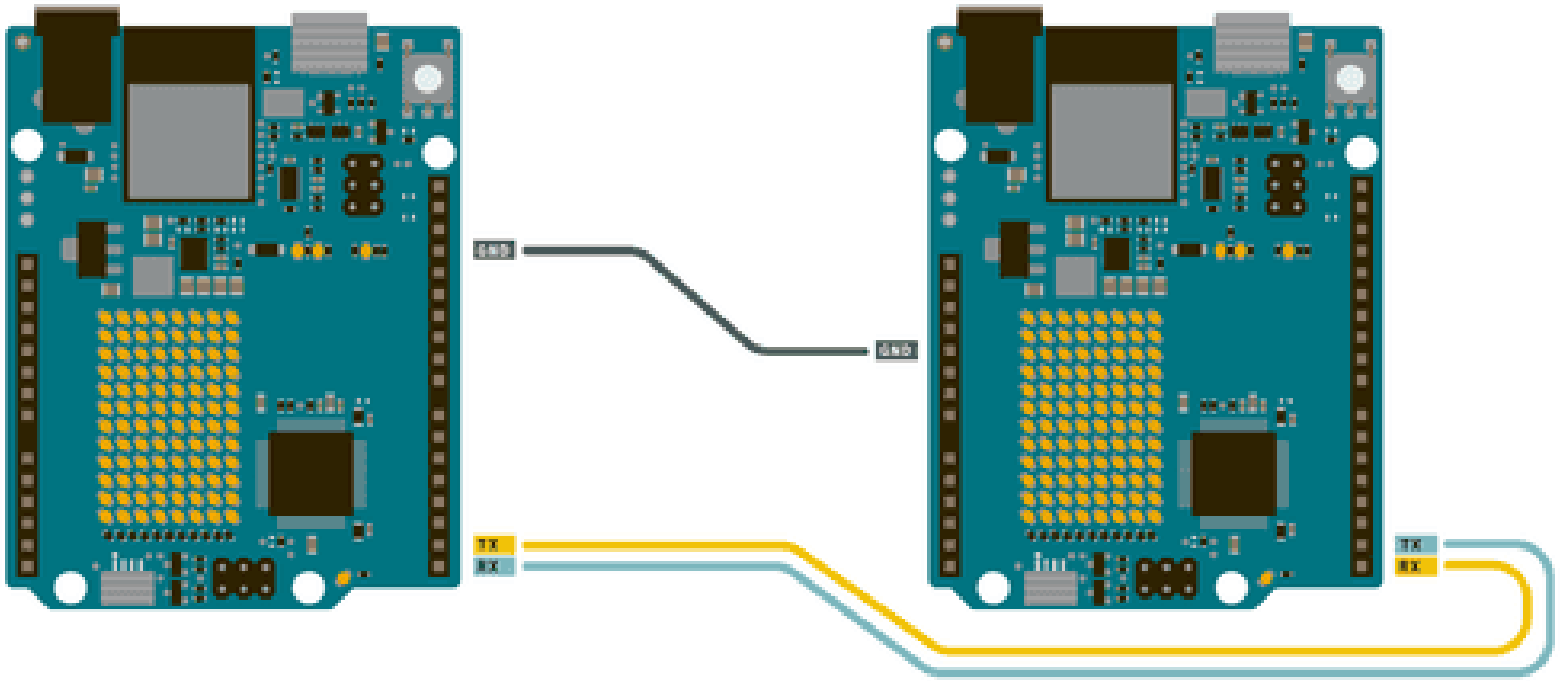
- Synchronous Vs Asynchronous



7. I/O Interfaces

- UART- Universal Asynchronous Receiver Transmitter

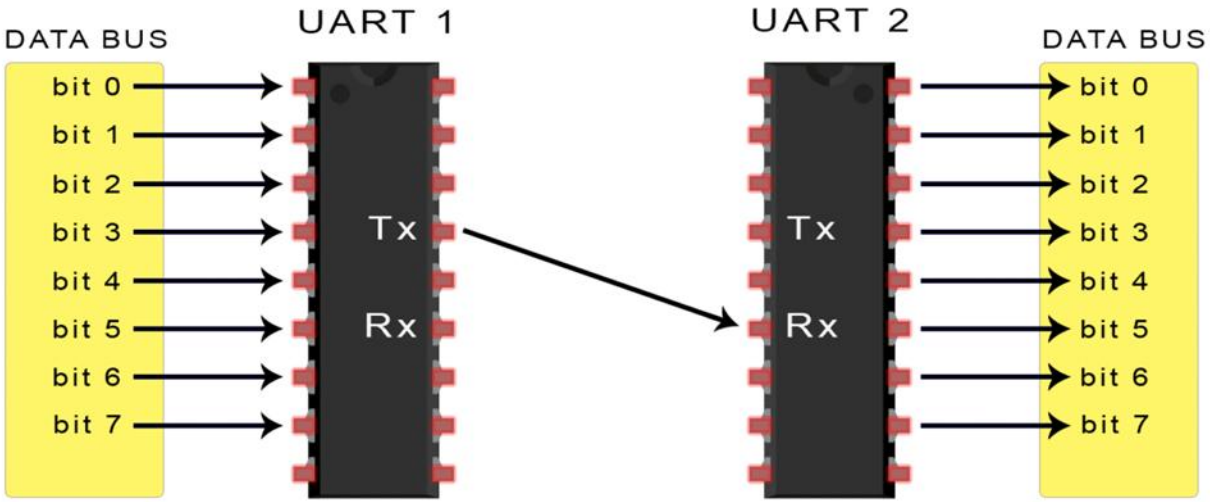




7. I/O Interfaces

- UART stands for universal asynchronous receiver / transmitter and defines a protocol, or set of rules, for exchanging **serial data between** two devices.

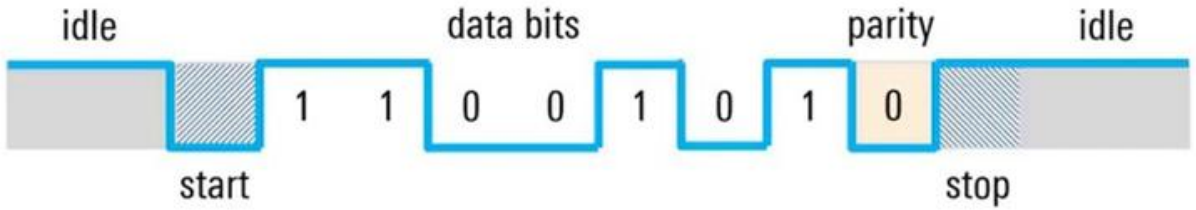
- UART is very simple **and only uses two wires** between transmitter and receiver to transmit and receive in both directions.



7. I/O Interfaces

UART Frame Format

- UART frames consist of:
  - Start / stop bits
  - Data bits
  - Parity bit (optional)
- High voltage (“mark”) = 1, low voltage (“space”) = 0



Start bit	data bits	Parity bits	stop bit

UART BAUD RATES

Baud rate
9600
19200
38400
57600
115200

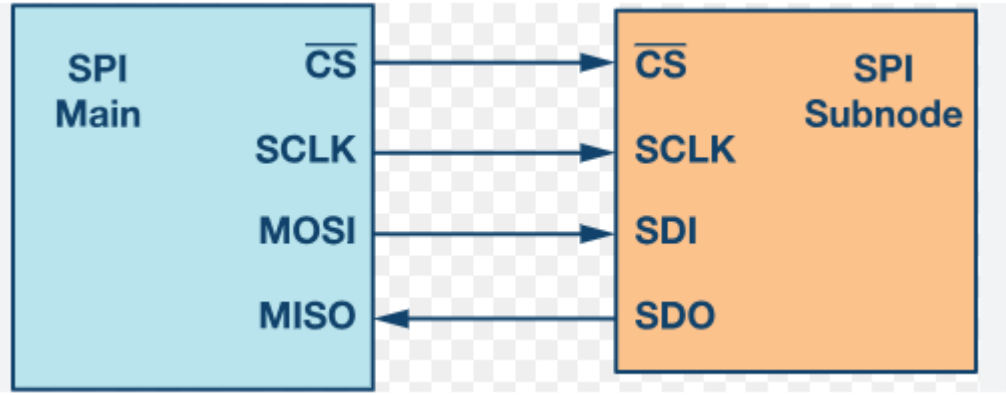
7.I/O Interfaces

- **SPI-** Serial Peripheral Interface
- Is an interface bus commonly used to send data between

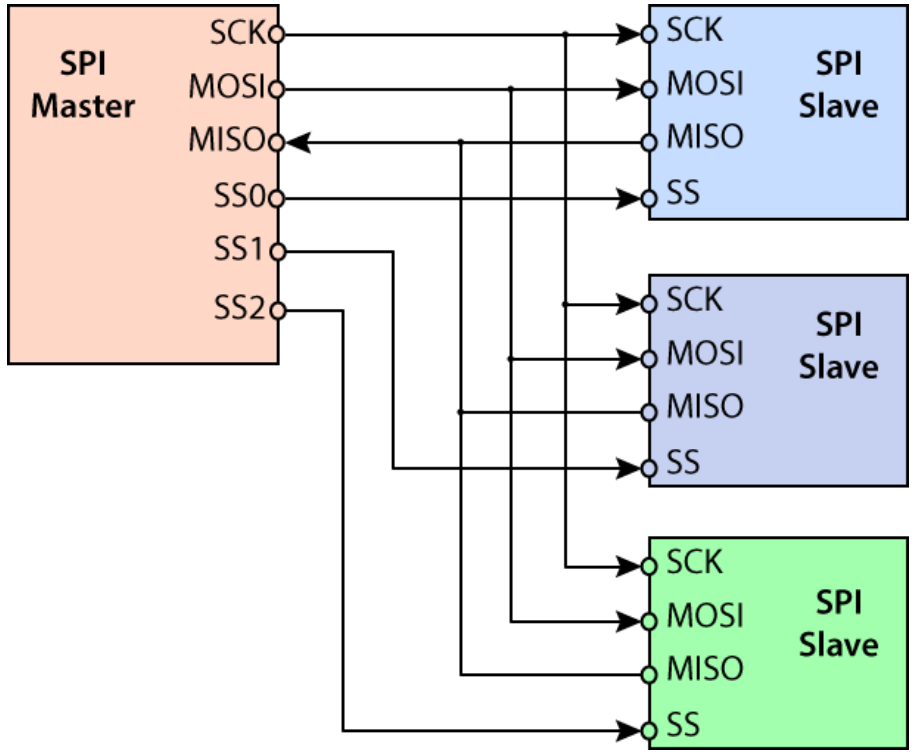
  microcontrollers and small peripherals such as shift registers, sensors, and SD cards
- Developed by Motorola in the 1980s
- It uses separate clock and data lines, along with a select line to choose the device you wish to talk to.

I/O Interfaces

- **SPI**



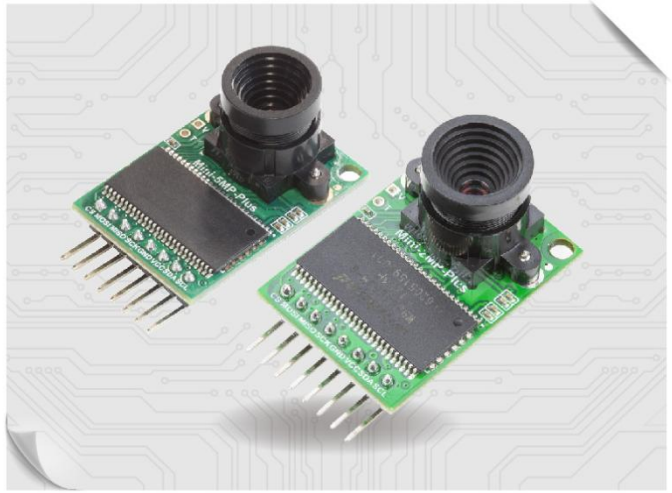
Single Master & single slave
Interface with **SPI**



Single Master & Multi slave interfaced
With SPI

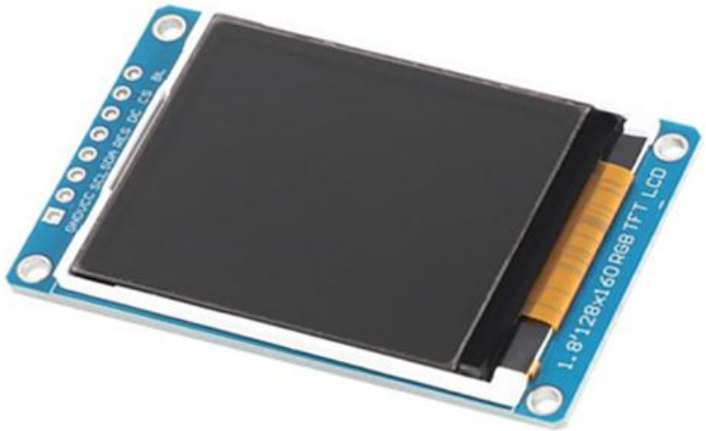
I/O Interfaces

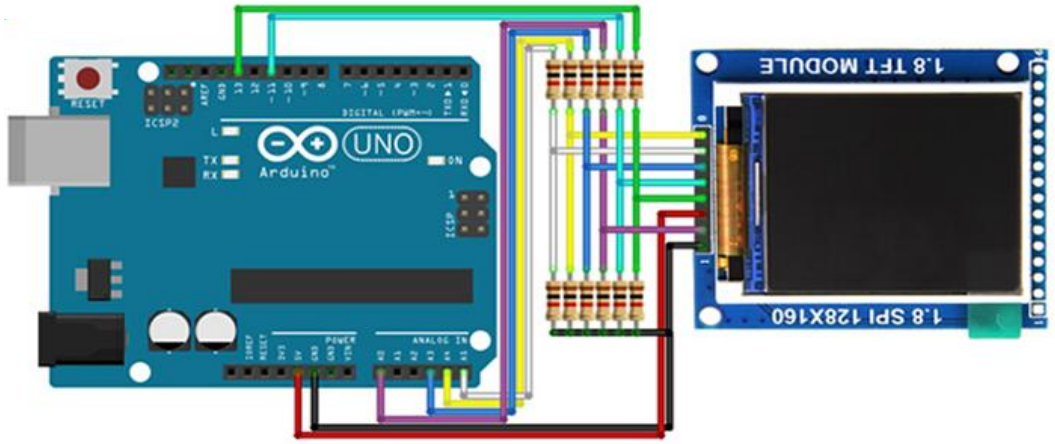
- **SPI**



SPI Camera

1.8” SPI TFT Color Display Arduino





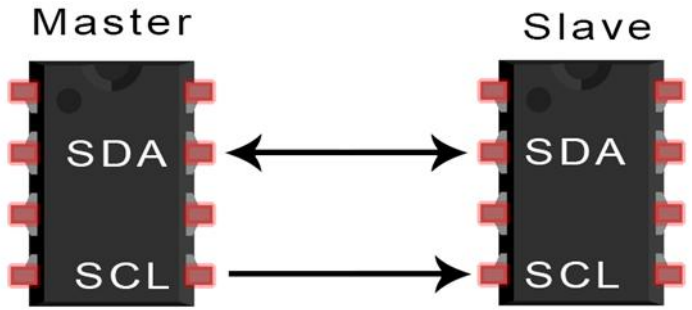
I/O Interfaces

- **SPI Specifications**

Wires Used	4
Maximum Speed	Up to 10 Mbps
Synchronous or Asynchronous?	Synchronous
Serial or Parallel?	Serial
Max # of Masters	1
Max # of Slaves	Theoretically unlimited*

SPI Transfers the data bit by bit

I/O Interfaces

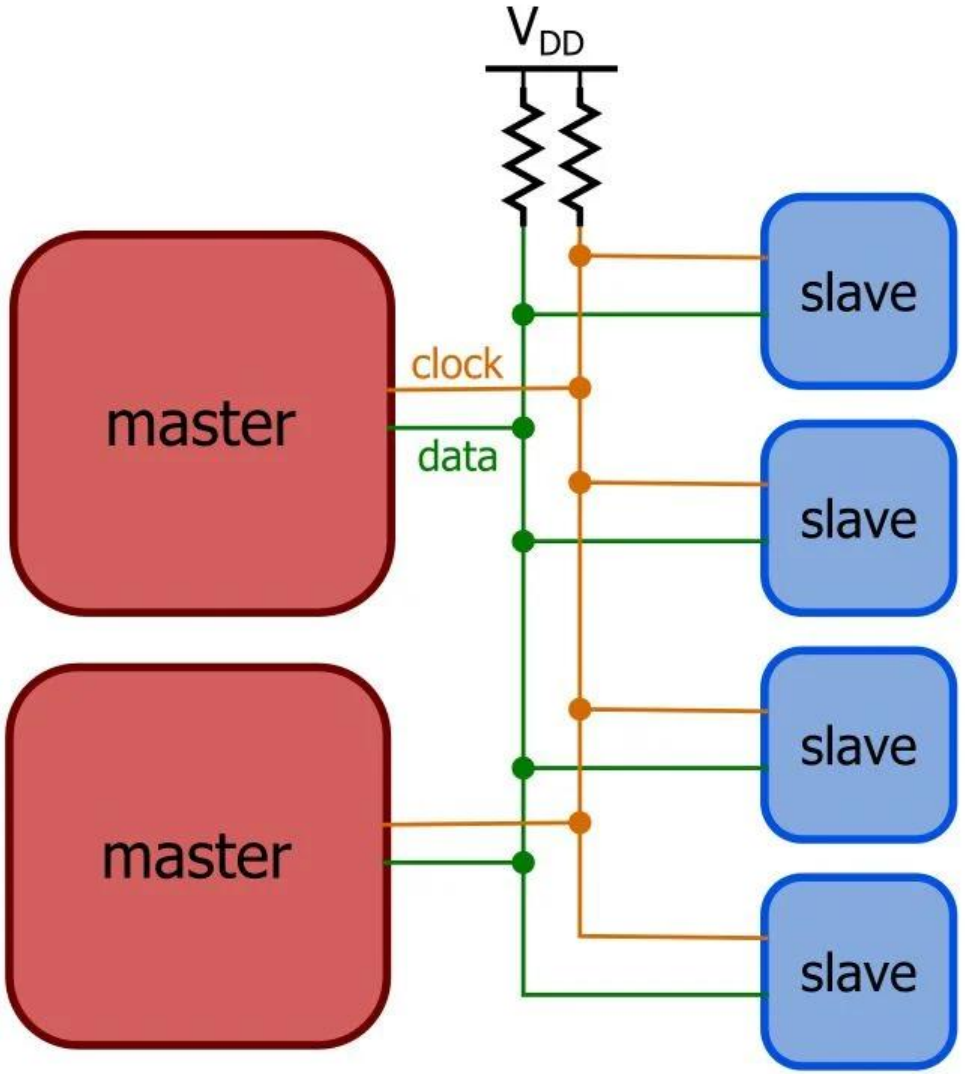


- IIC or I2C-**Inter Integrated Circuit**
- It is a bus interface connection protocol incorporated into devices for serial communication.
- It was originally designed by **Philips Semiconductor** in 1982.
- Recently, it is a widely used protocol **for short-distance** communication. It is also known as **Two Wired** Interface(TWI).

I/O Interfaces

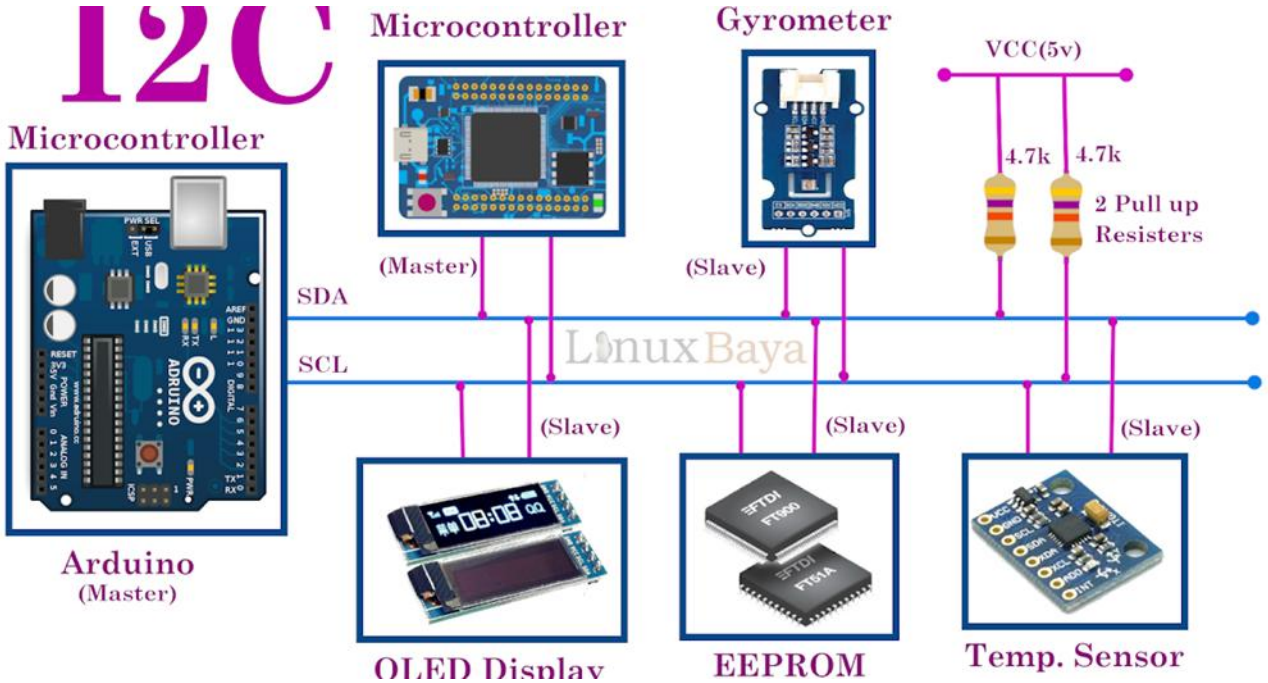
- IIC or I2C-**Inter Integrated Circuit**

Multi Master & Multi slave
Interface with **I2C**



I/O Interfaces

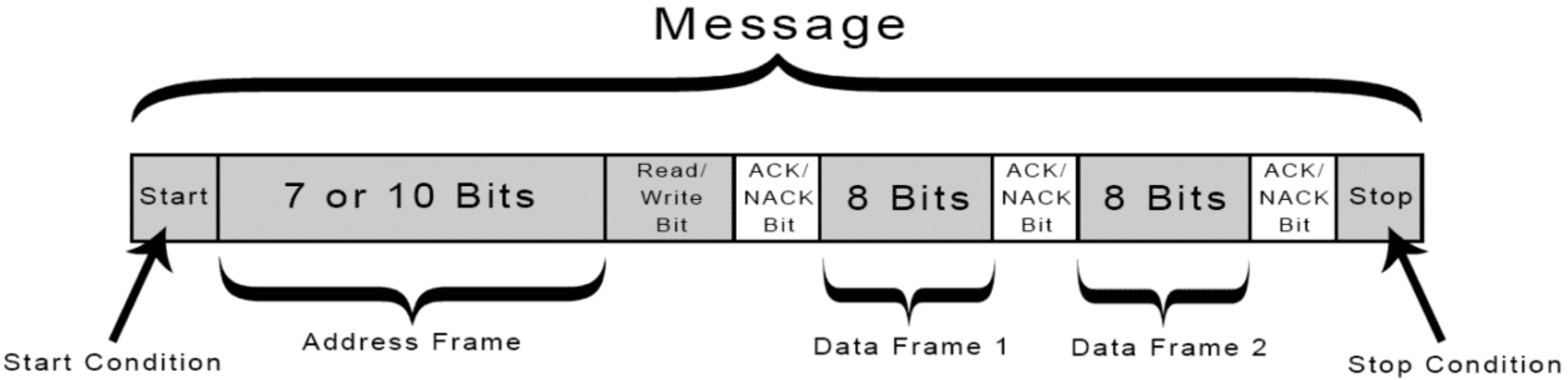
- IIC or I2C-**Inter Integrated Circuit**



Multi Master & Multi slave Interface with **I2C**

I/O Interfaces

I2C frame Format



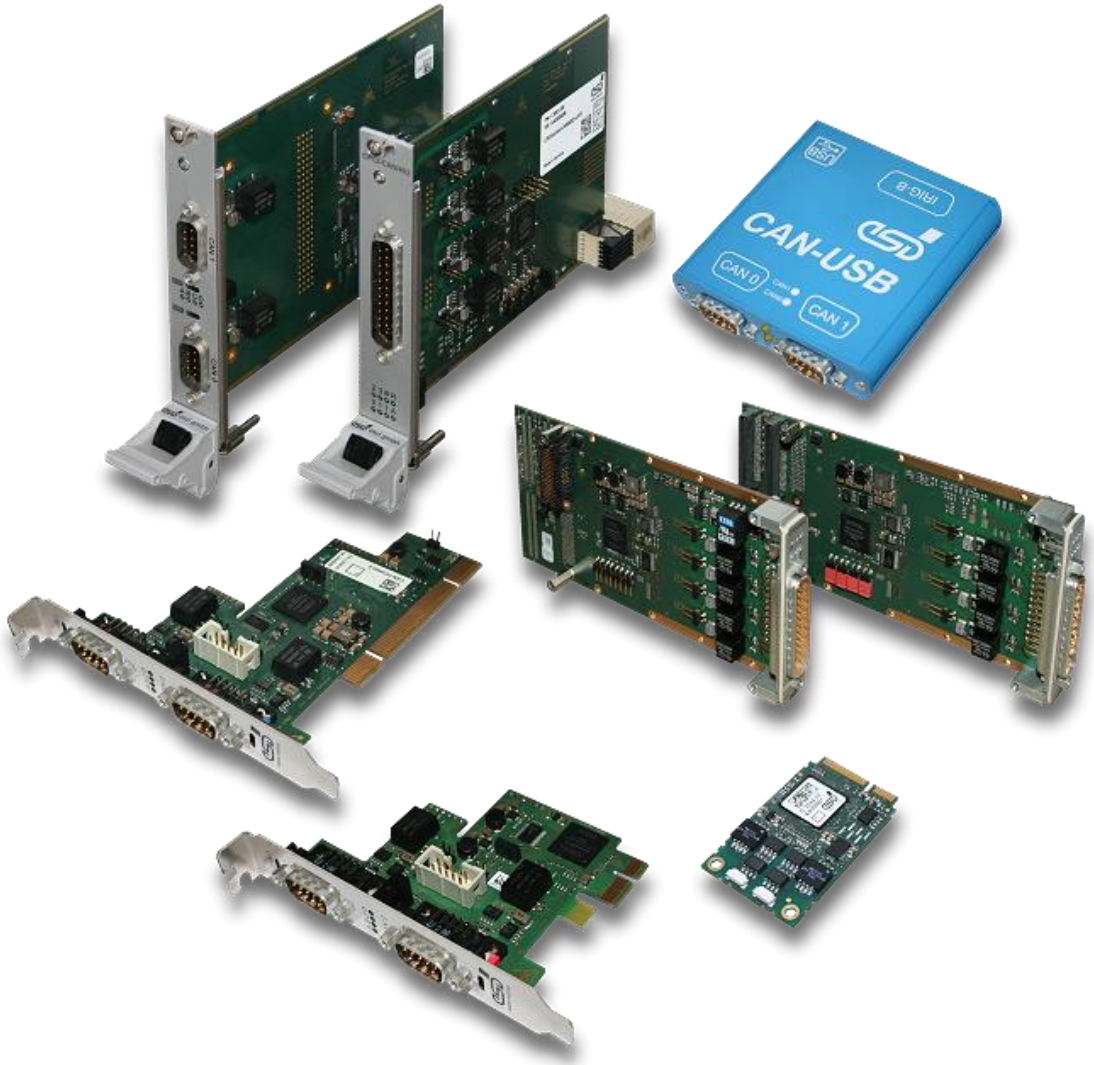
I/O Interfaces

- I2C-Specifications

Wires Used	2
Maximum Speed	Standard mode= 100 kbps
	Fast mode= 400 kbps
	High speed mode= 3.4 Mbps
	Ultra fast mode= 5 Mbps
Synchronous or Asynchronous?	Synchronous
Serial or Parallel?	Serial
Max # of Masters	Unlimited
Max # of Slaves	1008

I/O Interfaces

- CAN- Controller area network or
    - Campus area network

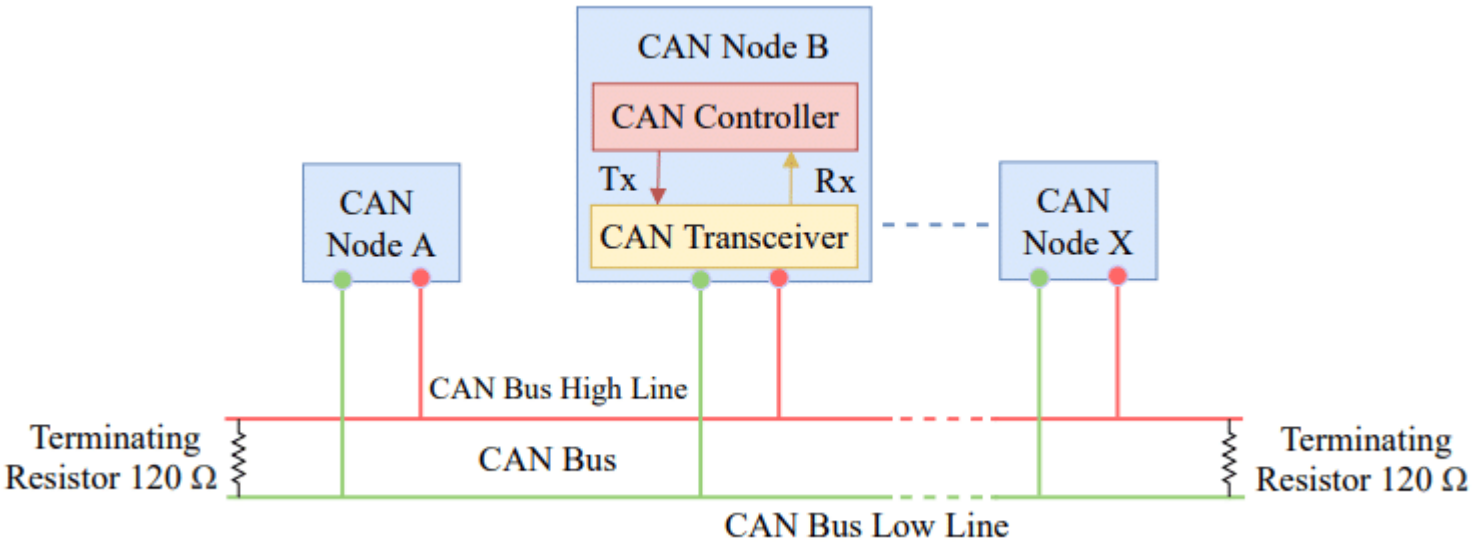


I/O Interfaces

- CAN- is a robust vehicle bus standard designed to allow microcontrollers and devices to communicate with each other within a vehicle without a host computer
- It was developed by Bosch in the mid-1980s and has since become a widely adopted standard in automotive and industrial applications due to its efficiency, reliability, and real-time capabilities.

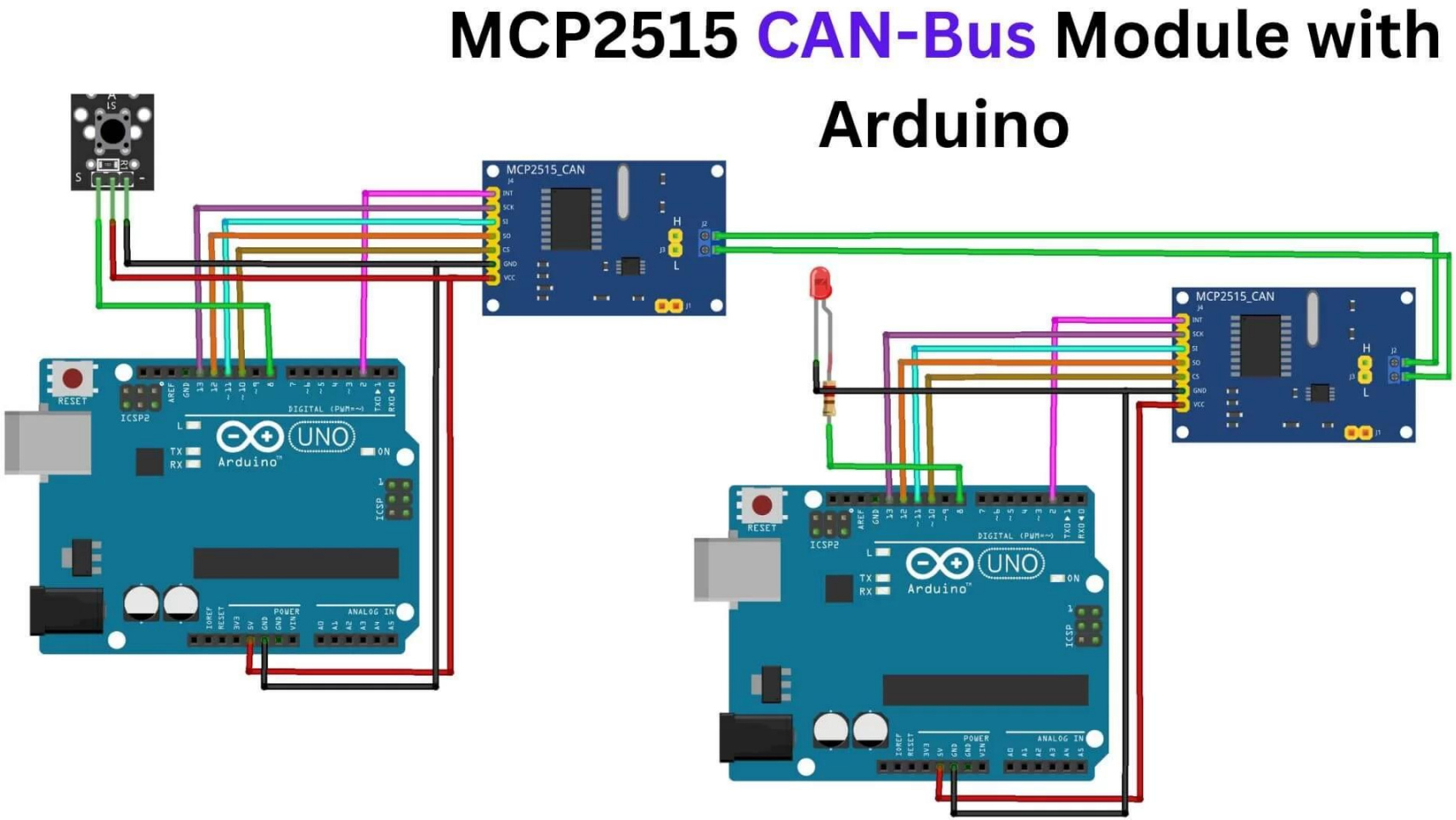
I/O Interfaces

- CAN



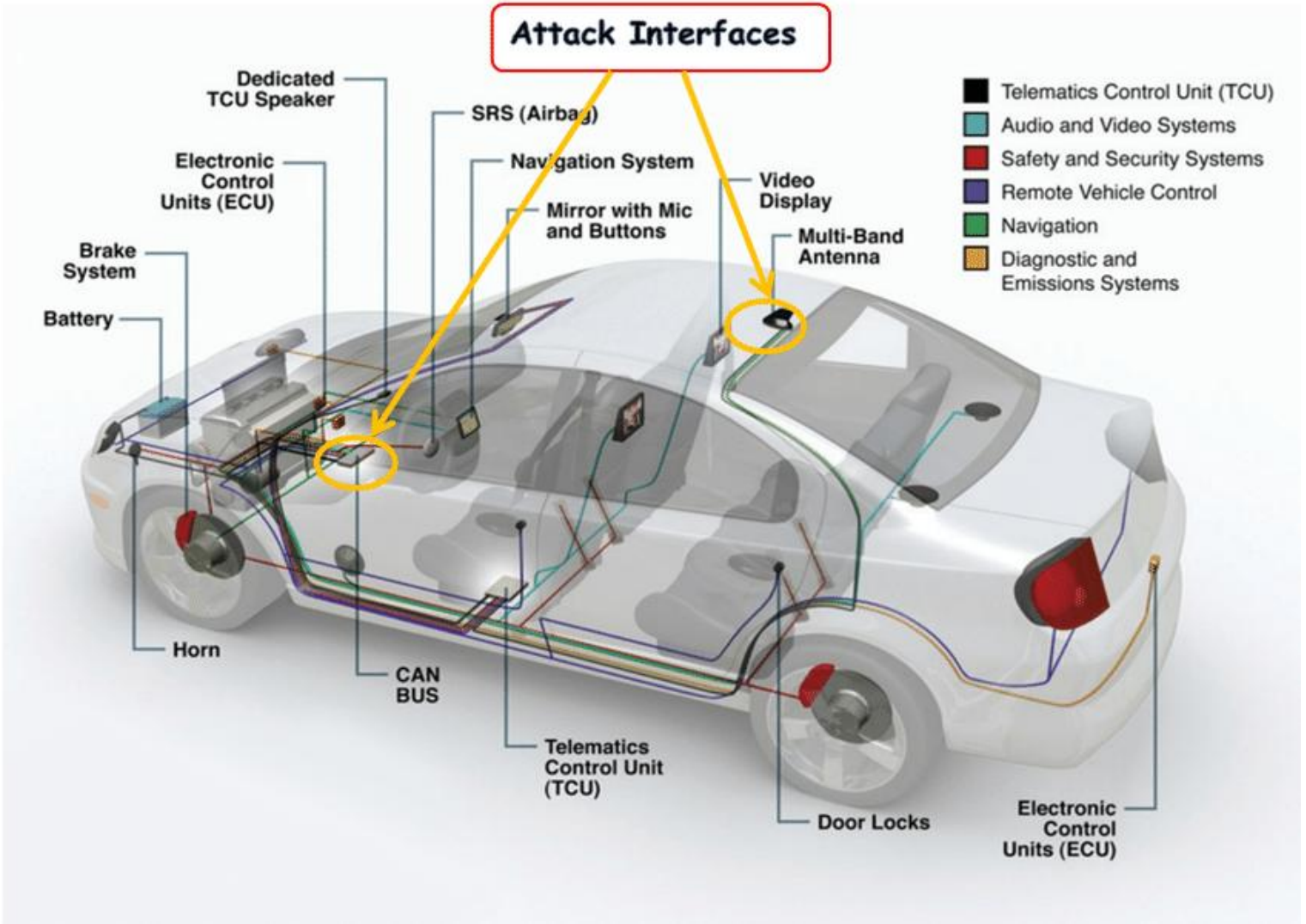
I/O Interfaces

- CAN



I/O Interfaces

- CAN-



Design of IoT

- Physical Design of IoT
  - Things in IoT
    - Generic Block Diagram of an IoT Device
  - IoT protocols
    - IoT Protocol Stack

- Logical Design of IoT- Functional diagram of IoT
  - Functional blocks
  - Communication models
  - Communication APIs

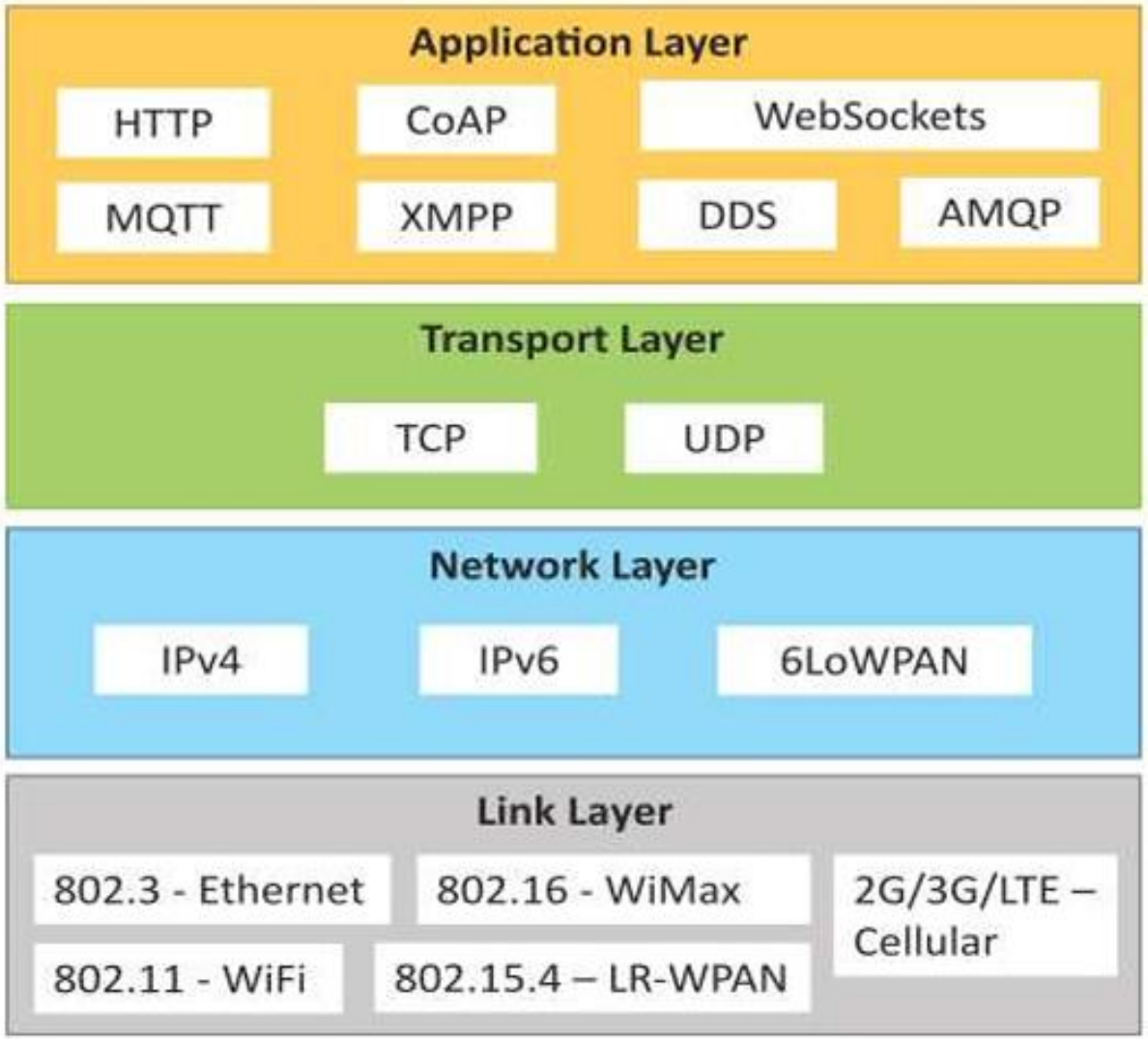
IoT Protocols

- The IoT devices are typically connected to the Internet via an IP (Internet Protocol) network.
- However, devices such as Bluetooth and RFID allow IoT devices to connect locally. In these cases, there's a difference in power, range, and memory used.
- Connection through IP networks, are comparatively complex, requires increased memory and power from the IoT devices while the range is not a problem.

IoT Protocols

- On the other hand, non-IP networks demand comparatively less power and memory but have a range limitation.
- As far as the IoT communication protocols or technologies are concerned, a mix of both IP and non-IP networks can be considered depending on usage.

IoT Protocols



IoT Protocols-Link Layer

- In computer networking, the link layer is the lowest layer in the Internet protocol suite, the networking architecture of the Internet.
- The link layer is the group of methods and communications protocols confined to the link that a host is physically connected to.
- The link is the physical and logical network component used to interconnect hosts or nodes in the network and a link protocol is a suite of methods and standards that operate only between adjacent network nodes of a network segment.

IoT Protocols-Link Layer

Ethernet Standard		
Sr.No	**Standard**	**Shared medium**
1	802.3	Coaxial cable
2	802.3.i	Copper Twisted pair
3	802.3.j	Fiber Optic
4	802.3.ae	Fiber…..10Gbits/s

IoT Protocols-Link Layer

WiFi Standard		
S.No	Standard	Operates in
1	802.11a	5 GHz band
2	802.11b& 802.11g	2.4GHz band
3	802.11.n	2.4/5 GHz bands
4	802.11.ac	5GHz band
5	802.11.ad	60Hz band

WiMax Standard		
S.No	Standard	Data Rate
1	802.16m	100Mb/s for mobile stations, 1Gb/s for fixed stations

Mobile Communication Standard		
Sr.No	Standard	Operates in
1	2G	GSM-CDMA
2	3G	UMTS and CDMA 2000
3	4G	LTE

IoT Protocols-Network/ Internet Layer

- The internet layer is a group of internetworking methods, protocols, and specifications in the Internet protocol suite that are used to transport network packets from the originating host across network boundaries; if necessary, to the destination host specified by an IP address.
- The internet layer derives its name from its function facilitating internetworking, which is the concept of connecting multiple networks with each other through gateways.

IoT Protocols-Network/ Internet Layer

- Responsible for sending of IP datagrams from source to destination network
- Performs the host addressing and packet routing
- Host identification is done using hierarchical IP addressing schemes such as IPV4 or IPV6

IoT Protocols-Network/ Internet Layer

- IPV4
  - Used to identify the devices on a network using hierarchical addressing scheme

    Uses 32-bit address scheme
  - This provides approximately **4.3 billion unique addresses** (2^{32} = 4,294,967,296).
  - Address format: Written in **dotted decimal notation** (four 8-bit fields separated by dots)
  - Example: 192.168.1.1

IoT Protocols-Network/ Internet Layer

IPV6 -Uses 128-bit address scheme

- This provides **340 undecillion addresses** ($2^{128} \approx 3.4 \times 10^{38}$), which is effectively an infinite number of addresses.
- Address format: Written in **hexadecimal** and separated by colons into eight 16-bit fields.
- Example: 2001:0db8:85a3:0000:0000:8a2e:0370:7334

IoT Protocols-Network/ Internet Layer

6LoWPAN (IPV6 over Low power Wireless Personal Area Network)

- Used for devices with limited processing capacity, Operates in 2.4 Ghz , Data Rates of 250Kb/s.
- 6LoWPAN allows IPv6 packets to be sent and received over constrained networks such as those that use the IEEE 802.15.4 standard (used in wireless sensor networks).

IoT Protocols-Network/ Internet Layer

6LoWPAN (IPV6 over Low power Wireless Personal Area Network)

- **Low Power**: It's designed for devices that have limited battery life and processing power.
- **Small Packet Sizes**: IPv6 headers are compressed to fit into smaller frames suitable for low-power networks.
- **Mesh Networking**: Supports multi-hop, mesh-based networking, ideal for IoT systems like smart homes and smart cities.

IoT Protocols- Transport Layer

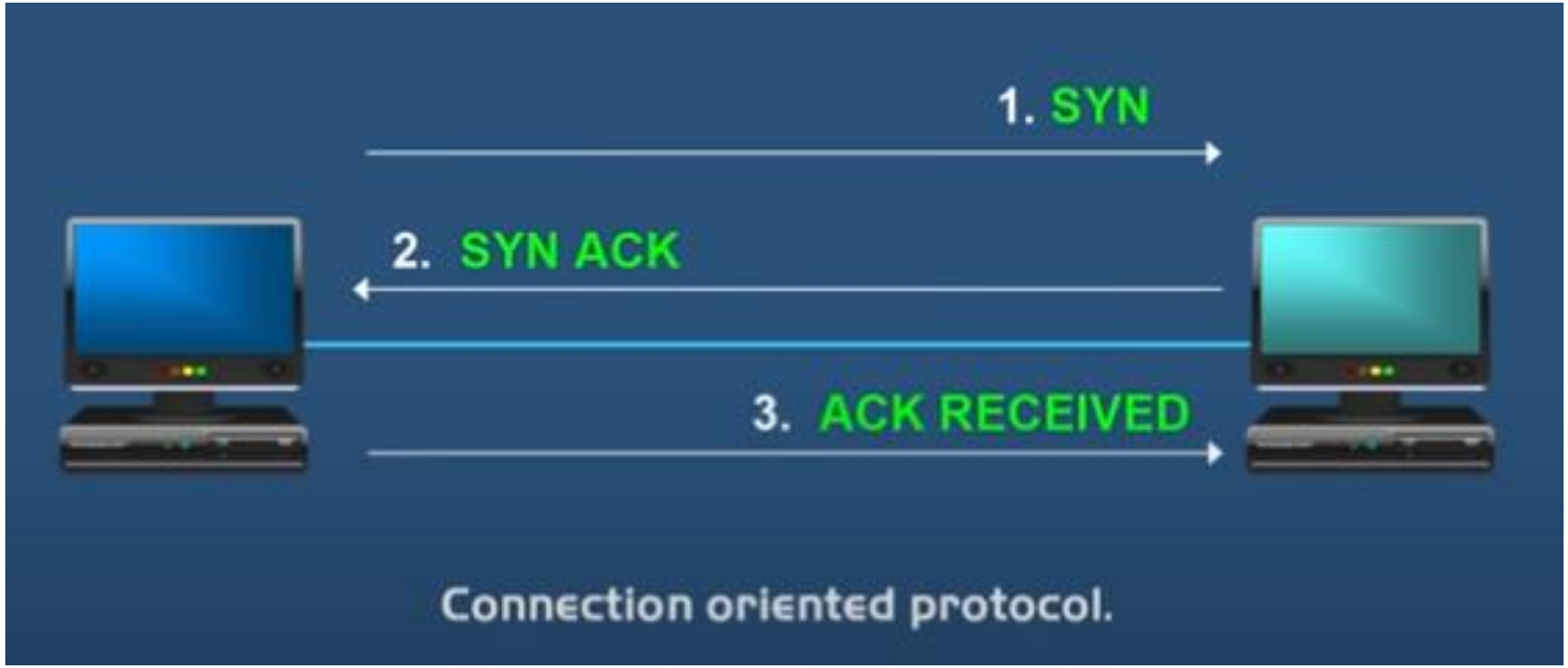
- In computer networking, the transport layer is a conceptual division of methods in the layered architecture of protocols in the network stack in the Internet protocol suite and the OSI model.
- The protocols of this layer provide host-to-host communication services for applications. It provides services such as connection-oriented communication, reliability, flow control, and multiplexing.

IoT Protocols- Transport Layer

- The best-known transport protocol of the Internet protocol suite is the Transmission Control Protocol (TCP).
- It is used for connection-oriented transmissions, whereas the connectionless User Datagram Protocol (UDP) is used for simpler messaging transmissions.
- Provide end-to-end message transfer capability independent of the underlying network
- It provides functions such as error control, segmentation, flow-control and congestion control

IoT Protocols- Transport Layer

Transmission Control Protocol (TCP):



IoT Protocols- Transport Layer

Transmission Control Protocol (TCP):



IoT Protocols- Transport Layer

Transmission Control Protocol (TCP):

- Connection Oriented
- Ensures Reliable transmission
- Provides Error Detection Capability to ensure no delicacy of packets and retransmit lost packets
- Flow Control capability to ensure the sending data rate is not too high for the receiver process
- Congestion control capability helps in avoiding congestion which leads to degradation of n/w performance

IoT Protocols- Transport Layer

User Datagram Protocol (UDP):



IoT Protocols- Transport Layer

User Datagram Protocol (UDP):

- Connectionless
- Does not ensures Reliable transmission
- Does not do connection before transmitting
- Does not provide proper ordering of messages
- Transaction oriented and stateless

IoT Protocols- Application Layer

- An application layer is an abstraction layer that specifies the shared communications protocols and interface methods used by hosts in a communications network.
- The application layer abstraction is used in both of the standard models of computer networking: the Internet Protocol Suite (TCP/IP) and the OSI model.
- Although both models use the same term for their respective highest level layer, the detailed definitions and purposes are different.

IoT Protocols- Application Layer

IoT-Specific Protocols

- **CoAP (Constrained Application Protocol)** 🔋 **:** Used for **low-power, resource-constrained IoT devices** that need to communicate with a web service.
- **MQTT (Message Queuing Telemetry Transport)** 🚀 **:** The most popular protocol for **machine-to-machine (M2M) communication** in IoT. It's a lightweight, publish-subscribe protocol ideal for high-latency, low-bandwidth networks.
- **DDS (Data Distribution Service)** 🚙 **:** Used for **real-time, mission-critical IoT systems** that require high performance and reliability.

IoT Protocols- Application Layer

General-Purpose Protocols Used in IoT

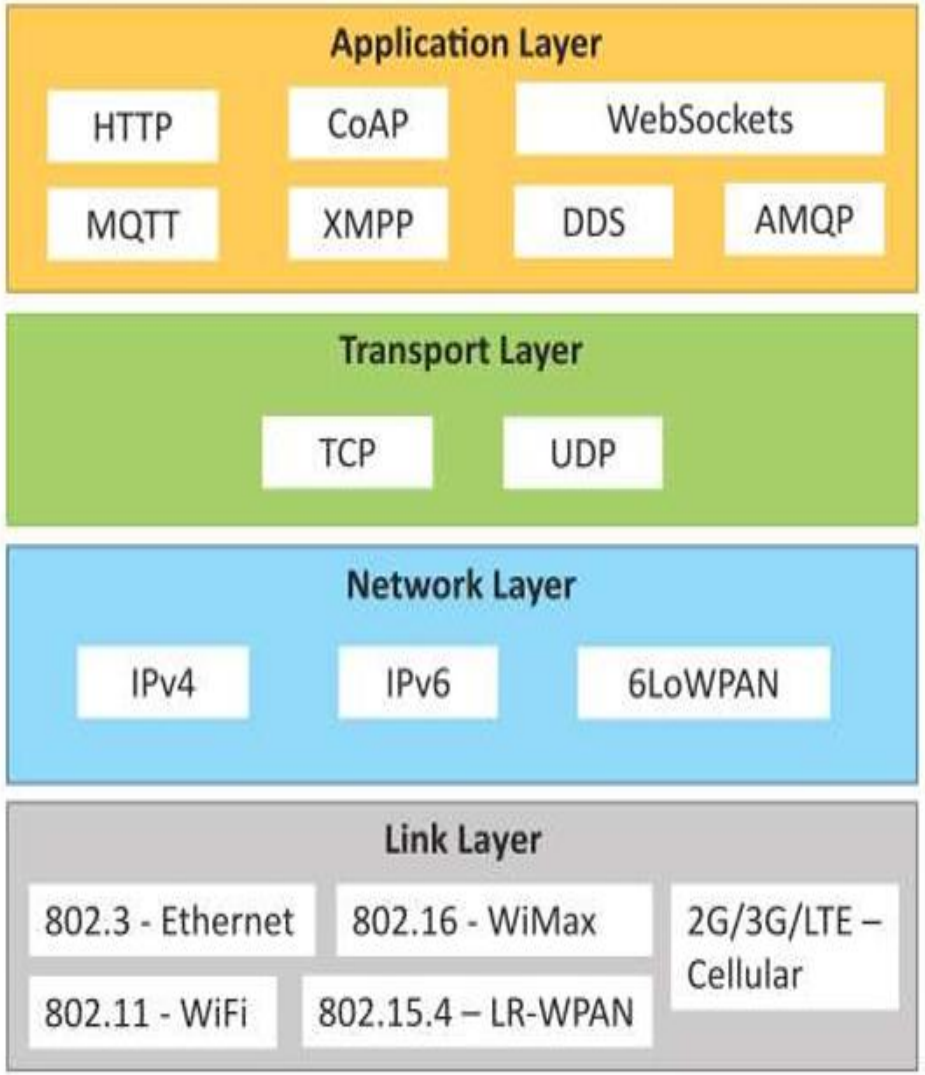
- **HTTP (Hypertext Transfer Protocol)** 🌐 **:** Used for **device configuration, management, and API calls** from a server to a device.
- **WebSocket** ⚡ **:** Used for **real-time, low-latency communication between a client (like a web dashboard) and an IoT device or gateway**.
- **AMQP (Advanced Message Queuing Protocol)** 📦 **:** Used in **complex, enterprise-level IoT solutions** that require advanced queuing, routing, and message delivery guarantees.

IoT Protocols- Application Layer

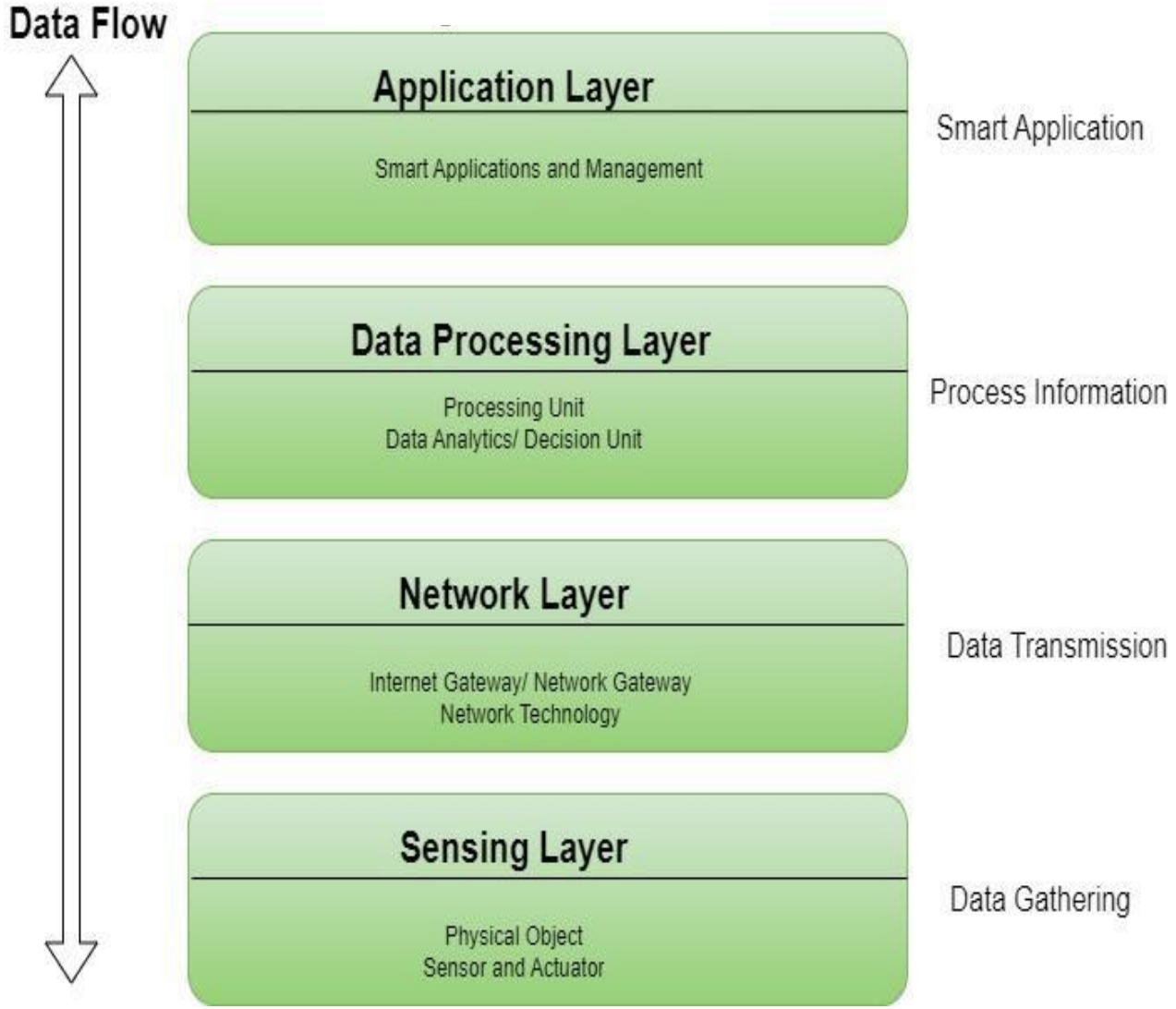
Less Common Protocols in IoT

- **XMPP (Extensible Messaging and Presence Protocol)**  **:** While it can be used for IoT, its XML-based structure and higher overhead make it less efficient than protocols like MQTT or CoAP for constrained devices.
- It's primarily used for instant messaging, not for a wide range of IoT applications.

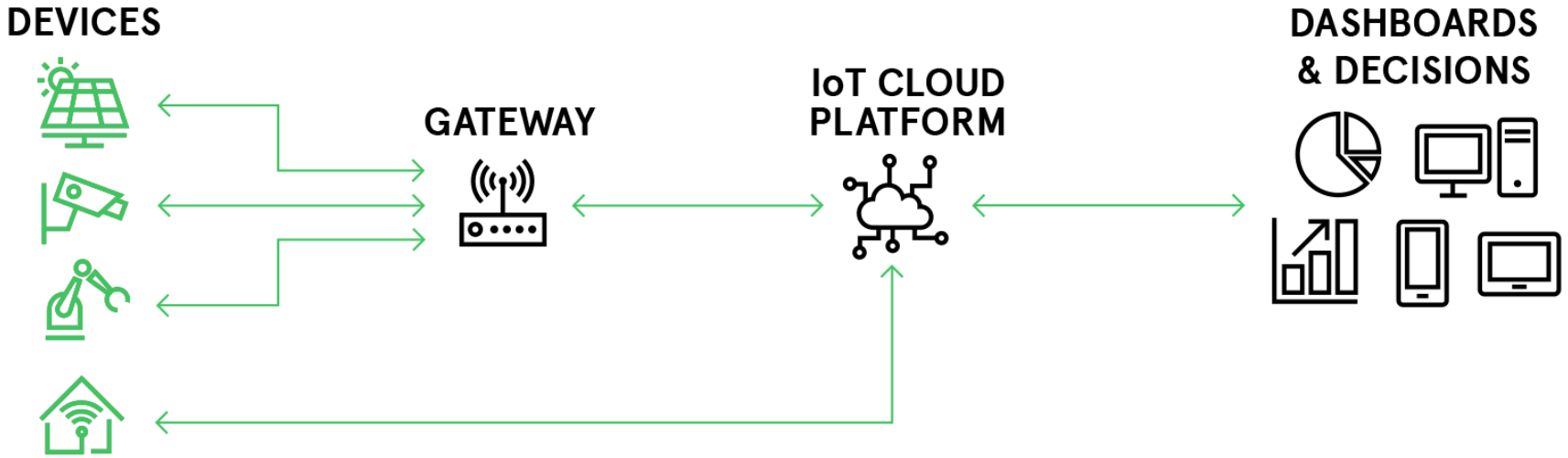
IoT Protocols

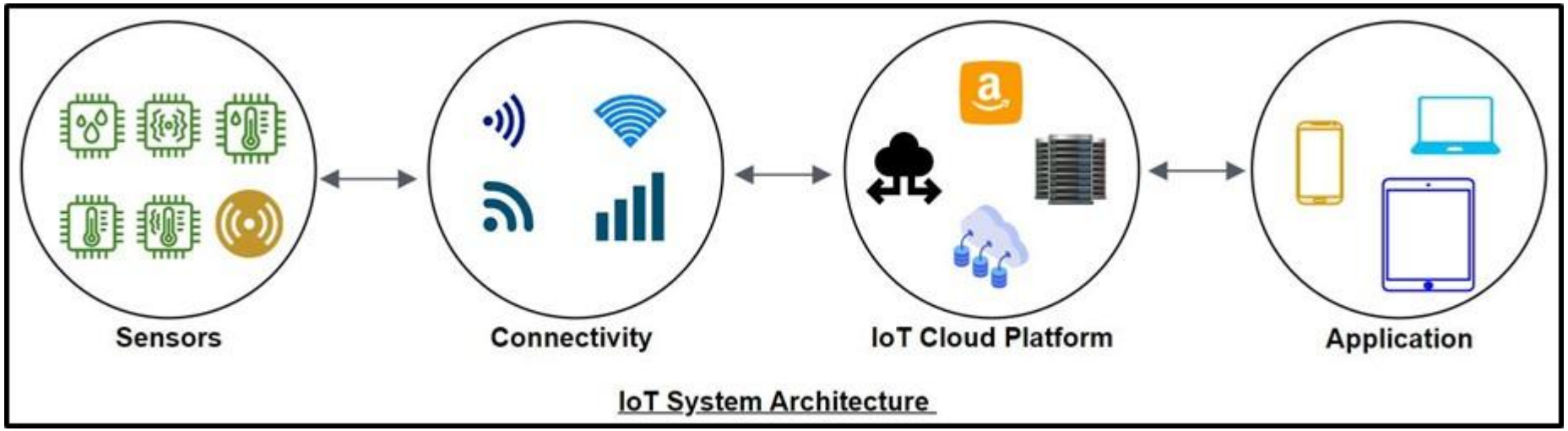


IoT Stack



IoT Stack/ IoT System Architecture





IoT Stack-Sensing Layer

1. The Things Layer (Perception/Sensing Layer)

- This is the lowest layer and the foundation of IoT.
- It comprises the **physical devices** and the components that interact with the real world.

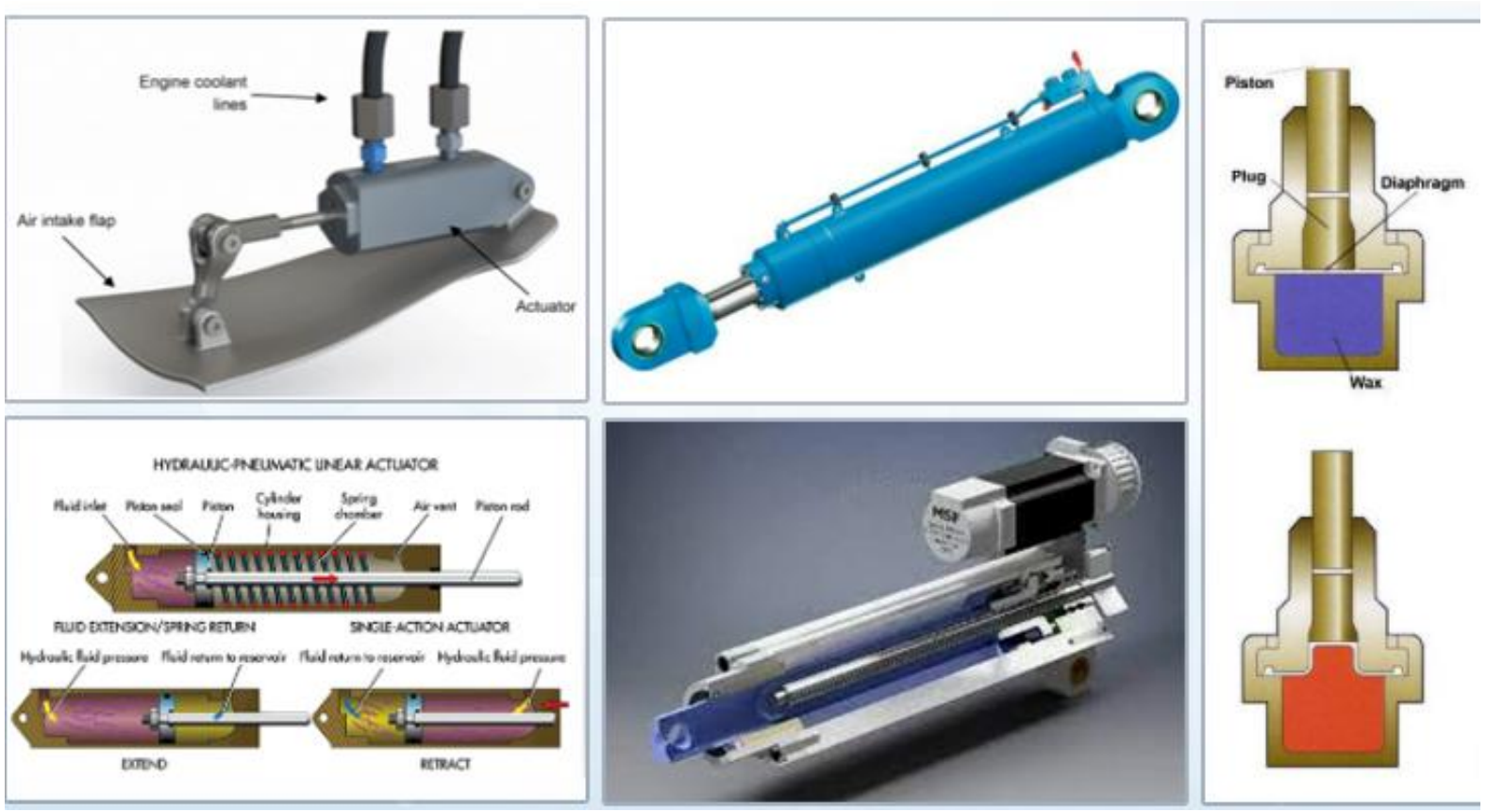
•Sensors: Collect data from the environment (e.g., temperature, humidity, motion).



IoT Stack-Sensing Layer

1. The Things Layer (Perception/Sensing Layer)

•**Actuators:** Perform physical actions based on instructions (e.g., turning a light on/off, adjusting a thermostat).

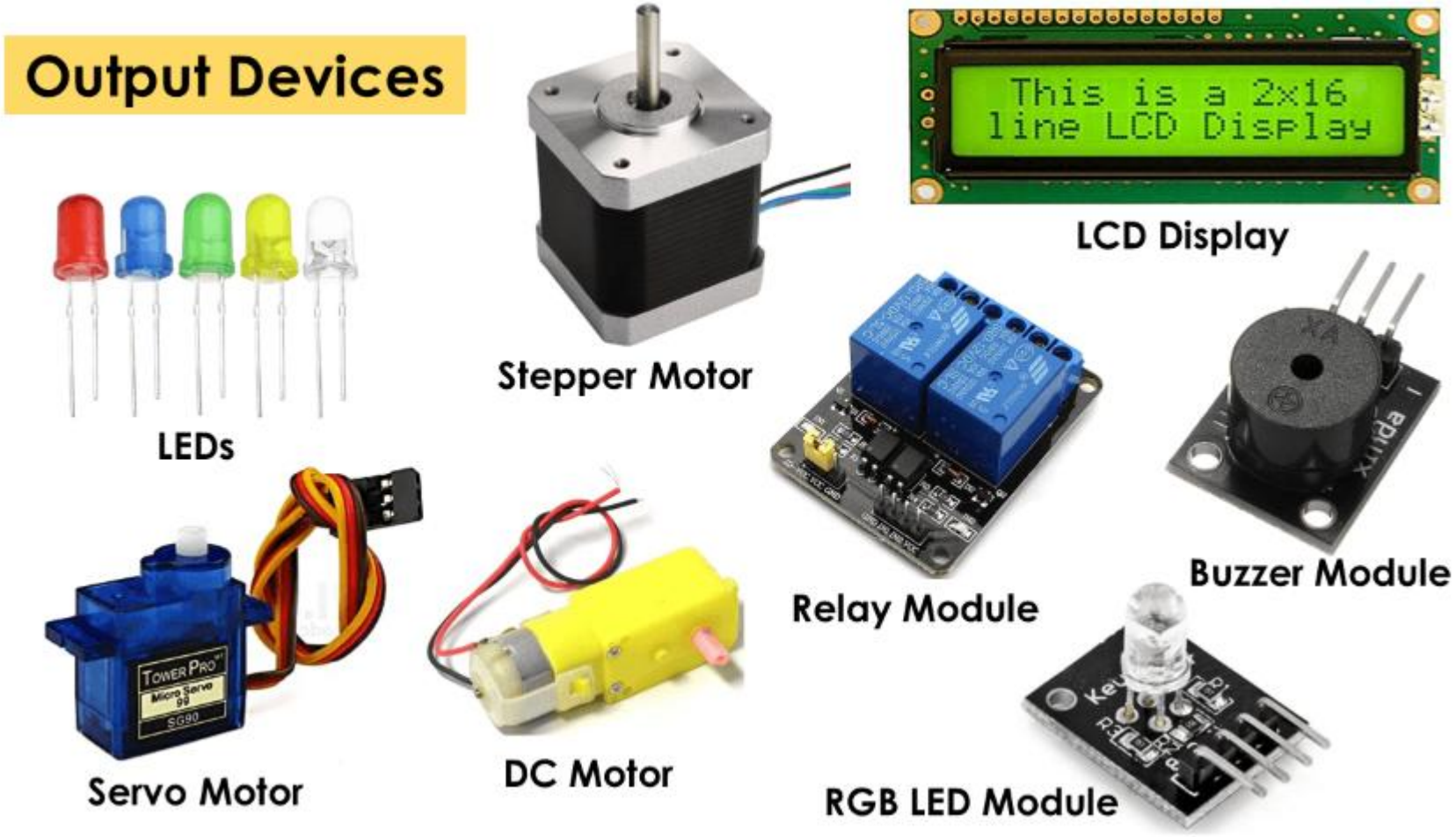


Actuators

IoT Stack-Sensing Layer

1. The Things Layer (Perception/Sensing Layer)

•**Actuators:** Perform physical actions based on instructions (e.g., turning a light on/off, adjusting a thermostat).



IoT Stack-Sensing Layer

1. The Things Layer (Perception/Sensing Layer)

•**Microcontroller/Microprocessor:** The "brain" of the device that processes sensor data and controls actuators.



Arduino Nano 33



Arduino Nano Every



Arduino Nano 33 Ble



Arduino UNO



Teensy 4.0



Tessel 2



Onion Omega2s+



ESP32



Particle Argon

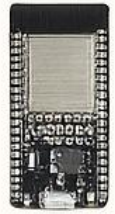
IoT Stack-Sensing Layer

1. The Things Layer (Perception/Sensing Layer)

- **Microcontroller/Microprocessor:**


Raspberry Pi 4 Model B

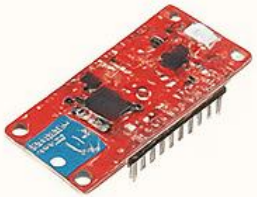

Arduino


ESP32 Devkit


Raspberry Pi Pico


Arduino Nano


BeagleBone Black


SparkFun Thing Plus


Arduino MKR WiFi 1010


Particle Photon


STM32 Nucleo


TTGO T-Display


BBC micro:bit


Onion Omega2+


Wio Terminal


Seeed Studio XIAO

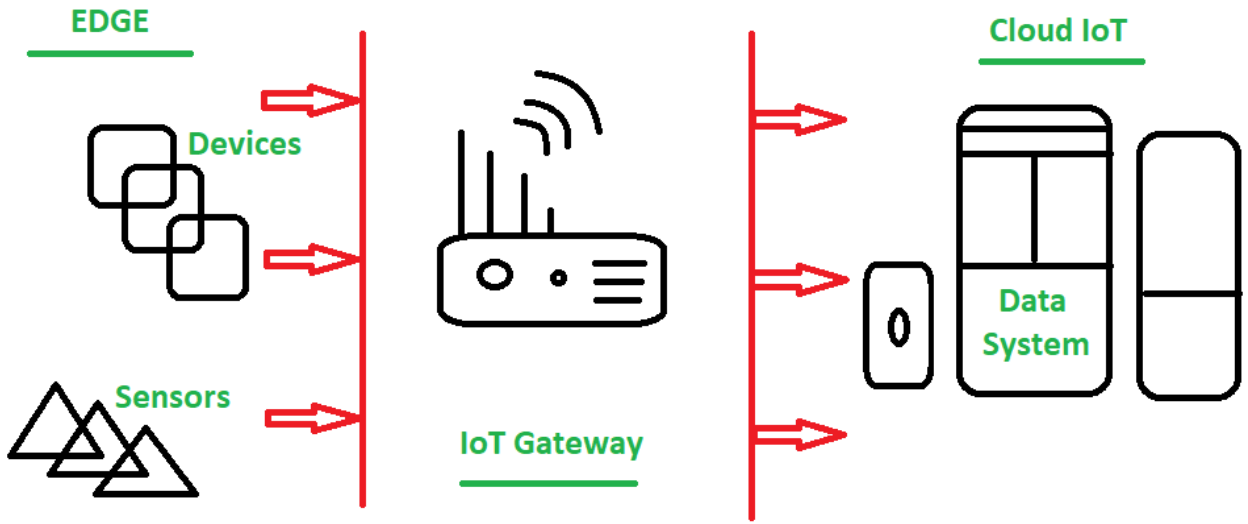

M5Stack Core2

IoT Stack-Connectivity Layer

2. The Network Layer (Connectivity Layer)

This layer is responsible for the **communication and connectivity** of the devices. It allows the "things" to connect and communicate with each other and with the broader internet.

•**Gateways:** Devices that bridge the gap between the local network of IoT devices and the internet. They often translate protocols and provide security.

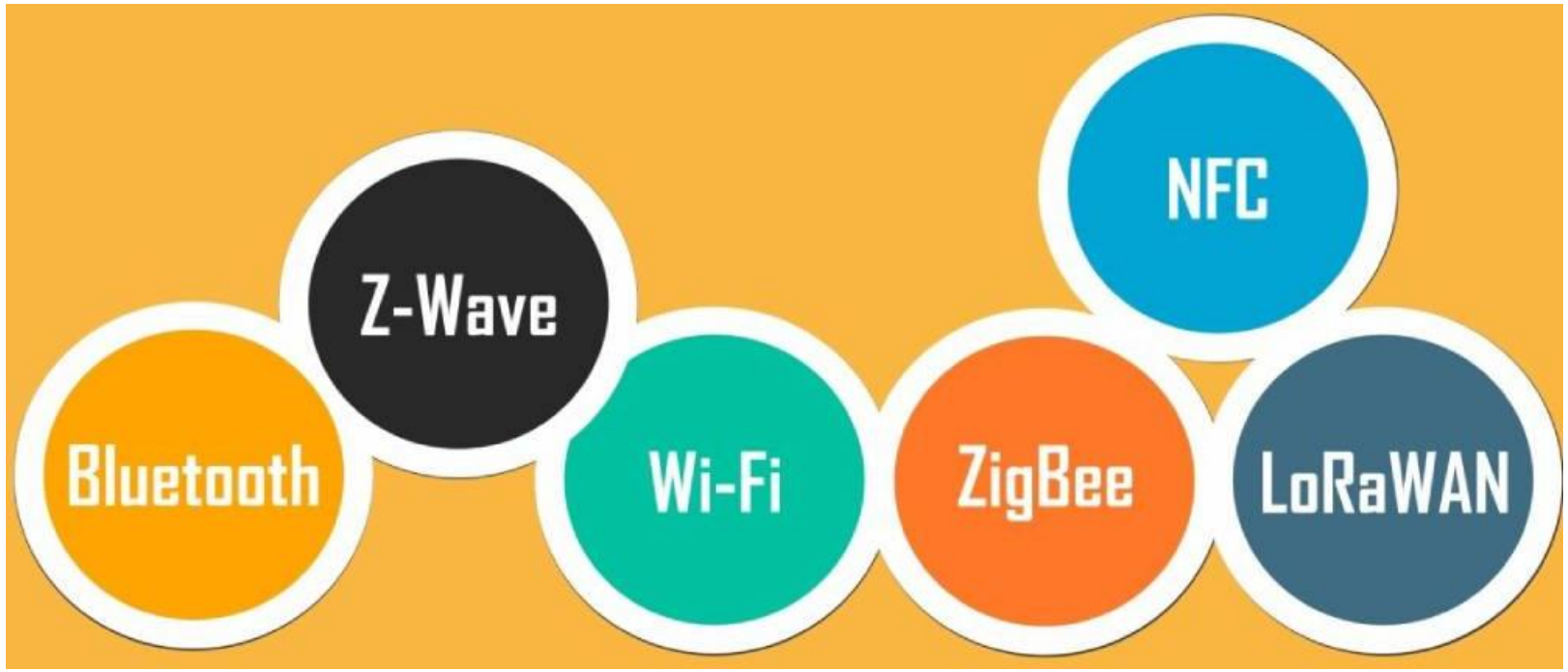


IoT Stack-Connectivity Layer

2. The Network Layer (Connectivity Layer)

•**Communication Protocols:** The rules for data transfer, such as Wi-Fi, Bluetooth, Zigbee, or cellular networks (4G/5G).

•**Cloud/Servers:** The infrastructure that provides the computing power and storage to handle the massive amounts of data generated by IoT devices.



IoT Stack-Data Processing Layer

3. The Data Processing Layer

Once the data from the devices reaches the cloud or a central server, this layer processes it. This is where raw data is transformed into useful information.

- **Data Aggregation and Filtering:** Cleaning and organizing the data to remove noise and redundancy.
- **Data Analytics:** Using algorithms to analyze the data, identify patterns, and generate insights. This can range from simple data analysis to complex machine learning.
- **Databases:** Storing the processed data for future use and analysis.

IoT Stack-Application Layer

4. The Application Layer:

This is the top layer and the one that the **end-user interacts with**. It provides the user interface and functionality to manage the IoT system.

- **User Interfaces:** This includes mobile apps, web portals, and dashboards that allow users to monitor devices, view data, and send commands.
- **Services:** The specific applications or services that use the processed data, such as a smart home automation system, a health monitoring app, or an industrial control system.

Design of IoT

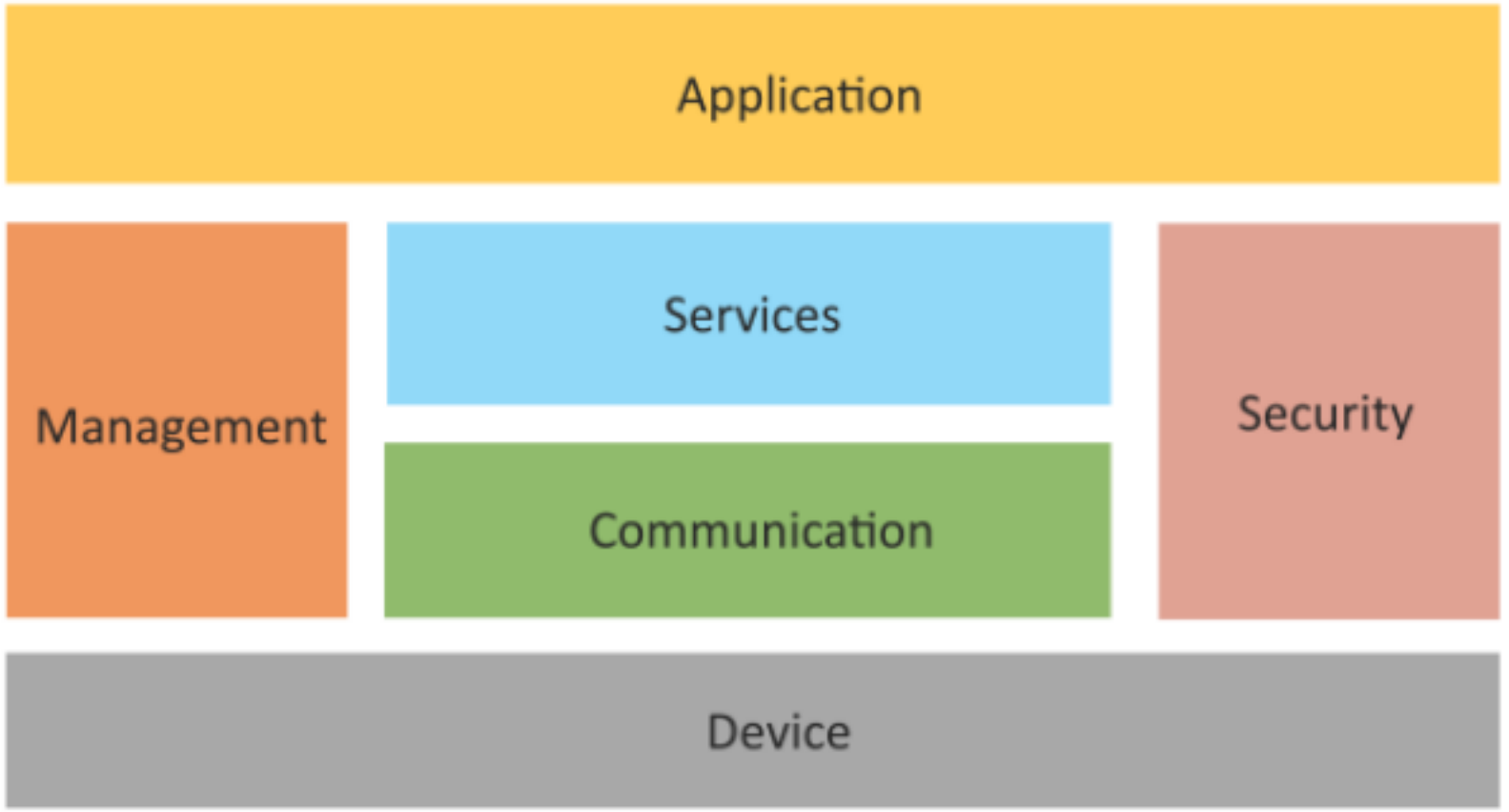
- Physical Design of IoT
  - Things in IoT
    - Generic Block Diagram of an IoT Device
  - IoT protocols

- Logical Design of IoT
  - Functional blocks
  - Communication models
  - Communication APIs

Logical Design of IoT

- The **logical design of IoT** refers to the abstract, conceptual structure that defines how the Internet of Things (IoT) works, including the flow of data, communication protocols, network topology, and interactions between various components.
- Unlike the physical design, the logical design focuses on how data is processed and transferred between different layers of the system.

Logical Design of IoT-Functional Block Diagram of IoT



Logical Design of IoT -Functional blocks

•Device : Devices such as sensing, actuation, monitoring and control functions.

- •This block represents the **physical layer** of the IoT ecosystem.
- •The devices are the point of interaction with the physical world.

Communication : IoT Protocols, This block covers the entire **connectivity layer** that enables data transfer. It includes:

- **Networking technologies** (e.g., Wi-Fi, Bluetooth, cellular) that connect the devices.
- **Protocols** (e.g., MQTT, CoAP) that format and transmit the data efficiently.
- **Gateways** that act as an intermediary, translating between different device-level and network-level protocols.

Logical Design of IoT -Functional blocks

•Services like device monitoring, device control services, data publishing services and device discovery.

This block corresponds to the **cloud and data processing layer**. It includes the core backend functionalities, such as:

- **Data storage** for collecting and archiving information from devices.
- **Analytics engines** that process and derive insights from the raw data.
- **Applications** that provide the logic and business rules for the IoT solution.

Logical Design of IoT -Functional blocks

•Management : Functions to govern the system

•Security : Functions as authentication, authorization, message and content integrity, and data security Applications.

•**Management** involves tasks like device provisioning, monitoring, and updating software on the devices (firmware over-the-air or FOTA).

•**Security** is crucial at every level—from securing the devices and communication links to protecting data in the cloud. It involves authentication, authorization, and data encryption to prevent unauthorized access.

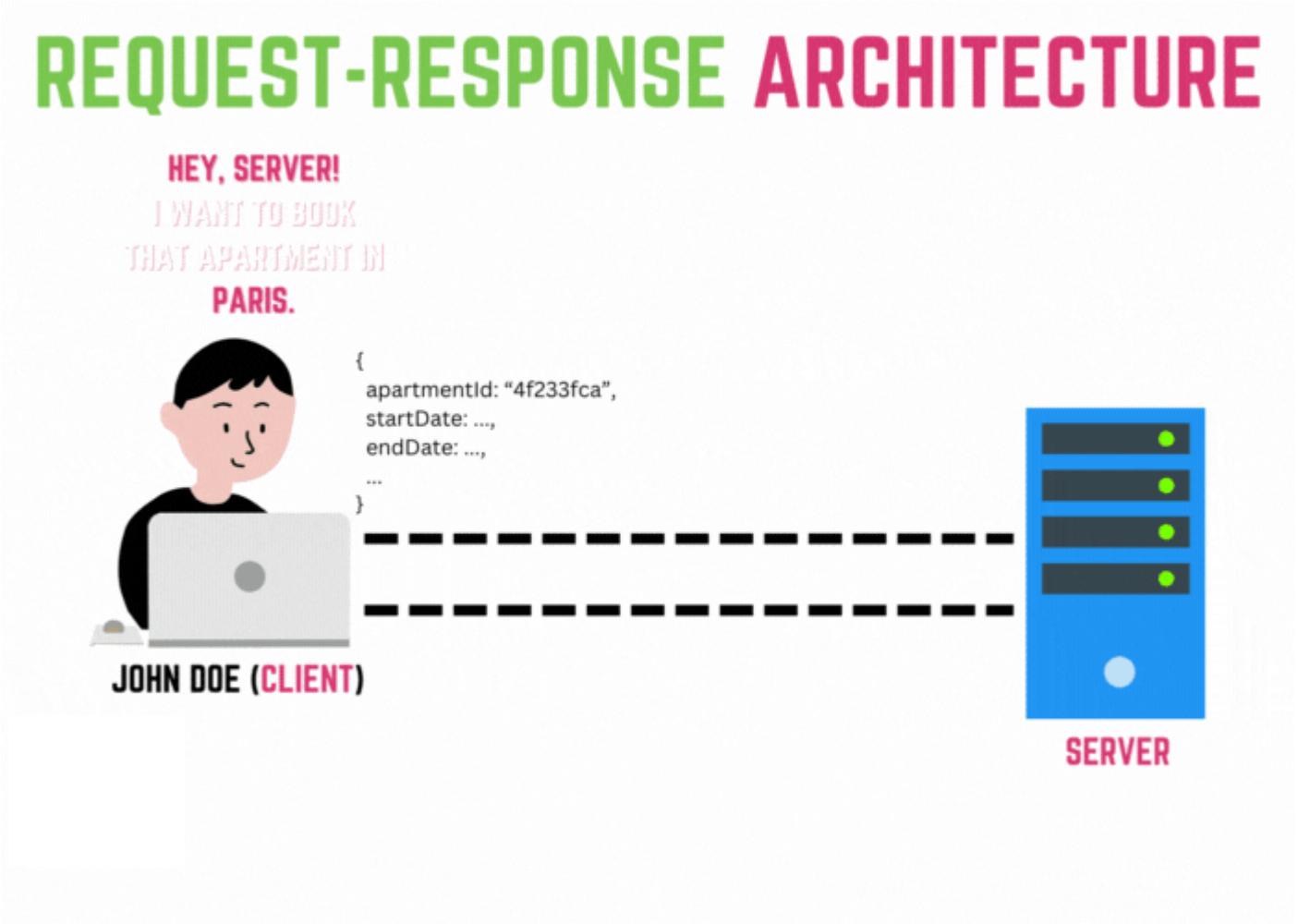
Logical Design of IoT

IoT Communication Models

- Request–Response Communication Model
- Publish–Subscribe Communication Model
- Push–Pull Communication Model
- Exclusive Pair Communication Model

Logical Design of IoT- Communication models

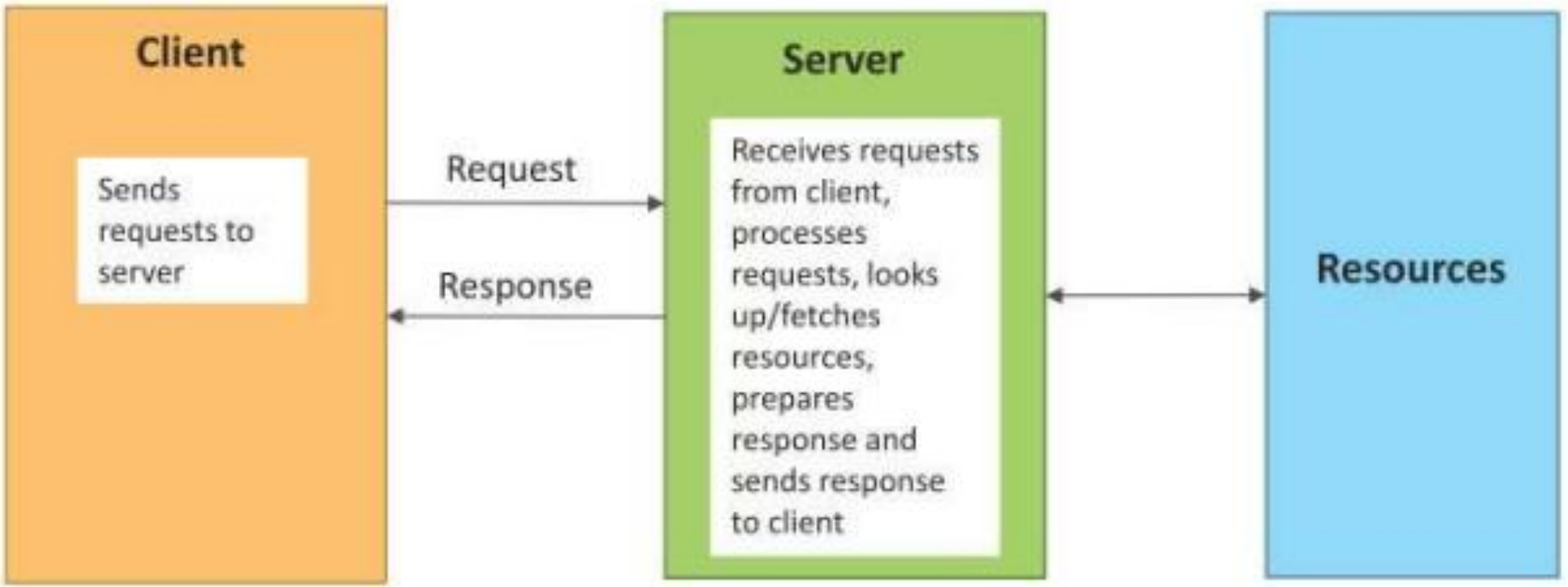
1. Request–Response Communication Model





Logical Design of IoT- Communication models

1. Request–Response Communication Model



Logical Design of IoT- Communication models

1. Request–Response Communication Model

- Request–Response is a communication model in which the client sends requests to the server and the server responds to the requests.
- When the server receives a request, it decides how to respond, fetches the data, retrieves resource representations, prepares the response and then sends the response to the client.

Logical Design of IoT- Communication models

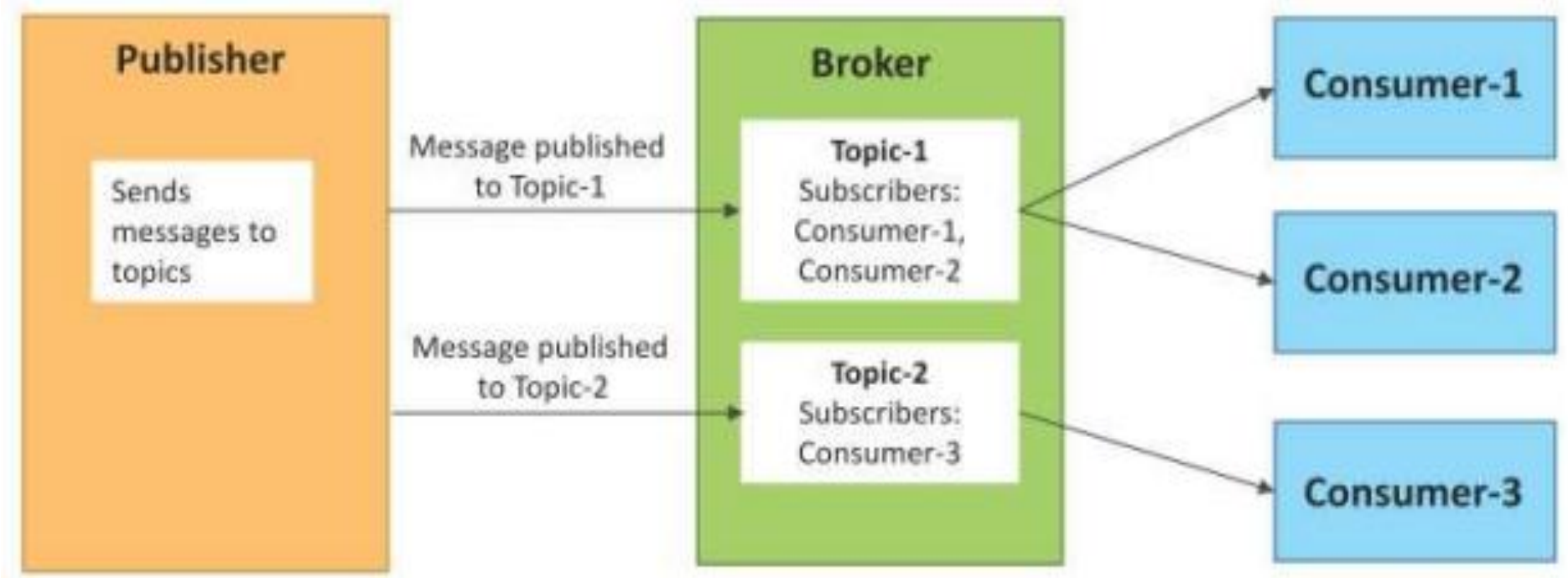
1. Request–Response Communication Model

- **Synchronous:** The client's process is paused, waiting for the server's response.
- **Stateless (typically):** Each request is treated as a new, independent transaction. The server doesn't inherently remember past interactions with the same client.
- **Asymmetric:** The client initiates the communication, and the server's role is to respond.

- Ex: Web browsing

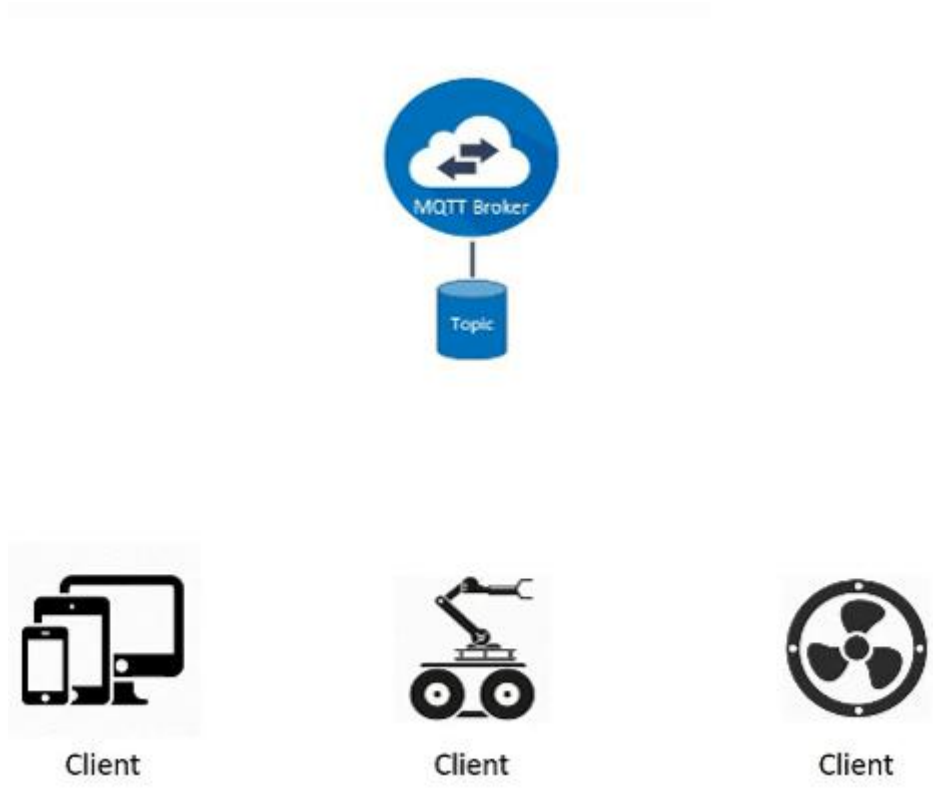
Logical Design of IoT- Communication models

•Publish–Subscribe Communication Model



Logical Design of IoT- Communication models

•Publish–Subscribe Communication Model



Logical Design of IoT- Communication models

- Publish–Subscribe Communication Model

- **Publisher:** An entity that creates and sends messages to a specific topic without needing to know who will receive them.
- **Subscriber:** An entity that registers its interest in a topic and receives all messages published to that topic.
- **Broker/Message Bus:** An intermediary service that manages the topics. It receives messages from publishers and forwards them to all relevant subscribers. The broker decouples publishers and subscribers from each other.

Logical Design of IoT- Communication models

•Publish–Subscribe Communication Model

•**Decoupling:** Publishers and subscribers are completely independent. They don't need to know about each other's existence, location, or how many there are. This makes the system very flexible and scalable.

•**Asynchronous:** The publisher sends a message and continues its work without waiting for a response from the subscribers. The broker handles message delivery.

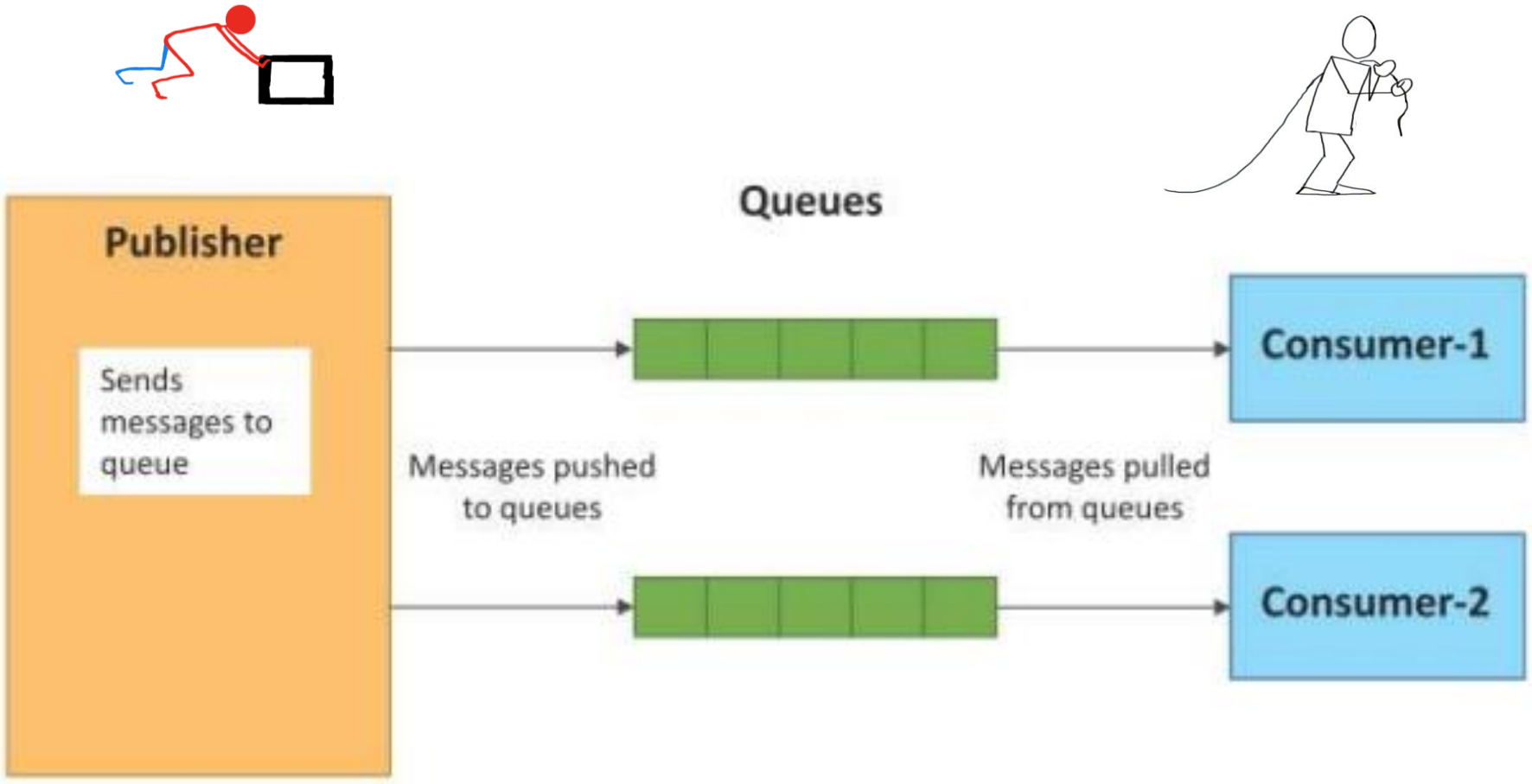
•**Scalability:** You can easily add new publishers and subscribers to a topic without affecting the rest of the system.

•**Many-to-Many:** Multiple publishers can send messages to the same topic, and multiple subscribers can receive messages from that same topic.

•Ex: Youtube, news paper subscription

Logical Design of IoT- Communication models

- Push–Pull Communication Model



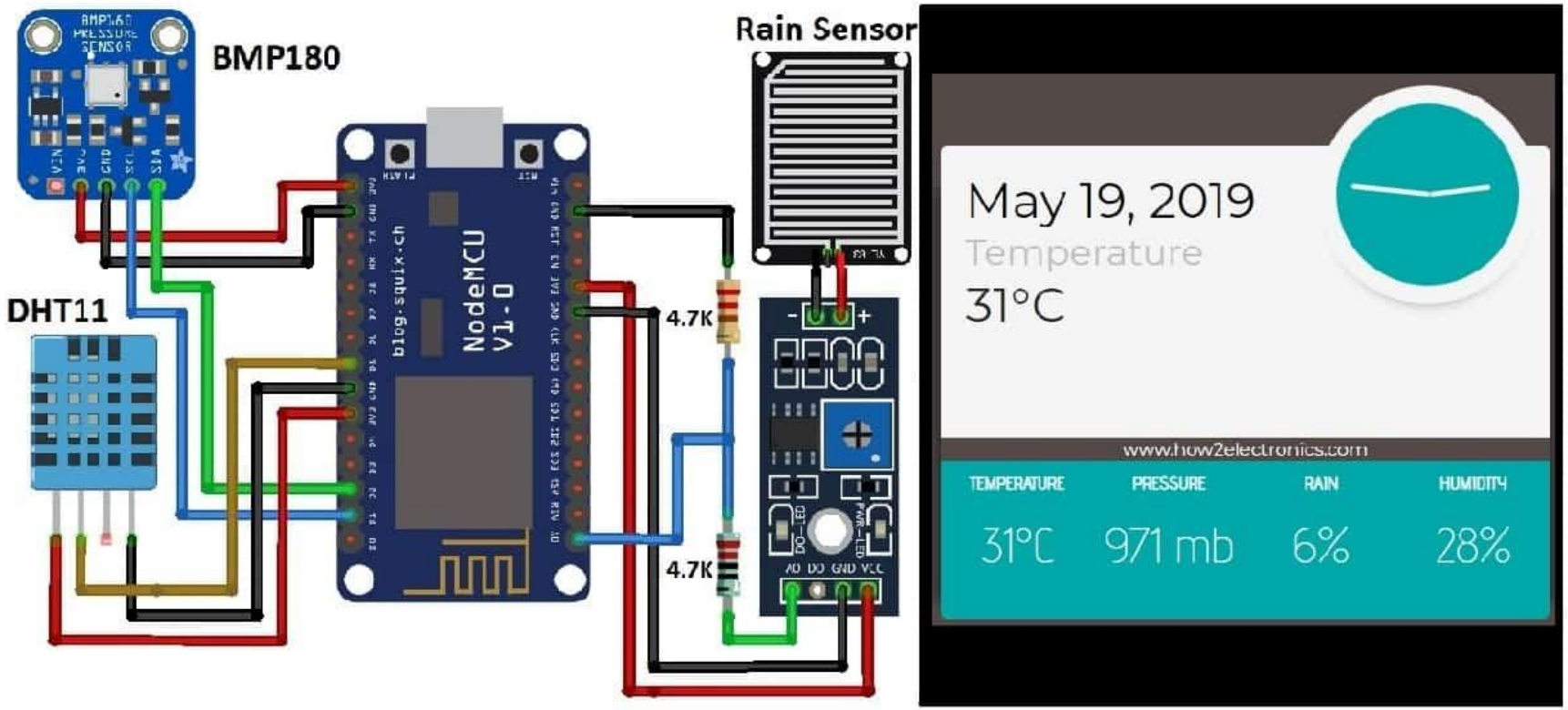
Logical Design of IoT- Communication models

- **Push–Pull Communication Model**
  - **Push:** In this part of the model, the server proactively **pushes** data or notifications to the client without an explicit request.
  - This is useful for real-time events or alerts.
  - For instance, a server might push a notification to a client when a sensor reading exceeds a certain threshold.
  - This is similar to the publish-subscribe model.

  - **Pull:** In this part, the **client** actively **pulls** or requests specific data from the server.
  - This is the traditional request-response model.
  - The client is in control and initiates the conversation to get the information it needs, such as retrieving a full set of historical data.

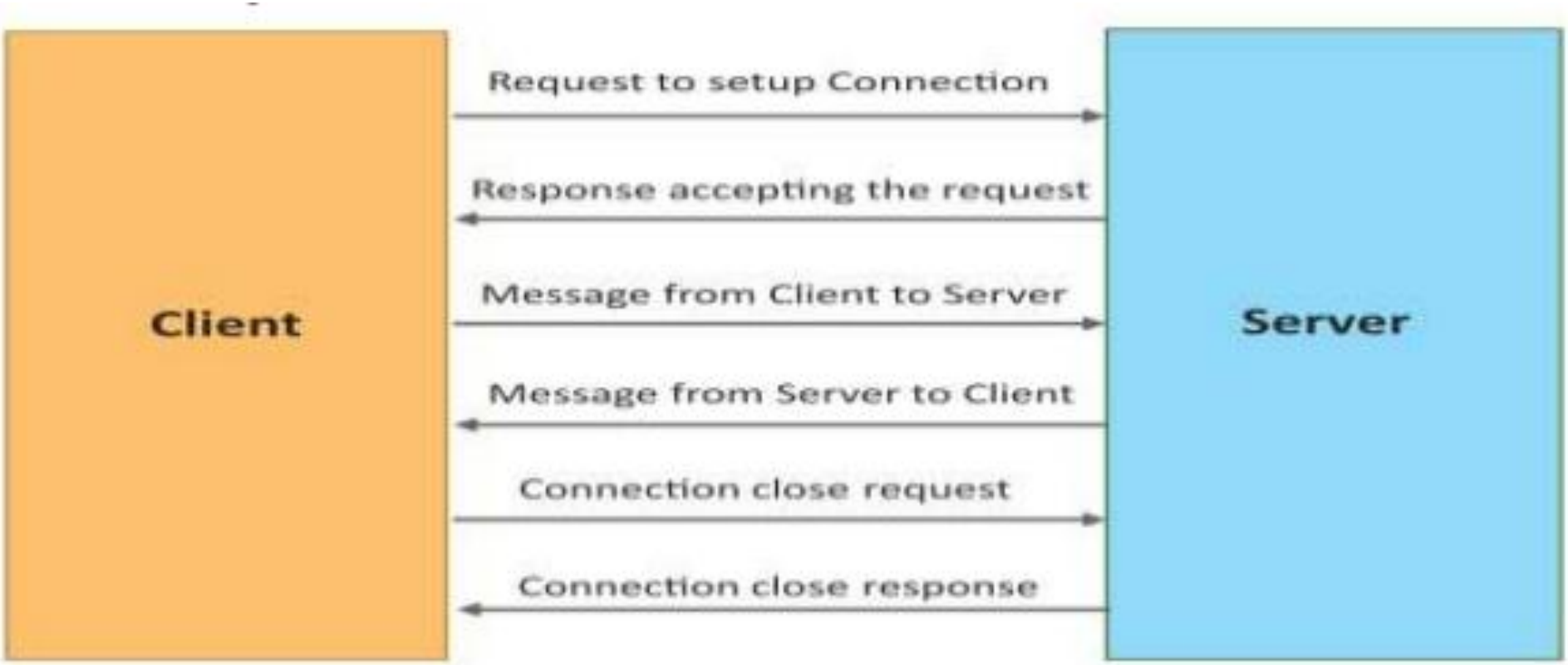
Logical Design of IoT- Communication models

- Push–Pull Communication Model



Logical Design of IoT- Communication models

- Exclusive Pair Communication Model



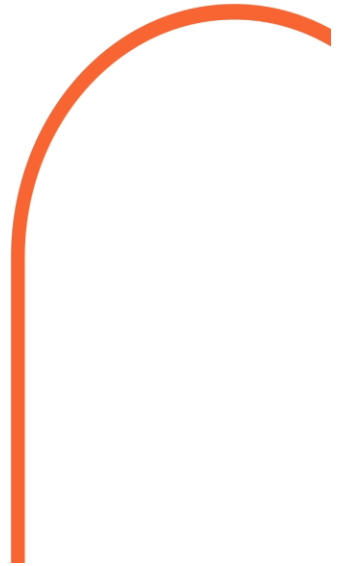
Logical Design of IoT- Communication models

- Exclusive Pair Communication Model









Logical Design of IoT- Communication models

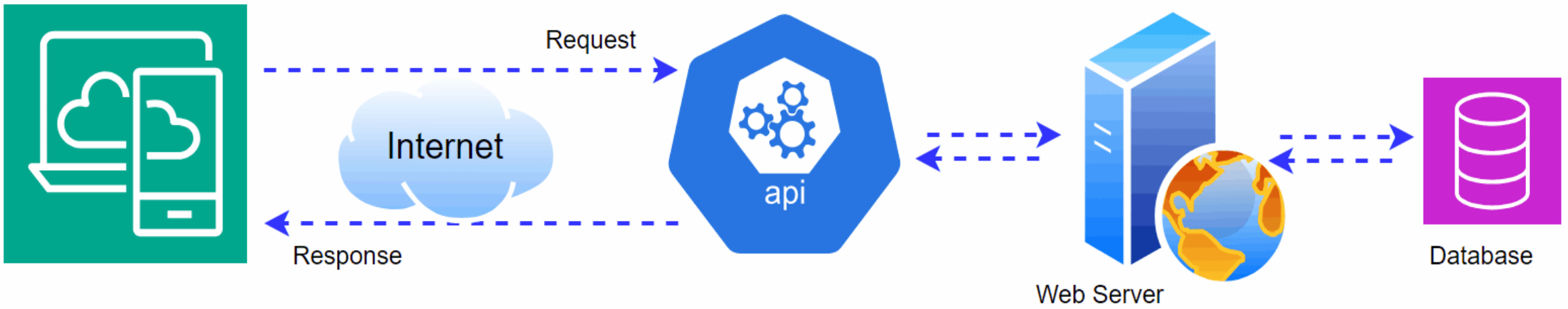
- Exclusive Pair Communication Model

- Exclusive Pair is a bidirectional, fully duplex communication model that uses a persistent connection between the client and the server.
- Once the connection is set up it, remains open until the client sends a request to close the connection.
- Client and server can send messages to each other after connection setup.
- Ex: ChatGPT, Bluetooth

Logical Design of IoT- Communication APIs

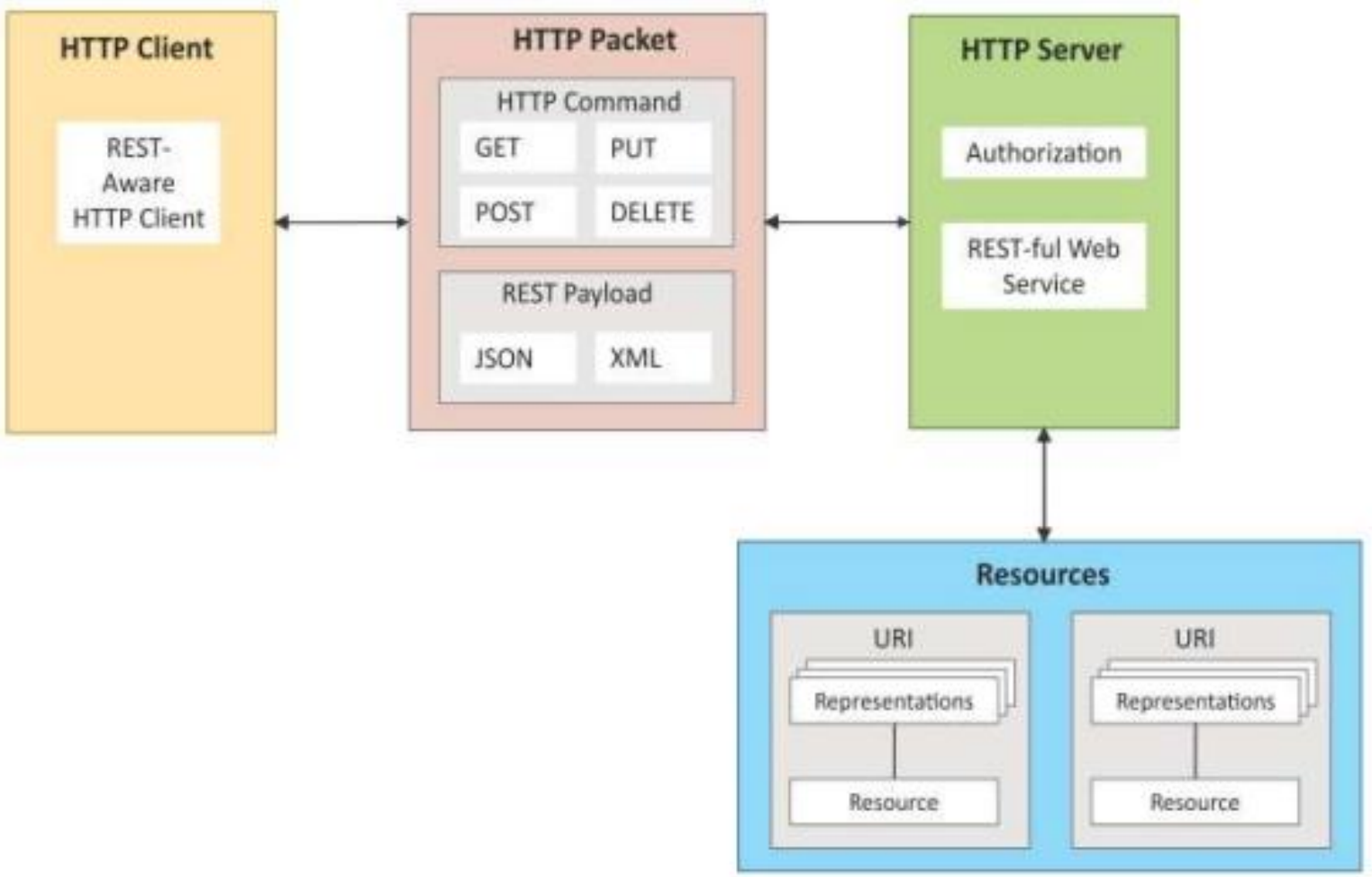
1. REST-based Communication APIs
2. Web Socket-based Communication APIs

Logical Design of IoT- Communication APIs



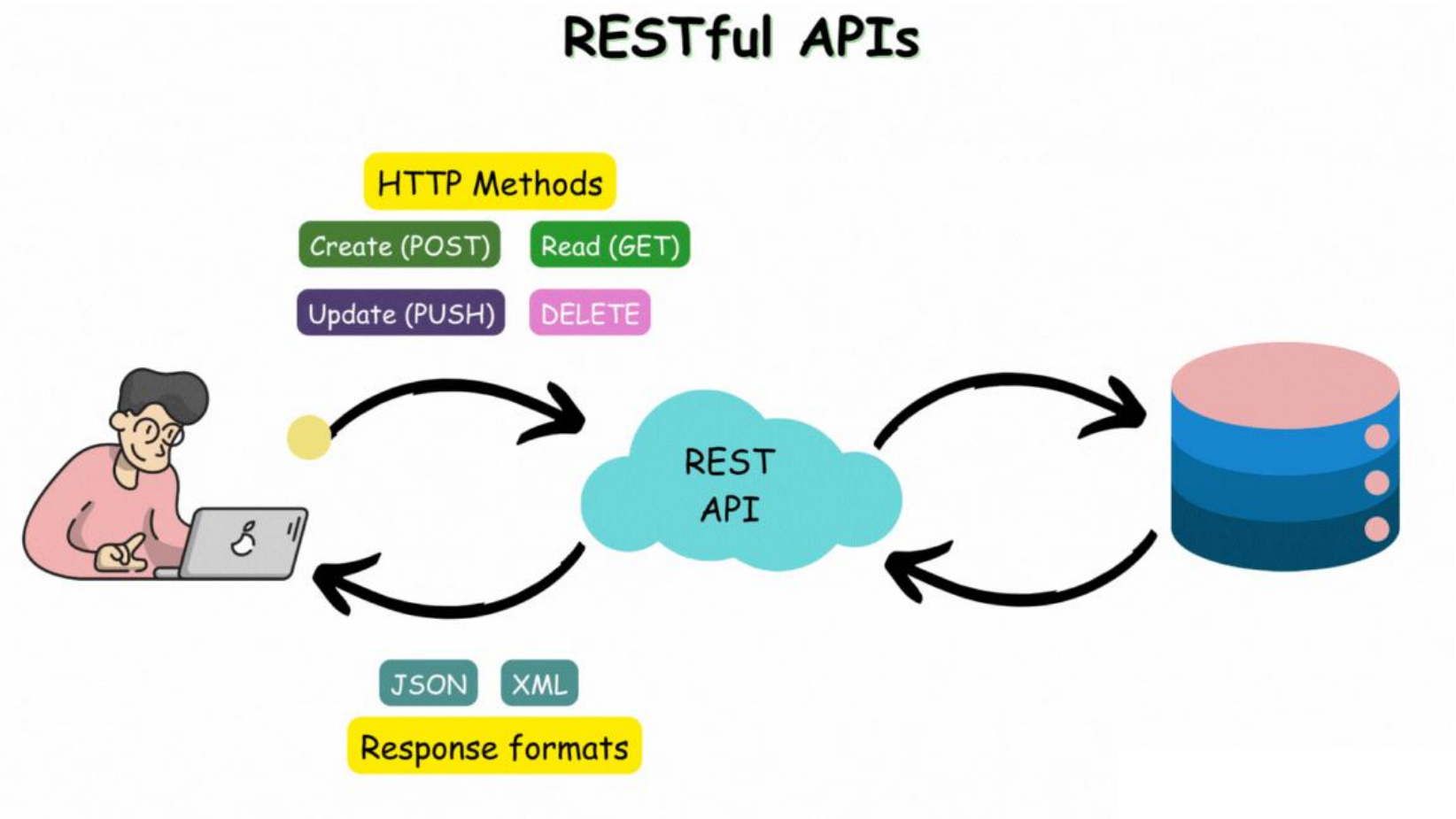
Logical Design of IoT- Communication APIs

- REST-based Communication APIs



Logical Design of IoT- Communication APIs

- REST-based Communication APIs



Logical Design of IoT- Communication APIs

- REST-based Communication APIs

•Representational State Transfer (REST) is a set of architectural principles by which you can design web services and web APIs that focus on a system's resources and how resource states are addressed and transferred.

•REST APIs follow the request–response communication model.

•REST architectural constraints apply to the components, connectors and data elements within a distributed hypermedia system

Logical Design of IoT- Communication APIs

- REST-based Communication APIs

Constraints:

•Client – Server

•Stateless

•Cacheable

•Layered System

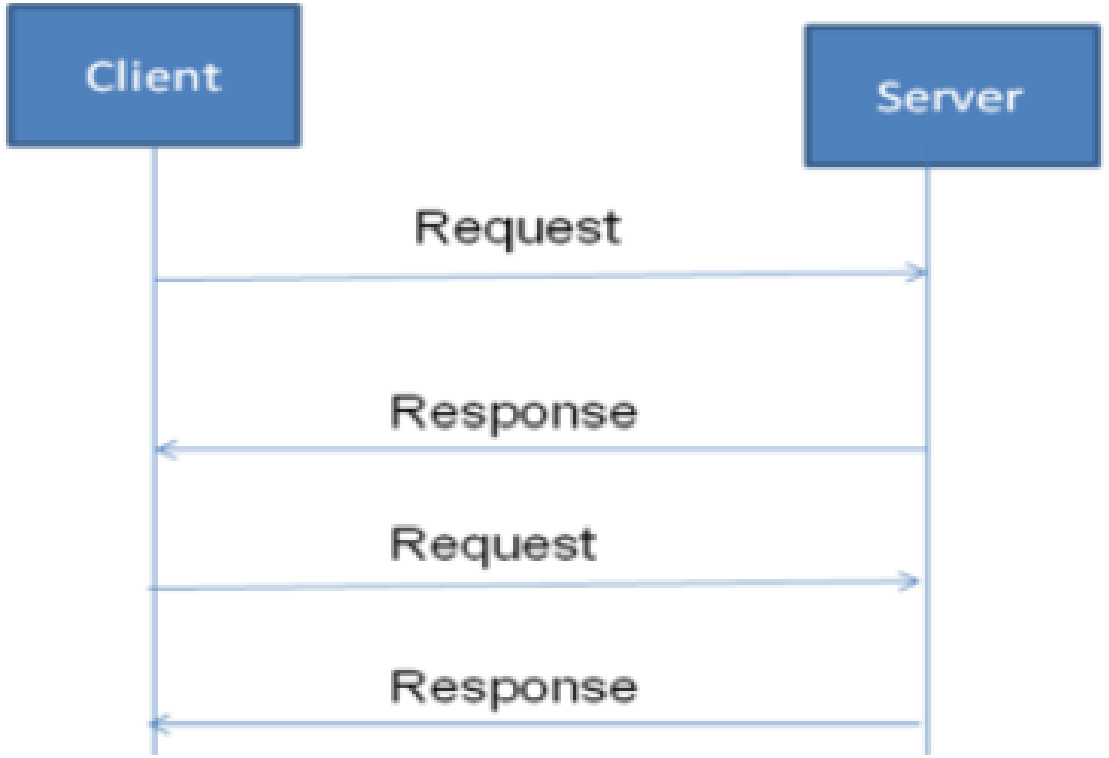
•Uniform Interface

•Code on demand

Logical Design of IoT- Communication APIs

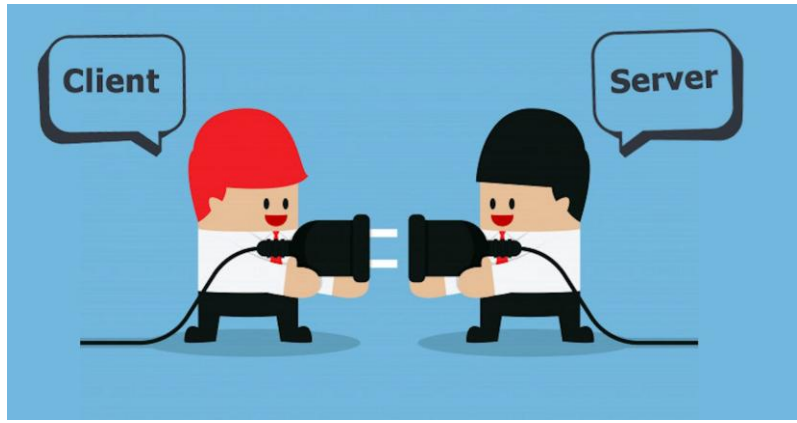
- REST-based Communication APIs

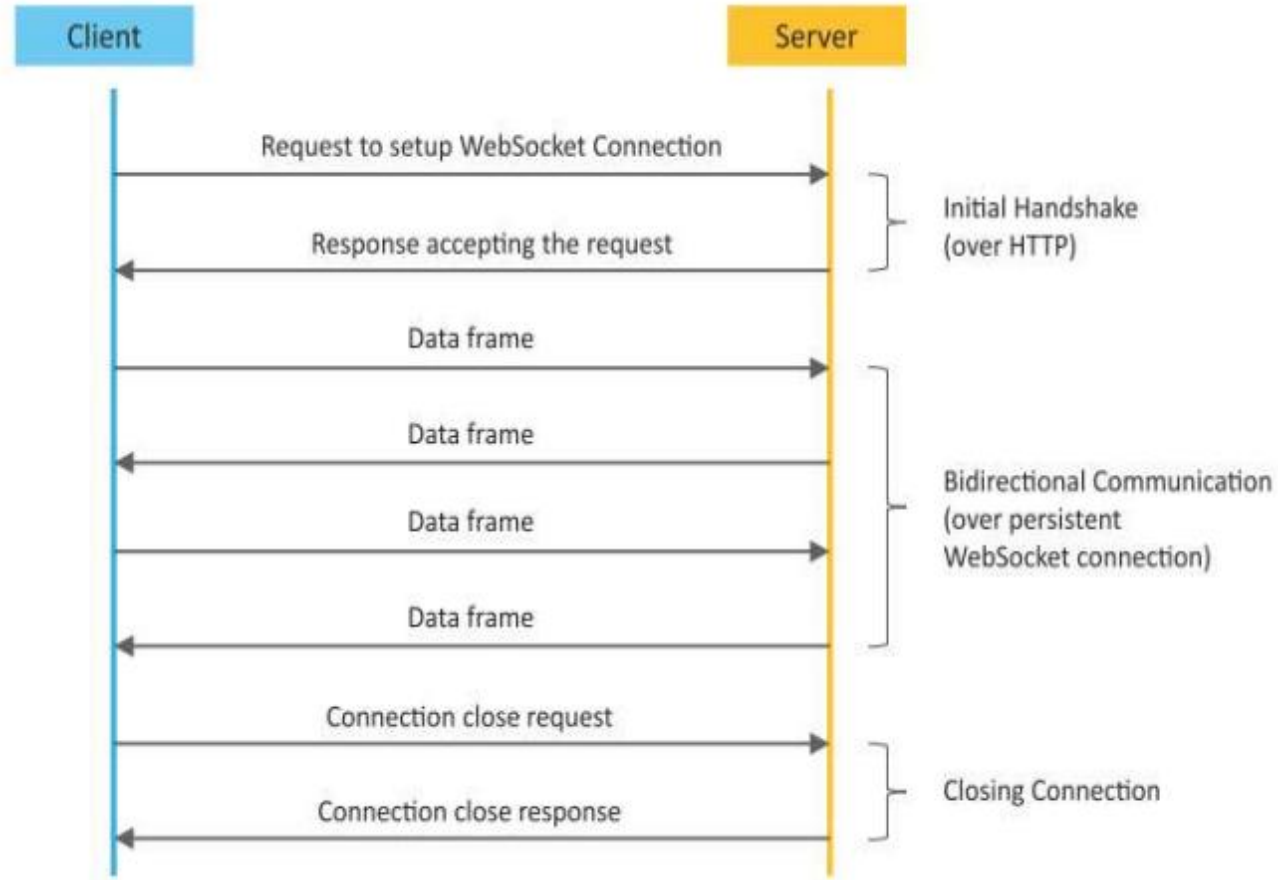
Constraints:



Logical Design of IoT- Communication APIs

- Web Socket-based Communication APIs





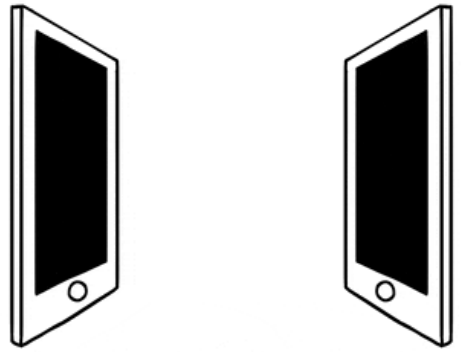
Logical Design of IoT- Communication APIs

- Web Socket-based Communication APIs



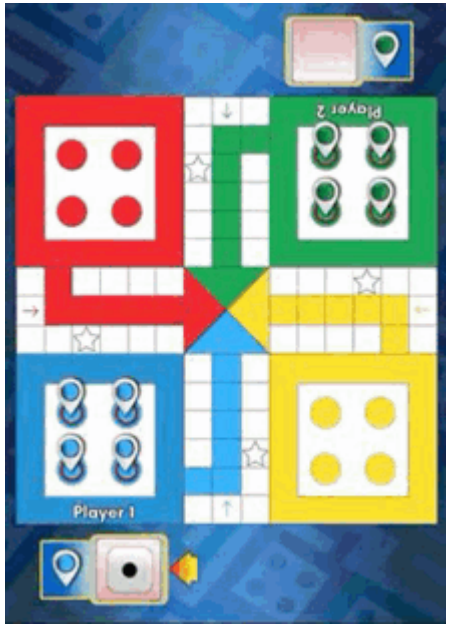
•WebSocket APIs allow bi-directional, full duplex communication between clients and servers.

•WebSocket APIs follow the exclusive pair communication model.

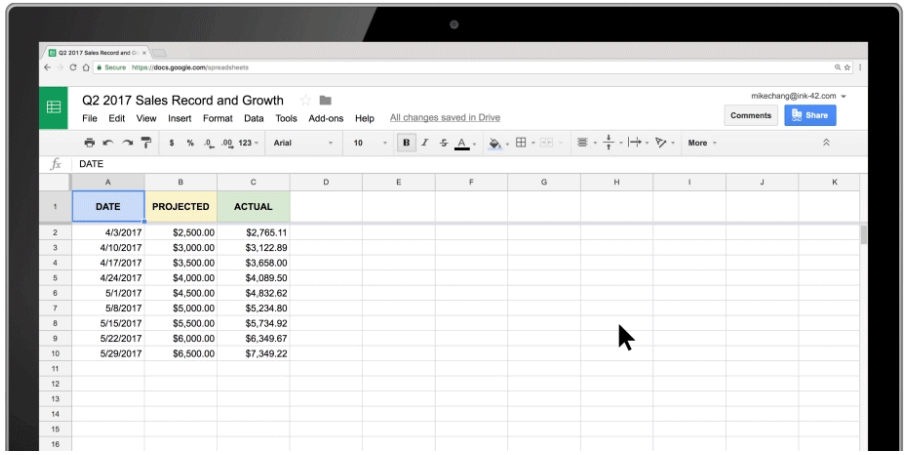


Logical Design of IoT- Communication APIs

- Web Socket-based Communication APIs





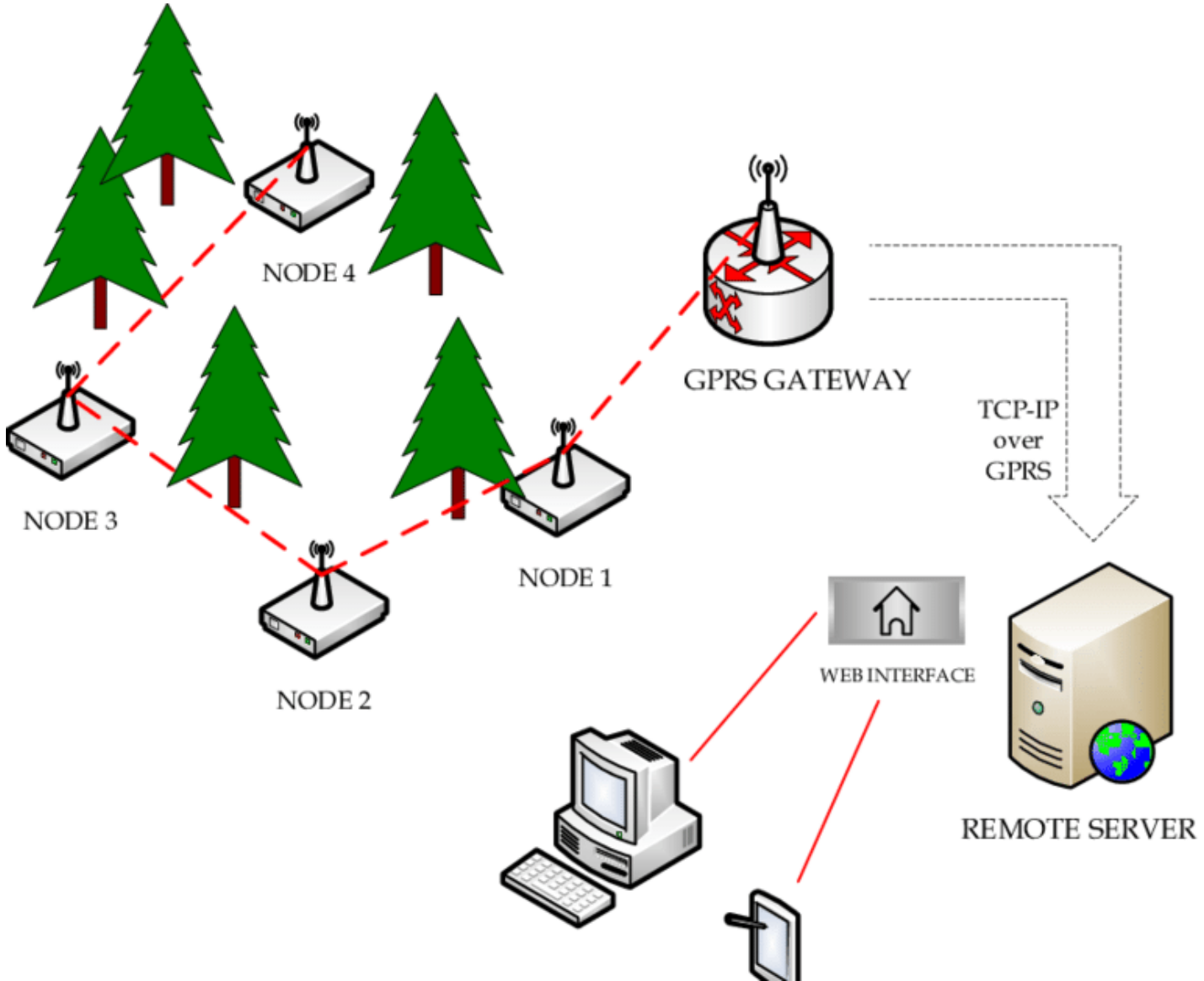


IoT Enabling technologies

• IoT (Internet of Things) enabling technologies refer to the set of tools, protocols, and innovations that allow IoT devices to connect, communicate, process data, and perform tasks.

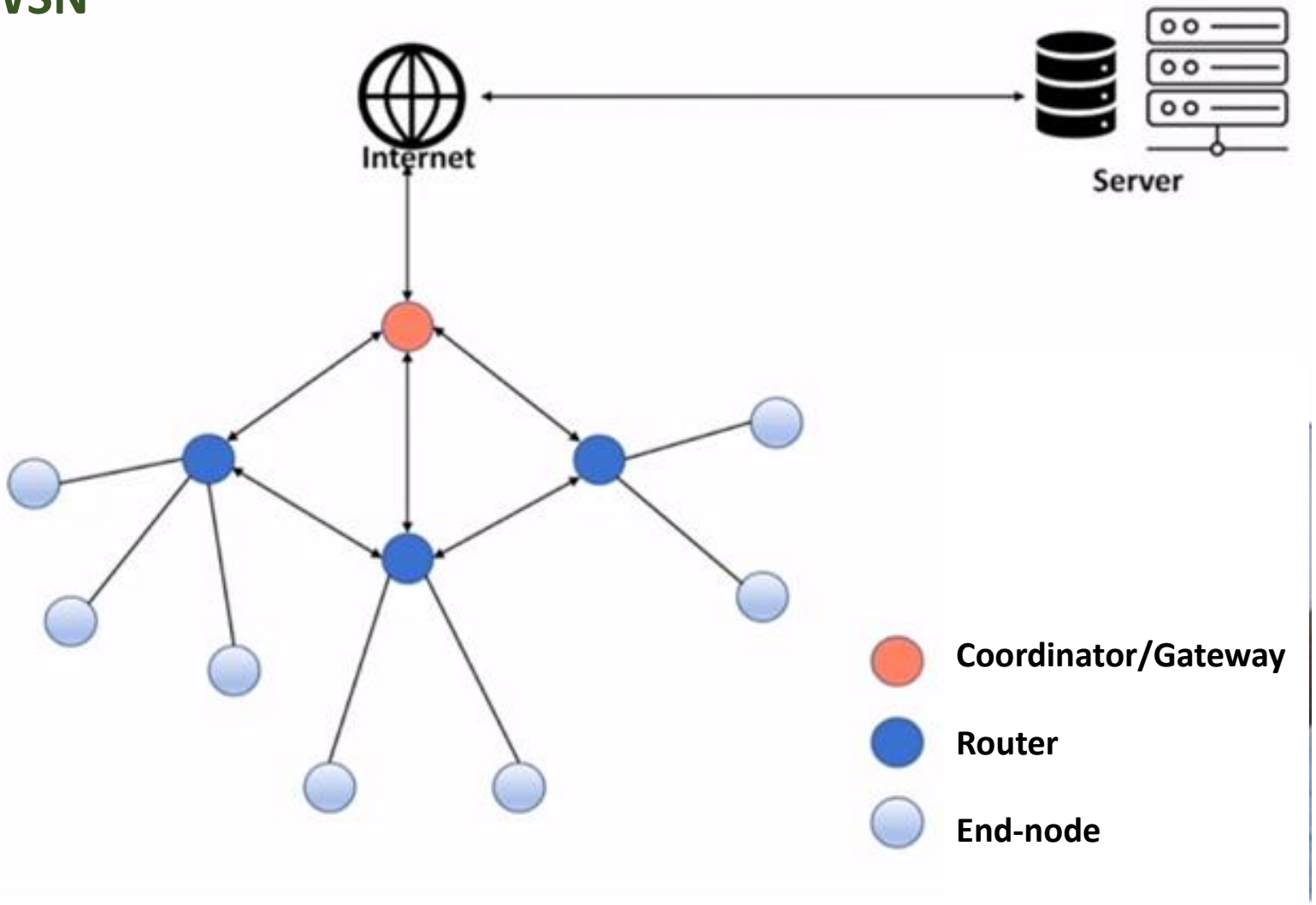
- Wireless Sensor Network(WSN)
- Cloud Computing
- Big Data Analytics
- Embedded Systems
- Communication Protocols

IoT Enabling technologies-Wireless Sensor Networks(WSN)



IoT Enabling technologies-Wireless Sensor Networks(WSN)

Basic Architecture of WSN



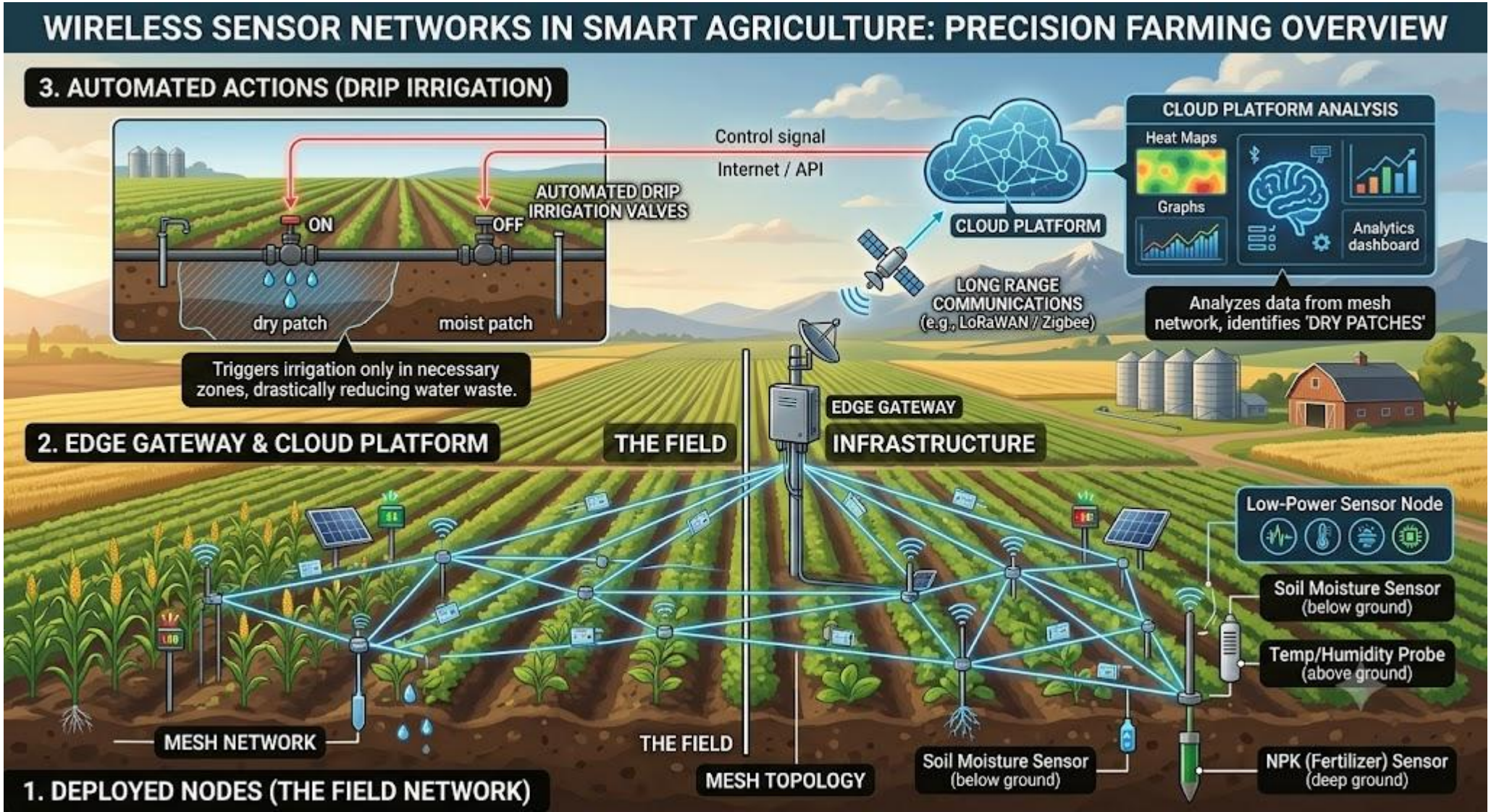
IoT Enabling technologies-Wireless Sensor Networks(WSN)

Characteristics of WSN

- Wireless Protocols(Zigbee, Bluetooth, Wi-Fi etc)
- Range(10m to 100m)
- Speed(100Kbps to few Mbps)
- Operating Frequency(ISM bands, 2.4GHz in majority devices)
- Low power Operating nodes
- WSNs are self-organizing networks
- Robust structure(Even if some nodes are failed, it does auto-reconfiguration can be done with working nodes)

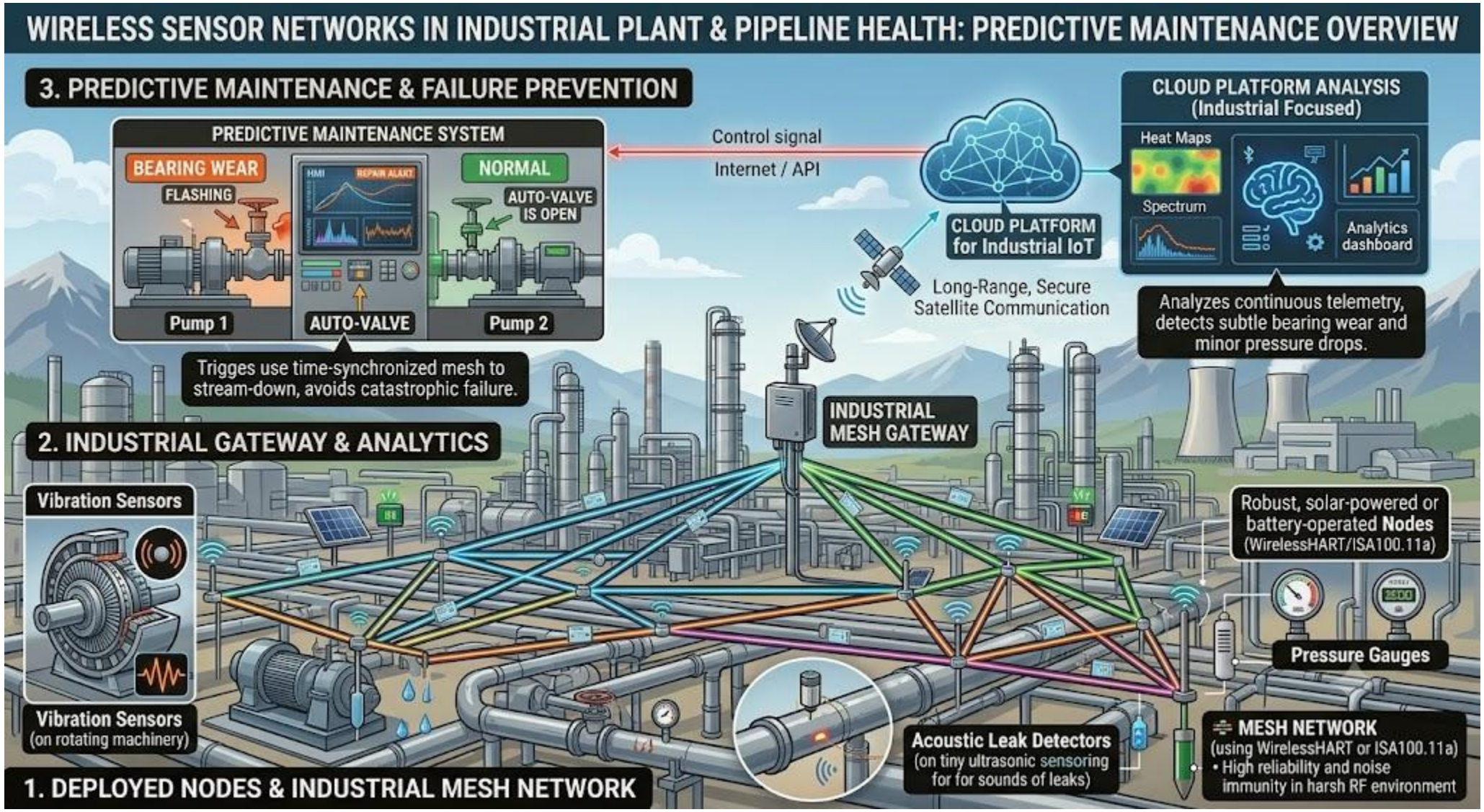
IoT Enabling technologies-Wireless Sensor Networks(WSN)

Examples of WSN



IoT Enabling technologies-Wireless Sensor Networks(WSN)

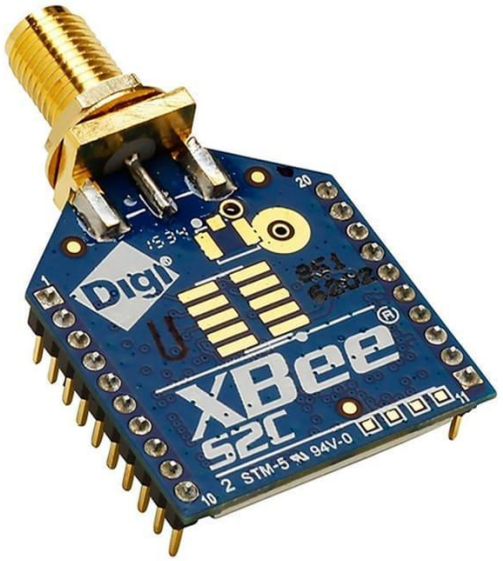
Examples of WSN



IoT Enabling technologies-Wireless Sensor Network(WSN)

Wireless Protocol modules used in WSN:

Zigbee Module



Bluetooth Module

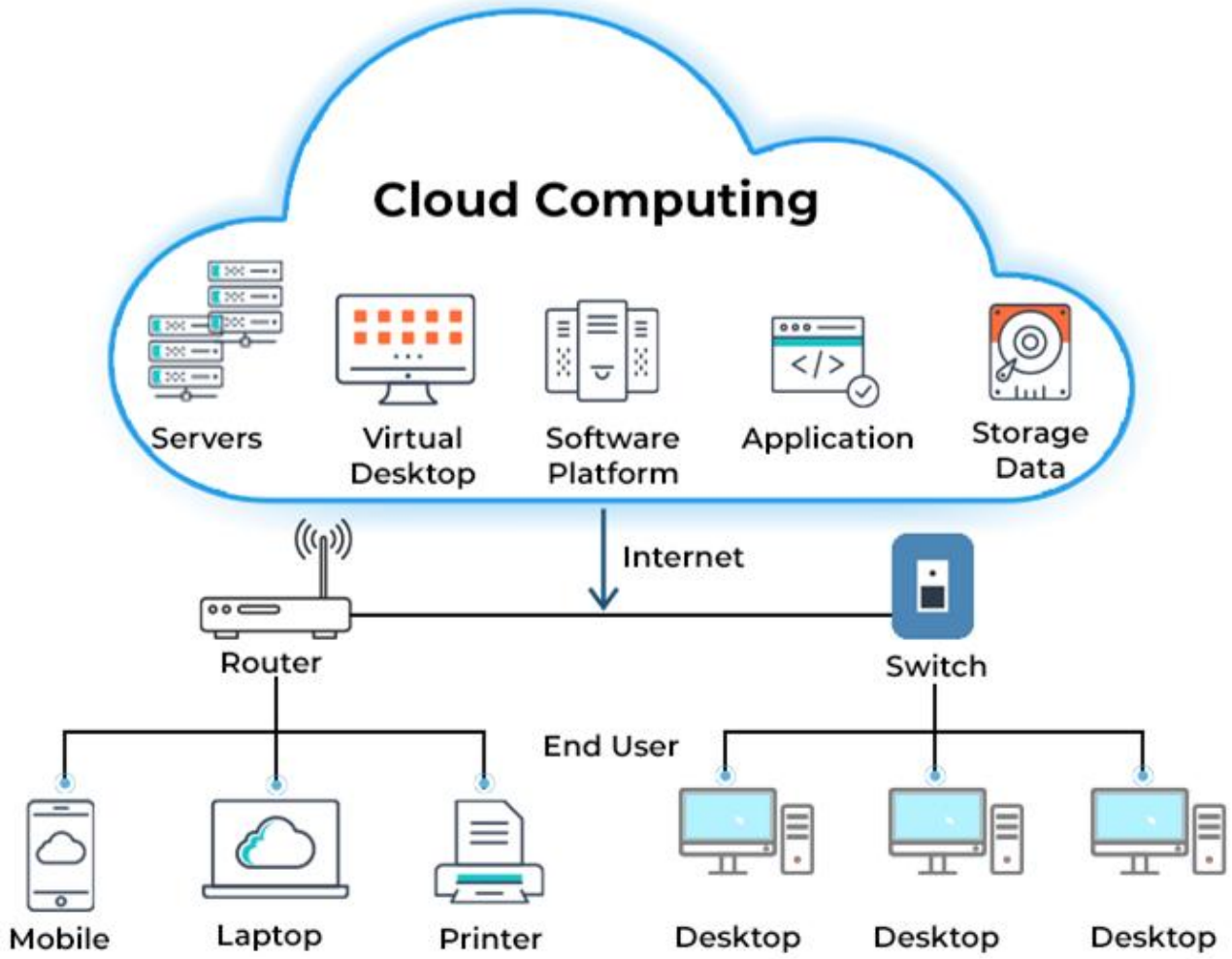


Wi-Fi Module



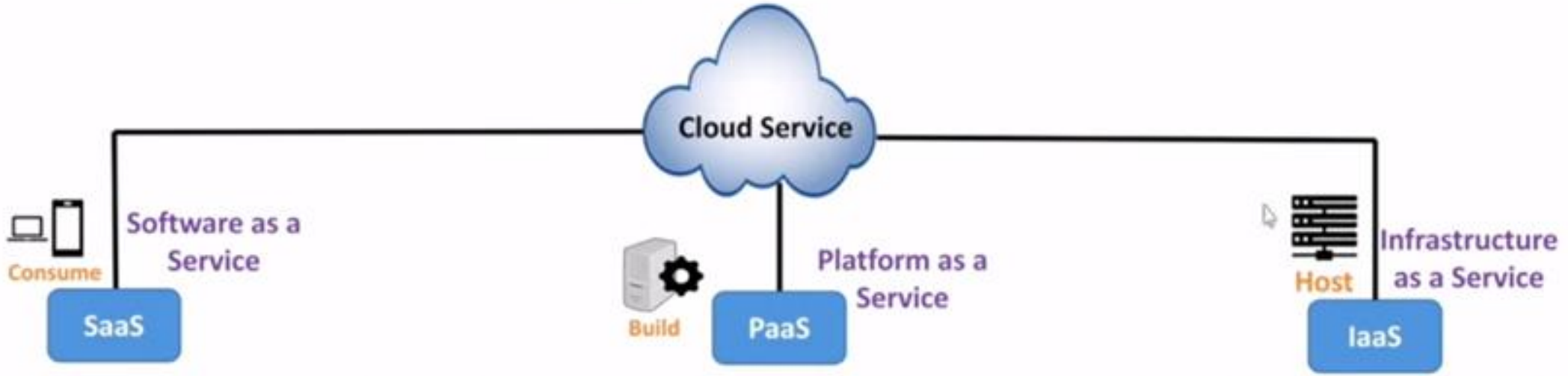
IoT Enabling technologies- Cloud Computing

- IoT devices generate massive amounts of data but have limited processing, storage, and power.
- Cloud computing fills this gap by providing the backend infrastructure to store, process, analyze, and act on IoT data.
- The three main service models
  - — IaaS,
  - — PaaS, and
  - —SaaS



IoT Enabling technologies- Cloud Computing

Service Model



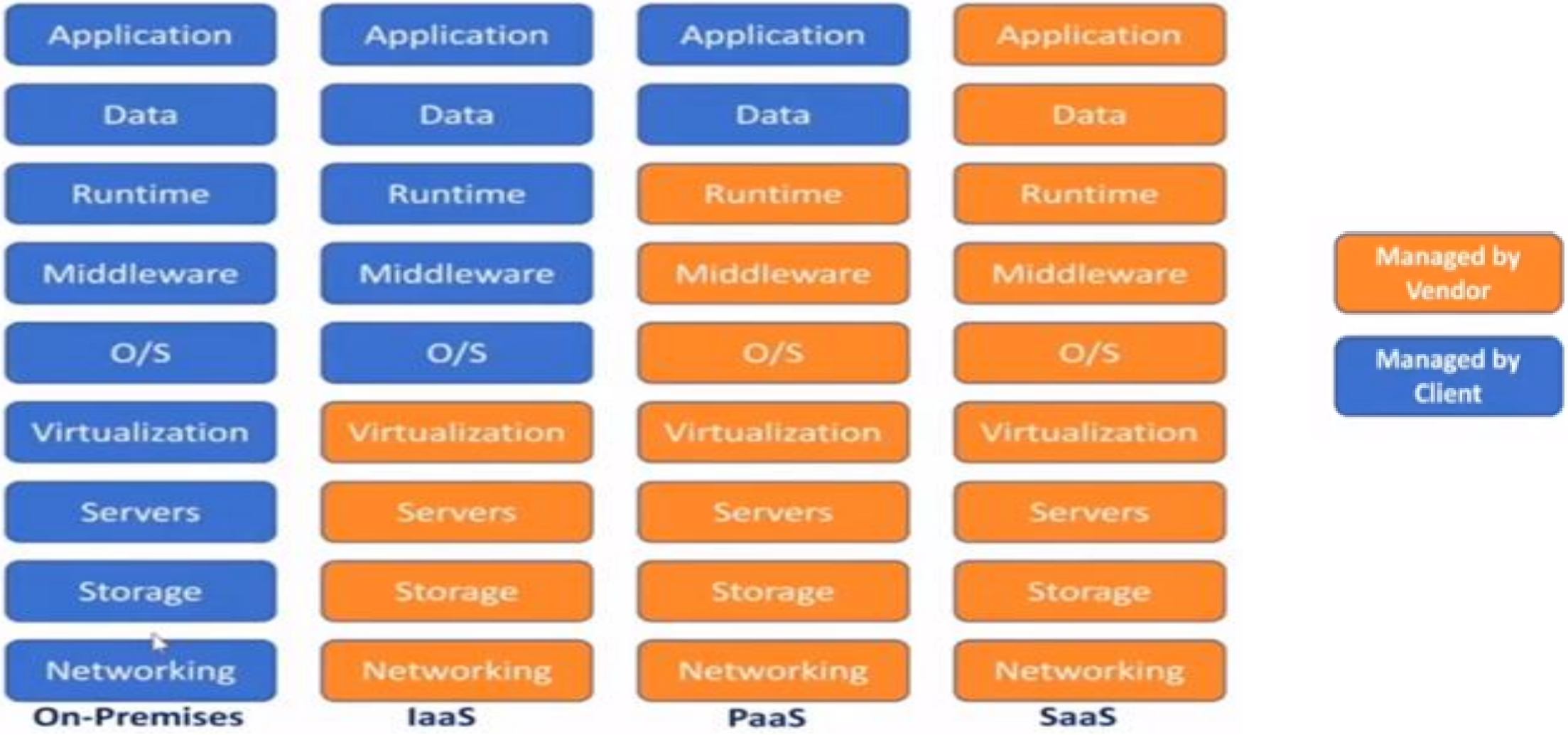
- Service Provider leases application or software service owned by them to the client
- The client does not have any back-end access.
- E.G. Google Workspace, Office 360, Google Drive.

- User has no control over underlying architecture like storage, OS, servers etc.
- Service provider gives ability to the user to develop and deploy customer created app using programming languages, tools etc
- E.G. Google App Engine

- Provides the users the ability to provision computing and storage resources.
- No worries about the underlying physical machine
- Users can deploy operating systems and applications of their choice on the virtual resources
- E.G. Amazon web service, Microsoft Azure

IoT Enabling technologies- Cloud Computing

Cloud Service Model Comparison



IoT Enabling technologies- Cloud Computing

Cloud Service Model	IoT Examples
IaaS	Azure VMs, GCE, IBM Virtual Servers, Alibaba ECS, DigitalOcean
PaaS	AWS IoT Core, Azure IoT Hub, ThingSpeak, Firebase
SaaS	Blynk, Wokwi, Tinkercad, Google AI Studio, Kaggle

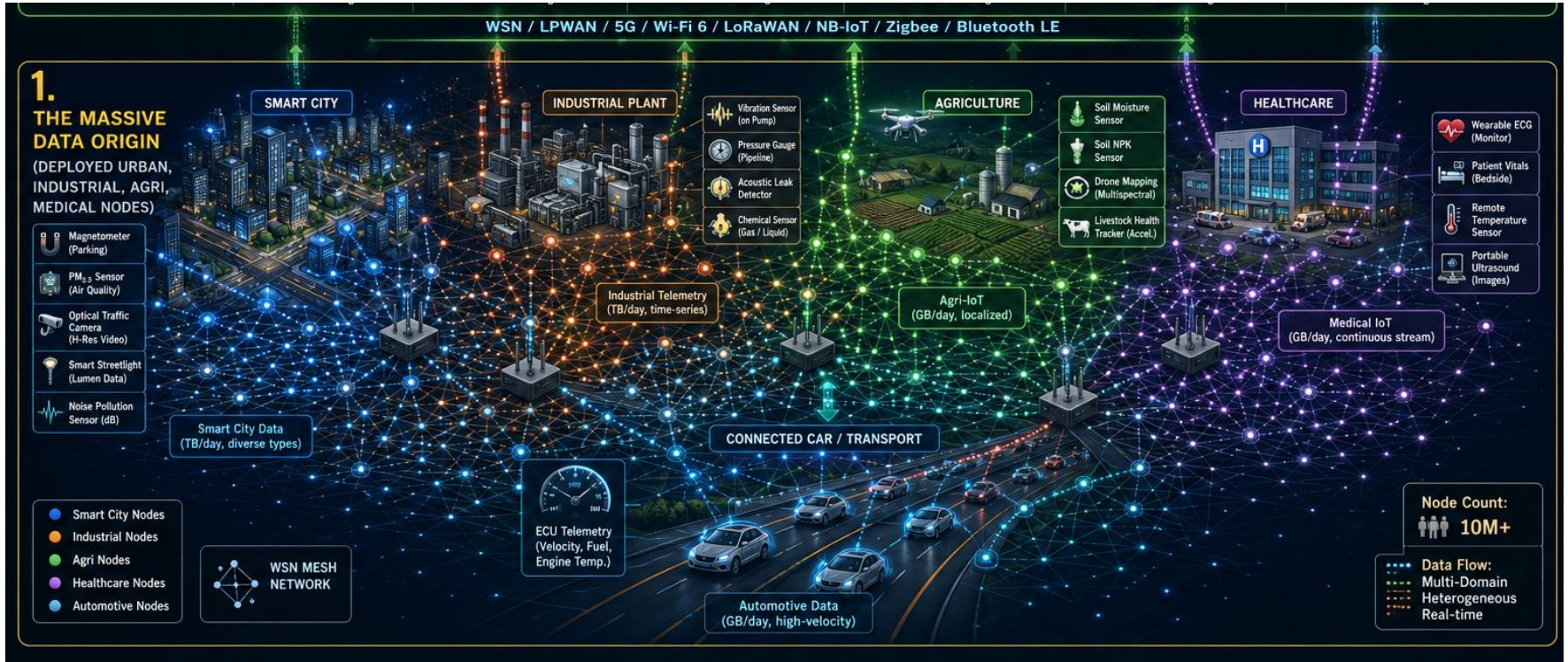
IoT Enabling technologies- Cloud Computing

Applications of Cloud Computing

- For Storage Purpose only E.G. Google Drive, Dropbox
- Provides Online editing software E.G. Canva, Invideo
- Offering Hosting services E.G. Hostinger, BigRock etc
- Online compilers services C, C++, JAVA etc E.G. Runmycode, Geeksforgeeks
- Marketing Cloud Platforms E.G. Maropost, Hubspot
- Education E.G. SlideRocket, Ratatype, Amazon Web Services

IoT Enabling technologies- Big Data Analytics





IoT Enabling technologies- Big Data Analytics

- IoT devices generate enormous volumes of data continuously — sensors, wearables, smart appliances, industrial machines, vehicles, all streaming data 24/7.

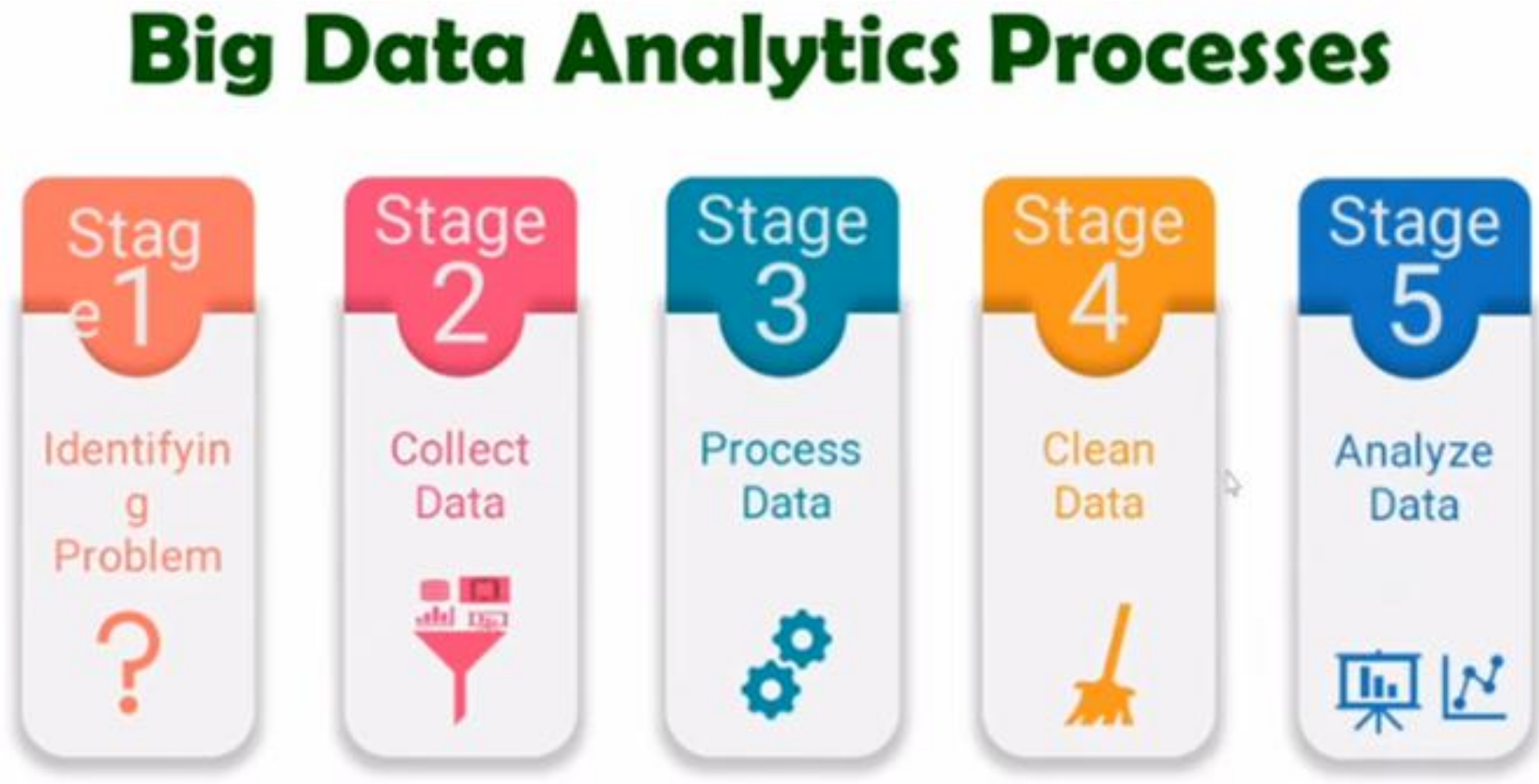
- This data is typically high in **Volume, Velocity, and Variety** (the 3 Vs of Big Data), which is exactly why traditional databases and analysis methods fall short and Big Data analytics becomes essential.

IoT Enabling technologies- Big Data Analytics

Why IoT Needs Big Data Analytics

Characteristic	IoT Reality
Volume	Billions of devices generating terabytes/petabytes of data daily
Velocity	Real-time, continuous streaming data (e.g., sensor readings every second)
Variety	Structured (numeric sensor data), semi-structured (JSON/XML logs), unstructured (video, audio, images)

IoT Enabling technologies- Big Data Analytics

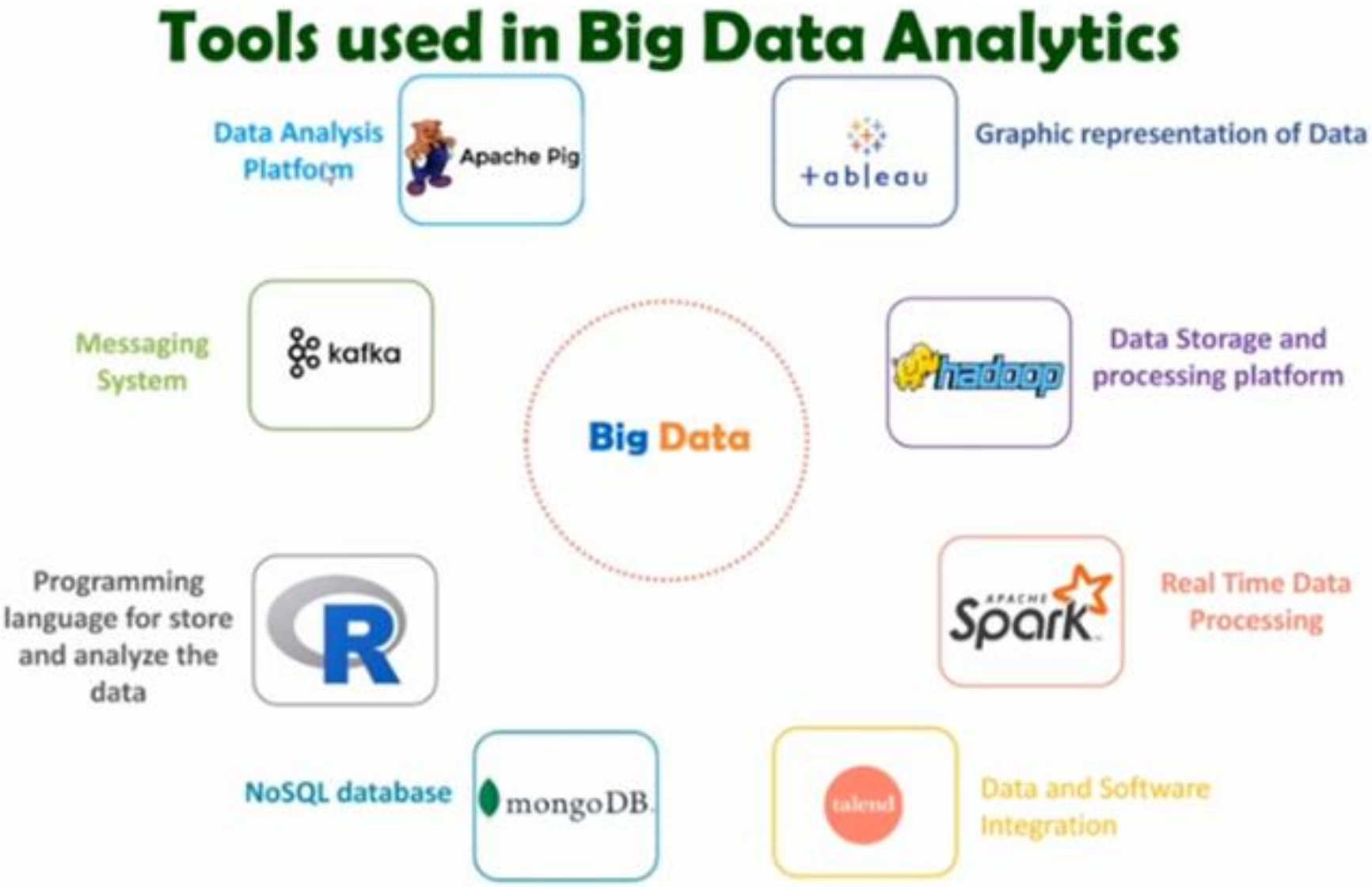


IoT Enabling technologies- Big Data Analytics

Uses of Big Data Analytics in IoT

1. Real-Time Monitoring & Alerts
2. Predictive Maintenance
3. Pattern Recognition & Anomaly Detection
4. Customer/User Behavior Insights
5. Energy Management
6. Supply Chain & Logistics Optimization
7. Healthcare Monitoring

IoT Enabling technologies- Big Data Analytics



IoT Enabling technologies- Big Data Analytics

Typical Big Data Tools Used in IoT Analytics

Purpose	Tools
Data ingestion/streaming	Apache Kafka, AWS Kinesis
Distributed storage	Hadoop HDFS, Amazon S3
Batch/stream processing	Apache Spark, Apache Flink
NoSQL databases	MongoDB, Cassandra (handle variety/velocity)
Visualization	Tableau, Power BI, Grafana
Machine learning	TensorFlow, Kaggle-trained models

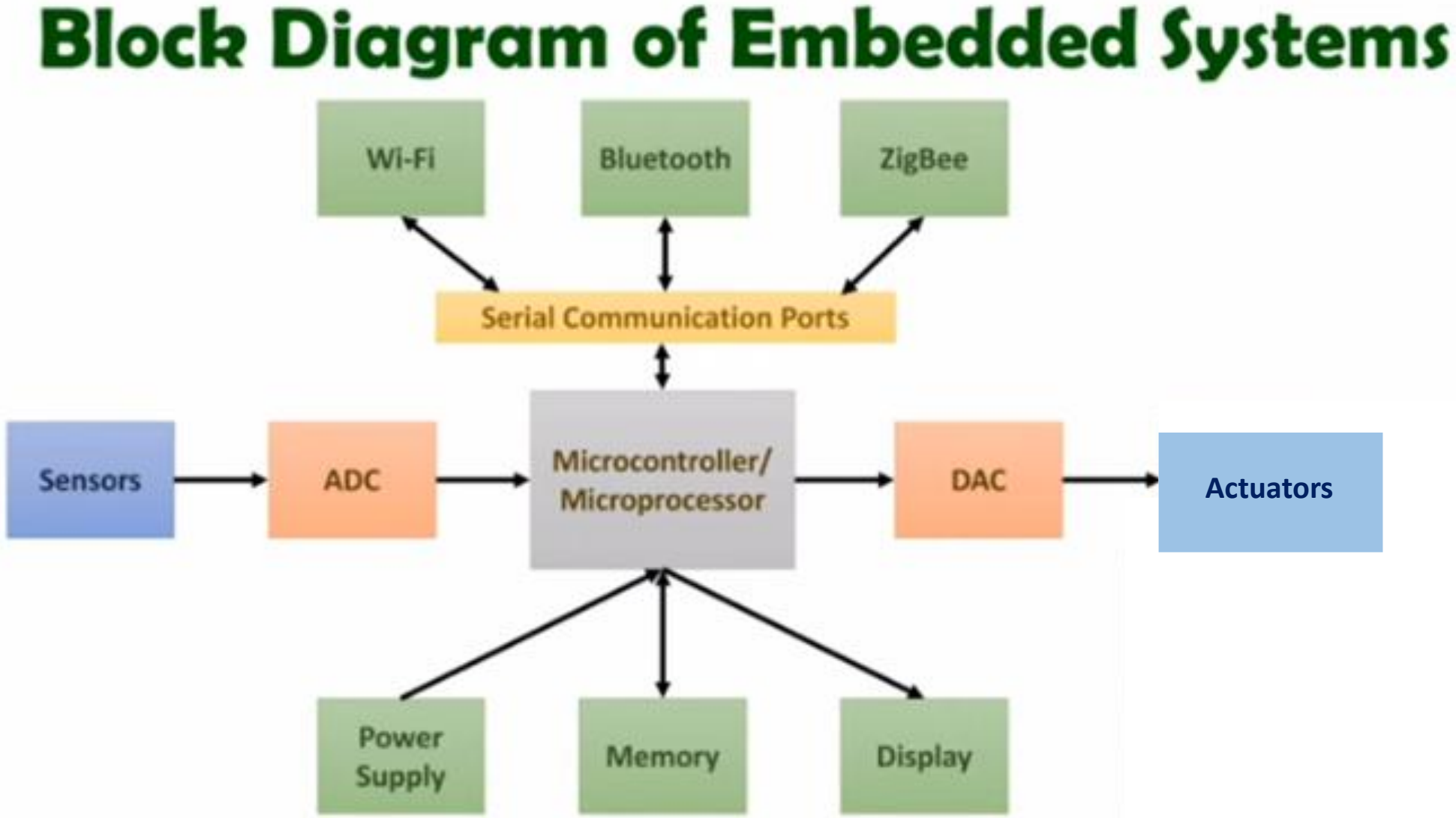
IoT Enabling technologies- Embedded Systems

- Embedded systems are the **foundation layer of IoT** — they're the actual hardware+software combination inside every "smart" device that senses the physical world, processes data locally, and communicates with the network.
- Without embedded systems, there would be no "Things" in the Internet of Things.

- **What is an Embedded System?**

- A dedicated computer system — combining a **microcontroller/microprocessor, sensors/actuators, memory, and software** — designed to perform a specific task, unlike a general-purpose computer (like a laptop) that runs many different applications.

IoT Enabling technologies- Embedded Systems



IoT Enabling technologies- Embedded Systems

Uses of Embedded Systems in IoT

1. Data Acquisition (Sensing)
2. Local Processing & Decision Making (Edge Computing)
3. Actuation (Controlling Physical Devices)
4. Communication & Connectivity
5. Real-Time Response
6. Power Management

IoT Enabling technologies- Embedded Systems

Common Embedded Platforms Used in IoT

Platform	Typical Use
Arduino (Uno, Nano, Mega)	Simple sensor/actuator projects, prototyping
ESP32 / ESP8266	Wi-Fi-enabled IoT nodes (smart home devices)
Raspberry Pi	More powerful edge processing, can run Linux + AI models
STM32	Industrial-grade, low-power embedded IoT applications
NodeMCU	Beginner-friendly Wi-Fi IoT development

IoT Enabling technologies- Communication Protocols

- Communication protocols are the **rules and standards that let IoT devices talk to each other and to the cloud**.
- Since IoT connects everything from tiny battery-powered sensors to powerful cloud servers, no single protocol fits all situations

  — different protocols are optimized for different combinations of range, power, bandwidth, and reliability.

Example: WiFi, BLE, LoRaWAN, Zigbee, Z-Wave, Cellular, MQTT, CoAP, HTTP…

IoT Enabling technologies- Communication Protocols

Why IoT Needs Multiple Protocols

Constraint	Why It Matters
Power	Many IoT devices run on batteries for years — protocols must be lightweight
Range	Devices vary from a few meters (Bluetooth) to kilometers (LoRaWAN, cellular)
Bandwidth	Sensor data is tiny (a few bytes), video streams are huge — one protocol can't serve both
Reliability	Critical systems (healthcare, industrial) need guaranteed delivery; casual sensors can tolerate loss
Device count	Networks may need to support thousands of nodes simultaneously

IoT Levels and Deployment Templates

What are IoT Levels?

•IoT Levels are **standard deployment templates** that define how an IoT system's components.

- Device,
- Resources ,
- Controller ,
- Storage/ database,
- Analytics ,
- Application are arranged



•Based on: number of nodes, where data is stored, where analysis happens, where the application is hosted

•Helps developers choose the right architecture based on **scale, complexity, and use case**

•6 commonly defined levels (Level 1 to Level 6)

IoT Levels and Deployment Templates

IoT Components

Component	Role	Example
Device	Sense/act on physical world	Temperature sensor, relay
Resources	Local software managing device	Firmware/drivers on ESP32
Controller Service	Manages communication to/from cloud	ESP32 controller/ MQTT client publishing data
Database/Storage	Stores collected data	Firebase, MongoDB, SD card
Analytics	Extracts insights from data	ML model detecting anomalies
Application	User interface	Blynk app, web dashboard

IoT Levels and Deployment Templates

1. Device

The **physical hardware** — sensors and/or actuators — that interacts directly with the physical world.

- **Sensors** collect data (temperature, humidity, motion, light, pressure, etc.)
- **Actuators** perform physical actions (turn on a motor, open a valve, switch a light)
- **Example:** A DHT11 temperature sensor and a relay-controlled fan in a smart room system.

IoT Levels and Deployment Templates

2. Resources

The **software components running on or near the device** that manage its operations — this includes things like the device's OS, drivers, and any local services needed to interact with sensors/actuators and communicate with the network.

- Includes **monitoring services** (checking device health/status) and **actuation services** (controlling actuators based on commands)
- **Example:** The embedded firmware on an ESP32 that reads sensor values and manages Wi-Fi connectivity.

IoT Levels and Deployment Templates

3. Controller Service

The **software that runs on the device (or gateway) and manages communication** between the device and the cloud/remote server — typically over protocols like MQTT or HTTP.

•Acts as the bridge: pulls data from the resources, sends it out to storage/cloud, and receives commands back to trigger actuators

Example: A control service on a Raspberry Pi that publishes soil-moisture readings via MQTT and listens for "turn on pump" commands from the cloud.

IoT Levels and Deployment Templates

4. Database/Storage

Where the **collected data is stored**, either locally (on the device) or remotely (in the cloud).

- **Local storage:** SD card, onboard flash memory — used in simple/low-cost setups (Level 1)
- **Cloud storage:** Scalable databases (SQL/NoSQL) hosted on cloud platforms — used when data volume is large or long-term storage/history is needed

Example: Sensor readings stored in a cloud database like Firebase or a NoSQL database like MongoDB, versus storing a day's readings on a local SD card.

IoT Levels and Deployment Templates

5. Analytics

The **component that processes stored data to extract meaningful insights, patterns, or predictions** — this is where Big Data analytics and machine learning come in.

•Ranges from simple threshold checks ("if temp > 40°C, alert") to complex ML-based predictions (predictive maintenance, anomaly detection)

•Can run locally (edge analytics) or in the cloud (heavier processing)

Example: A cloud analytics engine on ThingSpeak analyzing a week of temperature data to detect a rising trend and predict an equipment overheating risk.

IoT Levels and Deployment Templates

6. Application

The **user-facing interface** — a web dashboard or mobile app — through which people view data, monitor device status, and send control commands.

•Can be **local** (an app on the same network as the device) or **cloud-based** (accessible remotely from anywhere)

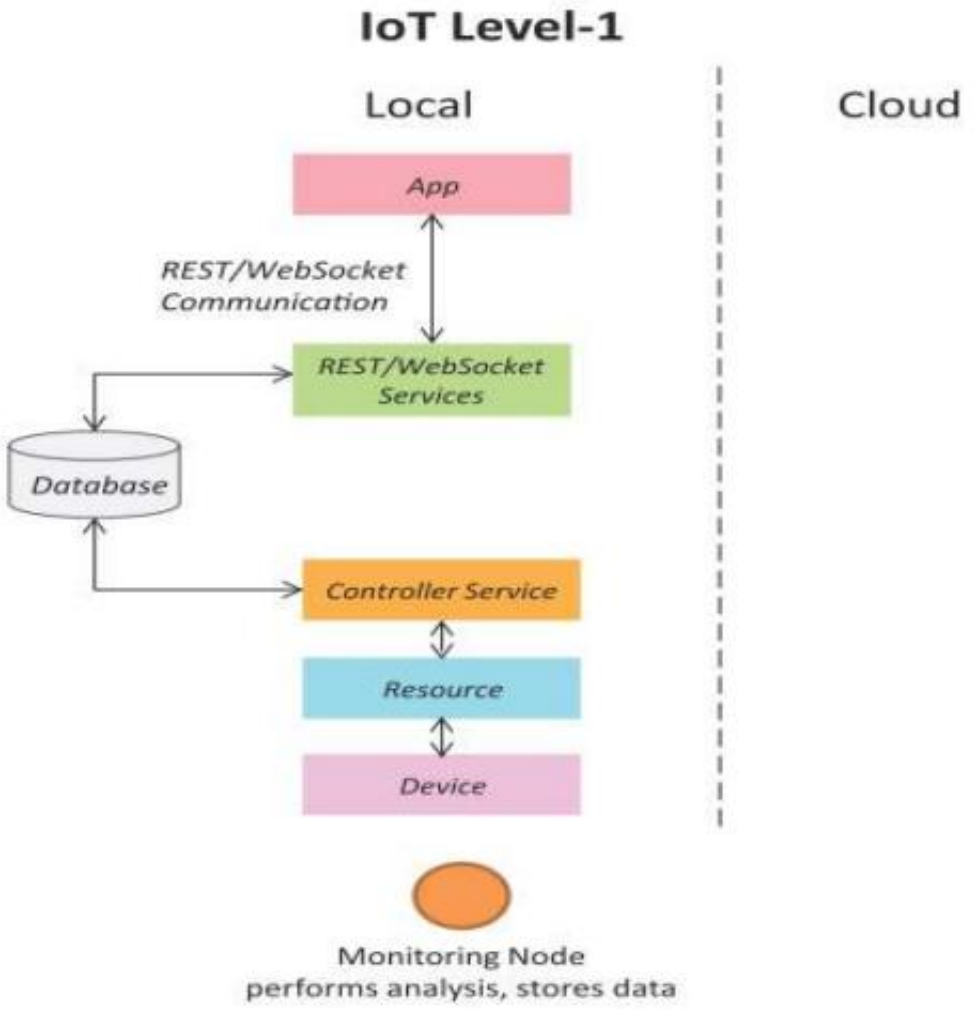
Example: A Blynk mobile app dashboard showing live temperature graphs and a button to remotely turn the fan on/off.

IoT Level-1

Single Node, Single Application/Service

- One device (node) handles sensing, actuation, monitoring, and control
- Data storage and analysis done **locally on the device**
- Suitable for **low-cost, low-complexity** solutions
- No cloud involved

Example: A home automation system where a single microcontroller monitors temperature and controls a fan — everything processed on the device itself.



IoT Level-2

Single Node + Cloud-Based Storage/Analysis

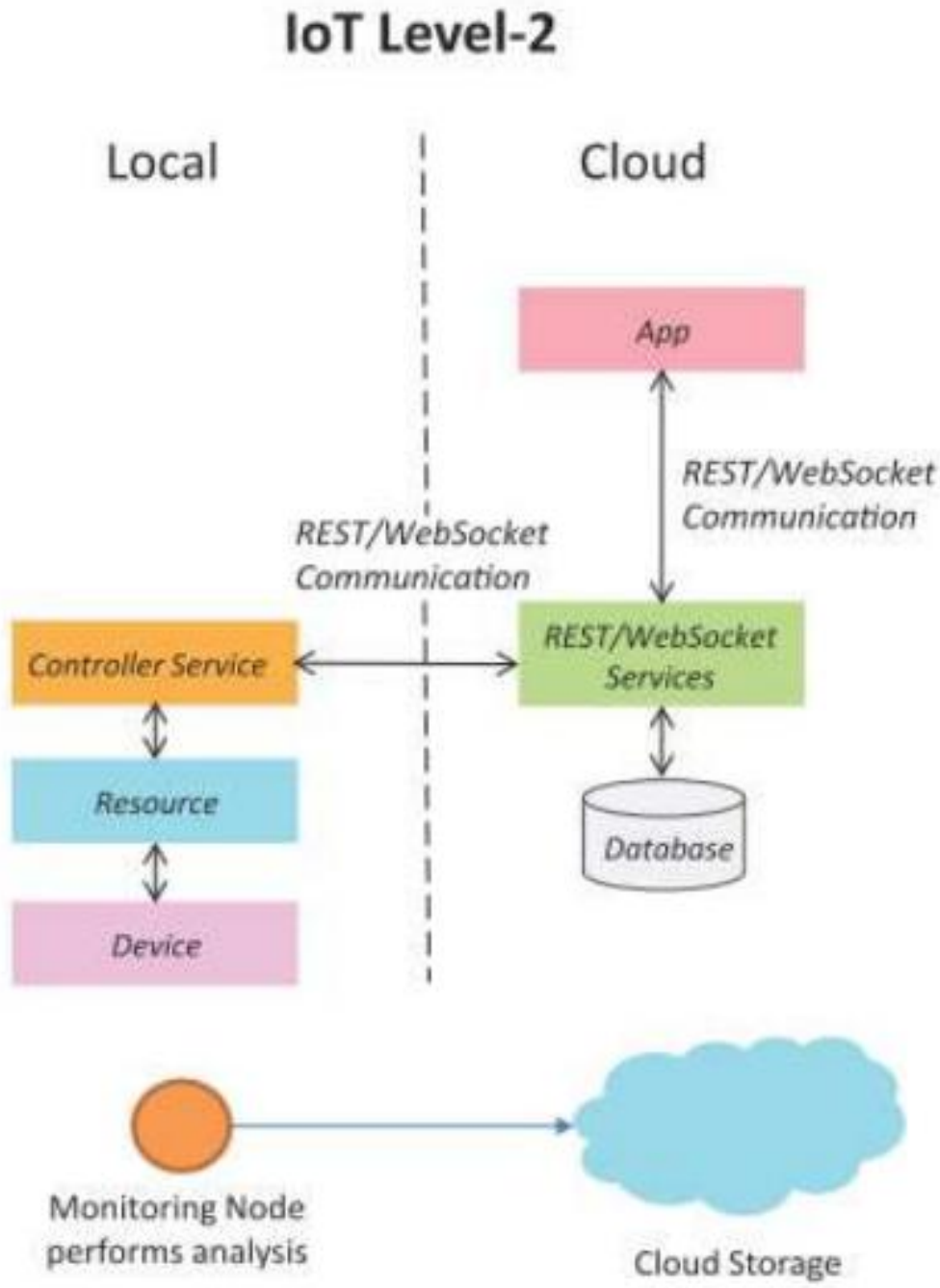
•Single device collects data

•Data is sent to the **cloud** for storage and

•Analysis, Application can be **cloud-based or local**

•Suitable when data volume is manageable but local storage is limited

Example: A smart weighing machine that sends inventory data to the cloud for analysis and generates alerts.



IoT Level-3

Single Node + Cloud Storage/Analysis + Remote Application

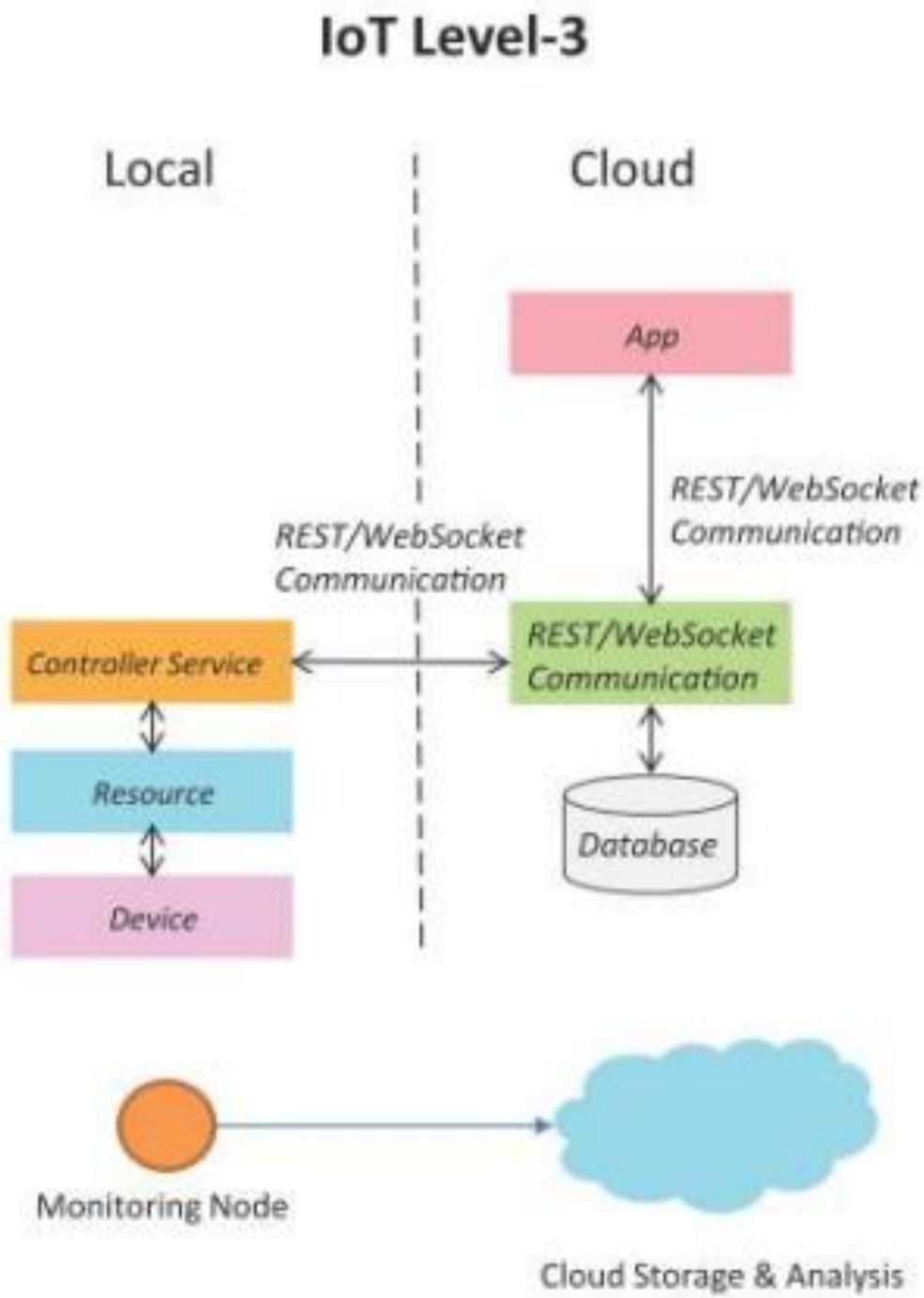
•Single device sends data to cloud

•Both **data analysis and application** are hosted on the cloud

•Application is accessed remotely (via web/mobile) — not tied to local device

•Good for **remote monitoring** systems

Example: A remote weather monitoring station sending data to the cloud, with a web dashboard accessible from anywhere.

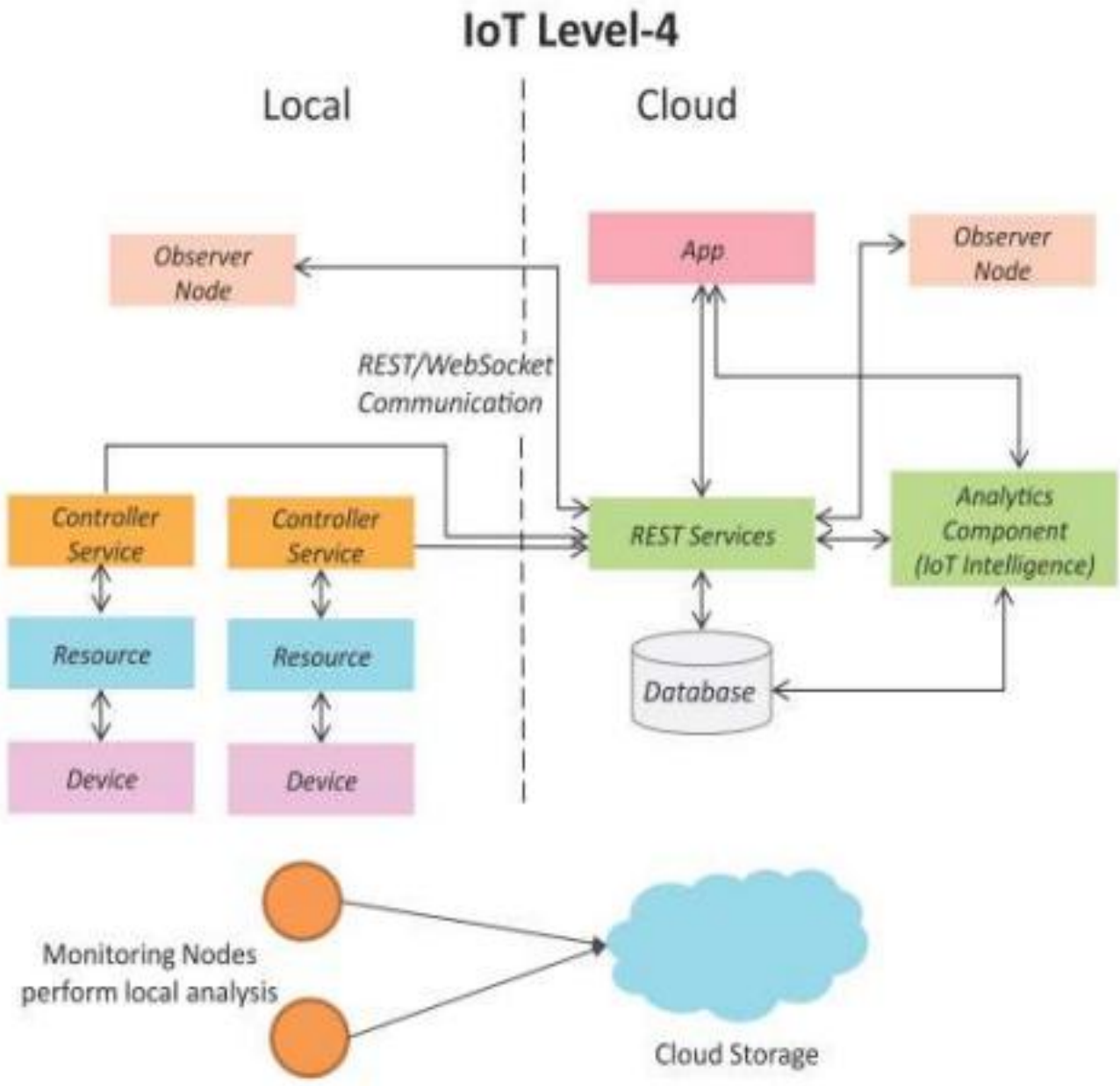


IoT Level-4

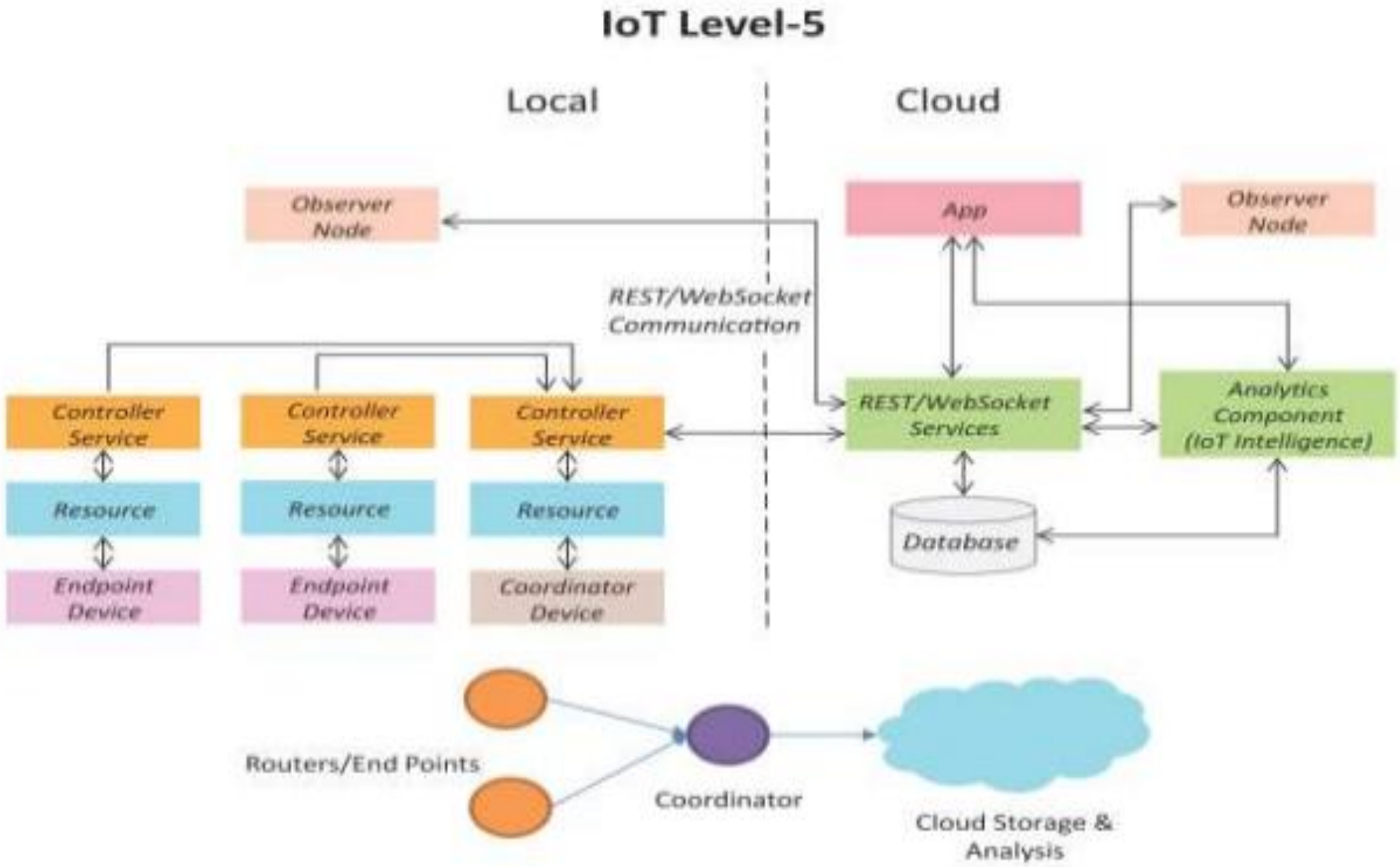
Multiple Nodes, Single Application (Local + Cloud)

- Multiple devices/nodes collect data independently
- Each node may have **local storage and analysis**
- Data also aggregated in the cloud for **broader analysis**
- Coordinator/observer node often manages multiple end nodes

Example: A noise-monitoring system with several sensor nodes across a city, each doing local processing, with results also aggregated centrally.



IoT Level-5


IoT Level-5
Local
Cloud
Observer Node
App
Observer Node
REST/WebSocket Communication
Controller Service
Controller Service
Controller Service
REST/WebSocket Services
Analytics Component (IoT Intelligence)
Resource
Resource
Resource
Database
Endpoint Device
Endpoint Device
Coordinator Device
Routers/End Points
Coordinator
Cloud Storage & Analysis

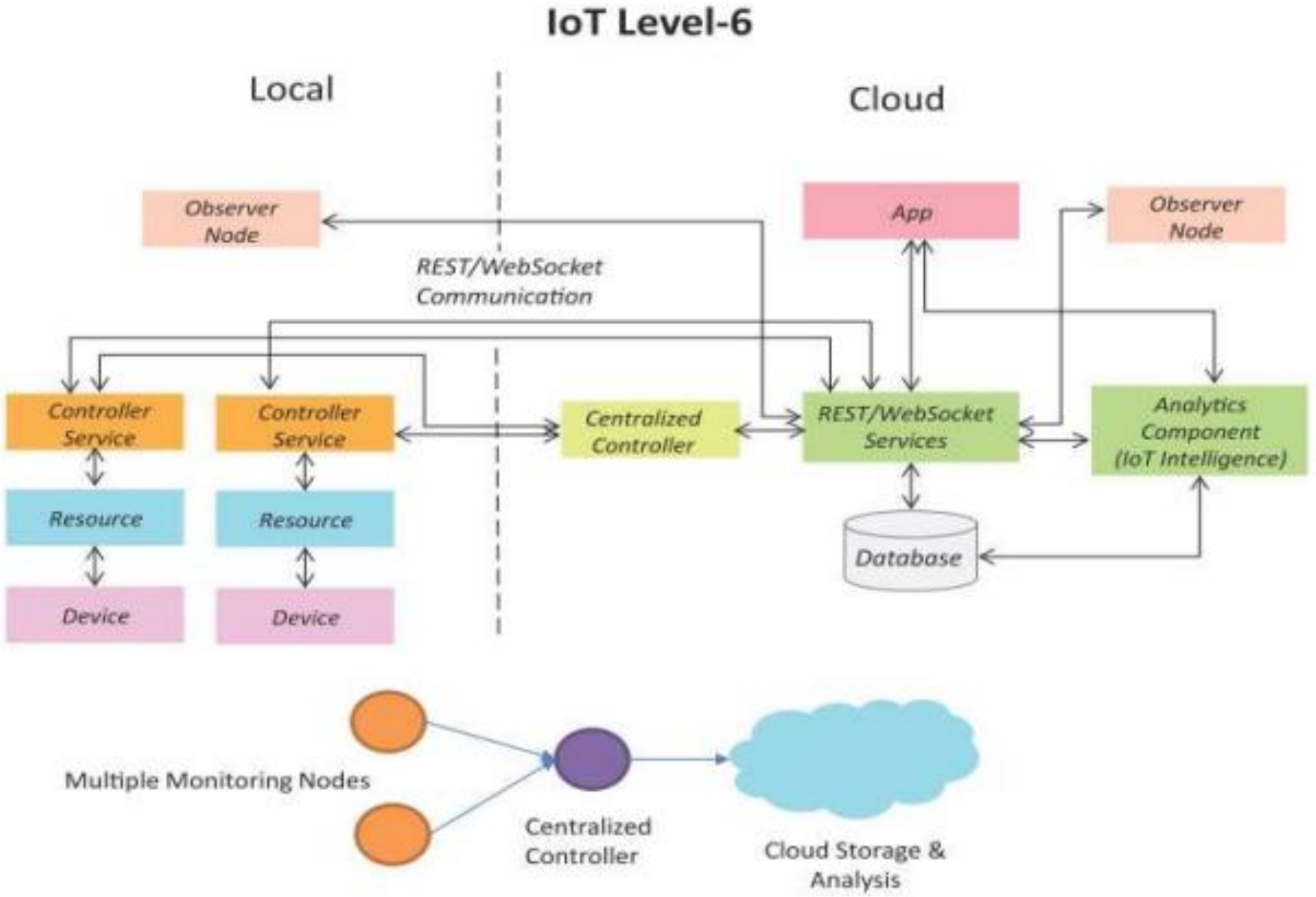
IoT Level-5

Multiple End Nodes + One Coordinator + Cloud Storage/Analysis

- Multiple end nodes send data to a **single coordinator node**
- Coordinator forwards aggregated data to the **cloud**
- Cloud handles storage, analysis, and hosts the application
- Common in **wireless sensor network (WSN)** based deployments

Example: Smart irrigation system — multiple soil-moisture sensor nodes report to a coordinator (gateway), which sends combined data to the cloud for analysis and control decisions.

IoT Level-6


IoT Level-6
Local
Cloud
Observer Node
App
Observer Node
REST/WebSocket Communication
Controller Service
Controller Service
Centralized Controller
REST/WebSocket Services
Analytics Component (IoT Intelligence)
Resource
Resource
Database
Device
Device
Multiple Monitoring Nodes
Centralized Controller
Cloud Storage & Analysis

IoT Level-6

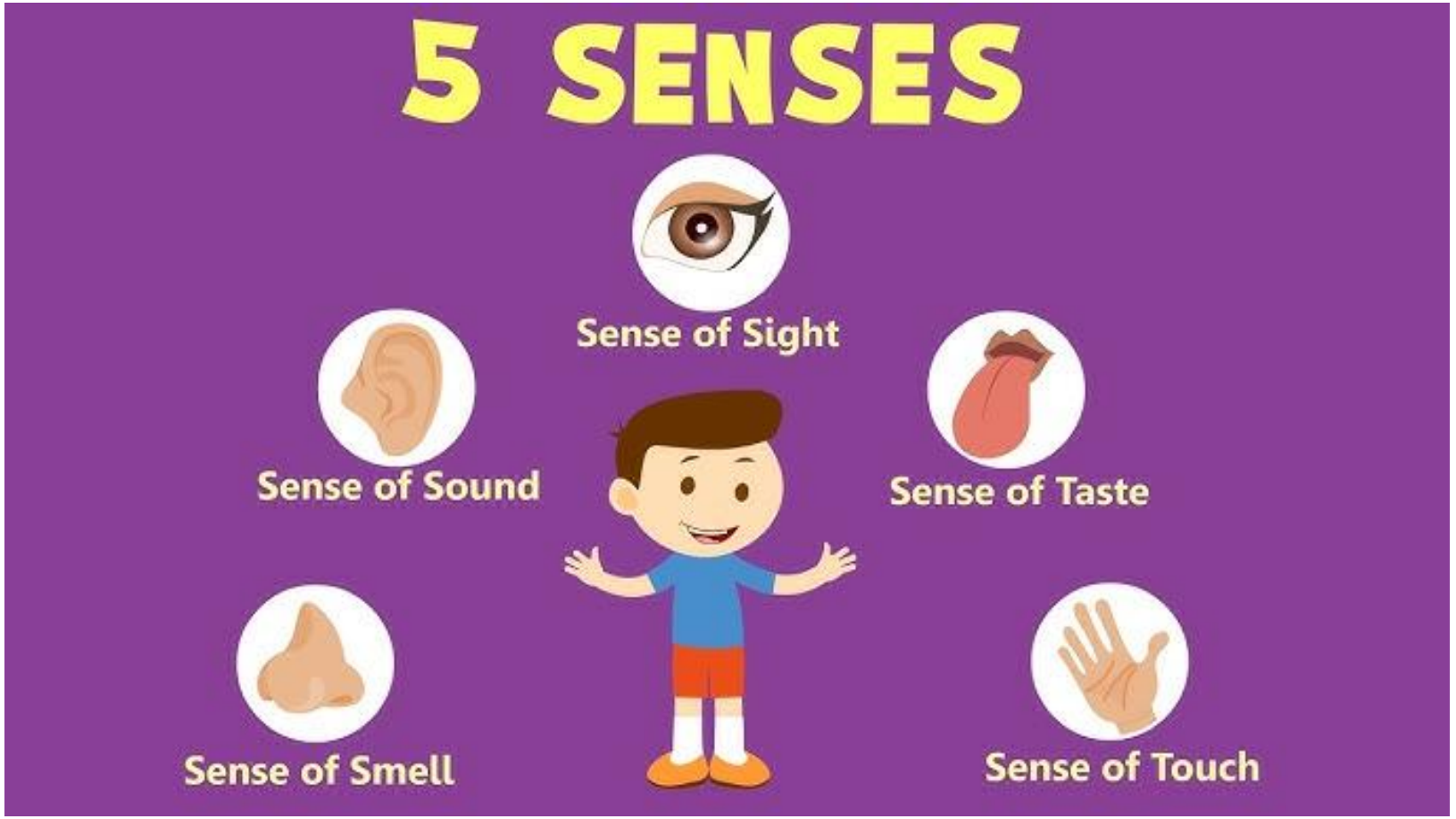
Multiple Independent Nodes + Cloud Storage/Analysis (Independent Data)

- Multiple nodes operate **independently**, each sending data directly to the cloud
- No single coordinator — each node has its own data path
- Cloud stores and analyzes data from each node **separately**
- Application on cloud provides a **unified view** of all nodes

Example: A fleet of delivery trucks where each vehicle's GPS/IoT device sends data independently to the cloud, and a central dashboard shows the whole fleet's status.

Sensors and actuators

Human Body Sensors



IoT Sensing and actuation

- **Sensors**
- Internet of Things (IoT) sensors are components of IoT devices that measure physical quantities, converting them into data that can be interpreted by humans or machines.
- The data gathered by IoT sensors enables various types of control and automation.
- Data from IoT sensors can be used to gain insights, improve decision-making, optimize processes, and automate tasks, across varied industries and use cases

IoT Sensing and actuation

Sensors

- Essentially, any sensor that is connected to the internet, or to an IoT device, is an IoT sensor.
- This means that there are as many types of IoT sensors as there are quantities or attributes of things to measure or detect in the world.

IoT Sensing

Classification of IoT Sensors

Sensing types are categorized based on how they quantify environmental properties and the nature of the data they produce.

IoT sensing classifications include

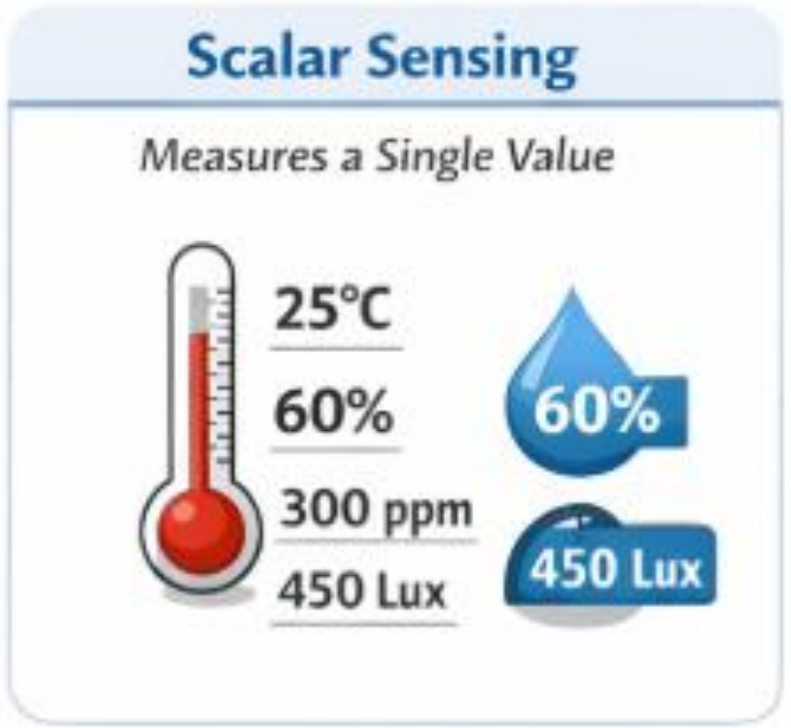
1. **Scalar**
2. **Vector**
3. **Multimedia**
4. **Hybrid**, and
5. **Virtual** sensing.

IoT Sensing

1. Scalar Sensing

It senses: A **single magnitude/value** (no direction)

- **Characteristics:**
  - Outputs one numerical value
  - Most common type in IoT
  - Low complexity and low power
- **IoT Examples:**
  - Temperature sensor → 25°C
  - Humidity sensor → 60%
  - Gas sensor → 300 ppm
  - Light sensor → 450 lux
- **Applications:**
  - Smart homes (temperature, humidity)
  - Environmental monitoring
  - Industrial safety



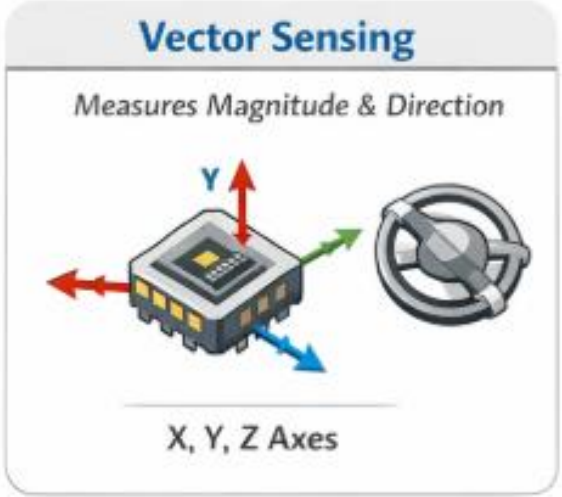
📌 *Think of scalar sensing as "How much?"*

IoT Sensing

2. Vector Sensing:

It senses: A value with **magnitude + direction**

- **Characteristics:**
  - Outputs multiple components (x, y, z)
  - Used for motion and orientation
- **IoT Examples:**
  - Accelerometer → acceleration in X, Y, Z axes
  - Gyroscope → angular velocity + direction
  - Magnetometer → magnetic field direction
- **Applications:**
  - Smartphones & wearables
  - Drones and robots
  - Vehicle navigation systems



📌 *Think of vector sensing as "How much and in which direction?"*

IoT Sensing

3. Multimedia Sensing

It senses: Rich data like images, audio, or video

- **Characteristics:**
  - High data rate
  - Requires more processing & bandwidth
- **IoT Examples:**
  - Camera → images/video
  - Microphone → sound
  - Smart doorbell camera
- **Applications:**
  - Smart surveillance
  - Healthcare monitoring
  - Traffic monitoring



📌 *Think of multimedia sensing as "Seeing and hearing"*

IoT Sensing

4. Hybrid / Composite Sensing

It senses: Combination of **multiple sensing types**

Characteristics:

Uses multiple sensors together

More accurate decision-making

IoT Examples:

Smartwatch → heart rate (scalar) + motion (vector)

Smart home → temperature + motion + light

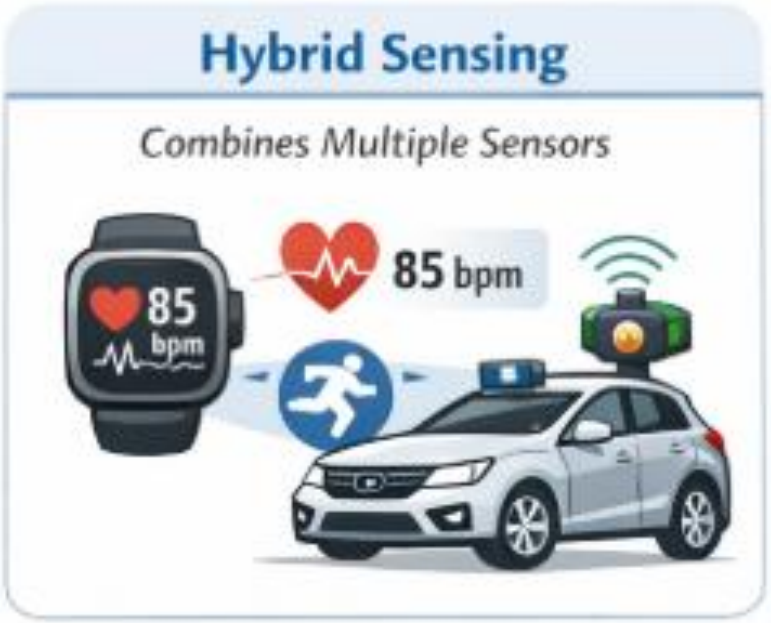
Autonomous vehicles → camera + LiDAR + radar

Applications:

Smart healthcare

Smart cities

Autonomous systems



📌 *Think of hybrid sensing as "Multiple senses working together"*

IoT Sensing

5. Virtual Sensing

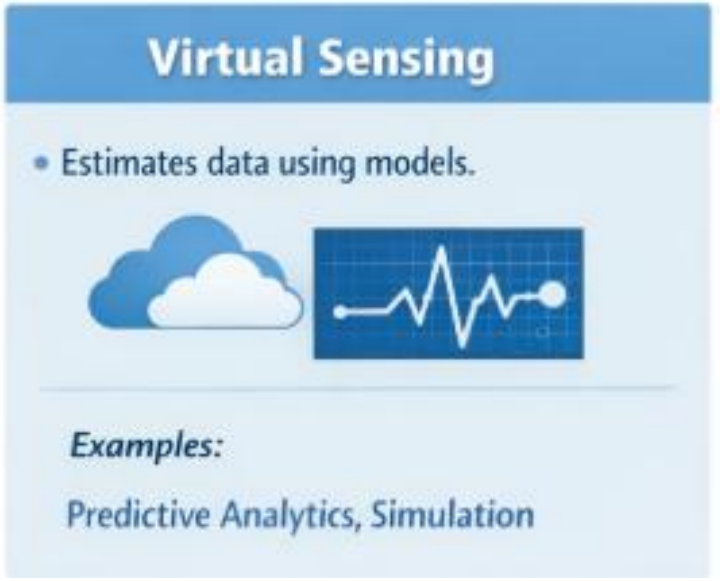
Does not use a physical hardware sensor. Instead, it uses software algorithms and mathematical models to **estimate** a value based on available data from other physical sensors.

Characteristics: Cost-effective as it replaces expensive physical hardware with software calculations.

•Example:

•**Estimating Flow Rate:** Calculating fluid flow speed using data from pressure sensors and temperature sensors, rather than installing a dedicated flow meter.

•**Battery Level:** Estimating remaining battery life based on voltage and current usage history.



Sensors and actuators

Classification of Sensors

Sensors can be categorized in a number of ways, including:

1. Energy Usage or Power Requirement
    i. Active
    ii. Passive
2. Output Type
    i. Analog
    ii. Digital

Sensors and actuators

Classification of Sensors

Sensors can be categorized in a number of ways, including:

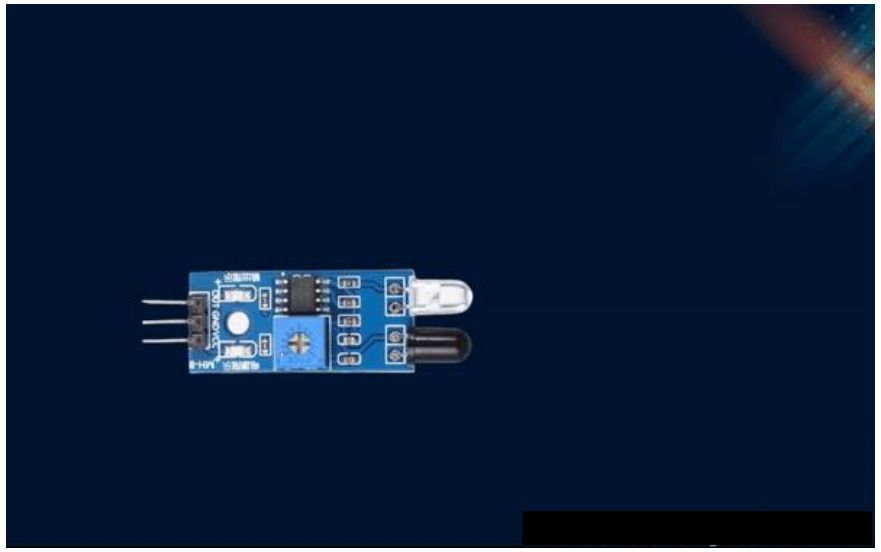
3. Sensing Principle/ Sensing Type
    i. Mechanical
    ii. Thermal
    iii. Electrical
    iv. Magnetic
    v. Radiant
    vi. Chemical

4. Input Type
    i. Sound
    ii. Light
    iii. Temperature
    iv. Optical
    v. colour
    …
    …

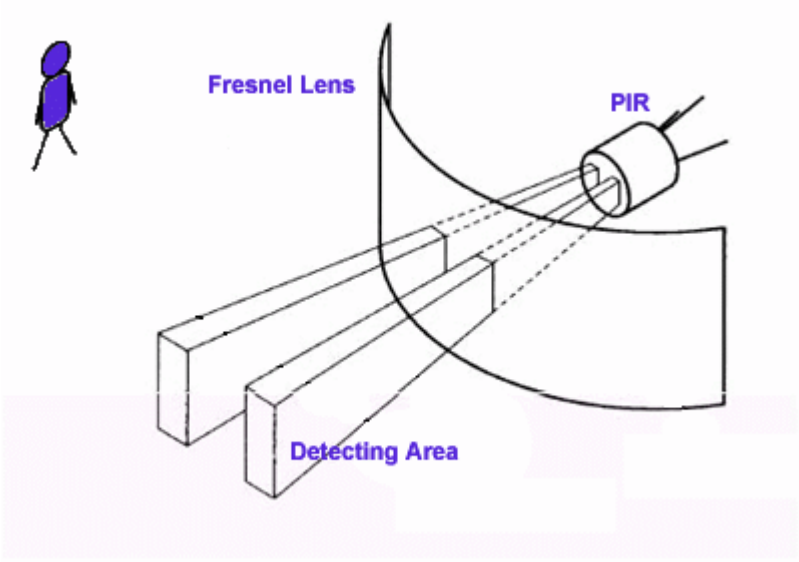
Sensors and actuators

Classification of Sensors

1. Energy Usage or Power Requirement



Active sensor



Passive sensor

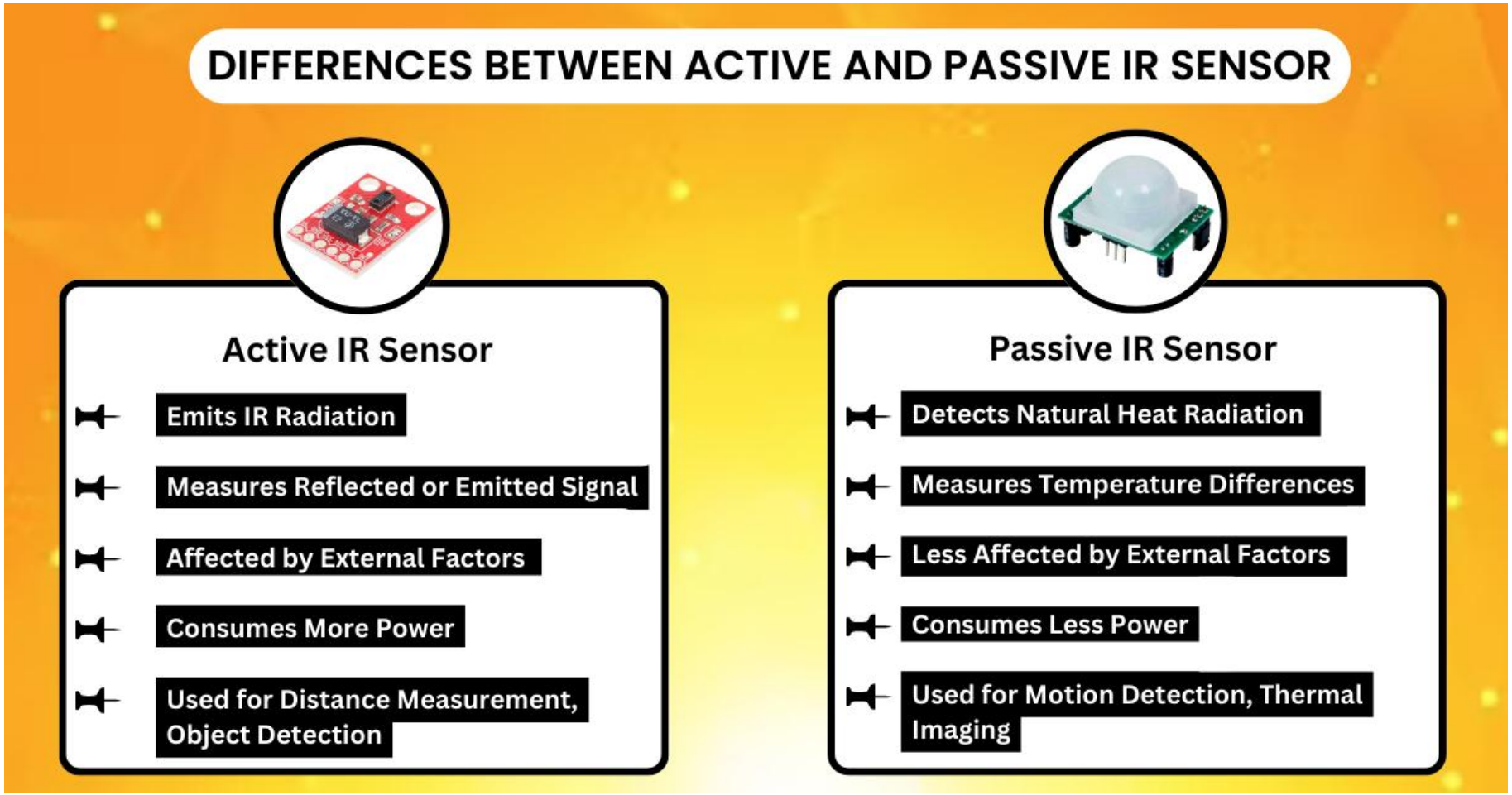
Active: Require power for the measurement

Passive: do not require an external power source for the measurement itself, but often need power to process or transmit the data

Sensors and actuators

Classification of Sensors

1. Energy Usage or Power Requirement



Sensors and actuators

Classification of Sensors

1. Energy Usage or Power Requirement

- based on whether or not they need an external power source, sensors are classed as active or passive sensors, respectively.
- PIR sensor, Radio-frequency identification (RFID) readers are examples of passive sensors that are also wireless sensors

Sensors and actuators

Classification of Sensors

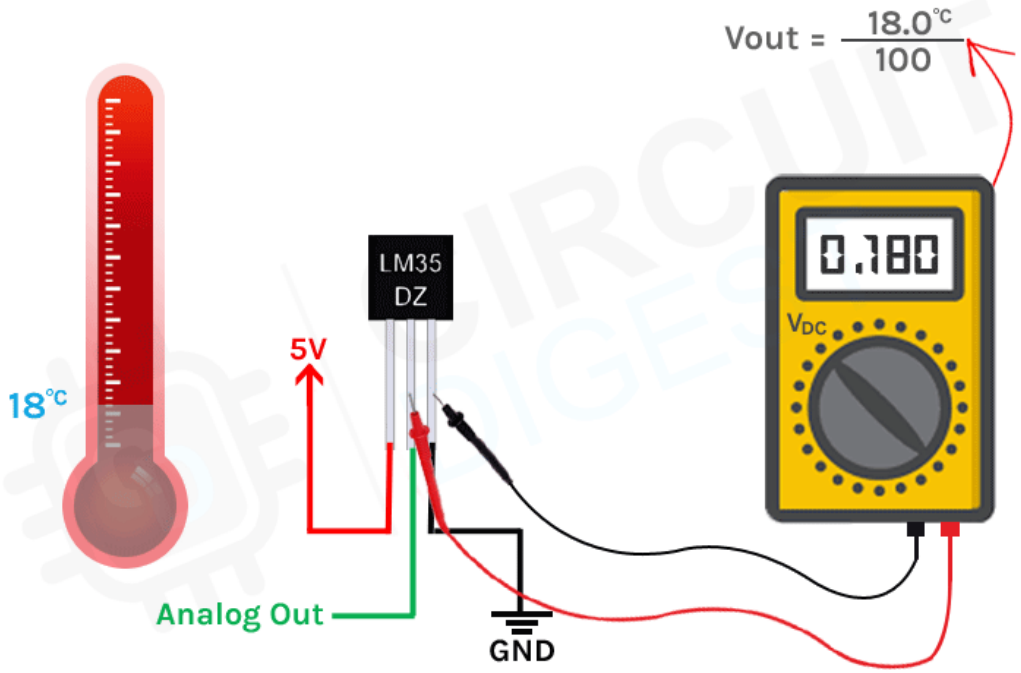
2. Output Type

- based on the type of output they produce, sensors can be categorized as analog or digital sensors. Analog sensors have a continuously variable output
- – *for example, an electrical voltage that varies continuously with temperature*.
- While digital sensors have a discrete output – *for example, pulse code modulation (PCM) sensors*

Sensors and actuators

Classification of Sensors

2. Output Type





Digital Temperature Sensor

Sensors and actuators

Classification of Sensors

3. Sensing Types/ Sensing Principle

a. Mechanical Sensors: Detect mechanical quantities like force, displacement, or acceleration using principles such as strain gauges, piezoelectric effects, or vibrating structures.

Example:

Strain gauge

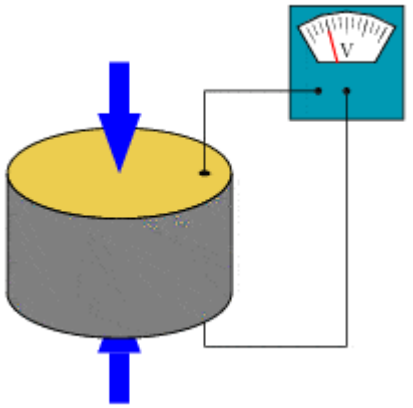
Piezoelectric sensor

Diaphragm/Bourdon tube pressure sensor

Load cell

Accelerometer

Tachometer



Piezoelectric pressure sensor

=>



Loadcell for measuring weight

Sensors and actuators

Classification of Sensors

3. Sensing Types/ Sensing Principle

b. Thermal Sensors: Measure temperature or heat changes based on temperature-dependent properties of materials. Examples include thermocouples, RTDs, and thermistors.





thermistors

Example:

Thermocouple
Temperature Detector (RTD)
Thermistor
Bimetallic strip
Pyrometer

Sensors and actuators

Classification of Sensors

3. Sensing Types/ Sensing Principle

c. Electrical Sensors: Utilize electrical properties and interactions to detect electrical quantities like current, voltage, or electric fields.

Example:
Resistive sensor
Capacitive sensor
Inductive sensor
Piezoresistive sensor



Potentiometer

Sensors and actuators

Classification of Sensors

3. Sensing Types/ Sensing Principle

d. Magnetic Sensors: Sense magnetic fields or changes in magnetic fields. Examples include Hall effect sensors and magneto resistors.

Example:

Hall effect sensor

Magnetoresistive sensor

Variable reluctance sensor

Fluxgate magnetometer



Hall Effect Magnetic Sensor



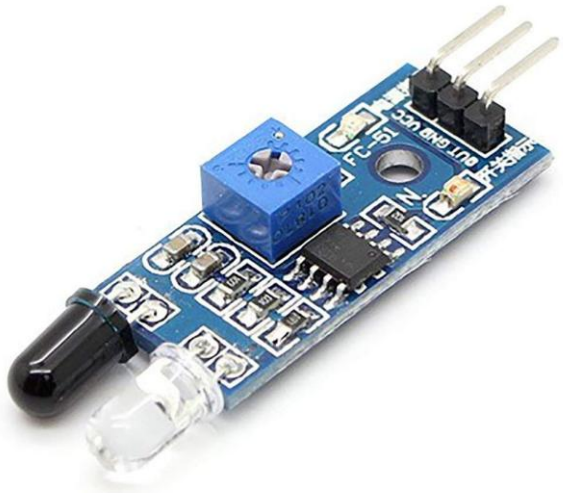
Magnetoresistive Element sensor

Sensors and actuators

Classification of Sensors

3. Sensing Types/ Sensing Principle

e. Radiant Sensors: Detect light, infrared radiation, or other forms of electromagnetic radiation using principles like photoconductivity or photoemission.



IR sensor

Example:

Photodiode / Phototransistor
Photovoltaic cell (solar cell) Infrared (IR) sensor
Optical encoder
LDR (Light Dependent Resistor)
UV sensor / Radiation detector (Geiger counter)

Sensors and actuators

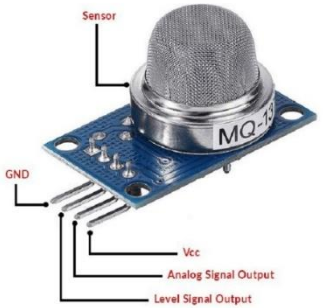
Classification of Sensors

3. Sensing Types/ Sensing Principle

f. Chemical Sensors: Detect specific chemical substances in the environment through chemical reactions or selective membrane properties.

Example:

pH sensor
Gas sensor (e.g., MQ series)
Electrochemical sensor
Chemiresistor
Biosensor
Humidity sensor (hygrometer)



MQ-137-Ammonia sensor



MQ-7-Carbon Monoxide sensor

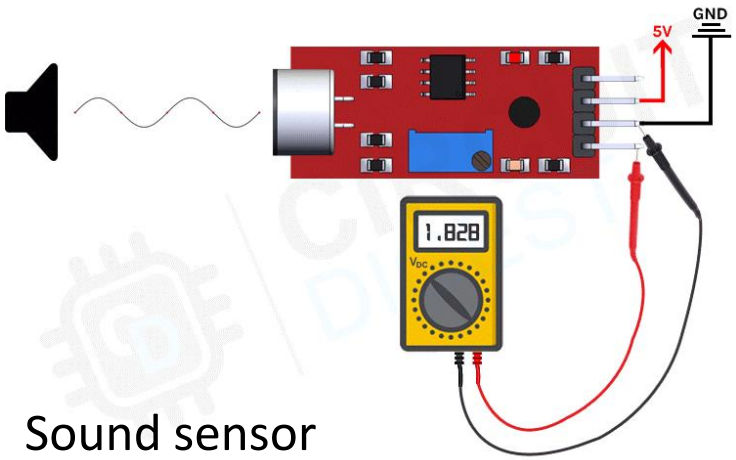


pH sensor-Chemical sensor

Sensors and actuators

Classification of Sensors

4. Input Type



Sound sensor



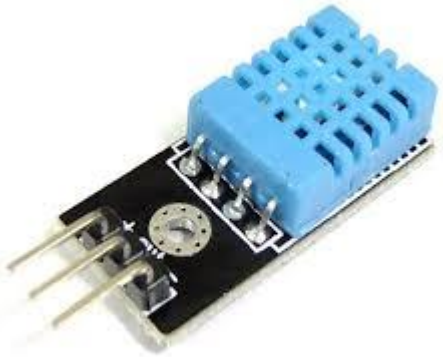
Light sensors

Sensors and actuators

Classification of Sensors

4. Input Type





Temperature sensors





Photo Diode

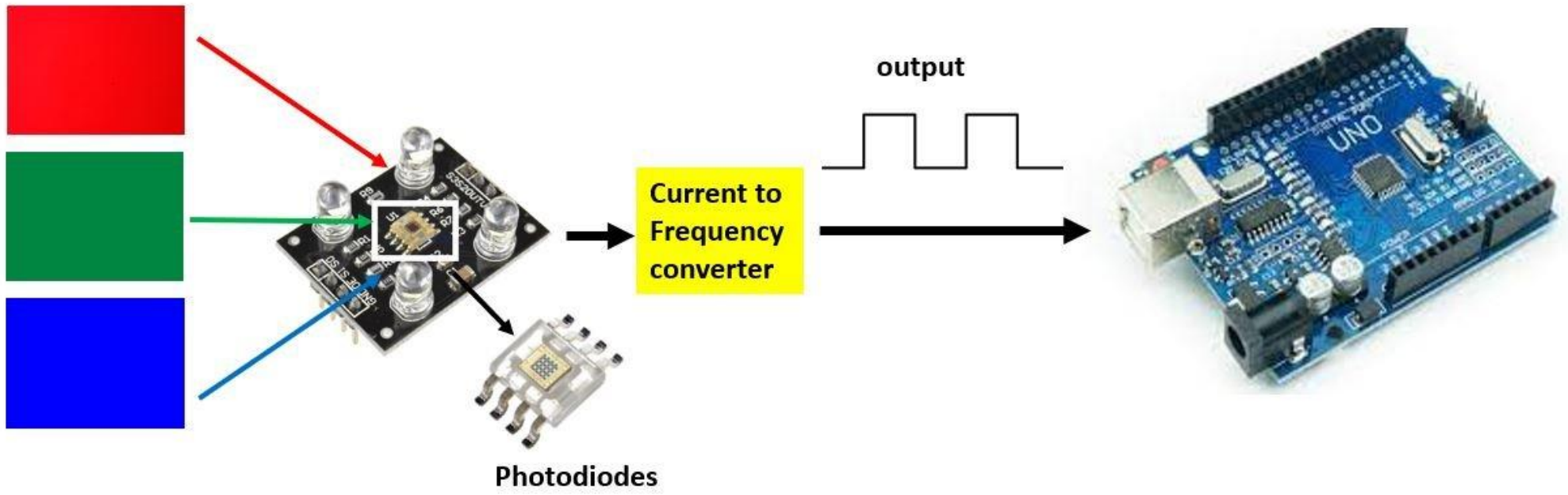
Photo Transistor

Optical Sensors

Sensors and actuators

Classification of Sensors

4. Input Type



Colour sensors

Sensors and actuators

Classification of Sensors

- 4. Input Type
- sound, light, temperature, electromagnetic radiation, acceleration, velocity, and color, are just a few examples of sensor types based on what they measure

Sensors and actuators

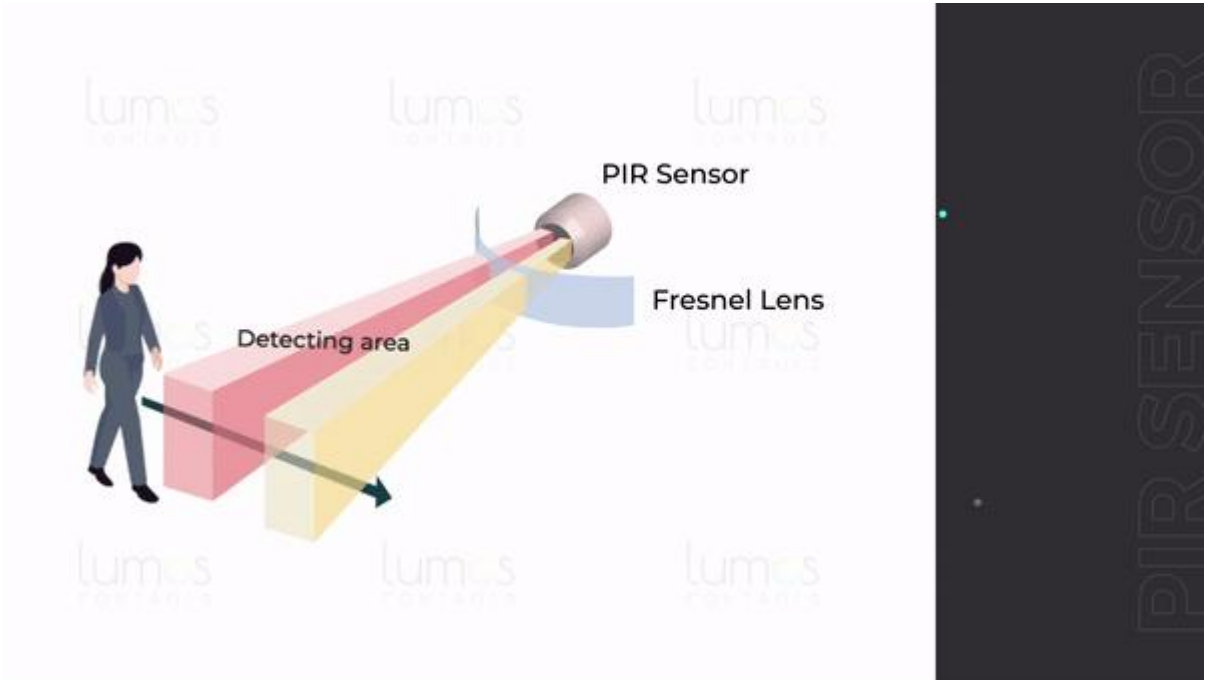
Characteristics of Sensors or Selection Criteria of Sensors

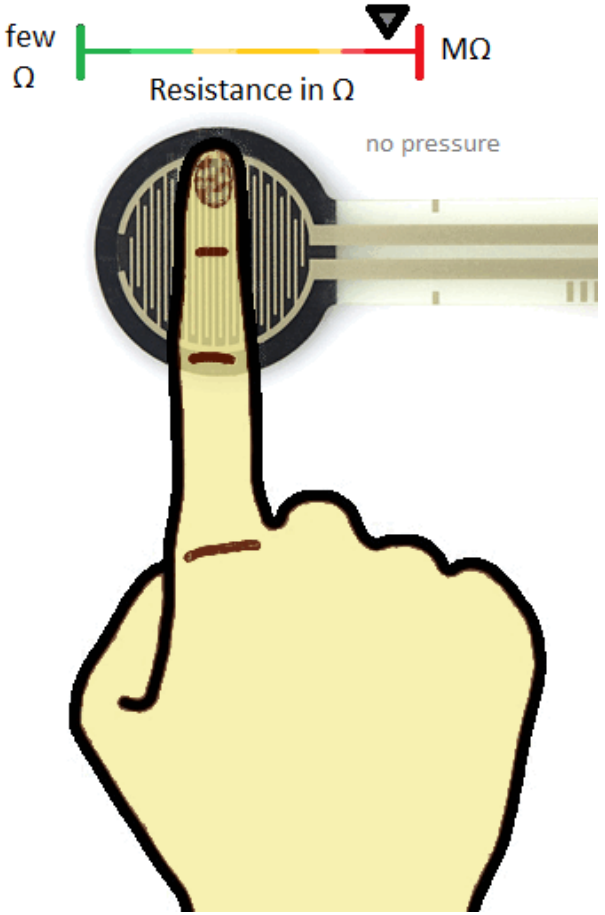
- Range
- Resolution
- Accuracy
- Precision
- Sensitivity
- Zero offset
- Linearity
- Zero Drift
- Response time
- Bandwidth
- Resonance
- Operating Temperature
- Deadband
- Signal to Noise ratio

Sensors and actuators

Range

Difference between the maximum and minimum value of the sensed parameter



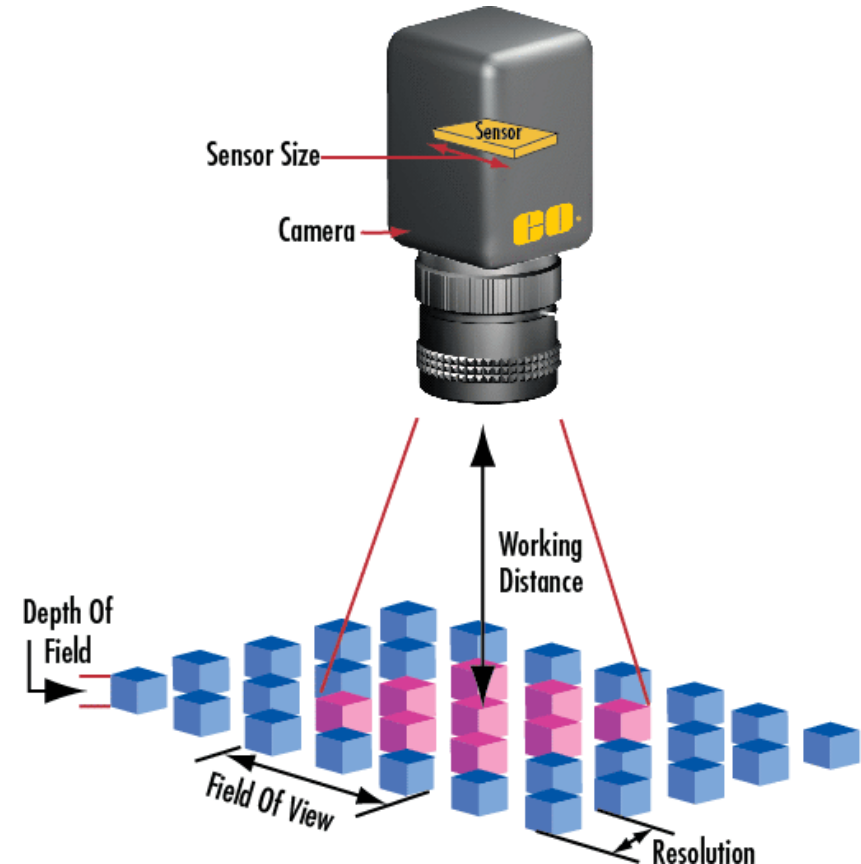


Sensors and actuators

Resolution

The smallest change the sensor can differentiate.

- For digital sensors, it is related to number of bits used
- For analog sensors, it is limited by low-level electrical noise

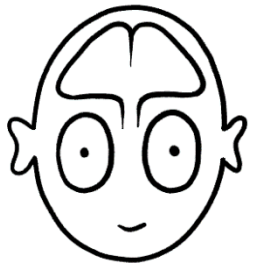


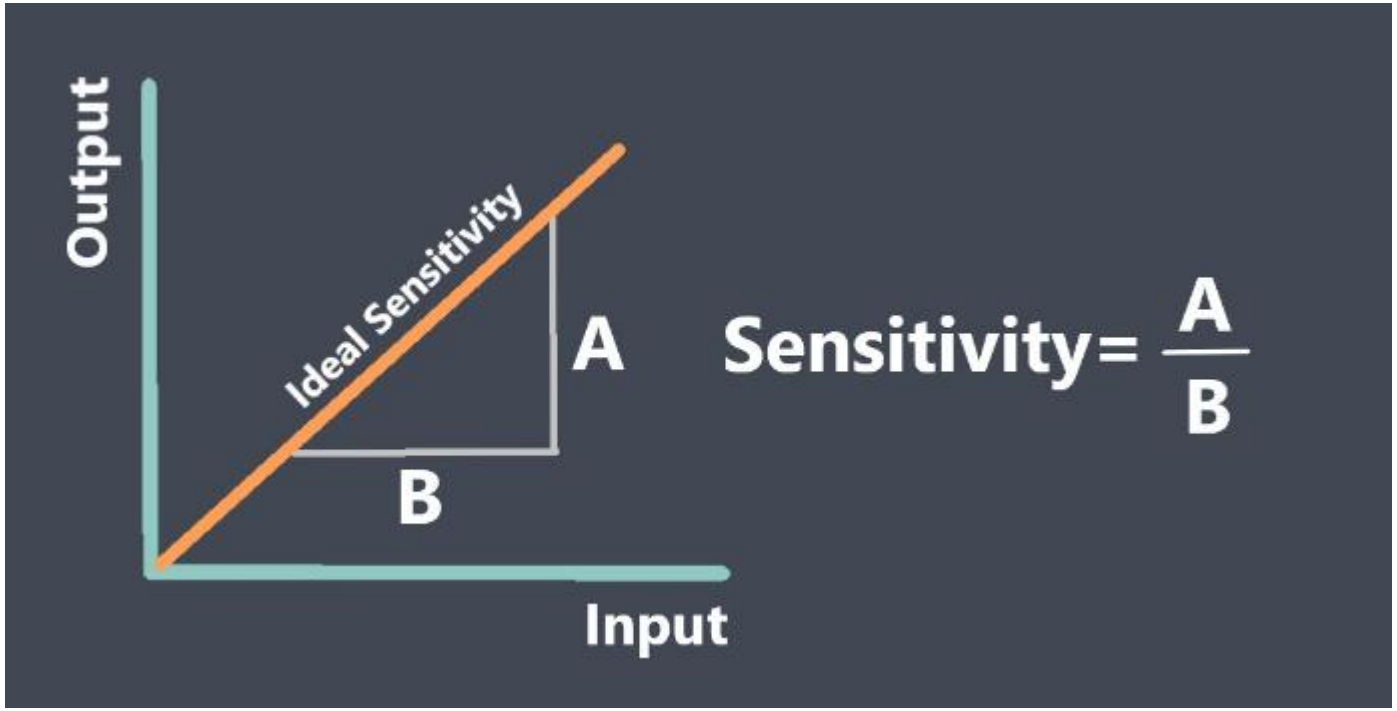
Sensors and actuators

Sensitivity

Ratio of change in output to a unit change of the input

- *For digital sensors, sensitivity is closely related to resolution*
- *For analog sensors, sensitivity is the output slope vs. input line*





Sensors and actuators

- Accuracy
  - Difference between the measured value and the true value



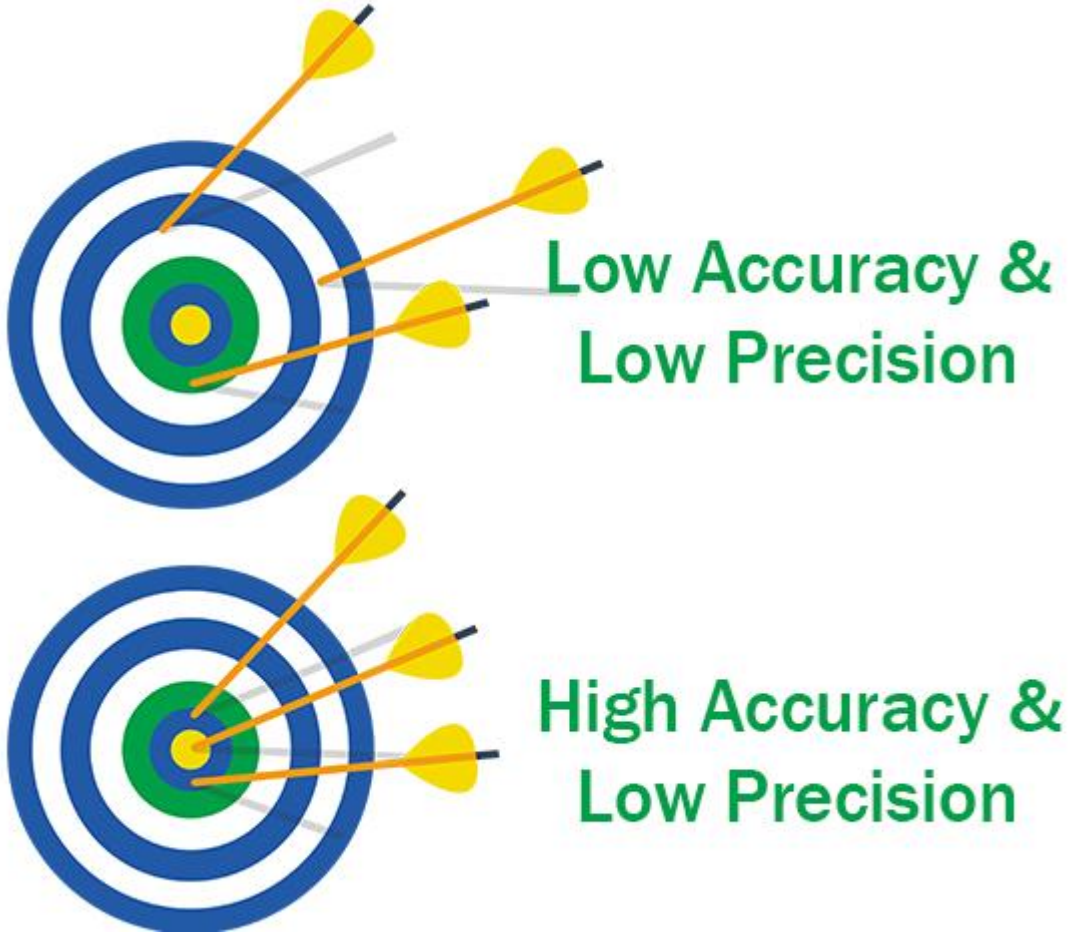
Sensors and actuators

- Precision
  - Ability to reproduce repeatedly with the given accuracy



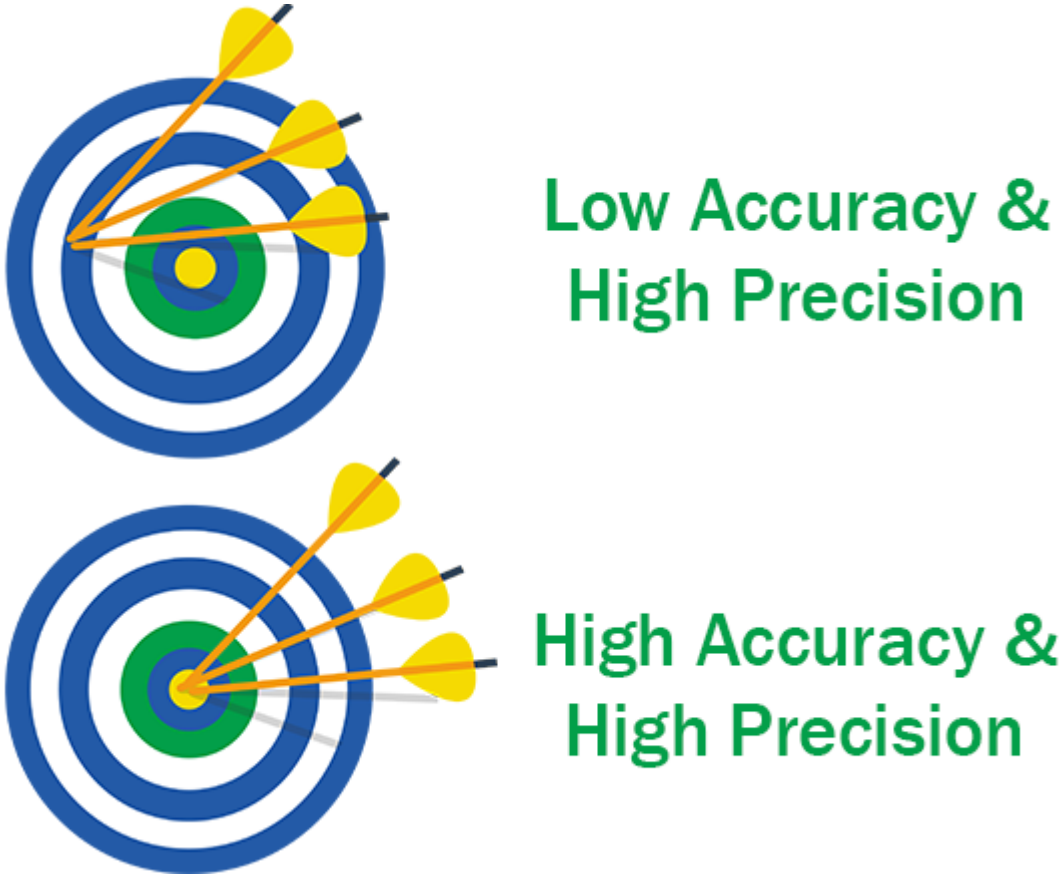
Sensors and actuators

Accuracy Vs Precision



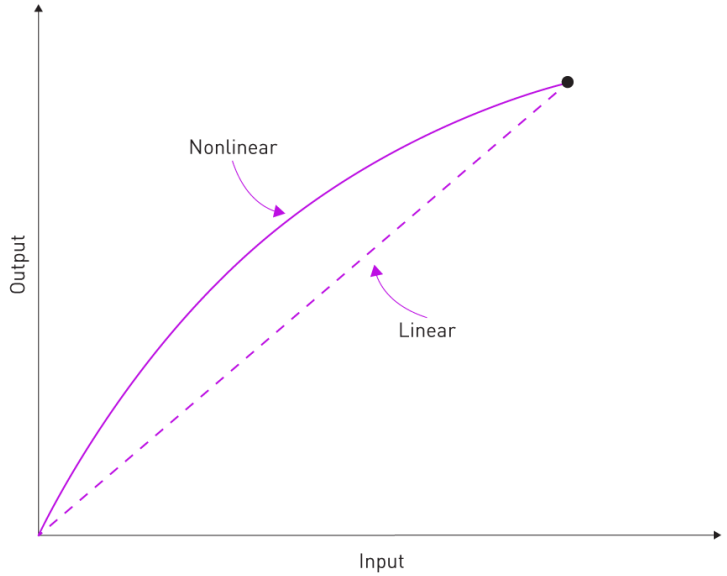
Sensors and actuators

Accuracy Vs Precision



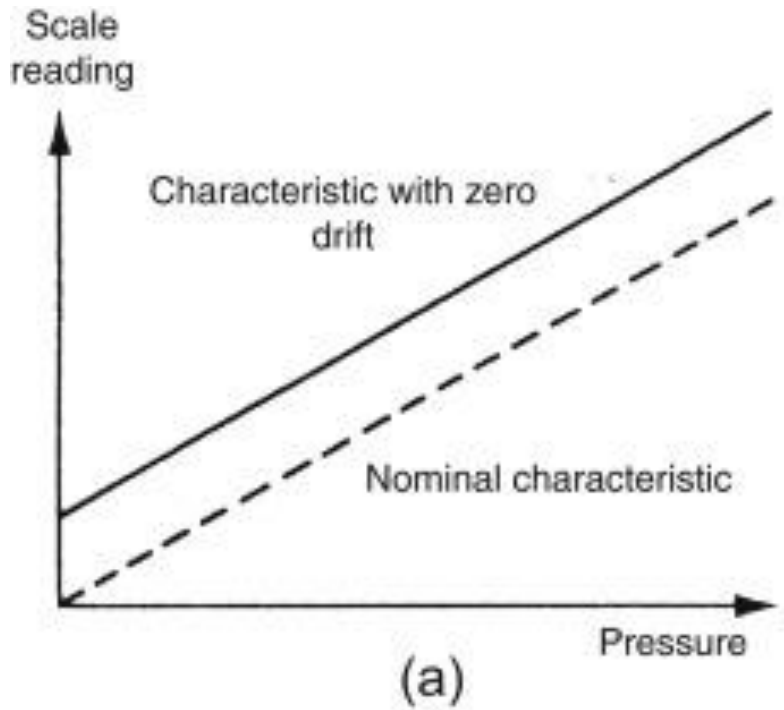
Sensors and actuators

- Zero offset
  - Non zero value output for no input
- Linearity
  - Percentage of deviation from the best fit linear calibration curve

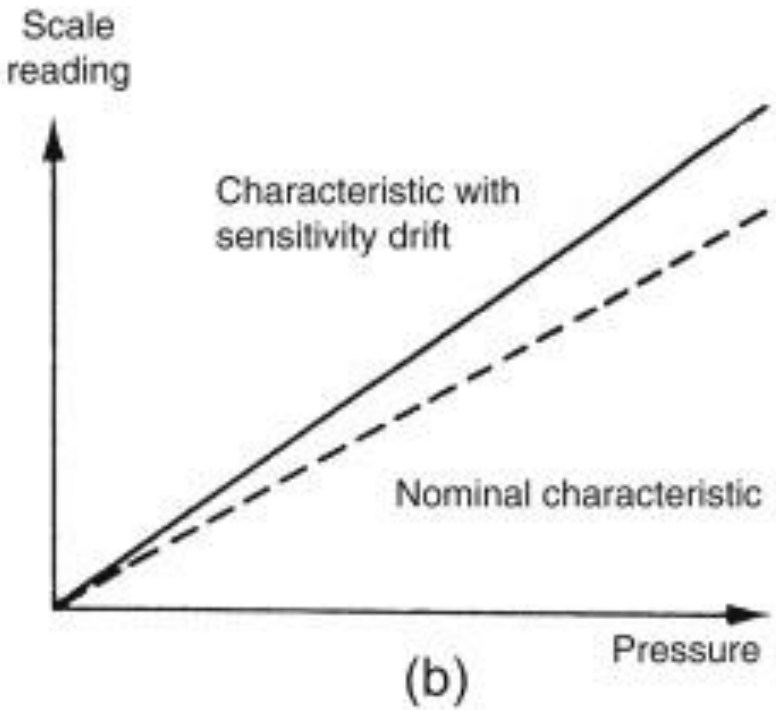


Sensors and actuators

- Zero drift
  - The departure of the output from zero value over period of time for no input



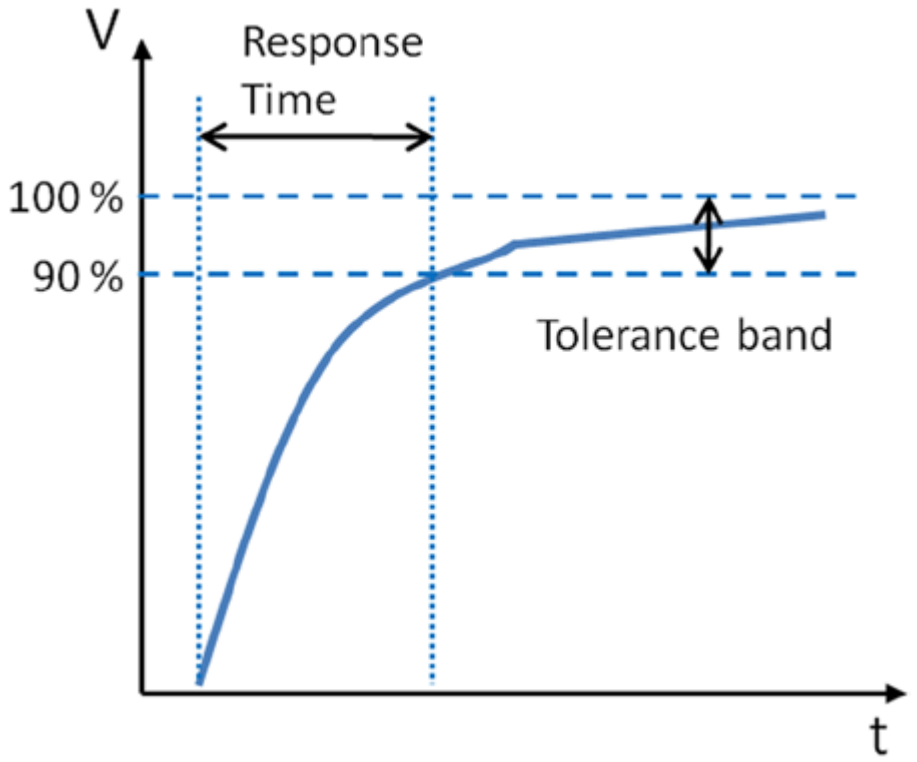
Zero drift



Sensitivity drift

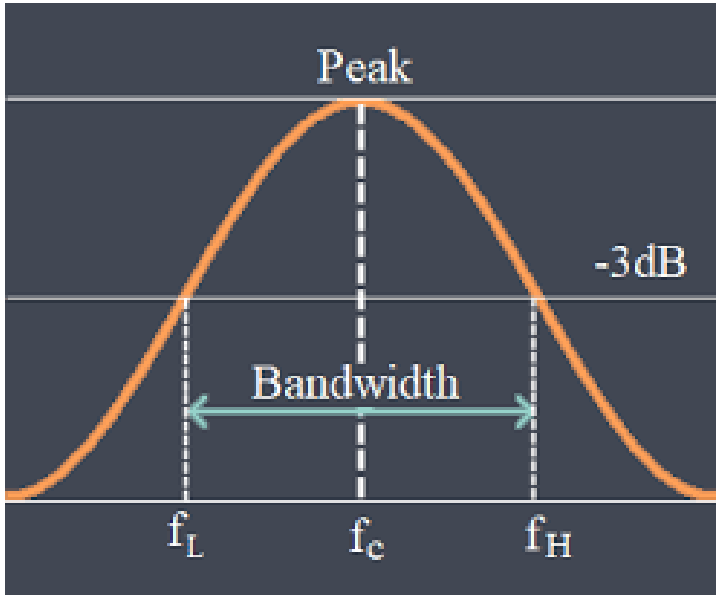
Sensors and actuators

- Response Time
  - The time lag between the input and output



Sensors and actuators

- Bandwidth
  - Frequency at which output magnitude drops by 3 dB
- Resonance
  - Frequency at which the output magnitude peak occurs



Sensors and actuators

- Operating temperature
  - The range in which the sensor performs as specified
- Deadband
  - Range of input for which there is no output
- Signal to noise ratio
  - Ratio between the magnitude of the signal and the noise at the output

Sensors and actuators

Various Sensors

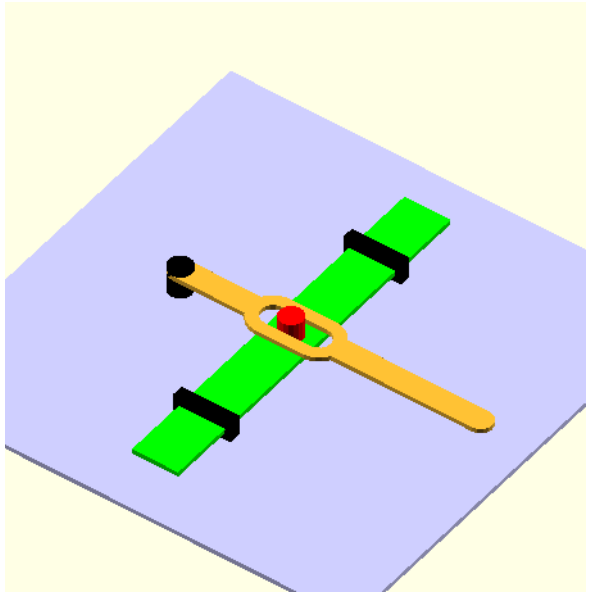


Sensors and actuators

Actuators

- Actuators are components that can take physical actions based on the input from the sensors.
- Actuators convert energy into mechanical motion.
- Types of Motions

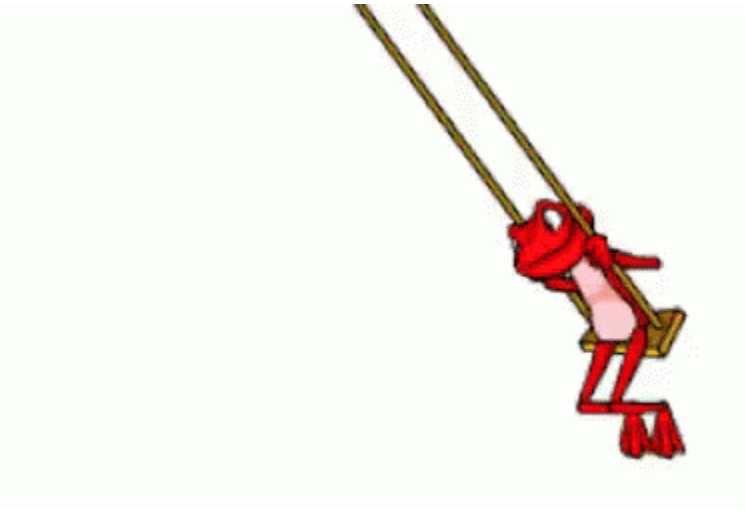
•Linear



•Rotary



•Oscillatory



Sensors and actuators

Types of Actuators



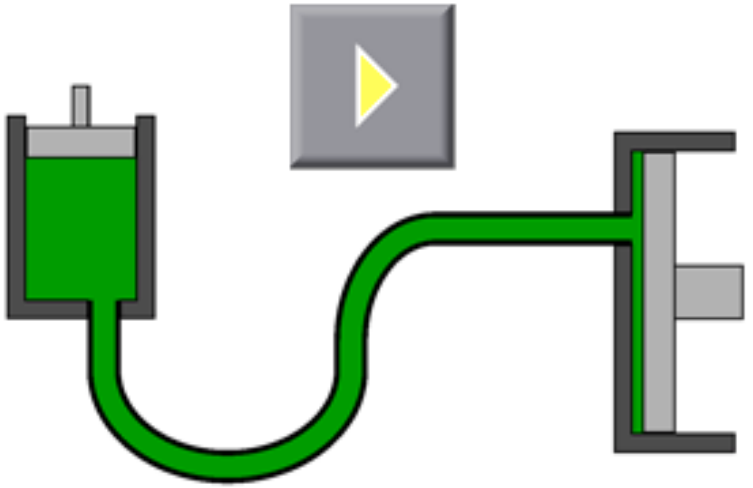
Type	Energy Source
Hydraulic	Fluid pressure
Pneumatic	Compressed air
Electric	Electricity
Thermal	Heat
Magnetic	Magnetic field

Sensors and actuators

Types of Actuators



1. Hydraulic:

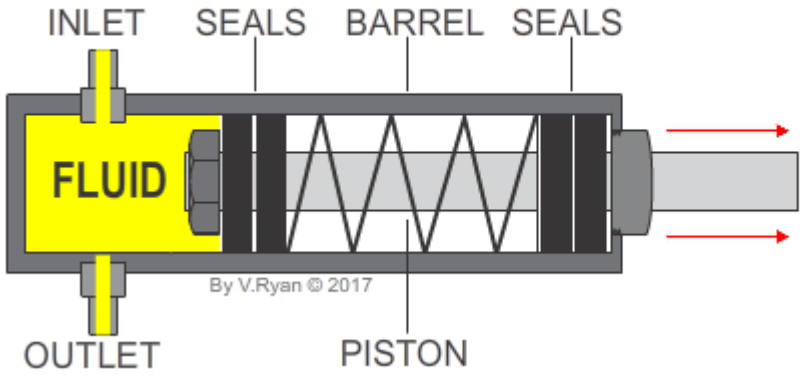


Basic Hydraulic System

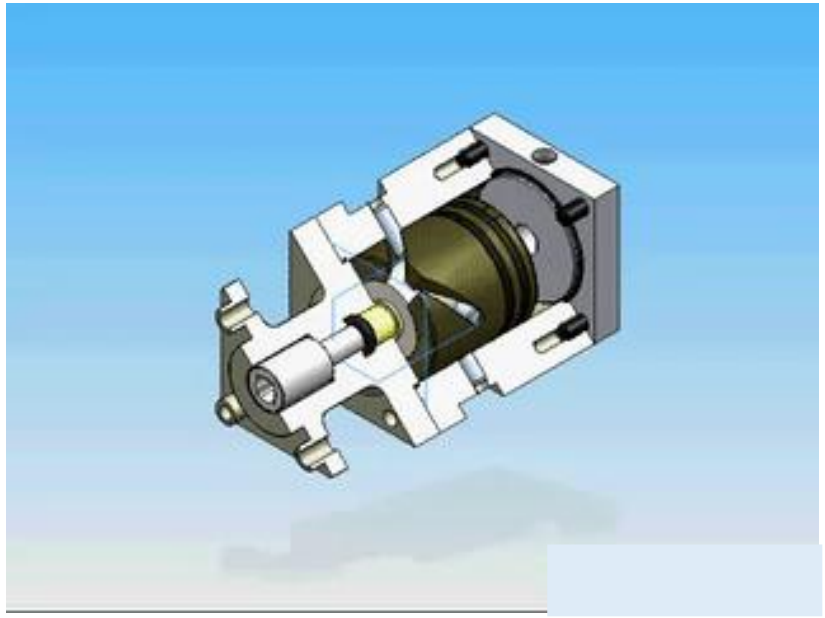
Sensors and actuators

Types of Actuators

1. Hydraulic:



Hydraulic Cylinder



Rotary Actuator

Sensors and actuators

Types of Actuators

1. **Hydraulic**: These actuators harness hydraulic power to perform mechanical functions and operations.

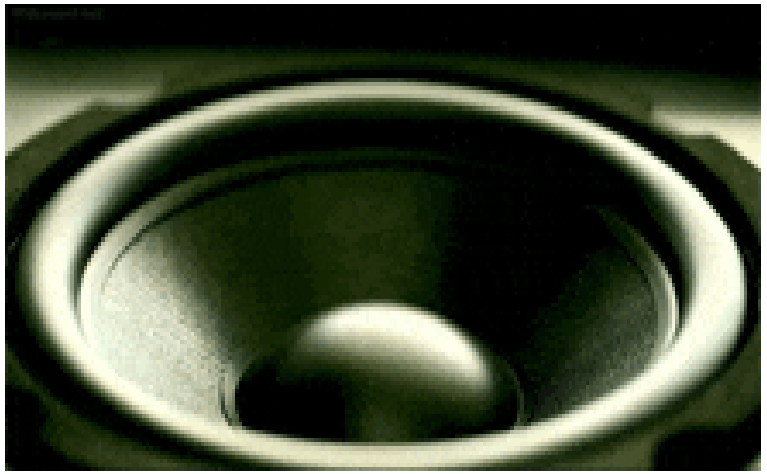
Example:

- Excavator arms/buckets,
- Aircraft landing gear,
- Car braking systems,
- Elevators (hydraulic lifts)

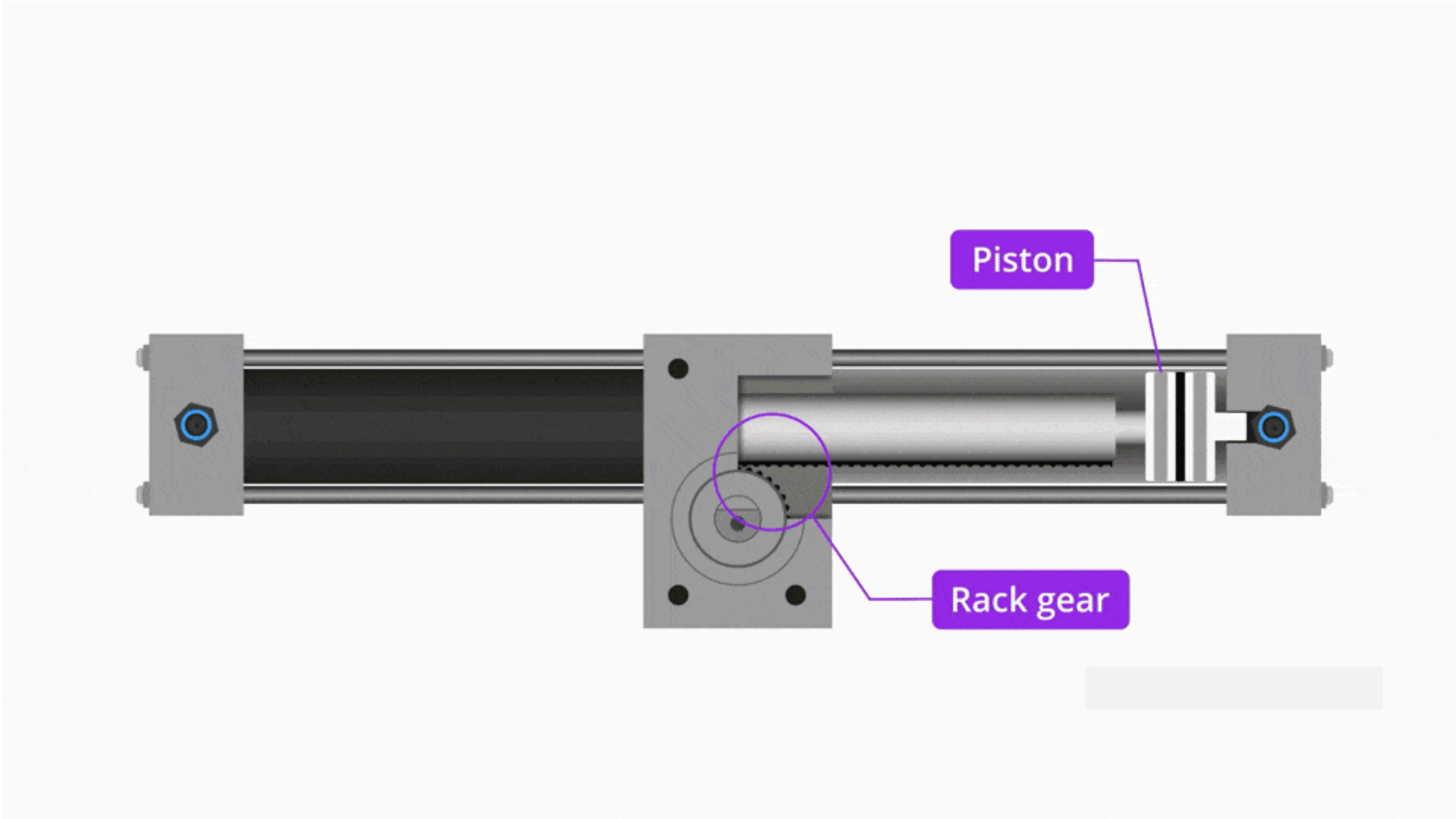
Sensors and actuators

Types of Actuators

2. Pneumatic:



Sound Speaker



Sensors and actuators

Types of Actuators

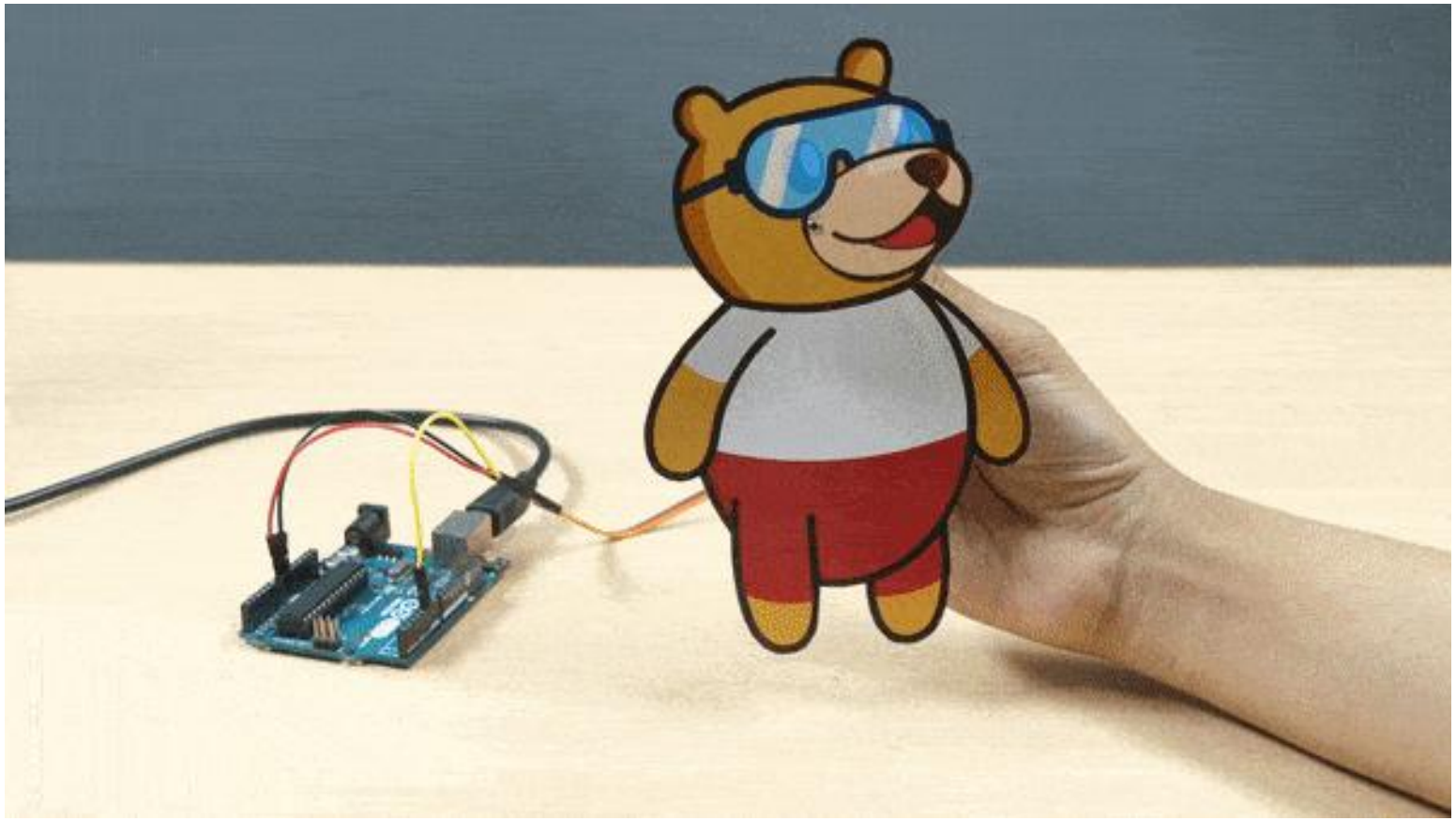
2. Pneumatic: They are powered by a vacuum or compressed air at high pressure to implement the required type of motion.

- Automatic doors (buses, trains, malls),
- Robotic pick-and-place arms,
- Dental chairs,
- Bottle filling/capping machines,
- Train brakes (air brakes)

Sensors and actuators

Types of Actuators

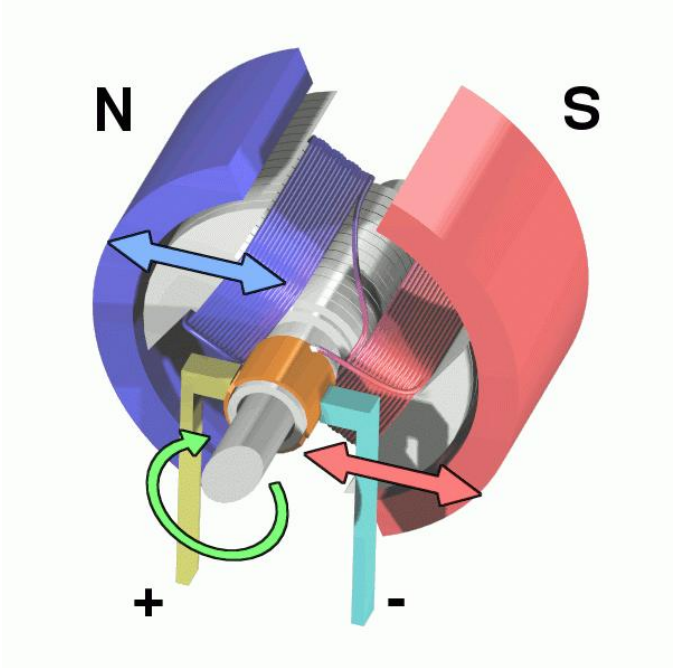
3. Electrical:

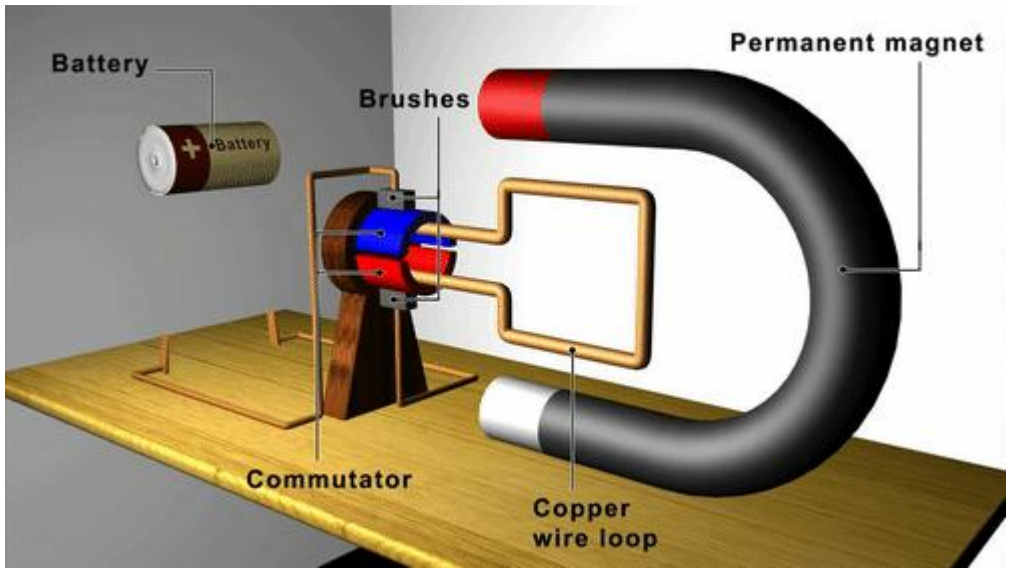


Sensors and actuators

Types of Actuators

3. Electrical:





Sensors and actuators

Types of Actuators

3. Electrical: In these actuators, a motor converts electrical energy into mechanical motion.

Example:

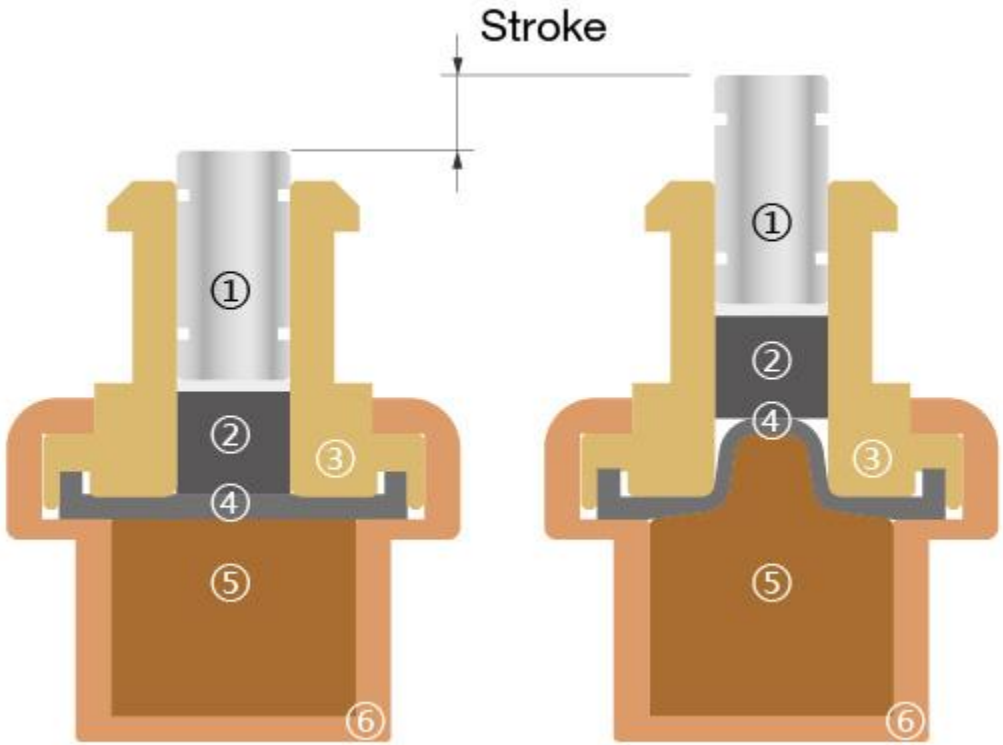
- Electric power windows/mirrors (cars)
- 3D printers
- Robotic arms (industrial robots)
- Electric door locks
- Camera autofocus lens

Sensors and actuators

Types of Actuators

4. Thermal:

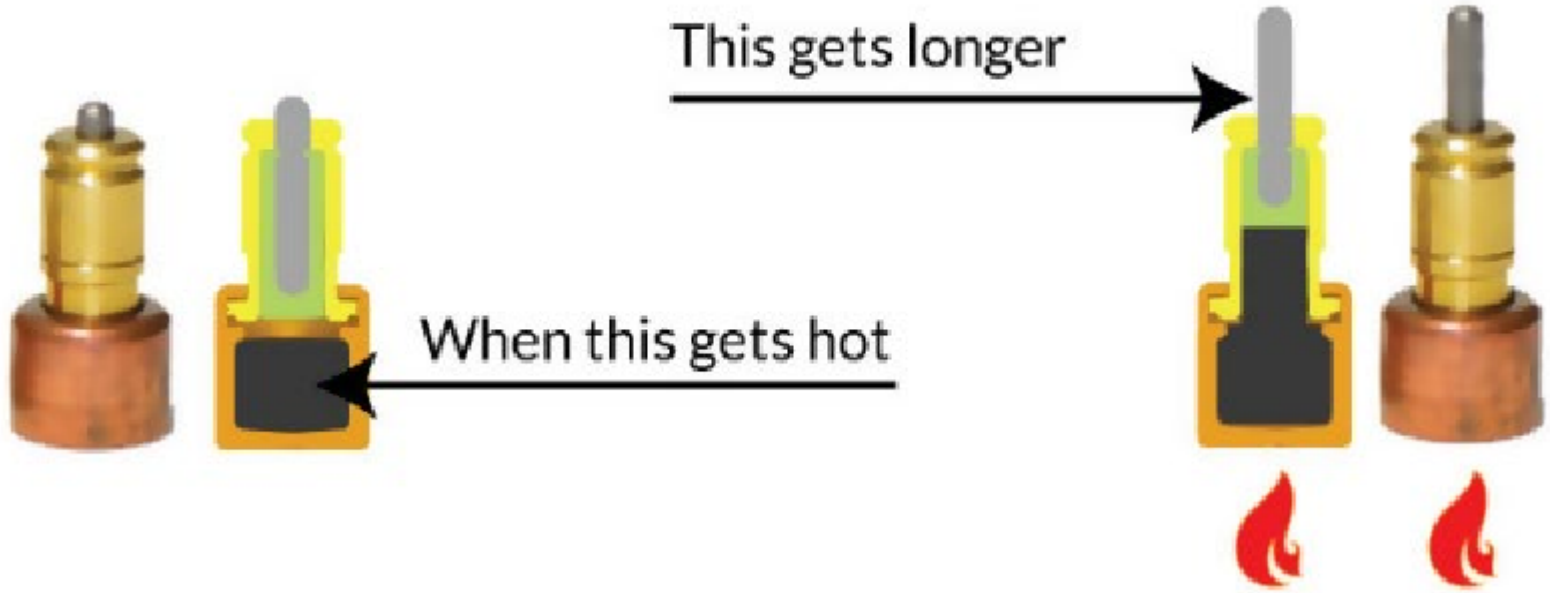




Sensors and actuators

Types of Actuators

4. Thermal:



Sensors and actuators

Types of Actuators

4. Thermal: The thermal actuators have thermal-sensitive material fitted inside, to produce motion.

Temperature-response systems.

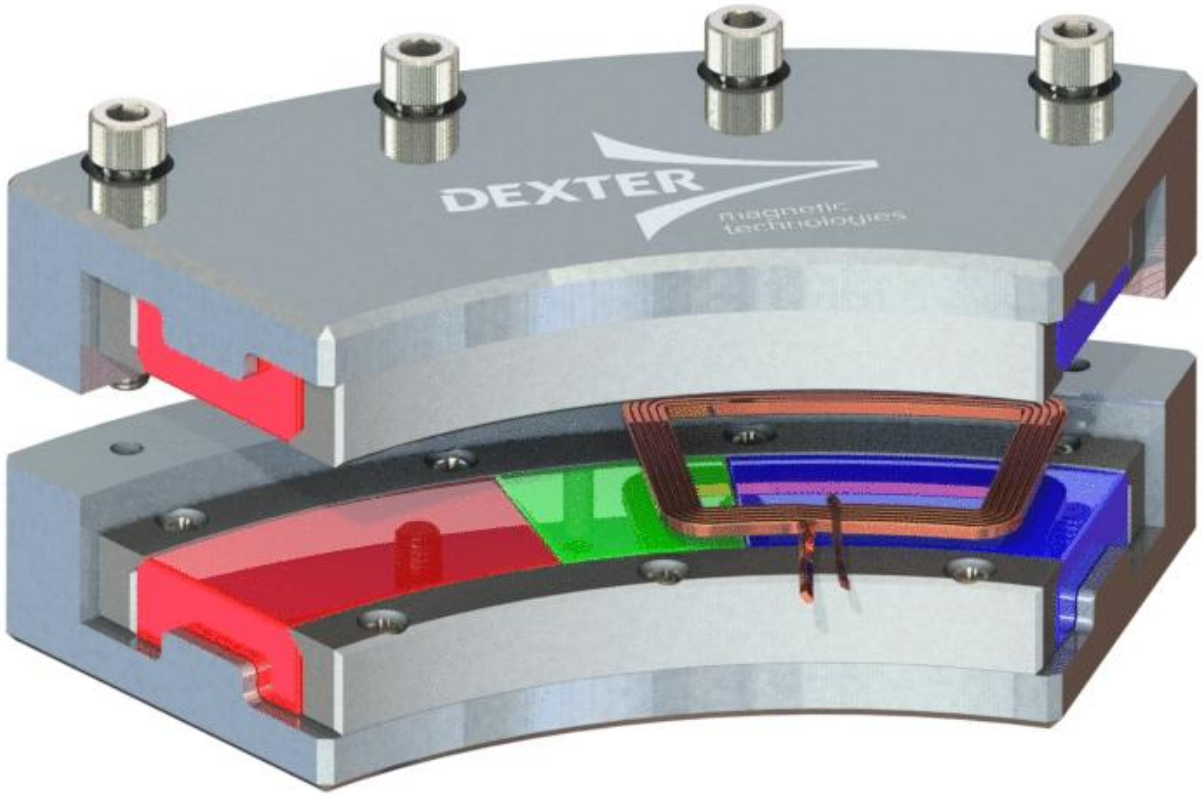
Example:

- Car radiator thermostat valve
- Fire sprinkler systems
- Circuit breakers (thermal-magnetic type)

Sensors and actuators

Types of Actuators

5. Magnetic:



Sensors and actuators

Types of Actuators

5. Magnetic: These types of actuators convert electromagnetic energy into mechanical output.

Example:

- Relays & contactors
- MRI machines
- Magnetic levitation (Maglev) trains
- Solenoid valves (industrial/automotive)
- Voice coil actuators in speakers

Sensors and actuators

Actuators-Solenoid- electromagnetic actuator

- A solenoid is a type of electromagnet formed by a helical coil of wire whose length is substantially greater than its diameter, which generates a controlled magnetic field.

